MRSpinDynamics

Python User Manual

Python magnetic-resonance spin-dynamics workspace

Simulation workflows for magnetic-resonance spin dynamics from experiment description to reproducible result: validated spin-1/2 Bloch kernels, small dense Hamiltonian models, an open sequence interchange layer, declarative experiment planning, and targeted models for NQR, ESR/EPR, relaxation, transport, fields, and instrument hardware.

July 2026

Contents

Part I

Getting Started

1 Start Here: Your First Experiment

This manual is written for two kinds of reader: someone who wants a reliable simulation running quickly, and someone who needs to understand or extend the physics underneath it. Both begin in the same place. Describe a small experiment, inspect what the package plans to run, execute it, and then follow the result down into the relevant model only when you need more detail.

In this chapter. Install the package, run a CPMG echo train, understand the returned objects, and choose a reading path through the rest of the manual.

1.1 The Shortest Useful Path

Create and activate a Python 3.10 or newer environment. From the `PythonSpinDynamics` directory, install the package with plotting and SciPy-backed analysis support:

```
python -m pip install -e ".[opt,plot]"
python examples/experiment_facade_quickstart.py --num-echoes 8
```

The quickstart prints a plan before it runs. Read that plan as a preflight check: it names the resolved numerical workflow, reports compatibility warnings, and estimates cost. A plan with errors will not execute. This separation is useful when an experiment includes a large walker ensemble, a dense powder average, or hardware settings that would otherwise fail late.

1.2 The Same Experiment in Python

The recommended high-level interface is `Experiment`. Its specifications say what should be simulated; the workflow registry decides which validated runner should do the work.

```
from spin_dynamics.experiment import (
    Acquisition, CPMGTrain, Experiment, Hardware, Sample,
)

study = Experiment(
    sequence=CPMGTrain(num_echoes=8),
    sample=Sample(t1_seconds=2.0, t2_seconds=0.25),
    hardware=Hardware(probe="ideal"),
    acquisition=Acquisition(numpts=101, maxoffs=10.0),
)

plan = study.plan()
print(plan.report())
if not plan.ok:
    raise RuntimeError("fix plan errors before running")

record = study.run()
print(record.result.echo.shape)
record.save("results/first_cpmg.npz")
```

There are three objects to keep straight:

`study` is the immutable experiment description. It is safe to serialize, compare, and reuse.

`plan` is the resolved workflow plus findings and a cost estimate. Inspect it before an expensive run.

`record` contains the native scientific result, the plan, and provenance such as package version, platform, workflow, and elapsed time.

If you already know the exact workflow you need, direct calls such as `run_tuned_cpmg(...)` remain public and fully supported. The facade adds planning, compatibility checks, configuration, and provenance; it does not replace the underlying dynamics.

1.3 Configuration and the Command Line

Experiments can also live in TOML or JSON. This is convenient for parameter studies, archived runs, and users who do not want to edit Python for every acquisition. The installed `spin-dynamics` command and the module form are equivalent:

```
spin-dynamics plan examples/experiment_config_cpmg.toml
spin-dynamics run examples/experiment_config_cpmg.toml -o results/cpmg.npz
spin-dynamics show results/cpmg.npz
spin-dynamics verify results/cpmg.npz
```

Use `plan` while editing a configuration, `run` when the plan is clean, `show` to inspect a saved run, and `verify` to rerun it and compare the result and its reproducibility metadata.

1.4 A Mental Map of the Package

Most tasks cross only three layers:

Layer	Start here	Use it when
Experiment	`spin_dynamics.experiment`	You want one plan/run/save interface, configuration files, or provenance.
Workflow	`spin_dynamics.workflows`	You know the experiment family and want direct control of its scientific inputs and outputs.
Model/kernel	Topic namespaces such as `nqr`, `esr`, `coupling`, `motion`, and `fields`	You are building a new workflow, validating an equation, or need a specialized model directly.

1.5 Choose Your Reading Path

You do not need to read the manual linearly.

- **New user:** read Chapters 1–5, then the unified experiment workflow in Chapter 10 and the relevant recipe in Chapter 11.

- **NQR, ESR, or coupled-spin user:** skim the physical scope and conventions, then go directly to Chapter 8, 7, or 6.
- **Sequence developer:** read Chapter 9, then the experiment facade and workflow guide.
- **Reviewer or validator:** begin with the validation chapter and follow each claim to its machine-readable evidence and reproducer.

Every chapter opens with an "In this chapter" box. The boxes are a fast route through the narrative; the API reference at the end is for lookup, not for first-time reading.

2 Physical Scope and Assumptions

Before choosing an API, choose the physical description. PythonSpinDynamics is not a single universal spin simulator. It is a set of closely related models, each chosen because it gives a clean description of a common magnetic-resonance experiment. The central question throughout the manual is therefore: what is represented explicitly, and what is represented by an effective parameter?

The MATLAB-compatible workflow layer answers that question in the classical Bloch language. It models a bath of uncoupled spin-1/2 nuclei evolving in a static, spatially non-uniform, or time-varying main field $B_0(\mathbf{r}, t)$ and applied RF fields. The elementary object is an isochromat: a subensemble that shares an offset, RF sensitivity, relaxation constants, and probe response. Other namespaces add small quantum Hamiltonians, quadrupolar sites, ESR spectra, exchange, and microscopic relaxation where the physics requires those variables to be explicit.

In this chapter. Match an experiment to the appropriate model layer and understand which effects are explicit, effective, or outside the package's scope.

Modeled directly in the core Bloch workflows. Independent spin-1/2 magnetization vectors; offset distributions from $B_0(\mathbf{r}, t)$; RF rotations; free precession; phenomenological T_1 and T_2 relaxation; probe transmit and receive filtering; optional deterministic radiation damping; optional motion through field maps.

Scalar-coupling extension. The `spin_dynamics.coupling` namespace adds small dense spin-1/2 systems with scalar J couplings, analytic low-field J-editing models, ideal TANGO-B filter curves, and initial SLIC simulations. These models are intentionally scoped and are documented in Chapter 6.

Pulsed NQR extension. The `spin_dynamics.nqr` namespace adds dense single-site quadrupolar spin tools for selective pulsed NQR, including spin-1 and spin-3/2 transition metadata, single-crystal and powder orientation averages, SLSE echo trains, two-frequency population transfer, EFG inhomogeneity, finite-window SLSE spectra, and weak-B0 perturbations. These models are documented in Chapter 8.

RFI and reference-channel rejection. The `spin_dynamics.interference` namespace models radio-frequency interference as a vector magnetic field, samples it with primary and reference coils, and removes it with masked, adaptive, windowed, or echo-aware joint estimators. The RFI tools are sequence agnostic; the NQR layer supplies SLSE and SORC acquisition masks on the same sample clock.

ESR/EPR extension. The `spin_dynamics.esr` namespace adds single-electron spin-1/2 ESR tools, including scalar or anisotropic g tensors, single-crystal and powder spectra, CW absorption/derivative lineshapes, static g-strain and field-strain distributions, rectangular pulsed ESR FID and Hahn-echo simulations with Liouville-space T_1/T_2 relaxation, and a first dense isotropic electron-nuclear hyperfine model. These tools are documented in Chapter 7.

Chemical / site exchange extension. The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange: a bath of magnetically distinct sites that swap magnetization at finite first-order kinetic rates while each site relaxes with its own T_1/T_2 and precesses at its own offset. It provides kinetic generators with detailed-balance checks, transverse lineshape coalescence, longitudinal mixing propagators, and encode-mix-detect T_2-T_2 relaxation exchange (REXSY) data that inverts to an exchange map. These tools are documented in Chapter 11.

Internal / susceptibility gradients. The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast in porous media. It provides analytic two-dimensional cylindrical-grain off-resonance maps that feed the moving-isochromat pipeline and pore-space internal-gradient distributions for diffusion-in-internal-gradient studies. These tools are documented in Chapter 11.

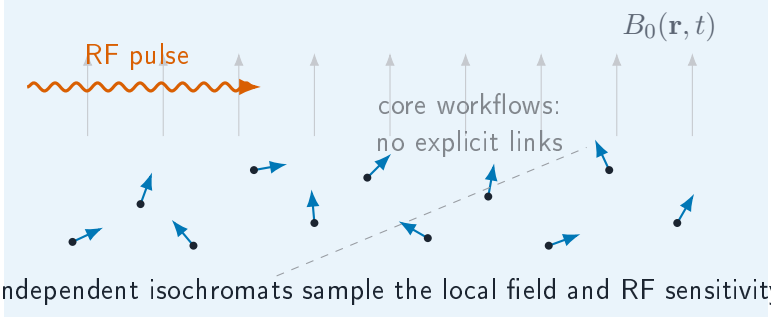
Coupled thermal modeling. The `spin_dynamics.thermal` namespace adds a thermal solver alongside the field solvers: duty-cycled heat sources (coil Joule power, gradient-waveform power, and sample SAR from the eddy solver), a lumped RC network with conduction/convection/radiation links, a quasi-static electro-thermal coupling loop that feeds coil $R(T)$, Q, and the sample temperature back into the noise and magnetization models, and 1D/axisymmetric conduction with an optional Pennes blood-perfusion sink. The solvers are validated analytically and against FEMM and are documented in Section 11.5.8.

Flowing samples. The `spin_dynamics.flow` namespace models measurements on a moving sample (inline pipe monitoring, stopped-flow, LC-NMR) in plug or laminar regimes: washout of excited spins during acquisition (flow-shortened apparent T_2), flux-weighted transit-time inflow polarization, and the matching advective heat removal in the thermal layer. These tools are documented in Section 11.5.9.

Still outside the package scope. Dipolar coupling networks, arbitrary nonselective multi-quantum pulse sequences, shaped sample microstructure beyond the supplied field and density maps, and large sparse coupled-spin simulations. Ensemble-level spin noise is modeled semiclassically by `spin_dynamics.spin_noise` (sample impedance in the receiver circuit and a stochastic Langevin source coupled to radiation damping); only the fully quantum measurement-back-action description remains out of scope. Thermal transport uses lumped and 1D/axisymmetric solvers with film-coefficient convection; full 3D conduction, CFD flow solving, and turbulent-flow washout are not modeled. Spin-spin effects enter the Bloch workflows only indirectly when represented by effective relaxation times such as T_2 or T_2^*. The Redfield/dipolar relaxation model treats fluctuating dipolar couplings as stochastic bath sources; it is not a coherent many-spin dipolar network. Site exchange is modeled phenomenologically by the `spin_dynamics.exchange` layer rather than from a microscopic exchange Hamiltonian.

This scope is deliberate. The package is strongest when the model boundary is chosen honestly: Bloch isochromats for independent subensembles, dense Hamiltonians for small quantum problems, and stochastic Redfield terms when the microscopic couplings are important but the bath itself is not being propagated.

2.1 Spin Bath Cartoon



2.2 Consequences for Users

- Use offset grids, field maps, or moving particles to represent spatially varying B_0, transmit B_1, and receive B_1 effects.
- Use relaxation constants to represent ensemble dephasing and energy exchange that are outside the explicit isochromat model.
- Use `spin_dynamics.coupling` when a small spin-1/2 scalar J-coupled model is the phenomenon of interest.
- Use `spin_dynamics.esr` for single-electron ESR spectra, pulsed ESR FID/echo simulations, static disorder studies, and simple isotropic electron-nuclear hyperfine doublets.
- Use `spin_dynamics.nqr` when selective pulsed NQR of a small quadrupolar site is the phenomenon of interest.
- Use `spin_dynamics.exchange` for Bloch-McConnell site exchange.
- Do not use the package for large coherent coupled-spin networks or arbitrary nonselective multi-quantum coherence pathways.

3 Overview

PythonSpinDynamics began as a parity-focused Python port of the MATLAB spin-dynamics simulation package and has grown into a broader, typed Python workspace. The MATLAB V3 tree remains an important numerical reference for the ported kernels; new Python-native layers now add experiment planning, sequence interchange, validation evidence, transport, hardware, and specialized NQR and ESR workflows.

In this chapter. See how the repository is organized and how the Bloch, quantum, sequence, experiment, and validation layers fit together.

Read the manual in layers. Chapters 2–5 define the model boundaries and shared conventions. Chapters 6, 7, and 8 introduce the explicitly quantum extensions. Chapter 9 describes the backend-neutral sequence representation, Pulseq interchange, compiler, and timeline visualizer. Chapter 10 presents the unified experiment facade – the recommended declarative entry point that ties the sample, hardware, sequence, acquisition, and transport model together, plans a run, and saves it. Chapter 11 then documents the underlying experiment-level workflows the facade delegates to: CPMG, FID, imaging, diffusion, motion, field sources, phase cycling, exchange, analysis, and optimization. The final chapters collect runnable examples, validation status, API tables, and known gaps.

The supported surface includes ideal, tuned, untuned, and matched-probe CPMG workflows; finite echo trains; FID; phase-encoded and frequency-encoded imaging; deterministic PGSE and seeded random-walker diffusion/flow; moving-isochromat simulations; Pulseq 1.4/1.5 interchange; WURST and UDD timing helpers; radiation damping; receiver noise; inverse-Laplace analysis; compact optimization scaffolds; small spin-1/2 scalar-coupling models; pulsed NQR; ESR/EPR; Bloch-McConnell exchange; susceptibility fields; and opt-in Liouville-space relaxation models.

3.1 Package Layout

Path	Purpose
`src/spin_dynamics/`	Installable Python package.
`examples/`	CLI examples and plotting scripts.
`tests/`	Smoke tests, validation tests, and unit tests.
`validation/fixtures/`	MATLAB/Octave fixture arrays.
`validation/octave/`	Fixture generation scripts.
`docs/python_api/`	Lightweight Markdown API notes.
`docs/validation_results.md`	Current validation status and tolerance notes.
`docs/radiation_damping.md`	Focused radiation-damping model notes.
`docs/spin_noise.md`	Spin-noise physics analysis and model options.
`docs/thermal_modeling.md`	Coupled thermal-modeling options and work plan.
`docs/flow_modeling.md`	Flowing-sample model design and validation.
`validation/femm/`	FEMM cross-validation scripts (thermal).

4 Installation and Test Runs

Installation has two goals: keep the numerical core lightweight, and let users opt into plotting, optimization, acceleration, or development tools only when they need them.

In this chapter. Choose an installation profile, verify the command-line entry point, and run the appropriate test tier for your work.

4.1 Editable Install

PythonSpinDynamics requires Python 3.10 or newer. The core install depends on NumPy 1.24 or newer and does not require MATLAB at runtime. MATLAB is only needed to regenerate the complete MATLAB reference fixture set; Octave can regenerate the smaller dependency-light fixtures.

From `PythonSpinDynamics`, install the package into an active environment:

```
python -m pip install -e .
```

Optional extras are provided for development, plotting, benchmarking, and SciPy-backed optimization:

```
python -m pip install -e ".[dev,opt,plot,bench]"
```

The core package depends only on NumPy. The opt extra enables SciPy-backed continuous optimizers, non-negative inverse-Laplace solves, and MATLAB `.mat` result import/export helpers. The `plot` extra enables the Matplotlib examples.

Confirm that the console entry point is available:

```
spin-dynamics --help
```

4.2 Tests

Use the smoke tier during normal editing:

```
python -m unittest tests.smoke_tests
```

Run the full validation suite before committing numerical or workflow changes:

```
python -m unittest discover -s tests
```

5 Model Conventions

This chapter collects conventions that appear across several workflows. The details are intentionally close to the equations, because most user mistakes in this package come from mixing coordinate conventions, timing conventions, or relaxation levels of description.

In this chapter. Learn the units, state ordering, coordinate systems, timing, phase, and relaxation conventions shared by otherwise different workflows.

5.1 Coherence Ordering

Low-level kernels preserve MATLAB's coherence ordering:

$$\mathbf{M} = \begin{bmatrix} M_0 & M_- & M_+ \end{bmatrix}^{\mathsf{T}}.$$

This ordering differs from many Bloch-vector presentations, so new kernels and validation tests should state the expected ordering explicitly.

5.2 Normalized Time

The core MATLAB routines usually normalize time to the nominal RF amplitude $\omega_1 = 1$. In these units, a 90-degree hard pulse has duration

$$t_{90} = \frac{\pi}{2},$$

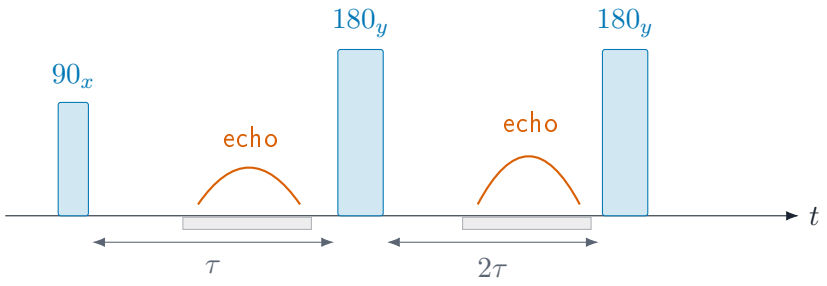
and a 180-degree hard pulse has duration

$$t_{180} = \pi.$$

Workflow helpers that expose physical seconds convert into these normalized units before calling the lower-level kernels.

5.3 Pulse Timing Cartoon

A minimal spin-echo or CPMG building block is a sequence of RF pulses, free precession windows, and acquisition windows. The package represents these as ordered intervals with pulse matrices and free-precession delays.



5.4 Effective Rotations

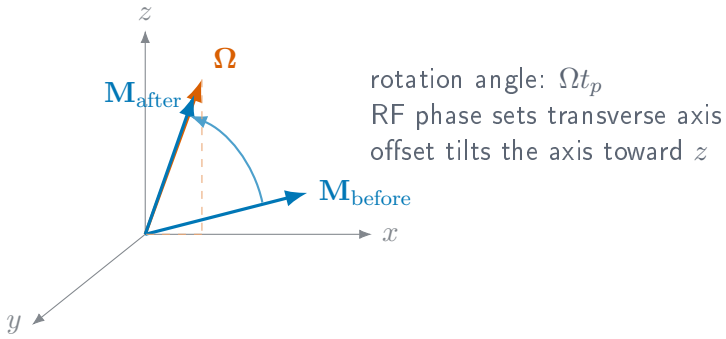
For an isochromat with normalized offset $\Delta\omega$, RF amplitude ω_1, phase ϕ, and pulse duration t_p, the rotating-frame effective field is

$$\boldsymbol{\Omega} = \begin{bmatrix} \omega_1 \cos\phi \\ \omega_1 \sin\phi \\ \Delta\omega \end{bmatrix}, \qquad \Omega = \sqrt{\omega_1^2 + \Delta\omega^2}.$$

The magnetization update is a rotation by angle Ωt_p about $\boldsymbol{\Omega}/\Omega$. The Python rotation helpers expose this calculation through `spin_dynamics.core.rotations` and return array shapes chosen to match the MATLAB reference fixtures.

5.5 Rotation Cartoon

In the rotating frame, a rectangular RF segment rotates the magnetization about an effective axis set by RF phase, RF amplitude, and off-resonance offset.



5.6 Initial Magnetization and Relaxation

Relaxation appears in PythonSpinDynamics at three different levels. The Bloch workflows use T_1 and T_2 factors attached to free-precession intervals. Prepolarization helpers use the same longitudinal recovery law to prepare a non-thermal starting magnetization. The shared `spin_dynamics.relaxation` module adds Liouville-space relaxation models, including opt-in Redfield/dipolar dissipators for spin-1/2 NMR and quadrupolar NQR, plus stochastic collision models for gas-wall relaxation. Keeping these levels separate is important: a fitted T_2, a BPP rate estimate, a Redfield bath model, and a wall-collision model can describe related decays, but they enter the sequence simulator at different points.

5.6.1 Bloch Relaxation in Isochromat Workflows

In the MATLAB-compatible Bloch kernels, relaxation is diagonal in the local magnetization basis. Free-precession intervals use the usual exponential factors:

$$M_{xy}(t + \Delta t) = M_{xy}(t) \exp\left(-\frac{\Delta t}{T_2}\right) \exp(-\mathrm{i}\Delta\omega\, \Delta t),$$

$$M_z(t+\Delta t) = M_{\mathrm{th}} + (M_z(t) - M_{\mathrm{th}}) \exp\left(-\frac{\Delta t}{T_1}\right).$$

Finite acquisition workflows assemble these intervals from pulse-sequence parameters and then call the `arb1O`-style kernels.

5.6.2 Preparing Longitudinal Magnetization

Many compact and low-field experiments do not begin at the thermal equilibrium magnetization of the detection field. The `spin_dynamics.prepolarization` module computes prepared longitudinal magnetization values in the same normalized units used by the sequence kernels. For a sample relaxing in a polarizing field B_{pol}, then detected at B_{det}, the full polarizer equilibrium in detection-field units is

$$M_{\mathrm{pol,eq}} = M_{\mathrm{det,eq}} \frac{B_{\mathrm{pol}}}{B_{\mathrm{det}}}.$$

Finite prepolarization time is then just the usual longitudinal buildup:

$$M_0(t_{\mathrm{pol}}) = M_{\mathrm{pol,eq}} + (M_{\mathrm{init}} - M_{\mathrm{pol,eq}}) \exp\left(-t_{\mathrm{pol}}/T_1\right).$$

For flow or sample transport, `prepolarized_flow_state` first converts a path length and local speed into a residence time. The returned `PrepolarizedMagnetization` object exposes `m0` and `mth` arrays that can be passed into the lower-level kernels or used to initialize moving isochromats.

5.6.3 BPP Scalar Rate Estimates

For prepolarization and compact low-field estimates, the same module also provides a BPP-style scalar relaxation model. The rotational spectral density is

$$J(\omega) = \frac{2\tau_c}{1+\omega^2\tau_c^2},$$

with an Arrhenius temperature dependence

$$\tau_c(T) = \tau_{\mathrm{ref}} \exp\left[\frac{E_a}{R}\left(\frac{1}{T} - \frac{1}{T_{\mathrm{ref}}}\right)\right].$$

The default rate ratios are the common dipolar-style combinations

$$R_1 = C\left[J(\omega_0) + 4J(2\omega_0)\right],$$

$$R_2 = C\left[\frac{3}{2}J(0) + \frac{5}{2}J(\omega_0) + J(2\omega_0)\right],$$

where C absorbs the spin-pair constants and convention-specific prefactors. Custom coefficient triples $(J(0), J(\omega_0), J(2\omega_0))$ can be supplied for other dipolar conventions or phenomenological fits.

```
import numpy as np
from spin_dynamics.prepolarization import prepolarized_state
from spin_dynamics.relaxation import BPPRelaxationModel

model = BPPRelaxationModel(
    angular_frequency_rad_per_s=2*np.pi*20e6,
    tau_ref_seconds=8e-9,
```

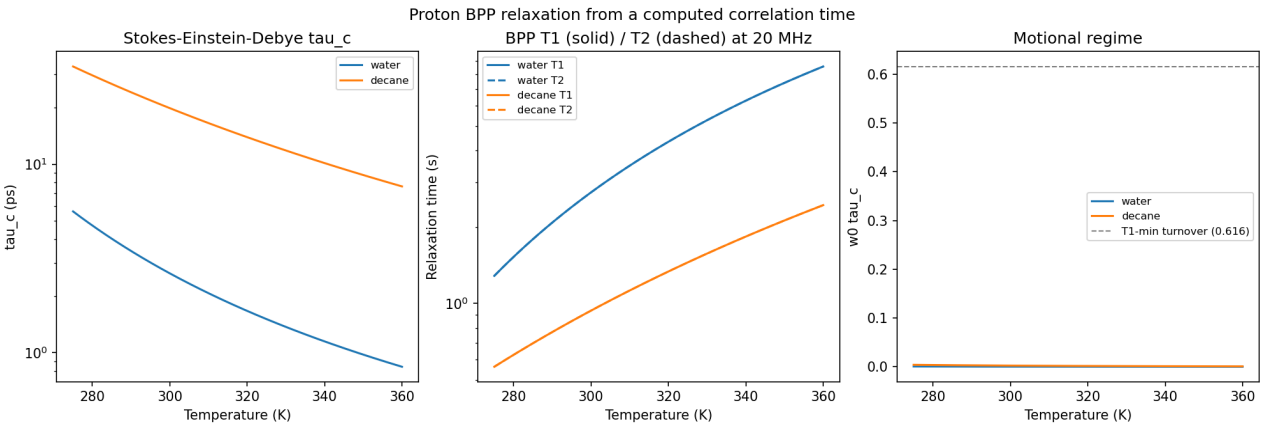


Figure 5.1: From molecular motion to relaxation. The Stokes–Einstein–Debye correlation time (left) feeds the BPP spectral density and produces the temperature-dependent T_1 and T_2 curves (center). The motional-regime panel (right) makes clear that both liquids remain deep in extreme narrowing at 20 MHz over this temperature range.

```
    coupling_scale_per_second2=5e8,
    activation_energy_j_per_mol=16e3,
)
rates = model.rates([280.0, 300.0, 320.0])
prepared = prepolarized_state(
    polarizing_field_tesla=0.1,
    detection_field_tesla=50e-6,
    prepolarization_time_seconds=2.0,
    t1_seconds=rates.t1_seconds,
)
```

Reproduce Figure 5.1 with:

```
python examples/plot_bpp_water_t1t2_temperature.py --output bpp.png
```

5.6.4 Field-Cycling and NMRD Workflows

An NMRD experiment asks how relaxation changes as the observation frequency moves across the molecular spectral density. In a field-cycling instrument, the sample relaxes at a sequence of magnetic fields and is usually detected at a fixed field afterward. The relaxation field sets the Larmor frequency $f_0 = |\gamma|B$, so field and frequency sweeps are two views of the same axis.

Use `run_nmrd` when correlation times and Larmor frequencies are already known. Use `run_field_cycling_nmrd` when the schedule is specified in tesla and the correlation time follows one Arrhenius law. Result arrays use the convention

$$\text{shape} = (\text{number of temperatures or conditions}, \text{number of fields}).$$

Each row is one NMRD curve. The result retains field and frequency axes, R_1/R_2, T_1/T_2, and $J(0)$, $J(\omega_0)$, and $J(2\omega_0)$, so the source of a dispersion turnover remains visible.

```
import numpy as np
from spin_dynamics.workflows import run_field_cycling_nmrd

fields_t = np.concatenate(([0.0], np.geomspace(1e-6, 3.0, 179)))
```

```
nmrd = run_field_cycling_nmrd(
    fields_t,
    temperature_kelvin=[280.0, 300.0, 320.0],
    gamma_hz_per_t=42.57747892e6,
    tau_ref_seconds=8e-9,
    reference_temperature_kelvin=300.0,
    activation_energy_j_per_mol=16e3,
    coupling_scale_per_second2=5e8,
)
# nmrd.r1_per_second.shape == (3, 180)
```

This workflow is a BPP baseline, not a universal inverse model. A measured dispersion may require several motions, chemical-shift anisotropy, exchange, surface relaxation, paramagnetic terms, or an instrument transfer function. Add missing physics rather than silently folding it into an "effective" activation energy. The example `plot_field_cycling_nmrd.py` shows the complete field-to-frequency workflow.

5.6.5 Transition-Resolved Quadrupolar Relaxation

For a quadrupolar nucleus, start with a `QuadrupolarSite`. The site fixes the isotope, spin, static quadrupole coupling, asymmetry, and NQR transition frequencies. Relaxation needs a separate description of the *time dependent* EFG. The static coupling C_Q is not the RMS fluctuation amplitude passed to the Redfield model.

`run_quadrupolar_relaxation` diagonalizes the site Hamiltonian, builds a secular fluctuating-EFG Redfield superoperator for each correlation time, and reports two local rates for every allowed transition:

R_1 initial decay of the transition population difference;

R_2 initial decay of its energy-basis coherence.

Arrays have shape `(n_correlation_times, n_transitions)`. Labels and frequencies stay in the same result so a rate column cannot silently become detached from its NQR line.

```
from spin_dynamics.nqr import quadrupolar_site
from spin_dynamics.workflows import run_arrhenius_quadrupolar_relaxation

site = quadrupolar_site("14N", cq_hz=3.0e6, eta=0.2)
quad = run_arrhenius_quadrupolar_relaxation(
    site,
    temperature_kelvin=[270.0, 300.0, 330.0],
    tau_ref_seconds=1e-9,
    reference_temperature_kelvin=300.0,
    activation_energy_j_per_mol=18e3,
    fluctuation_amplitude_hz=20e3,
)
print(quad.transition_labels)
print(quad.transition_frequencies_hz)
print(quad.r2_per_second)
```

These are initial single-rate summaries. Strongly coupled degenerate coherences can decay multiexponentially, in which case the full superoperator is the correct object. The validated `ZeroFieldRedfieldEFGModel` retains the non-diagonal Kramers-degeneracy terms required by the ^{209}Bi benchmark; the general workflow uses the completely-positive secular model. Use the specialized model for that zero-field coupled-coherence lineshape problem. The example `plot_quadrupolar_relaxation_workflow.py` plots all transitions and their spectroscopy metadata together.

5.6.6 What the Experimental Validation Establishes

Sodium chlorate tests weak-field powder splitting, spectral overlap, and a measured SLSE lifetime. The ^{209}Bi data test non-diagonal zero-field coherence relaxation. RDX tests an activated ^{14}N NQR branch ($91.8 \pm 1.4\,\mathrm{kJ\,mol^{-1}}$ from the digitized vertical uncertainty) and the alignment of proton NMRD minima with nitrogen transitions.

The read-only NQRDatabase audit shows why not every relaxation record is a fit target. Of 89 records, only two state a temperature and only nine preserve an uncertainty; none combines a complete series, pulse-sequence definition, uncertainty, and microscopic inputs. It also finds a factor-1000 unit conflict between duplicate sodium-nitrite sources. Consult `docs/relaxation_validation.md` before fitting a literature fixture.

5.6.7 Prepolarized Spin-Lock ($T_{1\rho}$) Dispersion

A spin-lock experiment measures relaxation in the presence of a resonant RF field: a $(\pi/2)_{-y}$ pulse tips the prepared magnetization onto the spin-lock axis, an on-resonance field of nutation frequency $\omega_1 = \gamma B_1$ holds it for a time T_{SL}, and the surviving locked magnetization is read out. The on-resonance dipolar rate is

$$\frac{1}{T_{1\rho}} = C\left[\tfrac{3}{2}J(2\omega_1) + \tfrac{5}{2}J(\omega_0) + J(2\omega_0)\right],$$

so, unlike T_2, $T_{1\rho}$ probes the spectral density at the operator-set spin-lock frequency $2\omega_1$ (typically 0.1–$3\,\mathrm{kHz}$). As $\omega_1 \to 0$ the expression reduces exactly to the transverse rate R_2 above, and it disperses downward once $\omega_1 \gtrsim 1/\tau_c$. Because the rotational correlation time grows with molecular size through the Stokes–Einstein–Debye relation $\tau_c = 4\pi\eta a^3 f/(3k_BT)$, sweeping the hydrodynamic radius a sweeps the dispersion: fast, small tumblers stay flat while slow, large ones show a strong $T_{1\rho}$ dispersion in the kHz band.

The helper `t1rho_relaxation_rate` returns $R_{1\rho}$, and `simulate_prepolarized_t1rho` composes prepolarization, the spin lock, and readout into a size × lock grid. The example `plot_t1rho_molecular_size_dispersion.py` maps a set of molecule radii to their dispersion curves and prepolarized contrast.

```
import numpy as np
from spin_dynamics.relaxation import (
    simulate_prepolarized_t1rho,
    stokes_einstein_debye_correlation_time,
)

radii_m = np.array([0.5, 2.0, 6.0, 20.0]) * 1e-9
tau_c = stokes_einstein_debye_correlation_time(
    radii_m, viscosity_pa_s=0.1, temperature_kelvin=310.0
)
experiment = simulate_prepolarized_t1rho(
    spin_lock_angular_rad_per_s=2*np.pi*np.geomspace(20.0, 20e3, 160),
    larmor_angular_rad_per_s=2*np.pi*20e6,
    correlation_time_seconds=tau_c,
    spin_lock_time_seconds=0.03,
    coupling_scale_per_second2=2e6,
    polarizing_field_tesla=0.3,
    detection_field_tesla=0.02,
    prepolarization_time_seconds=3.0,
    t1_seconds=np.array([8.7, 0.2, 0.008, 5.0]),
)
# experiment.r1rho_per_second and experiment.locked_signal are (n_sizes, n_locks)
```

5.6.8 Liouville-Space Relaxation Models

The `spin_dynamics.relaxation` module provides shared Liouville-space utilities for spin-1/2 NMR and quadrupolar NQR simulations. The historical scalar model remains available as the phenomenological relaxation class and is also re-exported as `NQRRelaxationModel` for compatibility. It damps populations and coherences in the Hamiltonian eigenbasis using supplied T_1 and T_2 values. This is still a phenomenological model, but it now lives in the shared relaxation layer rather than in NQR-specific code.

For microscopic relaxation, `RedfieldDipolarRelaxationModel` builds a secular Markovian dissipator from fluctuating dipolar bath sources. A source is specified by a target–bath vector, gyromagnetic ratios, bath spin, and optional explicit coupling; if the coupling is omitted it is computed from $\mu_0 h \gamma_a \gamma_b / (4\pi r^3)$. The motional model determines how the dipolar covariance is averaged before the Redfield spectral-density step: `RigidSolidMotionalAveraging` keeps the fixed-frame anisotropy and is the natural choice for NQR solids, while `IsotropicLiquidMotionalAveraging` rotationally averages the covariance for liquid-state NMR.

```
from spin_dynamics.relaxation import (
    DipolarRelaxationSource,
    IsotropicLiquidMotionalAveraging,
    RedfieldDipolarRelaxationModel,
)

source = DipolarRelaxationSource(
    vector_angstrom=(1.52, 0.0, 0.0),
    gamma_target_hz_per_t=42.57747892e6,
    gamma_bath_hz_per_t=42.57747892e6,
    bath_spin=0.5,
)
relaxation = RedfieldDipolarRelaxationModel.from_dipolar_sources(
    0.5,
    (source,),
    motion=IsotropicLiquidMotionalAveraging(correlation_time_seconds=2.5e-12),
)
```

Gas-wall relaxation is another microscopic Liouville model, but it is not a dipolar Redfield bath. The wall-collision model treats wall encounters as a Poisson sequence of quantum jumps. Kinetic theory supplies the encounter rate

$$k_{\mathrm{wall}} = p_{\mathrm{acc}} \frac{\bar{v}}{4} \frac{S}{V}, \qquad \bar{v} = \sqrt{\frac{8 k_B T}{\pi m}},$$

where p_{acc} is the probability that a kinetic wall encounter actually samples the relaxing surface. Each encounter applies an isotropic depolarization channel with probability p_{dep}, giving the continuous Liouville generator $k_{\mathrm{wall}}(\Phi - I)$. In the weak-loss limit this has the familiar surface-relaxivity form

$$R_{\mathrm{wall}} = p_{\mathrm{dep}} k_{\mathrm{wall}} = \rho_{\mathrm{eq}} \frac{S}{V}, \qquad \rho_{\mathrm{eq}} = \frac{p_{\mathrm{acc}} p_{\mathrm{dep}} \bar{v}}{4},$$

but the model exposes the microscopic ingredients instead of asking the user to choose T_1 directly.

```
from spin_dynamics.relaxation import (
    WallCollisionRelaxationModel,
    sphere_surface_to_volume_per_m,
)

surface_to_volume = float(sphere_surface_to_volume_per_m(10e-3))
relaxation = WallCollisionRelaxationModel.from_geometry(
```

```
    0.5,
    surface_to_volume_per_m=surface_to_volume,
    temperature_kelvin=295.0,
    mass_amu=128.9047808611,
    accommodation_probability=1.0,
    depolarization_probability=1e-8,
)
print(relaxation.t1_seconds, relaxation.collision_rate_per_second)
```

5.6.9 Choosing a Relaxation Description

The Redfield model is non-default and opt-in. Existing pulse-sequence simulators keep their historical behavior when `relaxation=None`. The Redfield plotting examples show the two main dipolar regimes: isotropic-liquid proton CPMG and rigid-solid ^{14}N SLSE. The wall-relaxation example covers a different microscopic regime, polarized gas whose dominant relaxation source is repeated surface contact. Use fitted T_1/T_2 values when the experiment only needs the observed decay law, use BPP rates when a compact temperature- or correlation-time trend is the goal, and use `run_field_cycling_nmrd` when frequency is the experiment. Use transition-resolved quadrupolar relaxation when EFG motion and the identity of each NQR line matter. Use Redfield/dipolar relaxation when bath geometry and motional regime matter, and use the wall-collision model when container size, gas temperature, and per-collision spin loss are the physics being tested.

5.7 Echo Conversion

The echo utilities direct-sum a received spectrum $S(\Delta\omega)$ over the offset grid:

$$e(t) = \sum_k S(\Delta\omega_k)\exp(\mathrm{i}\Delta\omega_k t)\,\Delta\omega.$$

The implementation uses the same direct-sum convention as the MATLAB `calc_time_domain_echo` reference path. FID traces use the corresponding FID conversion helper.

5.8 Radiation Damping

The deterministic radiation-damping back-action model follows the local Measurements text convention:

$$T_{\mathrm{rd}} = \frac{2}{\gamma\mu_0\eta M_0 Q},$$

where γ is the gyromagnetic ratio, η is the magnetic-energy fill factor, M_0 is the equilibrium magnetization density, and Q is the loaded probe quality factor.

The analytic on-resonance hard-pulse FID envelope used by the test suite is

$$|M_{xy}(t)| = \frac{M_0}{\cosh\left(t/T_{\mathrm{rd}} - \log\tan(\theta/2)\right)}.$$

The finite-sequence implementation can add feedback during free-precession and acquisition windows, and can optionally apply an operator-split approximation during RF pulse matrices.

Part II

Physics Models

6 Scalar J-Coupling Models

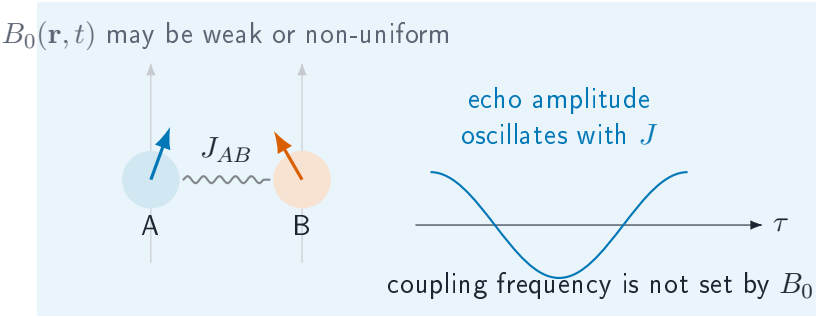
Scalar J coupling is field independent, so it can remain visible in weak or grossly inhomogeneous magnetic fields where chemical-shift resolution is compressed. The `spin_dynamics.coupling` namespace adds a scoped set of spin-1/2 tools for these cases. It is separate from the MATLAB-compatible Bloch workflow layer: Bloch workflows propagate independent isochromats, while the coupling layer propagates small dense Hilbert-space systems or evaluates closed-form J-editing response curves.

In this chapter. Choose between analytical J-editing responses and dense Hamiltonian propagation, then build a small scalar-coupled spin system.

Use this layer for: heteronuclear low-field J-editing curves, known-J spectrum fitting, ideal TANGO-B filter selectivity, exploratory two-spin or few-spin scalar-coupled Hamiltonian simulations, and multinuclear zero/ultra-low-field J-coupled spectra with mixed spin quantum numbers, quadrupolar nuclei, and per-spin relaxation (Section 6.6).

Current limits: dense matrices only (few-spin Hilbert spaces); no chemical exchange in this layer; no large sparse coupled-spin networks; no full pulse-sequence compiler for arbitrary multi-quantum pathways. The multinuclear extension models quadrupolar relaxation with a phenomenological per-spin "extended T1/T2" superoperator rather than a full quadrupolar Redfield tensor.

6.1 J-Coupled Spin Cartoon



6.2 Dense Spin-1/2 Hamiltonians

A coupled system is built from offsets ν_i in hertz and symmetric scalar couplings J_{ij} in hertz:

```
from spin_dynamics.coupling import (
    coupled_spin_system,
    isotropic_j_hamiltonian,
    zeeman_hamiltonian,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
```

```
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
    labels=["A", "B"],
)
hamiltonian = zeeman_hamiltonian(system) + isotropic_j_hamiltonian(system)
```

Hamiltonian builders return angular-frequency matrices in radians per second and use spin-1/2 operators with eigenvalues $\pm 1/2$. The rotating-frame offset Hamiltonian is

$$H_Z = 2\pi \sum_i \nu_i I_{iz}.$$

For weakly coupled systems, the secular scalar Hamiltonian is

$$H_J^{\mathrm{sec}} = 2\pi \sum_{i<j} J_{ij} I_{iz} I_{jz},$$

while the isotropic scalar Hamiltonian is

$$H_J^{\mathrm{iso}} = 2\pi \sum_{i<j} J_{ij} \left(I_{ix} I_{jx} + I_{iy} I_{jy} + I_{iz} I_{jz} \right).$$

The dense propagator evaluates

$$\rho(t) = U(t)\rho(0)U^{\dagger}(t), \qquad U(t) = \exp\left(-\mathrm{i}Ht\right),$$

and is intended for small systems where dense matrices are acceptable.

6.3 Inhomogeneous B0/B1 Isochromats

The bridge between the coupled-spin layer and the Bloch workflow layer is a coupled isochromat ensemble. Each isochromat contains the same scalar-coupled spin network, but samples a different local field and receive weight. For isochromat k, the local spin offsets are

$$\nu_{i,k} = \nu_i^{(0)} + s_i \, \Delta\nu_k,$$

where $\nu_i^{(0)}$ is the base spin offset, $\Delta\nu_k$ is the local B0 offset in hertz, and s_i is a per-spin offset scale. The default $s_i = 1$ adds the same field offset to every spin. Heteronuclear or carrier-specific models can supply different scales.

During an RF or spin-lock interval, the nominal nutation frequency is scaled by the local transmit field:

$$\omega_{1,k} = b_{1,k}^{\mathrm{tx}} \omega_1.$$

The ensemble signal is accumulated as

$$S = \sum_k w_k b_{1,k}^{\mathrm{rx}} \, \mathrm{Tr}(\rho_k D),$$

where w_k is the isochromat weight, $b_{1,k}^{\mathrm{rx}}$ is the local receive scale, and D is the detection operator.

```
from spin_dynamics.coupling import (
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
)

ensemble = coupled_isochromat_ensemble(
    system,
    b0_offsets_hz=[-2.0, 0.0, 2.0],
    weights=[0.25, 0.5, 0.25],
    b1_tx_scale=[0.9, 1.0, 1.1],
    b1_rx_scale=[1.0, 1.0, 0.8],
)

result = simulate_coupled_isochromat_sequence(
    ensemble,
    [
        rf_step(duration=0.25, nutation_hz=1.0, phase=1.5708),
        free_precession_step(duration=0.01),
    ],
    initial_axis="x",
    detect_axis="x",
)
```

For time-varying fields, each sequence step may override the ensemble B0 offsets or B1 transmit scale. This keeps the first implementation close to the existing isochromat approach: the coupled-spin Hilbert space is dense and small, while field inhomogeneity is represented by an outer ensemble sum.

6.4 Low-Field J-Editing

Analytic J-editing helpers model response curves as sums of field-independent cosine modulations:

$$s(\tau) = b + \sum_m a_m \cos^{p_m}(2\pi N J_m \tau)\,,$$

where τ is the encoding time, N is the number of encoding cycles, J_m is a known or hypothesized coupling in hertz, a_m is its amplitude, and p_m is an optional multiplicity exponent. Proton-detected models use $p_m = 1$. Carbon-detected CH_n groups use $p_m = n$.

```
from spin_dynamics.coupling import (
    carbon_detected_j_modulation,
    fit_known_j_spectrum,
    j_modulation_curve,
)

signal = j_modulation_curve(
    encoding_times,
    couplings_hz=[125.0, 160.0],
    amplitudes=[0.85, 0.15],
)

carbon_signal = carbon_detected_j_modulation(
    encoding_times,
```

```
    couplings_hz=[125.0, 160.0],
    abundances=[0.85, 0.15],
    proton_counts=[2, 1],
)

fit = fit_known_j_spectrum(
    encoding_times,
    signal,
    couplings_hz=[125.0, 160.0],
    include_background=False,
)
```

`fit_known_j_spectrum` solves a linear least-squares problem once the candidate J_m values are supplied. It returns amplitudes, optional background, fitted signal, residual, rank, and residual norm.

6.5 TANGO-B Filtering

The ideal TANGO-B helper evaluates the coupled-spin transverse filter envelope

$$A(J; t_d) = \sin^2(\pi J t_d)\,.$$

When a target coupling $J_\star$ and positive odd order n are supplied, the delay is selected as

$$t_d = \frac{n}{2J_\star}.$$

```
from spin_dynamics.coupling import tango_b_filter

response = tango_b_filter(
    couplings_hz,
    target_coupling_hz=160.0,
    order=3,
)
```

This model is idealized: it reports the scalar-coupling selectivity envelope, not RF imperfections, relaxation, diffusion, or probe filtering.

6.6 Zero/Ultra-Low-Field Multinuclear J-Coupled Spectra

At Earth's field ($\sim 50\,\mu$T) chemical shifts scale away and the spectrum is a *J-coupled spectrum* (JCS): every nucleus sits within a few kHz, so the frequency separations $|\nu_i - \nu_j|$ can be comparable to the couplings and spins are often *strongly* coupled. The multinuclear layer therefore works in the lab frame with absolute Larmor frequencies $\nu_i = \gamma_i B_0$ and the full isotropic coupling $2\pi J_{ij}\,\mathbf{I}_i{\cdot}\mathbf{I}_j$, over a tensor-product Hilbert space that mixes spin quantum numbers (e.g. spin-1/2 ^{1}H/^{19}F with spin-1 ^{14}N).

A quadrupolar nucleus such as ^{14}N relaxes quickly, and its rate $R(^{14}\mathrm{N}) = R_1 = R_2$ broadens the splittings of any spin J-coupled to it: as R grows the multiplet moves through resolved $\rightarrow$ coalesced $\rightarrow$ self-decoupled singlet (Altenhof et al., *J. Magn. Reson.* 355 (2023) 107540). The rate is set by molecular tumbling through the single-correlation-time quadrupolar model, which in the extreme-narrowing limit relevant at Earth's field is

$$R_1 = R_2 = \frac{3\pi^2}{10}\,\frac{2I+3}{I^2(2I-1)}\left(1+\tfrac{\eta^2}{3}\right) C_Q^2\,\tau_c,$$

with $C_Q = e^2qQ/h$ the quadrupole coupling constant. Relaxation is applied with a per-spin phenomenological superoperator that is exact for spin-1/2 (ladder channels $\sqrt{R_1/2}\, I^{\pm}$ and dephasing $\sqrt{2R_2 - R_1}\, I_z$).

```
import numpy as np
from spin_dynamics.coupling import (
    multinuclear_system,
    multinuclear_quadrupolar_rates,
    simulate_zulf_spectrum,
)

# 1H-19F-14N with a strong 2J(F,N); 14N is the last (quadrupolar) site.
couplings = np.zeros((3, 3))
couplings[0, 1] = couplings[1, 0] = 8.0    # J(H,F)
couplings[1, 2] = couplings[2, 1] = 37.0   # J(F,N)
system = multinuclear_system(["1H", "19F", "14N"], couplings, b0_tesla=50e-6)

# Derive R(14N) from the quadrupole coupling and a rotational correlation time.
r1, r2 = multinuclear_quadrupolar_rates(
    system,
    correlation_time_seconds=3e-12,
    quadrupole_coupling_hz=4.0e6,
    asymmetry=0.4,
    spin_half_rate_hz=0.3,
)
spectrum = simulate_zulf_spectrum(
    system, r1_per_second=r1, r2_per_second=r2,
    dwell_seconds=2e-4, n_points=32768,
    detect_indices=system.indices_for_isotope("19F"),
    apodization_hz=0.8,
)
# spectrum.frequencies_hz / spectrum.spectrum are the 19F J-coupled spectrum.
```

The examples `plot_zulf_quadrupolar_jcoupling.py` (sweeping $R(^{14}\mathrm{N})$ directly, with heteronuclear and homonuclear panels) and `plot_zulf_quadrupolar_relaxation.py` (deriving $R(^{14}\mathrm{N})$ from C_Q and τ_c) reproduce the coalescence and self-decoupling behaviour.

Run `python examples/plot_zulf_quadrupolar_jcoupling.py -output zulf_jcoupling.png` to reproduce Figure 6.1.

6.7 SLIC Response

Spin-lock induced crossing (SLIC) is modeled by adding an RF spin-lock Hamiltonian to the static coupled-spin Hamiltonian:

$$H_{\mathrm{SLIC}} = H_Z + H_J^{\mathrm{iso}} + 2\pi\omega_1 \sum_i I_{ix}.$$

The helper scans spin-lock nutation frequencies ω_1 in hertz and returns the remaining transverse magnetization and its corresponding dip:

```
import numpy as np

from spin_dynamics.coupling import (
    coupled_spin_system,
    simulate_slic_spectrum,
```

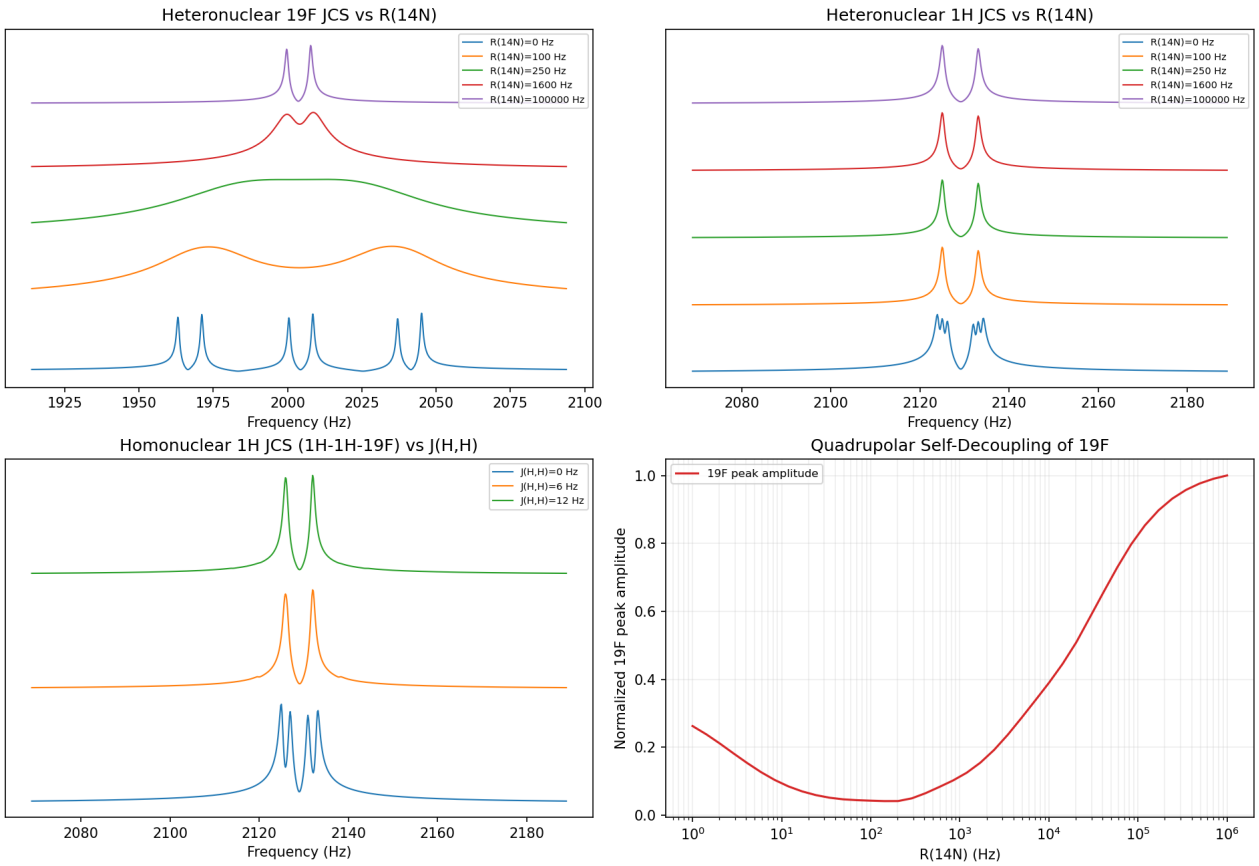


Figure 6.1: Earth-field ^{1}H/^{19}F spectra coupled through quadrupolar ^{14}N. Increasing the ^{14}N relaxation rate first broadens and suppresses the multiplet, then restores a narrow heteronuclear doublet through quadrupolar self-decoupling; the lower-left panel separates homonuclear J-structure.

```
    two_spin_slic_matching_nutation_hz,
    two_spin_slic_transfer_time,
)

system = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
)
duration = two_spin_slic_transfer_time(offset_difference_hz=0.7)
matching_nutation_hz = two_spin_slic_matching_nutation_hz(coupling_hz=7.0)
result = simulate_slic_spectrum(
    system,
    nutation_frequencies_hz=np.linspace(4.0, 10.0, 121),
    spin_lock_time=duration,
)
```

For a nearly equivalent two-spin system, the SLIC dip is expected near the coupling frequency: the spin-lock nutation matches $|J|$, while the resonance-offset difference breaks the singlet–triplet symmetry and sets the transfer time. At exact equivalence an ideal nonselective spin lock cannot produce transfer. The helper is exploratory and dense; broad homonuclear networks will need sparse or specialized methods.

6.8 Singlet and Parahydrogen State Primitives

The `spin_dynamics.hyperpolarization` namespace supplies the shared density-matrix foundation for long-lived singlet states, PHIP, and future SABRE workflows. For a selected spin-1/2 pair,

$$P_S = \tfrac{1}{4}\mathbf{1} - \mathbf{I}_1 \cdot \mathbf{I}_2, \qquad P_T = \mathbf{1} - P_S, \qquad Q_S = P_S - \tfrac{1}{3}P_T.$$

The projectors can be embedded at arbitrary pair indices inside a larger spin-1/2 system. Population helpers require a positive unit-trace physical density; `singlet_order_amplitude` also accepts the trace-zero deviation density used for NMR signal calculations.

```
from spin_dynamics.hyperpolarization import (
    parahydrogen_state,
    singlet_order_amplitude,
    singlet_projector,
)

h2 = parahydrogen_state(para_fraction=0.50)
projector = singlet_projector(nspin=4, pair=(1, 3))
order_amplitude = singlet_order_amplitude(h2.deviation_density)
```

The physical parahydrogen state is

$$\rho_{H_2} = f_p P_S + \frac{1 - f_p}{3} P_T = \frac{\mathbf{1}}{4} + \left(f_p - \frac{1}{4}\right) Q_S.$$

Thus 25% para-H2 is the statistical identity mixture and carries zero deviation singlet order; usable order scales with para excess above 25%. Physical and deviation densities are returned separately to keep this normalization explicit. Their model boundaries and validation plan are recorded in the PHIP/SABRE implementation plan shipped in the documentation directory. The singlet/parahydrogen example prints physical populations and deviation order versus para enrichment.

6.9 Long-Lived Preparation, Storage, and Readout

The dense two-spin reference workflow `simulate_slic_lls` connects the pieces that the response scan leaves separate. It begins with transverse magnetization, prepares singlet order using a spin lock at $\nu_1 = |J|$, applies an optional symmetry-selection purge, stores the retained order, and reconverts it with a second SLIC period:

```
import numpy as np
from spin_dynamics.coupling import coupled_spin_system
from spin_dynamics.hyperpolarization import simulate_slic_lls

pair = coupled_spin_system(
    offsets_hz=[-0.35, 0.35],
    couplings_hz=[[0.0, 7.0], [7.0, 0.0]],
)
lls = simulate_slic_lls(
    pair,
    storage_times_seconds=np.linspace(0.0, 54.0, 81),
    singlet_lifetime_seconds=18.0,
)
```

Storage uses a measured phenomenological T_S: the selected amplitude obeys $q_S(t) = q_S(0)\exp(-t/T_S)$. The map preserves trace and Hermiticity and, for a physical singlet density, relaxes the singlet population toward the statistical value $1/4$. This is intentionally not derived from independent spin T_1/T_2, which would omit the correlated symmetry protection that makes the state long lived. Microscopic correlated-noise relaxation is a later layer.

6.10 Hydrogenative PHIP: PASADENA and ALTADENA

Hydrogenative PHIP is modeled as an explicit source-to-product quantum-state map. If y_p is the fraction of products receiving both hydrogen atoms from the same parahydrogen molecule, the usable product order is

$$\Delta\rho_{\mathrm{product}} = y_p\left(f_p - \tfrac{1}{4}\right)Q_S.$$

The para fraction and pairwise-addition fraction are inputs: this reference model does not claim to predict catalyst chemistry, conversion, or scrambling.

`simulate_hydrogenative_phip` treats the two canonical experiments separately. For PASADENA, formation occurs at the detection field. The mapped state is projected onto its high-field secular product-basis order before weak-coupling evolution, a hard pulse, and FID acquisition. This secularization is essential: a perfectly coherent singlet is invariant to a nonselective hard pulse and would remain silent. For ALTADENA, formation begins at low field and the caller must provide every `PHIPFieldSegment` in the field trajectory; the API therefore never invents a transport time or adiabaticity assumption.

```
from spin_dynamics.hyperpolarization import (
    PHIPFieldSegment,
    simulate_hydrogenative_phip,
)

ramp = [
    PHIPFieldSegment(scale, 1.5e-3)
    for scale in np.linspace(0.02, 1.0, 48)
]
altadena = simulate_hydrogenative_phip(
    product_system,
    times_seconds=np.arange(1024) * 2.5e-4,
    protocol="altadena",
    para_fraction=0.90,
    pairwise_addition_fraction=0.72,
    reaction_time_seconds=0.05,
    field_trajectory=ramp,
    t2_seconds=0.12,
)
```

Pulse and receiver spin indices may be selected independently, providing the low-level machinery needed for heteronuclear products. A first-class, literature-validated PH-INEPT sequence is not yet included, nor are catalyst kinetics, reaction-time distributions, or SABRE exchange.

Run `python examples/plot_lls_phip_workflows.py` to reproduce the figure and vary T_S, para enrichment, and pairwise yield.

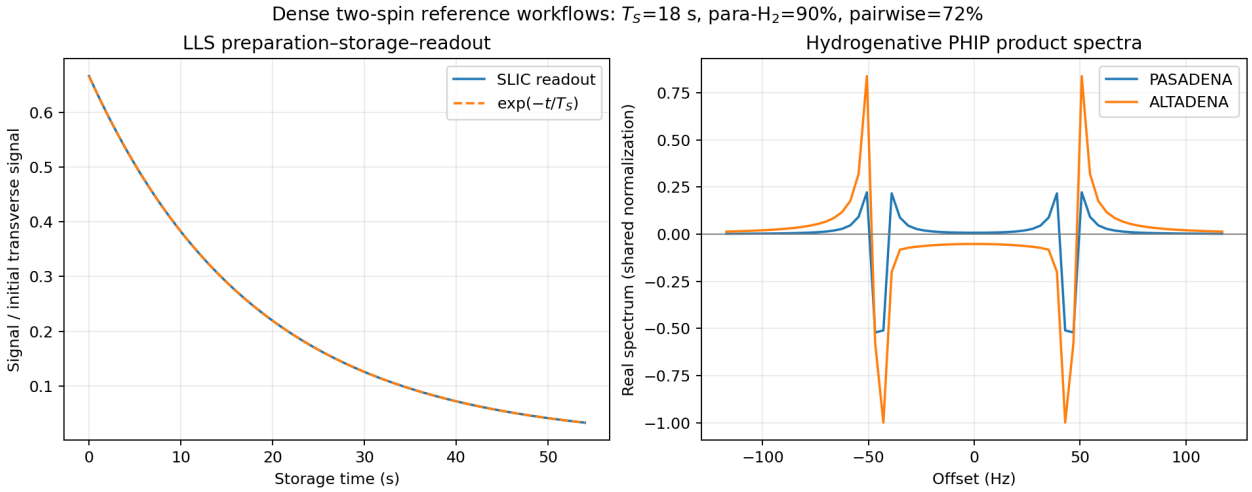


Figure 6.2: Complete dense reference workflows. Left: SLIC preparation, measured T_S storage, and reconversion. Right: PASADENA and explicit-trajectory ALTADENA spectra on a shared scale. These are algebraic/regression references, not fits to a particular catalyst or experiment.

7 Electron Spin Resonance Models

The `spin_dynamics.esr` namespace adds an ESR/EPR extension for single-electron spin-1/2 systems. The first layer is intentionally compact: it uses dense matrices, explicit g-tensor axes, and small product Hilbert spaces for hyperfine examples. Hamiltonians are expressed in radians per second, while public resonance frequencies, linewidths, and hyperfine constants are expressed in hertz.

In this chapter. Build ESR resonance, lineshape, pulsed-echo, hyperfine, DEER, and electron–nuclear modulation models while keeping their dense-system limits clear.

Use this layer for: single-electron ESR resonance calculations, anisotropic g-tensor single-crystal and powder spectra, CW absorption or first-derivative display, static g-strain and field-strain broadening, rectangular pulsed ESR FID and Hahn-echo simulations, Liouville-space T_1/T_2 relaxation, first electron-nuclear isotropic hyperfine doublets, four-pulse DEER/PELDOR distance measurement with $P(r)$ recovery (Section 7.6), and ESEEM, HYSCORE, and ENDOR of an $I = 1/2$, 1, or $3/2$ nucleus including the nuclear quadrupole interaction (Section 7.7).

Current limits: electron spin $S = 1/2$ only in the main ESR system; dense matrices only. The DEER model uses the secular point-dipole coupling in the weak-coupling observer/pump regime, and the ESEEM/HYSCORE/ENDOR model treats a single $I \leq 3/2$ nucleus with secular plus pseudosecular hyperfine and a field-collinear quadrupole tensor. No anisotropic A tensors, tilted quadrupole tensors, multiple coupled nuclei, exchange, higher-spin zero-field splitting, saturation, or microwave resonator model yet.

7.1 Zeeman Hamiltonian and g Tensors

The single-electron Zeeman Hamiltonian is

$$H_Z = 2\pi \frac{\mu_B}{h}\, \mathbf{B}_0^{\mathsf{T}}\, \mathbf{g}\, \mathbf{S},$$

where μ_B/h is exposed as `BOHR_MAGNETON_HZ_PER_T`. The g tensor may be supplied as a scalar, a three-component principal-axis vector, or a full 3×3 matrix. For a static-field direction $\hat{\mathbf{n}}$, the effective g value is

$$g_{\mathrm{eff}}(\hat{\mathbf{n}}) = \left| \mathbf{g}^{\mathsf{T}} \hat{\mathbf{n}} \right|,$$

and the spin-1/2 transition frequency is

$$\nu_{\mathrm{ESR}} = \frac{\mu_B}{h}\, B_0\, g_{\mathrm{eff}}.$$

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    effective_g_value,
    resonance_field_tesla,
    resonance_frequency_hz,
)
```

```
system = ESRSpinSystem(g_tensor=[2.00, 2.08, 2.24])
g_z = effective_g_value(system, [0.0, 0.0, 1.0])
nu = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
b_res = resonance_field_tesla(system, microwave_frequency_hz=9.5e9)
```

7.2 CW Spectra, Lineshapes, and Disorder

Frequency-swept and field-swept helpers broaden orientation-resolved ESR lines with Gaussian or Lorentzian profiles. The same helpers can return absorption or first-derivative CW-style spectra:

```
from spin_dynamics.esr import simulate_field_sweep

result = simulate_field_sweep(
    system,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.2e-3,
    lineshape="lorentzian",
    detection_mode="derivative",
)
```

Static disorder is represented by weighted samples. The convenience `static_disorder_grid` builds Gaussian quadrature-like samples for diagonal g-strain and applied-field offsets:

```
from spin_dynamics.esr import (
    simulate_field_sweep_distribution,
    static_disorder_grid,
)

samples = static_disorder_grid(
    system,
    g_std=[0.0, 0.0, 0.005],
    field_std_tesla=0.1e-3,
    g_points=5,
    field_points=5,
)

strained = simulate_field_sweep_distribution(
    samples,
    microwave_frequency_hz=9.5e9,
    orientations="powder",
    broadening_tesla=0.05e-3,
)
```

Run `python examples/plot_esr_powder_spectrum.py -output esr.png` to reproduce Figure 7.1 and change the g-strain or field-disorder sampling.

7.3 Pulsed ESR

The pulsed ESR helpers use a single rotating frame and a rotating-wave approximation. The `nutation_hz` parameter is the on-resonance spin-1/2 Rabi rate for the selected microwave B_1 direction, so a rectangular

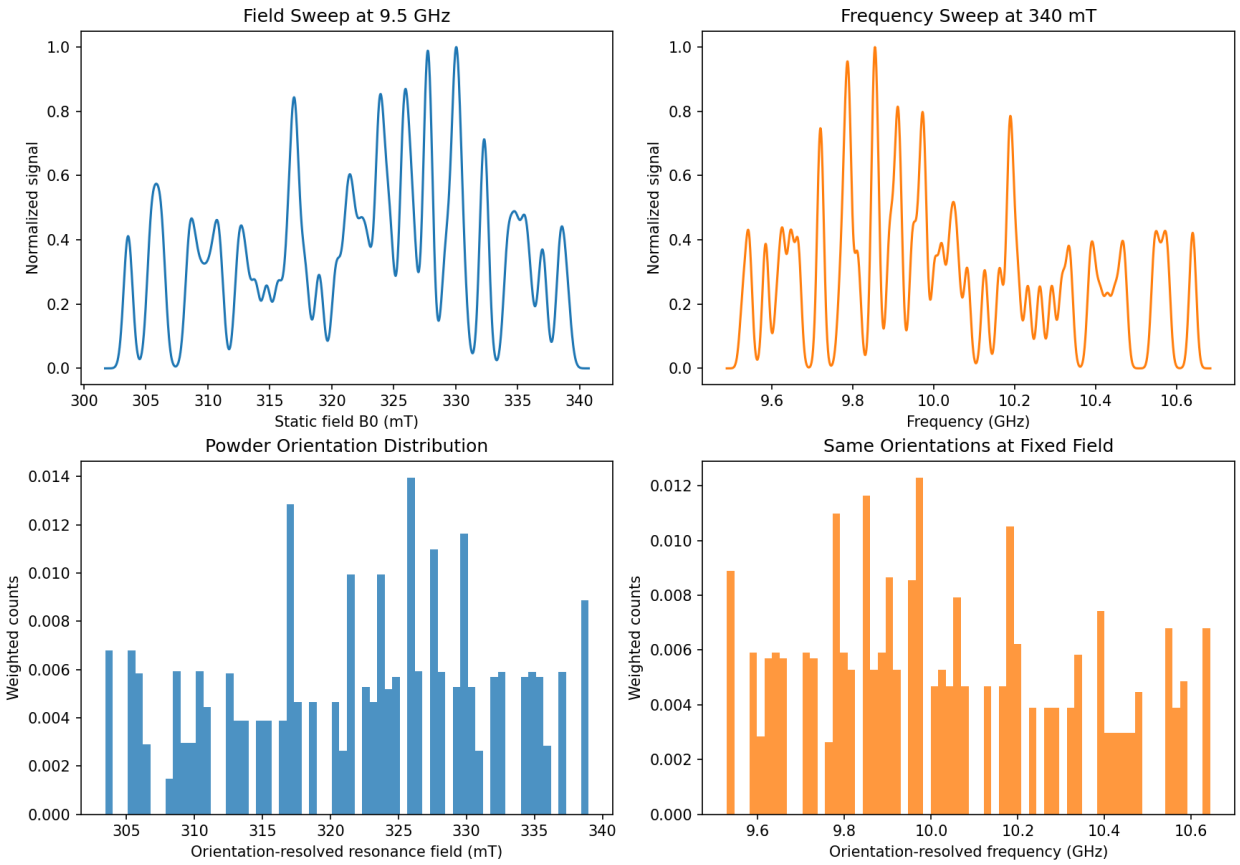


Figure 7.1: The same anisotropic g-tensor powder represented as a 9.5 GHz field sweep (left) and a 340 mT frequency sweep (right). The lower panels expose the weighted orientation-resolved resonance positions whose broadened sum gives the spectra above.

pulse of flip angle θ has duration

$$t_p = \frac{\theta}{2\pi\nu_1}.$$

In particular, a 90-degree pulse has duration $1/(4\nu_1)$.

```
import numpy as np

from spin_dynamics.esr import (
    flip_angle_duration,
    simulate_fid,
    simulate_hahn_echo,
)

carrier = resonance_frequency_hz(system, [0.0, 0.0, 0.34])
nu1 = 5e6
t90 = flip_angle_duration(np.pi / 2.0, nu1)
t180 = flip_angle_duration(np.pi, nu1)

fid = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
```

```
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
)

echo = simulate_hahn_echo(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    excitation_duration_seconds=t90,
    refocus_duration_seconds=t180,
    tau_seconds=2e-6,
    times_seconds=np.linspace(0.0, 4e-6, 256),
    rf_frequency_hz=carrier,
)
```

7.4 Pulsed Relaxation

For physical T_1/T_2 relaxation during pulses, free evolution, and acquisition, pass an `ESRRelaxationModel`. The model acts in Liouville space on the trace-zero high-temperature density-matrix deviation: T_1 damps population differences while preserving trace, and T_2 damps coherences.

```
from spin_dynamics.esr import ESRRelaxationModel

relaxed = simulate_fid(
    system,
    [0.0, 0.0, 0.34],
    nutation_hz=nu1,
    pulse_duration_seconds=t90,
    times_seconds=np.linspace(0.0, 5e-6, 256),
    rf_frequency_hz=carrier,
    relaxation=ESRRelaxationModel(t1_seconds=50e-6, t2_seconds=3e-6),
)
```

The older `t2_seconds` argument in the pulsed helpers is a simple post-propagation envelope retained for quick examples. When using `ESRRelaxationModel`, leave `t2_seconds=inf` to avoid double-counting coherence decay.

7.5 Isotropic Hyperfine Coupling

The first hyperfine layer models one electron spin-1/2 coupled isotropically to one or more nuclei:

$$H = H_{eZ} + H_{nZ} + 2\pi \sum_k A_k \, \mathbf{S} \cdot \mathbf{I}_k,$$

with A_k in hertz. The helper scans the applied field, diagonalizes the full dense electron-nuclear Hamiltonian at each point, and sums ESR-active transitions according to the microwave B_1 matrix element.

```
from spin_dynamics.esr import (
    NuclearSite,
    electron_nuclear_system,
```

```
    simulate_hyperfine_field_sweep,
)

hf_system = electron_nuclear_system(
    [20e6],
    nuclei=[NuclearSite("H1", isotope="1H", gamma_hz_per_t=42.577e6)],
    g_tensor=2.0023,
)

hf_sweep = simulate_hyperfine_field_sweep(
    hf_system,
    microwave_frequency_hz=9.5e9,
    broadening_hz=0.5e6,
)
```

For one spin-1/2 nucleus in the high-field limit, the ESR line splits by approximately A in frequency, or by

$$\Delta B \approx \frac{A}{(\mu_B/h)g}$$

in a field-swept spectrum.

7.6 Pulsed Dipolar ESR: DEER/PELDOR

Double electron-electron resonance (DEER, also PELDOR) measures the dipolar coupling between two electron spins, and hence the nanometre distance between them. The `spin_dynamics.esr.dipolar` and `spin_dynamics.esr.deer` modules provide the point-dipole coupling, the four-pulse DEER forward model, and regularized recovery of a distance distribution $P(r)$.

The secular dipolar coupling between two spectrally distinct electron spins is

$$\nu_{dd}(r,\theta) = \nu_\perp(r)\,(1 - 3\cos^2\theta), \qquad \nu_\perp(r) = \frac{\mu_0}{4\pi}\,\frac{g_a g_b \mu_B^2}{h\,r^3},$$

which for two free-electron g values gives the canonical $\nu_\perp \approx 52.04\ \mathrm{MHz}$ at $r = 1\ \mathrm{nm}$ (`dipolar_constant_hz_nm3`). A pump pulse on the partner spin inverts the local dipolar field, modulating the observer echo. For a single pair the DEER form factor is

$$F(t) = 1 - \lambda\big(1 - \cos(2\pi\nu_{dd}(r,\theta)\,t)\big),$$

where the modulation depth $\lambda = \sin^2(\beta/2)$ is set by the pump flip angle β. Averaging over an isotropic orientation distribution gives the powder kernel, whose dipolar (Pake) spectrum has its singularity at $\nu_\perp$ and an edge at $2\nu_\perp$.

```
import numpy as np

from spin_dynamics.esr import (
    deer_dipolar_spectrum,
    extract_distance_distribution,
    gaussian_distance_distribution,
    simulate_deer,
)

times = np.linspace(0.0, 2.5e-6, 100)
distances = np.linspace(1.5, 4.0, 60)
```

```
truth = gaussian_distance_distribution(distances, 2.5, 0.2)

form_factor = simulate_deer(times, distances, truth, lambda_depth=0.35)
freqs, pake = deer_dipolar_spectrum(times, form_factor)
recovered = extract_distance_distribution(
    times, form_factor, distances, lambda_depth=0.35, snr=300.0
)
```

Distance recovery solves the regularized non-negative least-squares problem $K\,p \approx F$ using the SNR-informed Tikhonov selector from `spin_dynamics.analysis.regularization` with the DEER powder kernel as the design matrix (SciPy required for the non-negative solve).

The analytic kernel is validated against an independent two-electron density-matrix simulation of the full four-pulse sequence (`deer_pair_trace_quantum`): with ideal spin-selective observer and pump pulses it reproduces $F(t)$ to floating-point precision for every orientation and pump flip angle, and the observer echo refocuses the observer offset.

7.7 ESEEM, HYSCORE, and ENDOR

While DEER measures electron-electron distances, ESEEM (electron spin echo envelope modulation), HYSCORE (hyperfine sublevel correlation), and ENDOR (electron-nuclear double resonance) measure the weak hyperfine coupling between an electron and a nearby nucleus. The modules `spin_dynamics.esr.eseem`, `spin_dynamics.esr.hyscore`, and `spin_dynamics.esr.endor` model an $S = 1/2$ electron coupled to a nucleus ($I = 1/2$, 1, or $3/2$) in the secular approximation,

$$H = -\omega_I I_z + S_z(A\,I_z + B\,I_x) + H_Q,$$

parameterized by `HyperfineCoupling(larmor_hz, secular_hz, pseudosecular_hz, nuclear_spin, quadrupole_hz, eta)`. For $I \geq 1$ the nuclear quadrupole term H_Q (quadrupole frequency ν_Q and asymmetry η; principal axis taken collinear with the static field) reuses the NQR quadrupole Hamiltonian. ESEEM requires the nuclear quantization axis to differ between the two electron manifolds, which the pseudosecular B and the quadrupole both provide. For $I = 1/2$ the two nuclear (ENDOR) frequencies and modulation depth have the closed forms

$$\nu_{\alpha,\beta} = \sqrt{(\omega_I \pm A/2)^2 + (B/2)^2}, \qquad k = \left(\frac{\omega_I B}{\nu_\alpha \nu_\beta}\right)^2;$$

for any spin, `manifold_frequencies` returns the nuclear transition frequencies in each electron manifold by diagonalization. The classic $^{14}\mathrm{N}$ ($I = 1$) *exact-cancellation* regime, where the hyperfine field cancels the nuclear Zeeman in one manifold and leaves the three pure-quadrupole (NQR) lines $\nu_\pm = \nu_Q(1 \pm \eta/3)$, $\nu_0 = \frac{2}{3}\nu_Q\eta$, is reproduced by this model.

ESEEM. `two_pulse_eseem` and `three_pulse_eseem` return the analytic two- and three-pulse envelopes, and `eseem_spectrum` their frequency spectrum (peaks at ν_α, ν_β, and – for two-pulse – their sums and differences). Each has a density-matrix counterpart (`*_quantum`) that propagates the spin Hamiltonian through ideal pulses and reproduces the analytic trace to floating-point precision.

```
import numpy as np

from spin_dynamics.esr import HyperfineCoupling, three_pulse_eseem, eseem_spectrum

coupling = HyperfineCoupling(larmor_hz=14.5e6, secular_hz=3e6, pseudosecular_hz=2e6)
```

```
times = np.linspace(0.0, 8e-6, 400)
trace = three_pulse_eseem(times, coupling, tau_seconds=200e-9)
freqs, spectrum = eseem_spectrum(times, trace)
```

Coherence selection and phase cycling. The stimulated-echo and HYSCORE pathways are isolated in simulation by electron coherence-order filtering. This is the idealized limit of an experimental phase cycle: shifting a pulse phase by φ multiplies a pathway of coherence-order change Δp by $e^{-i\Delta p\varphi}$, so cycling φ and weighting the record by $e^{+i\Delta p_{\mathrm{desired}}\varphi}$ selects that pathway. `three_pulse_eseem_phase_cycled` performs the explicit cycle (four steps per pulse suffice because the electron coherence order spans only $\{-1, 0, +1\}$) and reproduces the coherence-filtered result to numerical precision.

HYSCORE. `hyscore_signal` simulates the 2D four-pulse experiment and `hyscore_spectrum` returns its 2D spectrum, whose cross-peaks appear at (ν_α, ν_β) and (ν_β, ν_α) (`cross_peak_positions`). Choose the dwell time so the nuclear frequencies stay below the Nyquist limit, or the cross-peaks alias.

ENDOR. `endor_frequencies` returns the line positions, and `davies_endor_spectrum` / `mims_endor_spectrum` return the analytic Davies and Mims responses. Mims ENDOR carries the τ-dependent blind-spot factor $\sin^2(\pi\nu\tau)$, which suppresses a line whenever $\nu\tau$ is an integer.

The model covers a single nucleus of spin $I = 1/2$, 1, or $3/2$ in the secular regime, with the quadrupole principal axis collinear with the static field. Anisotropic hyperfine A tensors with powder averaging, an arbitrary quadrupole-tensor orientation, and multiple coupled nuclei are the natural next extensions.

8 Pulsed NQR Models

Nuclear quadrupole resonance is driven by the interaction between a nuclear quadrupole moment and the local electric-field-gradient (EFG) tensor. The `spin_dynamics.nqr` namespace adds a scoped quadrupolar extension for solid-state pulsed NQR experiments. It is intentionally separate from the spin-1/2 Bloch and scalar-coupling layers.

In this chapter. Select reduced or full quadrupolar dynamics, define a site and its orientations, and assemble FID, echo, SLSE, or population-transfer experiments.

Use this layer for: reduced two-level (selective) pulsed NQR of spin-1 sites; full $(2I+1)$-level density-matrix FID and echo dynamics required for spin-3/2; automatic reduced-vs-full model selection; spin-1 and spin-3/2 transition metadata; single-crystal and powder averages over local EFG orientations; SLSE echo trains; two-frequency population transfer; static EFG inhomogeneity; finite-window SLSE spectra; weak-B0 line splitting; optional Liouville-space phenomenological or Redfield/dipolar relaxation; and instrument-level polarization-enhanced NQR transport with CIF-based ^{1}H-quadrupolar dipolar-coupling estimates.

Current limits: dense single-site matrices only; two pulse models (the reduced spin-1 selective pulse and the full $(2I+1)$-level density-matrix pulse). Any integer or half-integer spin above 1/2 is supported: the level and transition structure is exact for all of them (spin-5/2, 7/2, 9/2 – ^{27}Al, ^{51}V, ^{93}Nb, ...), and FID/echo/SLSE run on the full model. The full-model *pulse* uses a single-carrier rotating-wave approximation, so it is faithful on the *fundamental* (lowest) NQR line; driving a higher/satellite line of a spin $> 3/2$ nucleus emits a warning (correct satellite pulsing needs exact time-domain propagation, a planned follow-up). The *reduced* SLSE and population-transfer workflows remain spin-1; a two-frequency 2D population-transfer workflow on the full model is still future work. Microscopic relaxation is a stochastic Redfield/dipolar treatment, not a coherent multi-site quadrupolar cluster; no multi-site coherent coupling between quadrupolar nuclei.

8.1 Quadrupolar Site Model

A quadrupolar site is described in the EFG principal-axis system. The zero-field Hamiltonian used by the Python helper is

$$H_Q = 2\pi\nu_{\text{scale}}\left[3I_z^2 - I(I+1)\mathbf{1} + \eta\left(I_x^2 - I_y^2\right)\right],$$

where I is the spin quantum number, ν_Q is the quadrupole-frequency parameter in hertz, and $0 \le \eta \le 1$ is the EFG asymmetry parameter. Hamiltonians are represented in radians per second. For spin-1, the package uses $\nu_{\text{scale}} = \nu_Q/3$, matching the nitrogen-14 convention where the zero-field lines are $\nu_x = \nu_Q(1+\eta/3)$, $\nu_y = \nu_Q(1-\eta/3)$, and $\nu_z = 2\eta\nu_Q/3$. For spin-3/2, $\nu_{\text{scale}} = \nu_Q/6$, so the zero-field chlorine-style NQR line is

$$\nu_{\text{NQR}} = \nu_Q\sqrt{1+\eta^2/3}.$$

Zero-frequency Kramers-doublet transitions are omitted from the transition inventory.

Weak Zeeman perturbations can be added as

$$H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I},$$

where $\mathbf{B}_0$ is expressed in the EFG principal-axis system for the current crystallite orientation. The default is $B_0 = 0$, which is the usual NQR case.

```
from spin_dynamics.nqr import QuadrupolarSite, diagonalize_site

site = QuadrupolarSite(
    spin=1,
    isotope="14N",
    quadrupole_frequency_hz=900e3,
    eta=0.3,
)

eigensystem = diagonalize_site(site)
for transition in eigensystem.transitions:
    print(transition.label, transition.frequency_hz)
```

For spin-1 at zero field, transitions are labeled by their dominant RF polarization in the EFG principal-axis system: `x`, `y`, and `z`. For the example above the line positions are approximately 990, 810, and 180 kHz.

Spin-3/2 transition metadata can be inspected with the same diagonalization helper:

```
chlorine = QuadrupolarSite(
    spin=1.5,
    isotope="35Cl",
    quadrupole_frequency_hz=30e6,
    eta=0.1,
    gamma_hz_per_t=4.171e6,
)

for transition in diagonalize_site(chlorine).transitions:
    print(transition.frequency_hz)
```

8.2 Two Modeling Regimes

Before propagating a pulse, the package chooses between two models. The choice is governed by how many states the RF pulse can connect, not by the number of spectral lines. The detailed reasoning is given in the technical note `References/Pulsed_NQR_Spin_Dynamics_Narrative_Rewrite.pdf`; the tracked `.tex` source is in the same directory.

- **Reduced two-level model** (`SelectivePulse`, `simulate_slse`): one pulse addresses one transition as an embedded fictitious spin-1/2 rotation. This is honest only when the selected transition is isolated. For a target transition t, with $\Delta_{\text{iso}} = \min_{j \neq t} |\omega_t - \omega_j|$ the spacing to the nearest RF-active competitor, the validity condition is

  $$\Delta_{\text{iso}} \gg \max(\Omega_1,\ \Delta\omega_{\text{pulse}},\ \Gamma)\,,$$

  where Ω_1 is the effective nutation rate, $\Delta\omega_{\text{pulse}} \sim 1/t_p$ is the pulse bandwidth, and Γ is the linewidth. A nonzero η is necessary but *not* sufficient: at $\eta = 0$ the spin-1 x and y lines coincide and the reduction fails. The reduced path is restricted to spin-1 for this reason.

- **Full density-matrix model** (`spin_dynamics.nqr.full_dynamics`): the complete $(2I+1)$-level density matrix is propagated. This is the correct general model and is required for spin-3/2, whose single zero-field line connects two Kramers doublets—four states hidden behind one frequency.

8.3 Selective Pulse Approximation

Most practical spin-1 pulsed NQR RF pulses are narrowband relative to the separation between quadrupolar transitions. The reduced model uses this selective limit: one pulse addresses one transition and acts as an embedded two-level rotation inside the full $(2I+1)$-level Hilbert space.

For a transition $|a\rangle \leftrightarrow |b\rangle$, the transition dipole vector is

$$\mathbf{d}_{ab} = (\langle a|I_x|b\rangle, \langle a|I_y|b\rangle, \langle a|I_z|b\rangle)\,.$$

For a local RF direction $\hat{\mathbf{b}}_1$, the complex drive projection is proportional to $\hat{\mathbf{b}}_1 \cdot \mathbf{d}_{ab}$. Preserving this signed complex projection is essential for powder averages because transmit and receive phases must pair consistently across crystallites.

```
from spin_dynamics.nqr import SelectivePulse, apply_selective_pulse

pulse = SelectivePulse(
    "x",
    duration_seconds=50e-6,
    nutation_hz=10e3,
)
```

The pulse helper also supports an optional RF frequency. If omitted, the RF is placed on the selected transition. If supplied, the difference between the RF frequency and transition frequency is treated as a two-level detuning.

The selective-pulse propagation routines support spin-1 sites only. Spin-3/2 nuclei such as $^{35}\mathrm{Cl}$ and $^{37}\mathrm{Cl}$ have degenerate doublets at zero field, so their pulsed response is handled by the full density-matrix model below rather than by an embedded two-level transition; calling the reduced selective-pulse routines on a non-spin-1 site raises a clear error.

8.4 Full Density-Matrix Model

The `spin_dynamics.nqr.full_dynamics` module propagates the complete $(2I+1)$-level density matrix in a rotating frame at the pulse carrier. It is the required model for spin-3/2 and runs for any spin from 1 to 9/2. The static Hamiltonian $H_Q + H_Z$ is diagonalized for the actual EFG and field orientation, the RF and detection operators $\hat{\mathbf{e}} \cdot \mathbf{I}$ are transformed into that eigenbasis, and the density matrix is propagated through each constant-Hamiltonian interval by $U_k = \exp(-iH_k\,\Delta t_k)$, $\rho_{k+1} = U_k \rho_k U_k^\dagger$. The free-evolution acquisition windows diagonalize the constant free generator once and reconstruct every sample, so a spin-9/2 FID with thousands of points stays a few milliseconds despite the 100×100 Liouville space.

A half-integer spin I has $I - 1/2$ zero-field lines; for an axial EFG they fall at $\nu_Q, 2\nu_Q, 3\nu_Q, \ldots$ (the $\frac{1}{2} \leftrightarrow \frac{3}{2}$, $\frac{3}{2} \leftrightarrow \frac{5}{2}$, … transitions), where ν_Q is `quadrupole_frequency_hz`, the fundamental line; a non-zero η shifts them and switches on a weak combination line. `quadrupolar_site` builds a named nucleus ($^{27}\mathrm{Al}$, $^{51}\mathrm{V}$, …) from its C_Q or ν_Q, filling the spin and gyromagnetic ratio from the isotope registry. `examples/plot_nqr_higher_spin_transitions.py` shows the level ladder, the transition ratios, and a pulsed FID/echo on the $^{27}\mathrm{Al}$ fundamental line.

Here `nutation_hz` is the bare field nutation $\gamma B_1/(2\pi)$, so the realized Rabi rate on a transition $a-b$ is $2\,\nu_1\,|\langle a|\hat{\mathbf{e}}_1 \cdot \mathbf{I}|b\rangle|$; the same pulse is a different flip angle on different transitions, which is the physical effect the full model captures. A single-carrier rotating-wave approximation is used: it is faithful on the

fundamental (lowest) line, which is the default carrier and the strongest line. Driving a higher/satellite line of a spin $> 3/2$ nucleus is flagged with a warning (the winding-number frame would otherwise select the wrong transition); correct satellite pulsing needs exact time-domain propagation, a planned follow-up.

```
import numpy as np
from spin_dynamics.nqr import QuadrupolarSite, simulate_full_fid

site = QuadrupolarSite(spin=1.5, isotope="35Cl",
                       quadrupole_frequency_hz=30e6, eta=0.1)

fid = simulate_full_fid(
    site,
    nutation_hz=20e3,            # bare field nutation gamma*B1/(2*pi)
    pulse_duration_seconds=10e-6,
    times_seconds=np.linspace(0.0, 200e-6, 1024),
)
print(fid.signal)               # complex baseband signal
```

A two-pulse (Hahn-style) echo is available through `simulate_full_echo`, and the spin-lock spin-echo (SLSE) detection train—the chlorine-style spin-3/2 measurement—through `simulate_full_slse`:

```
from spin_dynamics.nqr import QuadrupolarSite, simulate_full_slse

site = QuadrupolarSite(spin=1.5, isotope="35Cl",
                       quadrupole_frequency_hz=1.0e6, eta=0.0)

slse = simulate_full_slse(
    site, nutation_hz=10e3,
    excitation_duration_seconds=25e-6, refocus_duration_seconds=50e-6,
    echo_spacing_seconds=400e-6, num_echoes=12,
    orientations="powder",   # powder average; b0_tesla > 0 adds weak Zeeman
    t2e_seconds=3e-3,
)
```

`simulate_full_slse` returns a `FullNQRSLSEResult` with the orientation-weighted echo train, the per-orientation trains, and the weights. Pass `b0_tesla` > 0 for a weak Zeeman perturbation (the field direction follows each orientation sample; use `b0_powder_average_grid` for a Zeeman powder), and an optional `relaxation=NQRRelaxationModel(...)` adds Liouville-space T_1/T_2 decay. The lower-level primitives `pulse_hamiltonian`, `static_hamiltonian_rotating`, and `detection_operator` are exposed for custom sequences. The full model has been validated against an exact lab-frame propagator (the rotating-wave error scales as the Ω_1^2 Bloch-Siegert shift), and its spin-1 powder nutation curve reproduces the classic $119°$ maximum while the spin-3/2 curve peaks at a slightly smaller flip angle. The `plot_nqr_spin32_slse.py` example runs the ^{35}Cl powder SLSE train and shows how a weak Zeeman field reshapes the decay.

8.5 Model Selection

`select_nqr_model` reads the reduced-vs-full choice from the static Hamiltonian and the RF matrix elements for the coil polarization, not from the spin or the line count. It builds the pulse-addressed connected set (states the pulse can drive from the target pair, plus RF-driven near-degenerate partners) and the isolation ratio $\Delta_{\mathrm{iso}}/\max(\Omega_1, 1/t_p, \Gamma)$.

```
from spin_dynamics.nqr import QuadrupolarSite, select_nqr_model
```

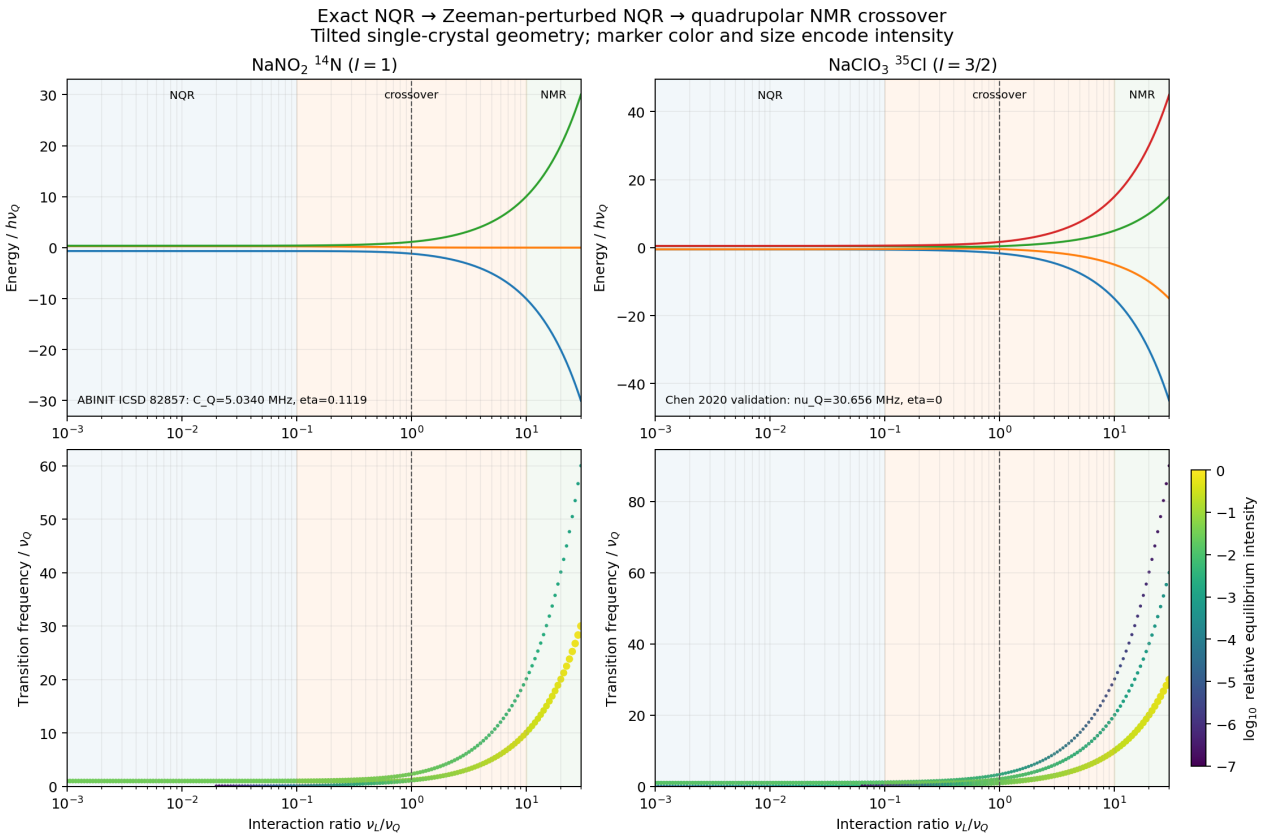


Figure 8.1: Exact energy levels (top) and observable transition frequencies (bottom) across the NQR, crossover, and quadrupolar-NMR regimes. Marker color and size encode equilibrium transition intensity. The spin-1 ^{14}N $NaNO_2$ and spin-3/2 ^{35}Cl $NaClO_3$ panels show why the intermediate regime requires exact diagonalization and state tracking.

```
site = QuadrupolarSite(spin=1.5, quadrupole_frequency_hz=30e6, eta=0.1)
choice = select_nqr_model(
    site, "x",
    nutation_hz=5e3,
    pulse_duration_seconds=50e-6,
    b1_direction_pas=(1, 1, 1),
)
print(choice.recommended_model)   # "full" -- spin-3/2 Kramers doublets
print(choice.describe())          # diagnostic report
```

The reduced model is recommended only when the addressed set is a single, non-degenerate, RF-active pair *and* the isolation ratio clears `isolation_threshold` (default 5). The returned `NQRModelSelection` reports the target states, nearest competing RF-active transition, isolation distance and ratio, pulse bandwidth, the addressed-state set, a degeneracy flag, and human-readable reasons. It handles the regime-defining cases: spin-1 $\eta = 0$ and unresolved small-η splittings go full via the isolation ratio; spin-3/2 Kramers doublets go full because the addressed set spans four states; a strongly Zeeman-resolved spin-3/2 line can return reduced as a verified special case; and a polarization that makes neighboring lines RF-dark keeps a crowded spectrum reducible. The check is currently advisory—the reduced workflows do not yet invoke it automatically.

Reproduce Figure 8.1 with `python examples/plot_nqr_nmr_crossover.py -output crossover.png`.

8.6 Orientations and Powder Averages

NQR is a solid-state experiment, so the signal depends on the orientation of the RF field relative to the local EFG principal axes. A single crystal is one fixed orientation. A powder is represented by a normalized spherical quadrature grid:

```
from spin_dynamics.nqr import powder_average_grid, single_crystal_orientation

single = single_crystal_orientation(alpha=0.0, beta=1.57079632679)
powder = powder_average_grid(n_theta=16, n_phi=32)
```

For weak-B0 calculations, each orientation may also specify the direction of B_0 in the EFG frame. If no separate B_0 direction is supplied, the current helper uses the local RF direction as the default field direction for that sample.

Weak-field spectra use a correlated powder grid for the static field and RF field directions. The helper `b0_b1_powder_average_grid` samples the orientation of B_0 in the EFG frame and the rotation angle of B_1 around B_0, preserving a chosen lab-frame angle between the two fields. For a conventional transverse coil one uses $B_1 \perp B_0$.

8.6.1 Realistic Relaxation in Crossover Powder Echo Trains

A realistic powder relaxation curve is a receiver calculation, not an average of local decay envelopes. For crystallite k, let $\rho_k(t)$ be the state, M_k the complex receive operator, and w_k its quadrature weight. The ideal coil voltage is

$$s(t) = \sum_k w_k \, \mathrm{Tr}[M_k \rho_k(t)] \, .$$

The complex sum must be formed before taking a magnitude. Averaging $|s_k|$ or $|s_k|^2$ removes physical phase cancellation and is not the voltage induced in one coil.

At zero field only the RF direction relative to the EFG tensor is physical, so the spherical `powder_average_grid` is sufficient. At nonzero field the fixed laboratory relationship between B_0 and B_1 requires a full SO(3) average, including rotation of B_1 about B_0. The prefix-nested `b0_b1_powder_average_halton` sequence is preferred for expensive convergence studies.

Each crystallite must use its own complete $H_Q + H_Z$, Gibbs state, and field-dependent relaxation generator. A single experimental carrier is then held fixed across the ensemble. In a broad nonzero-field powder, `select_powder_frequency_slice` can restrict propagation to RF-active crystallites near that carrier. It also returns the fraction of the full powder retained; multiply by this factor when an absolute full-powder amplitude is required.

```
from spin_dynamics.nqr import (
    FieldDependentNonsecularRelaxationModel,
    b0_b1_powder_average_halton, powder_carrier_frequency_hz,
    select_powder_frequency_slice, simulate_crossover_slse_powder,
)

candidates = b0_b1_powder_average_halton(8192)
carrier = powder_carrier_frequency_hz(
    site, b0_tesla, candidates, nutation_hz=100e3,
)
active, retained_weight = select_powder_frequency_slice(
```

```
    site, b0_tesla, candidates,
    carrier_frequency_hz=carrier,
    half_width_hz=300e3,
)
relaxation = FieldDependentNonsecularRelaxationModel(
    spin=site.spin, temperature_kelvin=300.0,
    magnetic_rate_per_second=3.48,
    efg_rate_per_second=0.87,
    correlation_time_seconds=0.2e-6,
    frequency_cluster_width_hz=20e3,
)
result = simulate_crossover_slse_powder(
    site, b0_tesla,
    nutation_hz=100e3,
    excitation_duration_seconds=1.77e-6,
    refocus_duration_seconds=3.54e-6,
    echo_spacing_seconds=2e-3,
    num_echoes=30,
    relaxation=relaxation,
    rf_frequency_hz=carrier,
    orientations=active,
    pulse_model="rwa",
    acquisition_duration_seconds=100e-6,
    acquisition_points=129,
    receiver_bandwidth_hz=200e3,
    num_workers=4,
    parallel_backend="process",
    retain_local_results=False,
)
measured_train = result.matched_echo_amplitudes
absolute_scale = retained_weight
```

The simulator samples the complete complex waveform around every echo, coherently averages crystallites, applies the finite-bandwidth receiver, and projects the filtered echoes onto the first-echo template. Echo-center values alone are acceptable only after demonstrating that they reproduce this waveform-derived observable; they generally fail once B_0 broadens the powder spectrum.

Convergence must be demonstrated independently against nested orientation prefixes, acquisition point count and window, receiver bandwidth, spectral slice width, RF propagation model, and nonsecular frequency-cluster width. Check trace, Hermiticity, positivity, and Gibbs stationarity as well. The process backend parallelizes independent crystallites; setting `retain_local_results=False` avoids returning their complete density histories. The detailed rationale, failure modes, validation thresholds, and recommended build sequence are preserved in `docs/nqr_powder_relaxation_methods.md`.

8.7 Weak Static B0 Spectra

Weak static magnetic fields are modeled by exact diagonalization of

$$H = H_Q + H_Z, \qquad H_Z = -2\pi\gamma\,\mathbf{B}_0 \cdot \mathbf{I}.$$

The helper is intended for the NQR regime

$$\epsilon_Z = \frac{|\gamma B_0|}{\nu_{\mathrm{ref}}} \ll 1,$$

and reports the largest perturbation ratio in the result. It can warn or raise if the requested field is no longer weak compared with the selected NQR line.

```
from spin_dynamics.nqr import simulate_weak_b0_spectrum

weak = simulate_weak_b0_spectrum(
    chlorine,
    b0_tesla=1e-3,
    orientations="powder",
    broadening_hz=200.0,
    weak_ratio_threshold=0.05,
)

print(weak.max_perturbation_ratio)
```

For each crystallite, the code diagonalizes $H_Q + H_Z$, keeps transitions near the selected zero-field NQR line, and weights them by the RF projection

$$I_{ab} \propto w_k \left| \hat{\mathbf{b}}_{1,k} \cdot \mathbf{d}_{ab,k} \right|^2 ,$$

where w_k is the orientation weight. This RF projection is important for spin-3/2 in weak fields: the eigenvalue splitting can contain several nearby transitions, but transitions dark to the coil should not contribute to the plotted spectrum. In the axial limit $\eta = 0$, $B_0 \parallel z_{\mathrm{PAS}}$, and $B_1 \perp B_0$, a spin-3/2 site gives the expected pair $\nu_Q \pm \gamma B_0$.

8.8 Polarization-Enhanced NQR Transport

Polarization-enhanced NQR is modeled at the instrument level following the Glickstein–Mandal automated pre-polarization workflow. A solid sample first sits in a permanent magnet long enough to build proton polarization. It is then translated through the falling fringe field into the NQR probe. Whenever the proton Larmor frequency

$$\nu_H(x) = \gamma_H B_0(x)$$

crosses a nitrogen NQR line, dipolar coupling can transfer polarization from the proton reservoir into the quadrupolar population difference. For a sample moving with speed v through a local gradient dB_0/dx, the available crossing time is approximately

$$t_{\mathrm{cp}} \approx \frac{\Delta\nu_H}{\gamma_H \, v \, |dB_0/dx|}.$$

The transfer is adiabatic when this time is long compared with the inverse ^{1}H-^{14}N coupling rate ν_{NH}^{-1}, or equivalently when

$$\frac{\gamma_H v |dB_0/dx|}{\Delta\nu_H \, \nu_{NH}} \ll 1.$$

The helper `simulate_adiabatic_polarization_transfer` evaluates this criterion along a user-specified transport path, using either a finite Halbach magnet model or any callable B0 sampler. It reports the level-crossing positions, local gradients, adiabatic ratios, transfer efficiencies, ideal spin-1 enhancement factors, and practical enhancement after finite proton build-up, transfer efficiency, finite sample size, and T_1 loss during transport.

```
from spin_dynamics.nqr import (
    CylindricalSampleGeometry, HalbachPrepolarizationMagnet,
    LinearTransportMotion, PolarizationEnhancedNQRSample,
    simulate_adiabatic_polarization_transfer,
)

sample = PolarizationEnhancedNQRSample(
    name="melamine-like",
    line_labels=("nu+", "nu-", "nu0"),
    line_frequencies_hz=(2.766e6, 2.034e6, 0.732e6),
    protons_per_molecule=6,
    nitrogens_per_molecule=6,
    proton_t1_seconds=48.6,
    proton_linewidth_hz=80e3,
    proton_nitrogen_coupling_hz=1.3e3,
)
magnet = HalbachPrepolarizationMagnet(
    rod_shape="square", rod_width=25.4e-3, length=101.6e-3,
)
geometry = CylindricalSampleGeometry(length=20e-3, diameter=8e-3)
motion = LinearTransportMotion(0.0, 0.10, velocity=0.1667, axis="z")

pe = simulate_adiabatic_polarization_transfer(
    magnet, sample, geometry, motion,
    prepolarization_time_seconds=100.0,
)
print(pe.practical_enhancement)
```

The model intentionally does not replace a microscopic coupled-spin density-matrix calculation of each avoided crossing. Instead, it provides a transparent engineering model for choosing magnet geometry, transport speed, sample size, pre-polarization time, and the uncertain coupling scale ν_{NH}.

8.8.1 Estimating ^{1}H-N Coupling from CIF Structures

The coupling scale can be estimated from a crystal structure when nearby proton positions are known. The helper `estimate_proton_dipolar_couplings_from_cif` reads a small-molecule CIF, applies the listed symmetry operations and periodic images, finds protons within a search radius of the quadrupolar atom label, and evaluates the point-dipole prefactor

$$d_{HQ}(r) = \frac{\mu_0}{4\pi}\frac{h\,|\gamma_H\gamma_Q|}{r^3}.$$

It returns individual proton distances and couplings, plus an RMS effective coupling useful as ν_{NH} in the adiabatic-transfer criterion. If a field direction is supplied, it also reports the secular angular factor $(1-3\cos^2\theta)d_{HQ}$.

```
from spin_dynamics.nqr import estimate_proton_dipolar_couplings_from_cif

estimate = estimate_proton_dipolar_couplings_from_cif(
    "../QuadrupolarDFT/structures/Melamine/237082.cif",
    "N101",
    proton_radius_angstrom=3.0,
)
print(estimate.effective_rms_hz)
```

```
for item in estimate.proton_couplings:
    print(item.proton_label, item.distance_angstrom, item.coupling_hz)
```

For the bundled melamine CIF, the ring nitrogen `N101` sees four nearby protons within 3 Å and gives an RMS coupling of about 1.3 kHz. Protonated amine nitrogens have direct N–H distances and therefore much larger couplings. The example `plot_nqr_polarization_enhancement.py` uses the melamine CIF estimate by default, but accepts `-nh-coupling-hz` to override it.

8.8.2 Using the Local NQR Data and Structure Workspaces

The example `plot_nqr_database_prepolarization.py` demonstrates how the separate workspaces can be used together. It reads measured transition frequencies, site metadata, T_1 values, and provenance from the SQLite export in `../NQRDatabase/data/exports/nqr.sqlite`; it then estimates the effective $^{1}\mathrm{H}$-$^{14}\mathrm{N}$ coupling from a CIF stored under `../QuadrupolarDFT/structures`; finally, it passes the resulting sample parameters to `simulate_adiabatic_polarization_transfer`. The default case uses the glycine database record and the bundled glycine CIF:

```
python examples/plot_nqr_database_prepolarization.py \
    --compound Glycine \
    --coupling-target N1 \
    --output results/nqr_database_prepolarization.png
```

This is a useful pattern for screening real compounds. The database supplies experimentally reported NQR lines and relaxation metadata; the structure workspace supplies atom positions for nearby-proton coupling estimates and can also hold first-principles EFG or resonant-frequency calculations when measured lines are absent or need comparison. The simulation layer then converts those inputs into transition-by-transition ideal and practical prepolarization signal gains for a chosen magnet, sample geometry, and transport speed.

8.9 SLSE Detection

The spin-lock spin-echo (SLSE) sequence is the classic pulsed NQR detection sequence. The helper models it as repeated selective pulses on a spin-1 detection transition and reports echo amplitudes at the requested echo spacing. Echo decay can be represented by a scalar T_{2e} envelope or by a phenomenological Liouville-space relaxation model with T_1 and T_2. For quantitative powder echo trains, especially when comparing coherent powder signal amplitudes, prefer the full density-matrix `simulate_full_slse` path described above.

Microscopic Redfield/dipolar relaxation can be supplied through the same `relaxation=` slot on the full-density-matrix path. The NaNO2 example reads the $^{14}\mathrm{N}$ EFG parameters from the QuadrupolarDFT summary, builds nearby $^{23}\mathrm{Na}$ and $^{14}\mathrm{N}$ stochastic bath sources from the CIF, and propagates a coherent SLSE train. The powder row uses equal SLSE pulse lengths with the spin-1 powder optimum near $119°$; the refocusing pulse phase is the full-model default $\pi/2$ shift. It plots the coherent measured powder voltage, not magnitude- or RMS-averaged local crystallite signals:

```
python examples/plot_redfield_nano2_slse.py --output nano2_redfield_slse.png
```

```
from spin_dynamics.nqr import simulate_slse, slse_sequence

sequence = slse_sequence(
    "x",
```

```
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=1e-3,
    num_echoes=16,
)

result = simulate_slse(
    site,
    sequence,
    orientations="powder",
    t2e_seconds=20e-3,
)
```

The returned `SLSEResult` includes echo times, powder-averaged echo amplitudes, per-orientation echo amplitudes, orientation weights, transition metadata, and the eigensystem used for the first orientation. When a relaxation model is supplied, the result also reports local cycle eigenvalues and cycle-derived effective decay times.

```
from spin_dynamics.nqr import NQRRelaxationModel

relaxed = simulate_slse(
    site,
    sequence,
    orientations="powder",
    relaxation=NQRRelaxationModel(t1_seconds=1.0, t2_seconds=20e-3),
)
```

Offset and pulse-period sweeps are available as compact diagnostics for the SLSE modulation:

```
from spin_dynamics.nqr import simulate_slse_offset_sweep

sweep = simulate_slse_offset_sweep(
    site,
    "x",
    offsets_hz=[-2e3, 0.0, 2e3],
    pulse_duration_seconds=25e-6,
    nutation_hz=10e3,
    echo_spacing_seconds=500e-6,
    relaxation=NQRRelaxationModel(t2_seconds=20e-3),
)
```

8.9.1 Circular and Multi-Axis Excitation

The examples above excite the powder with a single linearly-polarized coil. The full density-matrix engine keeps a general RF direction and phase per coil and is linear in the field, so several coils superpose as co-rotating RWA couplings. The helper `multi_axis_pulse_hamiltonian` sums a list of `CoilDrive` components (each a bare nutation, phase, and PAS direction); with one component it reproduces `pulse_hamiltonian` exactly. *Circular* polarization is the special case of two orthogonal coils driven $\pi/2$ out of phase (`circular_pulse_hamilto` producing a rotating RF field, with a matched quadrature receiver `quadrature_detection_operator`.

Because a multi-coil drive couples to two or three orthogonal lab axes at once, its powder response depends on the *full* crystallite orientation, not just one coil axis. `powder_frame_grid` provides the required SO(3) grid of orthonormal lab-to-PAS frames (Gauss–Legendre $\cos\beta$ × uniform α × uniform in-plane χ).

```
python examples/plot_nqr_circular_polarization_nutation.py --output circular.png
```

In zero-field NQR the transition has no intrinsic precession handedness, so a linear coil couples equally to both rotation senses whereas a circular coil couples to one. Across a powder this raises the *refocused* (SLSE) signal: neither circular excitation alone (single-coil detection) nor quadrature detection alone helps, but together, at matched per-coil B_1, they lift the powder echo train by $\sim 1.6\times$ (each scheme calibrated to its own optimal flip, so the trivial $\sim 2\times$ field efficiency of a rotating field is normalized out). Detecting with the *wrong* rotation sense nulls the powder signal — a direct check that the handedness is physical. Circular uses two coils, drawing $\sim 2\times$ peak RF power and adding a $\sqrt{2}$ two-channel receiver-noise penalty, so the $\sim 1.6\times$ signal gain corresponds to a $\sim 1.1\times$ net SNR gain — the slight increase reported in the powder NQR literature.

8.10 SORC Detection

The strong off-resonance comb (SORC) sequence is represented as $(\tau - \phi - \tau)^N$, with the observation sampled between repeated off-resonant pulses. The default `simulate_sorc` path follows the finite-N pathway recurrence used by Konnai, Odano, and Asaji (2008). As the number of pulses grows, this recurrence converges to the published steady-state powder expression and retains the expected $\Delta\omega\tau$ comb periodicity. Use `model="coherent"` only when the desired calculation is the closed unitary transient train rather than the dephased SORC pathway model.

```
from spin_dynamics.nqr import QuadrupolarSite, simulate_sorc, sorc_sequence

site = QuadrupolarSite(
    spin=1,
    isotope="14N",
    quadrupole_frequency_hz=4.2e6,
    eta=0.3,
)
sequence = sorc_sequence(
    "x",
    pulse_duration_seconds=20e-6,
    nutation_hz=16.5e3,
    half_spacing_seconds=0.8e-3,
    num_pulses=96,
    rf_frequency_hz=4.60425e6 - 2.05e3,
)

result = simulate_sorc(site, sequence, orientations="powder")
print(result.signal_amplitudes[-1])
```

For direct paper-style comparisons, `sorc_powder_pathway_signal` evaluates the finite-N recurrence in Konnai's one-dimensional powder angle, which avoids aliasing from coarse spherical orientation grids in long pulse-width sweeps. `sorc_powder_theory_signal` evaluates the corresponding infinite-N steady-state response, while `fid_powder_theory_signal` gives the spin-1 powder FID pulse-width response. The example `examples/plot_nqr_sorc_konnai2008.py` overlays finite-N SORC markers with the infinite-N theory curves for the offset, spacing, and pulse-width sweeps in the paper.

8.10.1 Sensitivity per Unit Time: SLSE vs. Steady-State SORC

The two schemes are often quoted with a $\sqrt{T_1/T_{2e}}$ advantage for SORC, but that law is an artifact of a T_1-independent steady-state model. Done honestly – both sequences fully optimized in one common full density-matrix framework – the schemes are *comparable*. The figure of merit is SNR per unit $\sqrt{\text{time}}$, because averaging N acquisitions improves SNR as $\sqrt{N}$ with $N = T/t_{\text{scan}}$.

`examples/plot_nqr_slse_sorc_sensitivity.py` grounds the comparison in a specific material, ^{14}N in $NaNO_2$, whose three relaxation times are not inserted by hand: T_1 and the spin-locked T_{2e} emerge from a microscopic dipolar Redfield model built from the actual neighbour spins (`RedfieldDipolarRelaxationModel.from_dipola`) and the fast reversible T_2^* is the Van Vleck rigid-lattice second moment of the same neighbour list ($\sim 1\,\text{ms}$ here – weak, because ^{14}N has a small gyromagnetic ratio, so it is set mostly by the ^{23}Na neighbours). The correlation time τ_c is the temperature knob; sweeping it walks the sample over $T_1/T_{2e} \sim 1$ to 100 while T_2^* stays fixed. Both sequences are propagated from the same equilibrium ρ_{eq} and read with the same detection operator (one common normalization, no per-scheme scaling); the SORC steady state is the affine fixed point of its cycle with an equilibrium-target source $(I - M)\rho_{\text{eq}}$, so it correctly carries the T_1 that replenishes the polarization. Sensitivity is the matched-filter energy $\sqrt{\int |s(t)|^2\, dt}$ of each acquisition window, and flip angle, RF offset, and pulse spacing are optimized for each scheme at each τ_c.

Result 1: comparable sensitivity, with a mild crossover. On a single crystal the two schemes stay within $\sim 15\%$ of each other across the entire T_1/T_{2e} range. SORC is marginally ahead when the sample is warm ($T_1/T_{2e} \lesssim 5$) and SLSE marginally ahead when it is cold, the ratio crossing unity near $T_1/T_{2e} \approx 5$ (Table 8.1). Neither grows a running advantage. The often-quoted $\sqrt{T_1/T_{2e}}$ law would predict SORC winning by more than an order of magnitude at the cold end; it does not, because that law comes from a T_1-*independent* steady-state model (the reduced Konnai pathway). Once the SORC steady state is forced to carry T_1 through its equilibrium-target source, its per-time signal falls off with relaxation at essentially the same rate as SLSE's. The mirror-image error – "SLSE wins increasingly" – comes from under-optimizing the SORC flip angle: its true optimum is a *small* tip (~ 10–$20°$), a balanced-SSFP-like high-retention comb, and coarse large-flip grids miss it.

T_1/T_{2e}	T_1 (ms)	T_{2e} (ms)	SNR/$\sqrt{\text{time}}$ (a.u.)		SORC / SLSE
			SLSE	SORC	
0.8	134	165	0.230	0.248	1.08
1.2	156	134	0.197	0.221	1.12
2.6	255	100	0.136	0.152	1.12
8.0	482	60	0.076	0.070	0.91
29	949	33	0.040	0.036	0.89
112	1891	17	0.020	0.018	0.88

Table 8.1: Optimized single-crystal sensitivity of SLSE and steady-state SORC for ^{14}N in $NaNO_2$, swept over the dipolar correlation time ($T_2^* \approx 1.07\,\text{ms}$ throughout). Both schemes are fully optimized over flip, offset, and spacing in one common normalization.

Result 2: SORC is more robust to powder averaging. The one robust *sensitivity* asymmetry appears when the single crystal is replaced by a powder. At $T_1/T_{2e} \approx 29$ the SLSE sensitivity drops by $\sim 30\%$ from crystal to powder while SORC barely moves, so SORC leads by $\sim 1.3\times$ on the powder (and $\sim 1.1\times$

with fixed operating points; the effect is grid-converged from a 4×6 to a 9×18 orientation mesh). The mechanism is the per-orientation spread of the signal: the SLSE $90/180$ echo has a strongly orientation-dependent refocusing efficiency – across the powder its per-crystallite echo energy varies by more than an order of magnitude, with some crystallites sitting near a nutation null – whereas SORC's small-tip steady state is nearly uniform across orientations (its per-crystallite energy varies by only a factor of a few). A powder therefore averages away much of the SLSE signal but little of the SORC signal. This is a genuine, potentially useful result for powder samples, not a grid artifact.

Result 3: SORC draws less average RF power. Because both schemes drive the same coil at the same nutation frequency, the peak (in-pulse) RF power is identical and the *average* RF power is peak power times duty cycle; the average-power ratio is thus a pure duty-cycle ratio. At the optima SORC draws only ~ 0.2–$0.4\times$ the average power of SLSE across the whole sweep. This is the opposite of the intuition that a *continuously* pulsing comb must dissipate more: SORC's small $\sim 15^\circ$ tip makes each pulse very short (here $\sim 1.7\,\mu\mathrm{s}$ every $260\,\mu\mathrm{s}$, a 0.6% duty), whereas the SLSE matched filter prefers *wide* pulses ($\sim 70^\circ$) at the *shortest* allowed echo spacing, giving a much higher in-train duty ($\sim 4\%$). Even after amortizing SLSE's duty over its T_1-recovery idle time, SORC stays the lower-power method. One caveat keeps this an operating-point statement rather than a fundamental bound: SLSE's sensitivity is nearly flat in echo spacing near the optimum, so it could be run at a longer spacing – and hence lower power – at little cost in SNR; the example simply reports the matched-filter optimum, which happens to be power-hungry.

Putting it together. For this material the two classic pulsed-NQR detection families are close enough in raw sensitivity that the decision is driven by the *secondary* properties the example quantifies: SORC is somewhat more forgiving on powders and runs at lower average RF power (attractive for portable, battery, or heating-sensitive work), but only as a continuously running steady state; SLSE is a simple pulse-and-recover experiment that reaches the same crystal sensitivity and is marginally better in the cold, long-T_1 regime. The broader lesson is methodological: a fair sensitivity comparison of two pulse sequences requires a common signal normalization, grounded (not hand-inserted) relaxation times, and a full optimization of *both* sequences – dropping any one of these reproduces one of the folklore "advantage laws."

8.11 EFG Inhomogeneity and SLSE Spectra

Static EFG disorder is modeled as an isochromat ensemble of independent quadrupolar sites. Each isochromat has its own ν_Q, η, transition frequency, and normalized weight. This covers impurity-induced broadening, static temperature gradients, and other inhomogeneous mechanisms analogous to NMR isochromat broadening.

```
from spin_dynamics.nqr import (
    SelectivePulse,
    gaussian_efg_distribution,
    simulate_fid_efg_distribution,
)
import numpy as np

distribution = gaussian_efg_distribution(
    site,
    quadrupole_std_hz=2e3,
    samples=41,
)
```

```
fid = simulate_fid_efg_distribution(
    distribution,
    "x",
    times_seconds=np.linspace(0.0, 20e-3, 512),
    excitation=SelectivePulse("x", duration_seconds=2.5e-6, nutation_hz=100e3),
)
```

The distribution simulators check whether a coarse EFG grid can artificially rephase within the simulated duration. Increase the number of isochromats when the warning appears, or pass `rephase_action="ignore"` only after a convergence check.

In an SLSE experiment, the spectrum is not the FFT of the echo train. It is the FFT of the averaged echo acquired over a receiver window T_{acq} centered on a selected echo, with T_{acq} shorter than the pulse spacing. The finite acquisition window itself broadens the spectrum:

```
from spin_dynamics.nqr import simulate_slse_acquisition_spectrum

slse_spectrum = simulate_slse_acquisition_spectrum(
    distribution,
    sequence,
    acquisition_duration_seconds=200e-6,
    acquisition_points=256,
    echo_index=-1,
    noise={"target_snr": 20.0, "seed": 123},
    deconvolution_strength=1e-2,
)
```

Noise is added to the complex time-domain acquisition before the FFT. The optional deconvolution performs a regularized inversion of the same finite-window FFT operator. It is useful for exploring acquisition-window broadening, but it can amplify noise and should be checked across plausible SNR values.

8.12 RFI Rejection and Echo-Aware SLSE Fitting

Low-field pulsed NQR is often limited by coherent radio-frequency interference (RFI), not only by receiver noise. This is especially true for $^{14}\mathrm{N}$ compounds whose transition frequencies can lie in or near the AM broadcast band. The new `spin_dynamics.interference` layer treats this as a reference-channel estimation problem on the same shot clock as the NQR sequence. It is deliberately split into two parts:

- sequence-independent RFI sources, pickup coils, cancellers, and diagnostics in `spin_dynamics.interference`;
- NQR-specific mask adapters, such as `slse_acquisition_mask`, in `spin_dynamics.nqr`.

The mask is essential. During transmit and ringdown the primary receiver is not a faithful linear measurement of the field, while the reference channels may continue to observe the environment. During quiet receive intervals the primary sample can be modeled as

$$y_n = s_n(\theta) + r_n + \epsilon_n,$$

where $s_n(\theta)$ is the NQR response, r_n is the RFI coupled into the primary channel, and ϵ_n is receiver noise. The reference array $X \in \mathbb{C}^{K \times N}$ contains K auxiliary pickup channels,

$$x_{k,n} = \mathbf{c}_k^{\mathsf{T}} \frac{\mathrm{d}}{\mathrm{d}t} \mathbf{B}_{\mathrm{RFI}}(\mathbf{p}_k, t_n) + \ell_k s_n + \xi_{k,n},$$

with pickup vector $\mathbf{c}_k$, coil position $\mathbf{p}_k$, optional NQR leakage ℓ_k, and reference noise $\xi_{k,n}$. Far-field RFI is represented by a nearly uniform vector field. Near-field RFI is represented by magnetic dipole sources with the usual $1/r^3$ spatial pattern, which makes multiple reference stations valuable because each station sees a different linear mixture of the interfering sources.

8.12.1 Masked and Windowed Reference Cancellation

The simplest stationary reference canceller predicts the RFI in the primary channel by a tapped linear model,

$$\widehat{r}_n = \sum_{k=1}^{K} \sum_{\ell=0}^{L-1} h_{k,\ell}\, x_{k,n-\ell}.$$

Let ϕ_n be the row vector of tapped reference samples at time n, and let B be the boolean fitting mask. Gated ridge fitting solves

$$\widehat{\mathbf{h}} = \arg\min_{\mathbf{h}} \sum_{n \in B} |y_n - \phi_n \mathbf{h}|^2 + \lambda \|\mathbf{h}\|_2^2.$$

For SLSE, B should exclude transmit, ringdown, and expected echo windows when the method does not explicitly model the NQR signal. The scalar canceller is the $L = 1$ case. The FIR form allows small group delays or front-end phase shifts between the reference and primary chains.

Time-varying interferers, such as randomly modulated AM-like carriers or moving near-field sources, are not well described by one global coefficient vector. Two alternatives are implemented. Mask-aware LMS/NLMS/RLS produces a continuous prediction but freezes coefficient updates outside trusted windows. The offline windowed ridge method fits one coefficient vector per window, $\mathbf{h}_w$, and regularizes adjacent windows:

$$\min_{\{\mathbf{h}_w\}} \sum_{w} \sum_{n \in B_w} |y_n - \phi_n \mathbf{h}_w|^2 + \lambda \sum_{w} \|\mathbf{h}_w\|_2^2 + \alpha \sum_{w>0} \|\mathbf{h}_w - \mathbf{h}_{w-1}\|_2^2.$$

This is the recommended first offline method when the front end has not saturated. It does not need to run in real time, and it can use future samples to estimate slowly varying coupling.

Impulsive pickup—Poisson-timed switching transients whose bursts are large and rare—violates the Gaussian-residual assumption of squared-error fitting: a handful of spiky samples dominate the normal equations and drag $\widehat{\mathbf{h}}$ away from the coherent coupling. The robust canceller `robust_fir_canceller` replaces the ℓ_2 loss with a Huber loss, minimized by iteratively reweighted least squares. Each iteration solves the weighted normal equations $\Phi^{\mathsf{H}} W \Phi\, \mathbf{h} = \Phi^{\mathsf{H}} W \mathbf{y}$ with $W = \mathrm{diag}(w_n)$, where $w_n = 1$ for $|r_n| \le \delta s$ and $w_n = \delta s / |r_n|$ otherwise, s is a robust (MAD-based) scale of the residual re-estimated each iteration, and δ (default 1.345) is the transition point. Samples in the impulsive tail are down-weighted so the fit tracks the stationary RFI path; the returned `fit_weights` expose which samples were treated as outliers.

The robust fit protects the coherent-transfer estimate but leaves the impulses in the cleaned output. `sparse_reference_canceller` instead models them, minimising $\|y - \Phi h - s\|^2 + \lambda\|h\|^2 + \mu\|s\|_1$ by alternating a ridge least-squares update of the FIR coefficients (fitted on the mask with s removed, so the transfer stays unbiased) with a soft-threshold update of the sparse impulse estimate s over the whole record. With μ set above the NQR echo amplitude and below the switching-transient amplitude, the bursts are captured in s while the signal passes through, and `cleaned` has both the coherent RFI Φh and the impulses s removed.

When the reference-to-primary coupling is strongly frequency dependent – a resonant probe or a long impulse response – a compact FIR would need many taps. `frequency_domain_canceller` instead estimates a

complex transfer $W_k(f)$ in every DFT bin from averaged cross-spectra over Hann-windowed segments that lie inside the fit mask,

$$W(f) = \big(S_{xx}(f) + \lambda I\big)^{-1} s_{xy}(f), \qquad \widehat{Y}(f) = W(f)^{\mathsf{H}} X(f),$$

with $S_{xx}(f) = \sum_{\text{seg}} X X^{\mathsf{H}}$ and $s_{xy}(f) = \sum_{\text{seg}} X\,\overline{Y}$. The prediction is returned to the time domain by weighted overlap-add. Because the NQR signal is uncorrelated with the references it does not bias W, only adds estimator variance, so gating to the baseline set is helpful but not required. The result also carries the multiple-coherence spectrum $\gamma^2(f) = s_{xy}^{\mathsf{H}} S_{xx}^{-1} s_{xy} / S_{yy} \in [0, 1]$, the fraction of primary power the references explain at each frequency – a direct readout of which parts of the band are cancellable. Application is exact for a pass-through and approximate when the coupling impulse response approaches the segment length (per-segment circular convolution); the first and last segment are edge-tapered.

All of the above need a correlated reference. When the interferer is narrowband and no clean reference exists, `kalman_harmonic_canceller` exploits its structure directly. Each carrier f_j is modelled as $I_j \cos(\omega_j n) - Q_j \sin(\omega_j n)$ with the quadratures $[I_j, Q_j]$ following a random walk, and a Kalman filter tracks them from the (scalar) primary. The subtracted estimate is the *a priori* prediction, so the current sample's NQR content never enters the carrier estimate, and measurement updates are frozen wherever `update_mask` is false, letting the tracker coast through the echo windows rather than absorb them. The knobs are the process standard deviation (how fast the amplitude may drift) and the measurement standard deviation (the observation-noise scale). Because it needs only the carrier frequencies, it removes a drifting amplitude-modulated broadcast line sitting on top of the NQR line with no reference channel at all.

8.12.2 Echo-Aware Joint Fitting

For SLSE, the echo train itself has strong temporal structure. Reference-only cancellation uses this structure only indirectly through masks and diagnostics; it does not prevent the RFI model from absorbing echo-like primary-channel structure. The echo-aware joint estimator adds an explicit signal subspace. Let $\Psi \in \mathbb{C}^{N \times M}$ be a basis for plausible NQR echo trains. The fixed joint model is

$$\mathbf{y} \approx \Phi \mathbf{h} + \Psi \mathbf{a},$$

where Φ is the tapped reference design matrix and $\mathbf{a}$ are signal-basis coefficients. The fitted reference contribution is still the RFI estimate, while $\Psi\widehat{\mathbf{a}}$ is the fitted NQR component.

The practical SLSE example uses the windowed form,

$$\begin{aligned}\min_{\{\mathbf{h}_w\},\mathbf{a}} \quad & \sum_w \sum_{n \in B_w} |y_n - \phi_n \mathbf{h}_w - \psi_n \mathbf{a}|^2 \\ & + \lambda_{\text{ref}} \sum_w \|\mathbf{h}_w\|_2^2 + \alpha \sum_{w>0} \|\mathbf{h}_w - \mathbf{h}_{w-1}\|_2^2 + \lambda_s \|\mathbf{a}\|_2^2.\end{aligned}$$

Here ψ_n is the row of the SLSE basis at sample n. Unlike the reference-only methods, this fit may include the echo windows because the echo has its own model block. The implementation is `windowed_joint_signal_reference_canceller`. It returns both a cleaned waveform $y - \widehat{r}$ and `signal_estimate`, the fitted NQR component. For parameter estimation, the example scores `signal_estimate` because that is the quantity the joint estimator is designed to recover.

The default SLSE basis in `examples/plot_nqr_rfi_statistical_study.py` is a small grid of echo trains,

$$\psi_{\nu, T_{2e}}(t_n) = \sum_j g(t_n - \tau_j) \exp(-\tau_j / T_{2e}) \exp(\mathrm{i} 2\pi \nu \tau_j),$$

where g is the echo-window template and τ_j are the expected echo centers. The command-line options `-echo-aware-frequency-span-hz`, `-echo-aware-frequency-points`, and `-echo-aware-t2e-grid-ms` control this dictionary. This is still a linear ridge problem because the frequencies and decay constants are gridded. A future nonlinear refinement step can use the best grid component as an initial guess for continuous ν and T_{2e}.

8.12.3 Parameter Endpoints and Diagnostics

RFI suppression in dB is a useful engineering diagnostic, but it is not the experimental endpoint. For NQR detection, the important quantities are the resonant frequency, initial echo amplitude, and decay constant T_{2e}. The statistical example projects the recovered record onto the echo templates, producing complex responses c_j at echo times τ_j. It then estimates

$$\widehat{\nu} = \frac{1}{2\pi}\frac{\mathrm{d}}{\mathrm{d}\tau}\,\mathrm{unwrap}\arg c(\tau),$$

and fits

$$\log|c_j| \approx \log A_0 - \tau_j/T_{2e}.$$

The plotted endpoints are therefore $|\widehat{\nu}-\nu|$, relative A_0 error, and relative T_{2e} error, all averaged over Monte Carlo trials with approximate 95% confidence intervals of the mean.

The diagnostics layer also provides:

- RFI suppression using the clean signal when available;
- matched-filter SNR improvement;
- amplitude and phase bias of the recovered signal;
- prominent residual spectral lines;
- reference design rank and condition number;
- the reference-noise-injection estimate, i.e. the white noise a fitted canceller passes from noisy reference channels into the cleaned output ($\mathrm{var} = \sum_k \sigma_k^2 \sum_l |h_{k,l}|^2$), the "noise figure" that decides whether suppression is a net win;
- saturation flags for front-end clipping checks.

Saturation is a hard boundary. If the primary channel clips before digitization, linear reference cancellation cannot reconstruct the lost waveform. Offline methods are appropriate only when the RFI remains inside the linear dynamic range of the receiver.

8.12.4 Active Feedforward Cancellation

When the interferer would saturate the receiver, the cure is to cancel it in the analog domain, before the amplifier and ADC, so the digitizer only ever sees the residual. The `spin_dynamics.interference.active` module models this with a `CompensationActuator` and `feedforward_cancel`. A fitted linear model (the same `LinearCancellerModel` used for digital cancellation, typically calibrated on the reference chain) supplies the commanded compensation field $\widehat{r}_n$ from the continuous references. The actuator realizes it imperfectly, and the residual seen by the ADC is

$$y_n^{\mathrm{res}} = y_n - g\,\widetilde{r}_{n-L},$$

where g is the analog gain/phase mismatch, L the causal actuator latency, and $\widetilde{r}$ the commanded field after any drive-range clip and bandwidth limit. The ADC then clips y^{res} rather than y. This is the decisive difference from digital cancellation `model.apply(clip(y))`, which subtracts *after* the clip and cannot recover saturated peaks.

Two physical limits bound feedforward. The causal latency limits the cancellation bandwidth: a tone at angular frequency ω is only suppressed by the factor $|1 - g\,e^{-i\omega L}|$, so cancellation fails (and eventually *adds* interference) once ωL approaches π. The finite drive range `max_field` caps how much of a large interferer the coil can oppose, so deep nulls require both low latency and adequate headroom. The example `examples/plot_nqr_rfi_active_feedforward.py` drives the primary well past full scale and contrasts digital-too-late against active feedforward, then sweeps both limits. Because the digitized active residual is again linear, a digital canceller can be run on it afterward for the last few decibels.

8.12.5 Loading Measured Records

A measured acquisition is just windows of ADC samples: a primary channel and one or more reference channels on a shared clock. The gating the cancellers need is not recorded per sample – it is reconstructed from the pulse-sequence timing. `slse_mask_from_metadata` and `sorc_mask_from_metadata` rebuild an acquisition mask of exactly the recorded sample count from the echo spacing, detection duration, pulse count, ringdown, and baseline offsets; `nqr_recording_from_samples` pairs the sample windows with that mask and returns an `RFIRecording` that drops straight into the `(primary, references, mask)` contract. `save_rfi_recording` and `load_rfi_recording` round-trip a recording through a NumPy `.npz` archive, storing the reconstructed mask as its label array so no re-derivation is needed on reload.

8.12.6 Typical Monte Carlo Results

The example `examples/plot_nqr_rfi_statistical_study.py` synthesizes five independent near-field magnetic-dipole RFI sources, tri-axial reference stations, a complex SLSE echo train, receiver noise calibrated by initial echo SNR, and randomly modulated AM-like interference. It compares one-station scalar cancellation, all-station scalar cancellation, offline windowed ridge, NLMS adaptation, and the echo-aware joint fit.

The most important control is spectral overlap. With the command-line default `-rfi-spectral-mode in-band`, the RFI carrier is placed near the echo phase frequency and contaminates the NQR parameter estimator. With `-rfi-spectral-mode out-of-band`, the same machinery is a null/control case: the interference mostly averages out of the echo estimator, so parameter error is dominated by receiver noise.

For 48 in-band Monte Carlo trials, the default example produced:

Method	Frequency error (Hz)		Relative T_{2e} error	
	SNR 4	SNR 35	SNR 4	SNR 35
1 station scalar	25.09	19.76	4.659	3.901
all stations scalar	27.21	8.03	3.507	0.467
windowed ridge	26.05	3.19	3.584	0.195
NLMS adaptive	29.26	8.05	6.041	0.345
echo-aware joint	5.08	2.31	7.813	0.258

The echo-aware method gives the clearest improvement in resonant-frequency accuracy when the RFI overlaps the NQR estimator, especially at low initial echo SNR. Its low-SNR T_{2e} estimates remain heavy-tailed because decay fitting is sensitive to failed or over-regularized late echoes; this is a real limitation of the first linear-grid method, not a plotting artifact. At high SNR, windowed ridge and echo-aware joint fitting are both useful, with the joint method favoring frequency recovery and the windowed method sometimes giving a slightly cleaner decay estimate.

The same run also tests near-field reference-count scaling at initial echo SNR 12. For windowed ridge, going from one to five tri-axial stations improved mean frequency error from 15.44 to 8.43 Hz and relative T_{2e} error from 2.848 to 0.877. For the echo-aware joint fit, the corresponding frequency error was already much lower, 3.38 to 2.57 Hz, and relative T_{2e} error improved from 2.959 to 0.547. The suppression diagnostic is much larger for the joint fit because the reported endpoint is the fitted NQR component rather than only the waveform after subtracting predicted RFI.

In the out-of-band control, all methods become mostly noise-limited at high SNR: frequency errors collapse to roughly 0.2–0.4 Hz and relative T_{2e} errors to about 0.05. That null result is valuable. It shows why RFI rejection must be evaluated with spectral overlap and NQR parameter endpoints; residual power alone can make a benign out-of-band interferer look experimentally important, or make an in-band interferer look acceptable when it is biasing the frequency and decay estimates.

```
python examples/plot_nqr_rfi_cancellation.py \
    --output results/nqr_rfi_cancellation.png

python examples/plot_nqr_rfi_statistical_study.py \
    --rfi-spectral-mode in-band \
    --output results/nqr_rfi_statistical_study.png

python examples/plot_nqr_rfi_statistical_study.py \
    --rfi-spectral-mode out-of-band \
    --output results/nqr_rfi_statistical_outofband_control.png
```

8.13 Two-Frequency Population Transfer

The two-frequency NQR workflow follows the perturbation-plus-detection idea used in 2D NQR. A selective pulse on one transition changes populations in the shared three-level spin-1 manifold; a later SLSE block detects the changed amplitude on another transition. The normalized diagnostic is

$$\Delta(p, q) = \frac{S(p, q)}{S_0(q)} - 1,$$

where p is the perturbation transition and q is the detection transition.

```
from spin_dynamics.nqr import SelectivePulse, simulate_population_transfer

transfer = simulate_population_transfer(
    site,
    SelectivePulse("x", duration_seconds=50e-6, nutation_hz=10e3),
    sequence,
    orientations="powder",
)

print(transfer.normalized_difference)
```

The diagnostic plotting example constructs a compact 3×3 map over the spin-1 x, y, and z transitions. Off-diagonal features indicate that perturbation and detection frequencies are connected through the same quadrupolar site.

Part III

Sequences and Experiments

9 Sequence Interchange and Compilation

Pulse programs are easiest to debug when their timing has one explicit, inspectable representation. The `spin_dynamics.sequences` package provides that representation as a small intermediate representation (IR) patterned after the open Pulseq block model. It separates sequence construction and file interchange from a particular simulator backend: import or build the timeline once, validate it, plot it, and then compile it into the arrays a backend needs.

In this chapter. Represent timing once, exchange Pulseq files, validate and plot the timeline, and compile it for a supported simulation backend.

9.1 The Block Model and Units

A `SequenceIR` contains sequential `SequenceBlock` objects. Within a block, an RF event, physical x/y/z gradient events, and an ADC event may overlap. Every block has an explicit duration, and construction fails if an event extends beyond it. RF envelopes use cycles per second (Hz), gradients use cycles per second per meter (Hz/m), phases use radians, and all timing uses seconds. These are Pulseq-native units; backend adapters own conversions to angular-frequency or field units.

Object	Meaning
`RFPulse`	Sampled complex RF envelope plus dwell, delay, frequency and phase offsets, pulse use, and optional center time.
`GradientWaveform`	Sampled physical gradient channel with dwell, delay, and boundary amplitudes.
`ADCEvent`	Uniform receive samples, including frequency/phase offsets and optional per-sample phase.
`SequenceBlock`	Concurrent events with an explicit duration, label, and retained extension metadata.
`SequenceIR`	Ordered blocks, definitions, and source-format/version provenance, plus independent transmit/receive hardware-effects policy.

For example, this constructs one RF block followed by a receive block:

```
import numpy as np
from spin_dynamics.sequences import ADCEvent, RFPulse, SequenceBlock, SequenceIR

sequence = SequenceIR(blocks=(
    SequenceBlock(
        duration_seconds=1e-3,
        rf=RFPulse(250.0 * np.hanning(100), dwell_seconds=10e-6),
        label="excitation",
    ),
    SequenceBlock(
        duration_seconds=4e-3,
        adc=ADCEvent(num_samples=200, dwell_seconds=20e-6),
        label="readout",
```

```
    ),
))
```

Probe effects are selected without modifying the nominal samples. A `HardwareEffectsPolicy` sets `transmit` and `receive` independently to `"ignore"` or `"apply"`. Compilation preserves the policy. A backend must reject an applied path it cannot implement rather than silently falling back to ideal execution. The policy is MRSpinDynamics execution metadata, not a non-standard Pulseq extension, so imported Pulseq files default to ignoring both paths and the policy is attached after import.

9.2 Pulseq Import and Export

Pulseq is used instead of introducing another sequence file format. The importer accepts core Pulseq 1.4.x and 1.5.x files: blocks, RF, arbitrary or default-raster gradients, trapezoids, ADC, and compressed or uncompressed shapes. The exporter writes signed core Pulseq 1.5.0 files.

```
from spin_dynamics.sequences import read_pulseq, write_pulseq

sequence = read_pulseq("protocol.seq")
print(sequence.source_version, sequence.duration_seconds)
write_pulseq(sequence, "protocol_roundtrip.seq")
```

Timing is not rounded silently on export: blocks, delays, and waveform dwells must align with their declared rasters. Required extensions and explicit non-default time shapes raise `PulseqFormatError`. Optional extensions are retained as block metadata but are not yet executed or exported. PPM offsets are preserved; compiling them requires the system frequency.

9.3 Compiler and Motion Adapter

`compile_sequence()` collects waveform raster edges and ADC sample centers into a piecewise-constant `CompiledSequence`. It reports absolute interval starts and durations, complex RF in Hz, three gradient channels in Hz/m, block provenance, and exact ADC times and offsets.

```
from spin_dynamics.sequences import compile_sequence, compiled_to_motion_steps

compiled = compile_sequence(sequence, system_frequency_hz=42.58e6)
steps = compiled_to_motion_steps(compiled, spatial_dimensions=2)
```

The motion adapter converts frequencies to angular units and maps ADC-centered interval boundaries onto the existing moving-isochromat engine. It is the first execution target for the IR, not a claim that every high-level workflow can yet consume arbitrary Pulseq. RF use labels, extension semantics, scanner safety limits, and sequence-specific coherence pathways still require backend-aware interpretation.

9.4 General Execution Through the Facade

The same compiler target is available through the unified experiment facade. `SequenceIRExecution` owns the IR, system frequency, walker settings, random seed, boundary condition, and integration substeps. A required `SequenceDomain` on the sample supplies one to three physical axes, spin density, optional B0/B1 maps, constant velocity, and the mapping from simulated dimensions to physical gradient channels.

```
import numpy as np
from spin_dynamics.experiment import (
    Experiment, Sample, SequenceDomain, SequenceIRExecution,
)
from spin_dynamics.sequences import read_pulseq

ir = read_pulseq("protocol.seq")
x = np.linspace(-10e-3, 10e-3, 65)
study = Experiment(
    sequence=SequenceIRExecution(
        ir=ir, system_frequency_hz=42.58e6, seed=17,
    ),
    sample=Sample(
        t1_seconds=1.0, t2_seconds=0.1,
        diffusion_coefficient=1.5e-9,
        sequence_domain=SequenceDomain(
            axes=(x,), density=np.ones_like(x),
            gradient_channels=("x",),
        ),
    ),
)
print(study.plan().report())
record = study.run()
record.save("results/protocol_run.npz")
```

Planning compiles the complete timeline without initializing a stochastic ensemble, checks the spatial/gradient mapping, reports interval, ADC, and particle counts, and rejects unsupported policies. Execution converts RF and gradients to the moving-isochromat convention, applies relaxation, diffusion, velocity and boundaries, samples with the receive B1 map, and demodulates each ADC sample by its Pulseq frequency and phase offset. The result retains both clean and noisy complex signals, ADC and step metadata, and the final weighted ensemble. Seeds and white receive-noise settings participate in the normal archive provenance and exact-rerun check.

This target is intentionally ideal-hardware-only. An IR whose `HardwareEffectsPolicy` requests transmit or receive `"apply"` fails at plan time, as does probe noise; those requests need a future probe-aware compiler target and are never approximated silently. Pulseq extensions are not executed. Embedded block/event IR round-trips in JSON config; TOML remains for the flatter hand-authored experiment specs.

9.5 Timeline Visualization

Both `SequenceIR.plot()` and `CompiledSequence.plot()` draw aligned RF magnitude/phase, x/y/z gradient, and ADC lanes. Block shading and labels make timing discontinuities, accidental overlaps, missing delays, and receive-window placement visible before a simulation is run. Matplotlib is imported lazily.

```
figure, axes = sequence.plot(time_unit="ms", show_blocks=True)
figure.savefig("results/protocol_timeline.png", dpi=150)
```

The command-line example accepts either a Pulseq file or its built-in spin echo:

```
python examples/plot_sequence_timeline.py protocol.seq \
  --system-frequency-hz 42580000 \
  --output results/protocol_timeline.png
```

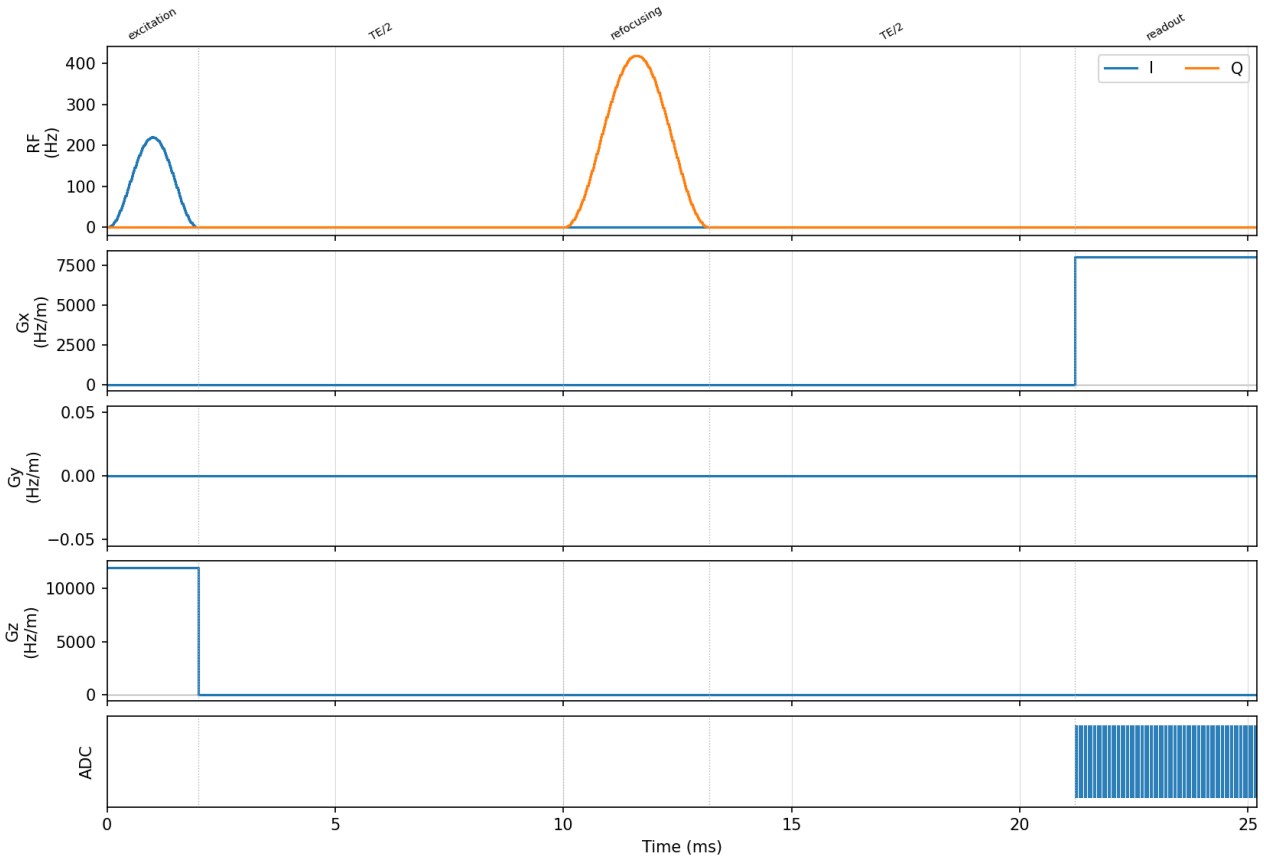


Figure 9.1: Built-in spin-echo `SequenceIR` rendered as synchronized RF, gradient, and ADC channels. Block boundaries and labels make overlap, dead time, gradient polarity, and receive timing inspectable before compilation.

```
python examples/plot_sequence_timeline.py \
  --export-pulseq results/demo_spin_echo.seq \
  --output results/demo_spin_echo.png
```

This visualization is deliberately part of the normal import/compile workflow, not a separate plotting data format: it consumes the same object that is passed to the compiler.

10 Unified Experiment Workflow

The `spin_dynamics.experiment` package is the recommended entry point for new code. It is a thin, declarative layer over the validated `run_*` and `simulate_*` workflows: you describe an experiment with frozen-dataclass specs, front-load validation with `plan()`, execute with `run()`, and save the result with provenance. The facade never re-implements any dynamics – a default `Experiment` reproduces the direct workflow call bit for bit – so it costs nothing in fidelity and adds compatibility checking, runtime estimation, automatic hardware wiring, and serialization on top. If you already know which `run_*` function you want, calling it directly (Chapter 11) remains fully supported; reach for the facade when you want that surrounding scaffolding or a single interface across the NMR, imaging, NQR, and ESR engines.

In this chapter. Describe an experiment with immutable specifications, inspect a plan, run the resolved workflow, and save a provenance-rich result.

The current catalog contains 31 registered sequence/probe routes across ten sequence specifications. Each route delegates to an existing public workflow and declares exactly which sample, hardware, acquisition, and execution fields it consumes.

10.1 The Eight-Step Workflow

The facade organizes an experiment into the same stages every example follows: (1) define the sample; (2) define transmitters/receivers; (3) define the sequence; (4) define the acquisition; (5) `plan()` – resolve the workflow, check compatibility, estimate cost; (6) solve fields (done automatically inside `plan()`/`run()` when coils are given); (7) `run()`; and (8) view, analyze, and save.

```
from spin_dynamics.experiment import (
    Experiment, CPMGTrain, Sample, Hardware, Acquisition,
)

study = Experiment(
    sequence=CPMGTrain(num_echoes=8),
    sample=Sample(t1_seconds=2.0, t2_seconds=2.0),
    hardware=Hardware(probe="tuned"),
    acquisition=Acquisition(numpts=501, maxoffs=10.0),
)

plan = study.plan()
print(plan.report())
record = study.run()
record.save("run1.npz")
```

10.2 Planning Before Running

`plan()` is the front-loaded validation stage. It resolves which workflow will run and returns an `ExperimentPlan` whose `errors` and `warnings` are also available structurally as `plan.findings`. `run()` refuses to execute a plan with errors; warnings are surfaced but do not block. The current rules cover isochromat-grid rephasing (with the recommended `numpts`, respecting `Acquisition.rephase_action`), noise spec validity, coil-field

wiring, and reduced-vs-full NQR model selection. Setting a spec field the resolved workflow does not consume produces an "ignored" warning, so mismatched specs never fail silently.

10.3 Current Facade Coverage

Sequence specification	Probe routes	Delegated model
`CPMG`	ideal, tuned, untuned, matched	Asymptotic CPMG.
`CPMGTrain`	ideal, tuned, untuned, matched	Finite echo train, including supported probe and radiation-damping options.
`CPMGIRTrain`	ideal, tuned, untuned, matched	Inversion-recovery echo trains.
`CPMGImaging`	ideal, tuned, matched	Phase-encoded imaging with phantom and optional solved coil/field maps.
`PGSE`	ideal	Closed-form gradient-moment attenuation with diffusion and transverse relaxation.
`PGSEWalkers`	ideal	Explicit seeded 2-D diffusion, restricted or periodic boundaries, and optional uniform flow.
`NQRFID`, `NQRPopulationTransfer`, `NQRSLSE`, `NQRSORC`	ideal	Full FID, reduced spin-1 transfer, reduced/full SLSE, and reduced SORC.
`ESRCWSweep`, `ESRDEER`, `ESRFID`, `ESRHahnEcho`	ideal	CW field sweep, distance-distribution DEER, and pulsed single-electron ESR.
`ESRTwoPulseESEEM`, `ESRThreePulseESEEM`, `ESRHYSCORE`	ideal	Electron–nuclear modulation traces and the full 2-D HYSCORE plane.
`ESRDaviesENDOR`, `ESRMimsENDOR`	ideal	Analytic pulsed-ENDOR radiofrequency sweeps.

Cost models estimate work and memory for finite trains, inversion-recovery, closed-form PGSE, and walker transport. Estimates are planning aids calibrated to a machine, not promises about wall-clock time.

10.4 Diffusion and Flow Through the Facade

Use `PGSE` when free-diffusion attenuation is described adequately by the gradient moment. The facade passes the sample diffusion coefficient and transverse relaxation to the closed-form workflow:

```
from spin_dynamics.experiment import Experiment, PGSE, Sample

study = Experiment(
    sequence=PGSE(gradient_amplitude=0.05,
                  gradient_duration=2e-3,
                  diffusion_time=20e-3),
    sample=Sample(diffusion_coefficient=2.1e-9, t2_seconds=0.25),
)
result = study.run().result
```

Use `PGSEWalkers` when geometry, boundaries, or advection make particle paths explicit. `TransportDomain2D` supplies density and physical axes; `UniformFlow2D` adds a constant velocity in m/s. A fixed seed makes the facade/config round-trip reproducible.

```
import numpy as np
from spin_dynamics.experiment import (
    Experiment, PGSEWalkers, Sample, TransportDomain2D, UniformFlow2D,
)

domain = TransportDomain2D(
    rho=np.ones((3, 2)),
    x_axis=np.array([-1e-3, 0.0, 1e-3]),
    z_axis=np.array([-0.5e-3, 0.5e-3]),
)
study = Experiment(
    sequence=PGSEWalkers(seed=17, walkers_per_cell=32,
                         boundary="periodic", jitter=True),
    sample=Sample(diffusion_coefficient=1.2e-9, t2_seconds=0.15,
                  transport_domain=domain,
                  flow=UniformFlow2D((0.002, 0.0))),
)
print(study.plan().report())
record = study.run()
```

This walker route represents uniform translation plus Brownian displacement. It does not solve a velocity field; use the specialized pipe-flow and thermal-flow models in Chapter 11 when washout, inflow polarization, or advective heat removal is the quantity of interest.

10.5 Automatic Hardware Wiring

For imaging sequences, describe the transmit coil as geometry in SI meters and the facade solves its Biot–Savart B_1 on the phantom grid at plan time, projects it transverse to B_0, normalizes it to a sample-weighted mean of one, and passes it to the workflow in place of the synthetic default. The solve is cached by a geometry hash and reused by `run()`; `plan()` reports the transverse fraction of the coil's B_1 over the sample and warns when most of it is parallel to B_0.

10.6 NQR and ESR

The same interface drives the NQR and ESR engines. For NQR, `plan()` picks the reduced two-level or full density-matrix engine via `select_nqr_model` and reports why; `NQRSLSE.nutation_hz` uses the reduced engine's effective two-level Rabi convention and the adapter converts to the bare $\gamma B_1/2\pi$ the full engine needs. For pulsed ESR, set `Sample.esr_system` and a `Hardware.b0 = UniformB0(field_tesla=...)` so the electron Larmor frequency is fixed, then use an `ESRFID` or `ESRHahnEcho` sequence. `ESRCWSweep` constructs its field axis and does not require fixed B0. `ESRDEER` consumes a serialized `DEERDistribution`. NQR also exposes full-model `NQRFID` and reduced spin-1 `NQRPopulationTransfer`; the FID's nutation rate follows the full engine's bare $\gamma B_1/2\pi$ convention.

ESEEM, HYSCORE, and ENDOR share a serialized `Sample.hyperfine_coupling`. The ESEEM specs select the analytic spin-1/2 expressions automatically when valid and otherwise use the density-matrix engine. `ESRHYSCORE` returns both evolution-time axes, the 2-D time signal, centered frequency axes, the 2-D

spectrum, and predicted cross-peak positions; its plan estimates the nested grid and zero-filled FFT cost. The Davies and Mims ENDOR specs construct an automatic frequency window when explicit bounds are omitted.

10.7 Saving and Reloading

`record.save(path)` writes an NPZ archive holding every array field of the native result plus a JSON document with the full experiment spec, versioned provenance, and scalar result fields. Provenance records canonical SHA-256 identities for the complete experiment and native result, resolved callable, module, and package-tree source hashes, Git revision and dirty state, the Python/NumPy/SciPy and NumPy-build environment, thread settings, and deterministic/seeded/unseeded randomness status. `load_run(path)` returns a `LoadedRun` exposing the raw arrays, the reconstructed native result, and – via `loaded.experiment` – the original spec, so a saved run can be re-run or tweaked. Specs are JSON-serializable directly with `Experiment.to_json()` and `Experiment.from_json(...)`. Runnable versions of the stages above are grouped in these examples:

- `examples/experiment_facade_quickstart.py`
- `examples/experiment_imaging_with_coil.py`
- `examples/experiment_nqr_auto_model.py`

`loaded.specification_matches` and `loaded.result_matches` check saved content against its identities. `loaded.verify_reproduction()` reruns the stored spec with its execution options and returns a structured `ReproductionReport`; `report.require_match()` makes an exact result identity a test assertion. Implementation and numerical-environment changes are reported separately, and unseeded stochastic inputs are called out rather than implying reproducibility. Legacy version-1 archives remain readable but cannot acquire a result identity retroactively.

10.8 Config-Driven Runs (CLI)

For runs you would rather describe in a file than in Python, the facade reads a human-friendly TOML or JSON config in which each spec is a table tagged by its `type`. The `sample`, `hardware`, and `acquisition` sections may omit `type` (their class is implied) and may be dropped entirely to accept the defaults, so a minimal config is just the `[sequence]` table.

```
[sequence]
type = "CPMGTrain"
num_echoes = 8

[sample]
t1_seconds = 2.0
t2_seconds = 2.0

[hardware]
probe = "tuned"

[acquisition]
numpts = 501
maxoffs = 10.0
```

A command-line runner drives the same plan/run/save flow via `python -m spin_dynamics.experiment`:

Command	Effect
`plan CONFIG`	Resolve and validate the config; print the plan. Exits non-zero if the plan has errors.
`run CONFIG [-o NPZ]`	Plan, then run (refusing an erroring plan), and optionally save the result archive.
`show RUN.npz`	Print a saved run's spec and provenance.
`verify RUN.npz`	Rerun a saved experiment and compare canonical result, implementation, and environment identities.
`convert CONFIG OUT`	Rewrite a config between `.toml` and `.json`.

In Python, `save_config` / `load_config` and `experiment_to_config` / `experiment_from_config` expose the same round-trip. This friendly form covers spec fields with scalar or array values, including imaging phantoms and coil geometry; a fully general result archive still uses the NPZ/JSON form of the previous section. Ready-to-run configs ship at:

- `examples/experiment_config_cpmg.toml`
- `examples/experiment_config_pgse.toml`
- `examples/experiment_config_pgse_walkers_flow.toml`

Part IV

Practical Workflows

11 Workflow Guides

This chapter is the practical half of the manual. The earlier chapters define the model layers; the sections below show how those layers are assembled into experimental workflows. The order is roughly from the most common NMR building blocks to more specialized field, motion, exchange, and analysis tools.

In this chapter. Use task-oriented recipes for CPMG, FID, imaging, diffusion, motion, fields, exchange, analysis, hardware response, and optimization.

11.1 CPMG

Application code should normally start with the public workflow runners:

```
from spin_dynamics.workflows import run_tuned_cpmg

result = run_tuned_cpmg(numpts=101, maxoffs=10)
print(result.del_w.shape)
print(result.echo.shape)
print(result.snr)
```

The asymptotic CPMG runners return `CPMGResult` objects with the offset grid, asymptotic magnetization, received spectrum, time-domain echo, acquisition time vector, optional SNR, probe label, and the default phase-cycle metadata.

Finite and diagnostic CPMG examples use the same public timing vocabulary: `num_echoes` and `echo_spacing_seconds`. The ideal convention is a 90_y excitation that prepares $+I_x$, followed by 180_x refocusing pulses and the default two-step CPMG/PAP excitation phase cycle. The microscopic bulk-water example keeps that convention but replaces fitted scalar T_2 with a shared Redfield/dipolar Liouville propagator for one observed proton relaxed by an isotropically tumbling partner:

```
python examples/plot_redfield_water_cpmg.py \
    --echo-spacing-seconds 0.02 \
    --correlation-time-seconds 2.5e-12 \
    --output water_redfield_cpmg.png
```

Run `python examples/plot_probe_cpmg.py -output probe_cpmg.png` to reproduce Figure 11.1 with different probe choices.

The older unit-suffixed plotting conveniences, `-echo-spacing-ms` and `-tau-c-ps`, remain aliases, but new examples should prefer the seconds form used by the workflow API.

11.1.1 Uhrig Dynamical Decoupling

The sequence utility layer also provides ideal CPMG and Uhrig dynamical decoupling timing helpers. For n refocusing pulses in a total evolution window T, UDD places the jth pulse at

$$t_j = T \sin^2\left(\frac{j\pi}{2n+2}\right), \qquad j = 1, \ldots, n.$$

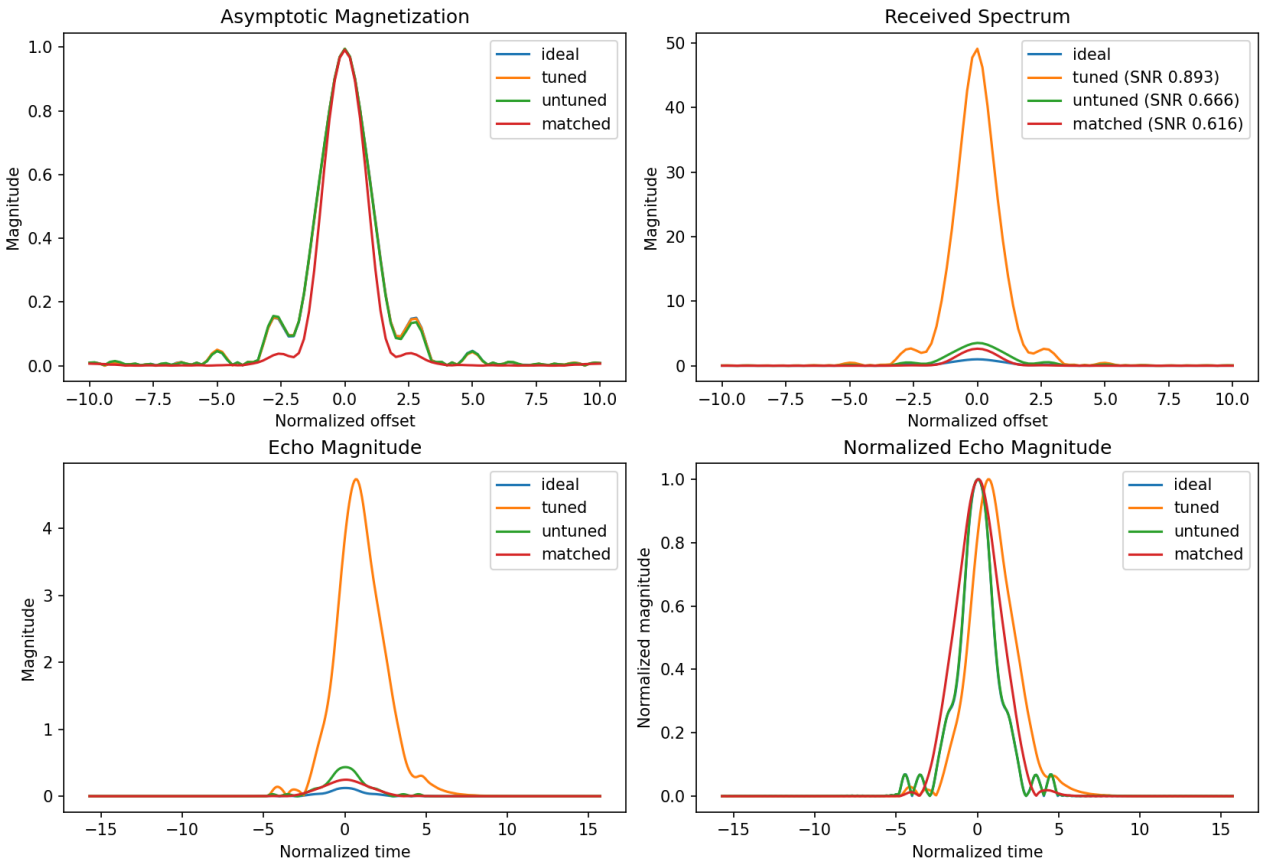


Figure 11.1: Ideal, tuned, untuned, and impedance-matched CPMG workflows. The normalized magnetization profiles remain similar, while the received spectrum and absolute echo amplitude expose the probe transfer function and noise penalty. Normalizing the echo (lower right) separates bandwidth distortion from overall sensitivity.

The pulses cluster near the beginning and end of the window. This does not usually improve simple unrestricted diffusion in a static gradient, but it can strongly suppress low-frequency correlated detuning noise.

```
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    dephasing_filter_function,
    udd_pulse_times,
)

duration = 1.0
omega = 0.1 / duration
cpmg = cpmg_pulse_times(8, duration)
udd = udd_pulse_times(8, duration)

print(dephasing_filter_function(omega, cpmg, duration))
print(dephasing_filter_function(omega, udd, duration))
```

11.2 Finite Echo Trains

```
from spin_dynamics.workflows import run_matched_cpmg_train

train = run_matched_cpmg_train(
    numpts=51,
    num_echoes=8,
    auto_refine_grid=True,
    num_workers=None,
)
```

Finite train results contain one acquired spectrum and one echo per echo index. They also include trapezoidal echo integrals and physical echo-center times. The default CPMG/PAP phase-cycle table is available as `train.phase_cycle`. For long trains, use the rephasing checks or enable `auto_refine_grid=True` so the offset grid is refined before RF matrices are built.

11.2.1 CPMG-IR Trains

Finite inversion-recovery trains add an inversion delay axis before the CPMG readout:

```
from spin_dynamics.workflows import run_matched_cpmg_ir_train

ir = run_matched_cpmg_ir_train(
    numpts=31,
    num_echoes=4,
    tauvect=[0.5e-3, 2e-3, 6e-3, 12e-3],
    auto_refine_grid=True,
)
```

The returned arrays are indexed by inversion delay first and echo number second. This makes the workflow useful for compact T_1-encoded echo-train tests without manually assembling the inversion pulse, delay, excitation, and refocusing blocks.

11.2.2 Probe Parameter Sweeps

Probe-aware CPMG workflows can also be swept over resonator Q or mistuning:

```
from spin_dynamics.workflows import run_tuned_q_sweep

sweep = run_tuned_q_sweep(q_values=[20, 50, 100], numpts=51)
```

Use the finite-train sweep variants when echo-to-echo evolution matters. The finite sweep results keep one `CPMGTrainResult` per parameter value and also stack echo integrals for quick heat-map style summaries.

11.3 FID

```
from spin_dynamics.parameters import set_params_ideal_fid
from spin_dynamics.workflows.fid import sim_fid_ideal

sp, pp = set_params_ideal_fid(numpts=101)
macq, fid, tvect = sim_fid_ideal(sp, pp)
```

11.4 Imaging

```
import numpy as np
from spin_dynamics.workflows import (
    make_imaging_field_maps,
    run_tuned_phase_encoded_cpmg_imaging,
    form_imaging_image,
)

rho = np.eye(8)
fields = make_imaging_field_maps(
    rho,
    b0_map=np.zeros_like(rho),
    b1_tx_map=np.ones_like(rho),
    b1_rx_map=np.ones_like(rho),
)
result = run_tuned_phase_encoded_cpmg_imaging(
    fields,
    num_echoes=4,
    ny=9,
    receive_mode="weighted",
)
image = form_imaging_image(result, mode="echo_sum")
```

Imaging workflows return raw k-space, reconstructed per-echo images, field maps, relaxation maps, gradient metadata, and sequence timing. Image formation is intentionally separated from acquisition so selected-echo, echo-summed, fitted-ρ, and fitted-T_2 displays can be compared. `make_imaging_field_maps` accepts spatial B0, transmit-B1, and receive-B1 maps, and the ideal imaging path can include an inversion-recovery preparation through `run_t1_encoded_phase_encoded_cpmg_imaging`. When `noise` is supplied, clean k-space and image fields remain in the result while noisy counterparts are stored in `kspace_noisy`, `image_noisy`, and `magnitude_noisy`.

11.4.1 Frequency-Encoded Imaging: Spin-Warp and RARE

The phase-encoded workflows above fill k-space one point per phase-encode step. `run_spin_warp_imaging` and `run_rare_imaging` add a readout (frequency-encode) gradient during acquisition, so each spin echo samples a whole k-space line. They are built on the Lagrangian motion engine—one static isochromat per voxel—which supports the two-axis gradient waveforms that the scalar-gradient arbitrary-pulse kernels cannot express; they are ideal-probe and reuse `reconstruct_image_from_kspace`.

```
import numpy as np
from spin_dynamics.workflows import run_spin_warp_imaging, run_rare_imaging

rho = np.zeros((32, 32)); rho[8:24, 10:18] = 1.0
t2 = np.full((32, 32), 60e-3)

# Spin-warp: one spin echo per phase-encode line (pz excitations, no blurring).
ref = run_spin_warp_imaging(rho, fov=(0.02, 0.02), t2_map=t2)

# RARE / fast spin echo: each echo reads a different line, so a train of length 8
# needs only ceil(pz / 8) excitations, at the cost of T2 blurring.
fse = run_rare_imaging(rho, fov=(0.02, 0.02), t2_map=t2, echo_train_length=8)
```

Readout is along x and the phase encode along z. Each echo is gradient-balanced (an x pre-dephase and rewind return k-space to the origin before each 180-degree pulse), so the train stays a clean Meiboom-Gill CPMG. The T2 decay across the echo train weights the phase-encode lines—`line_echo_time` records the echo time of each k_z line—which broadens the point-spread function (the RARE blurring, strongest for short T_2). `echo_train_length=1` recovers the spin-warp reference. The example `plot_rare_imaging.py` contrasts the two on a two-T_2 phantom.

Both workflows accept the same inputs as the phase-encoded path so the effects of inhomogeneity can be evaluated: pass `rho` with per-voxel `b0_map` (absolute angular off-resonance, rad/s), `b1_tx_map`, `b1_rx_map`, `t1_map`, and `t2_map`, or pass an `ImagingFieldMaps` container directly (with the map keywords omitted). B0 inhomogeneity distorts geometry along the readout axis and B1 maps shade the image. An unresolved sub-voxel B0 spread is modelled with `num_offsets` > 1 isochromats per voxel spread over $\pm$`offset_spread` (rad/s), the counterpart of the `ny`/`maxoffs` samples of the phase-encoded path; a spin echo refocuses the static spread at each echo, so it blurs the readout axis (the T_2^* point-spread function) without decaying the train. The example `plot_imaging_inhomogeneity.py` shows B0 distortion, sub-voxel T_2^* blur, and B1 shading.

A practical issue in inhomogeneous-field imaging is that the excited slice is neither flat nor uniform in real space: an RF pulse excites the spins whose frequency falls in its band, so the slice follows the curved iso-B0 contours and is shaded by B1. `imaging_slice_sensitivity` maps this sensitive slice from the same B0/B1 maps by computing, for every voxel at its own off-resonance (`b0_map - center_frequency`, rad/s) and transmit-B1, the excited transverse magnetization of a rectangular RF pulse (so the slice profile follows the pulse bandwidth $\sim 1/$`excitation_duration`), weighted by the receive B1; `refocusing=True` also applies the 180-degree refocusing efficiency for the spin-echo sensitive volume. The example `plot_sensitive_slice.py` visualizes the curved, non-uniform slice in a single-sided-like field.

11.4.2 Balanced SSFP: Steady-State Imaging and Off-Resonance Banding

Balanced SSFP (bSSFP; also TrueFISP, FIESTA, balanced FFE) is the workhorse steady-state gradient echo. Unlike the spin-echo imagers above, it never lets the magnetization recover to equilibrium between excitations: a train of small-flip pulses at a short, fixed T_R drives a *steady state*, and every gradient moment—readout and phase encode—is rewound to zero each T_R (the "balanced" condition). `run_bssfp_imaging` builds exactly this on the same moving-isochromat engine and the same `(B0, B1)` field maps as `run_spin_warp_imaging`, so the two are directly comparable: it is a continuous train of balanced T_Rs with a phase-cycled RF phase, one k-space line acquired per T_R, reconstructed with the same `reconstruct_image_from_kspace`.

```
import numpy as np
from spin_dynamics.workflows import run_bssfp_imaging, bssfp_optimal_flip_deg

rho = np.zeros((32, 32)); rho[8:24, 10:22] = 1.0
t1 = np.full((32, 32), 0.3); t2 = np.full((32, 32), 0.1)

img = run_bssfp_imaging(
    rho, t1_map=t1, t2_map=t2,
    b0_map=b0_rad_per_s,             # off-resonance -> banding
    b1_tx_map=b1_relative,           # flip shading
    flip_angle_deg=bssfp_optimal_flip_deg(0.3, 0.1),
    num_dummy_tr=200,                # dummy TRs to reach the steady state
    phase_increment_deg=180.0,       # phase cycling: passband centred on resonance
)                                    # img.magnitude has shape (px, pz, 1)
```

The appeal is SNR. On resonance the steady-state signal is high—at the optimal flip $\cos\alpha^\star = (T_1/T_2 -$

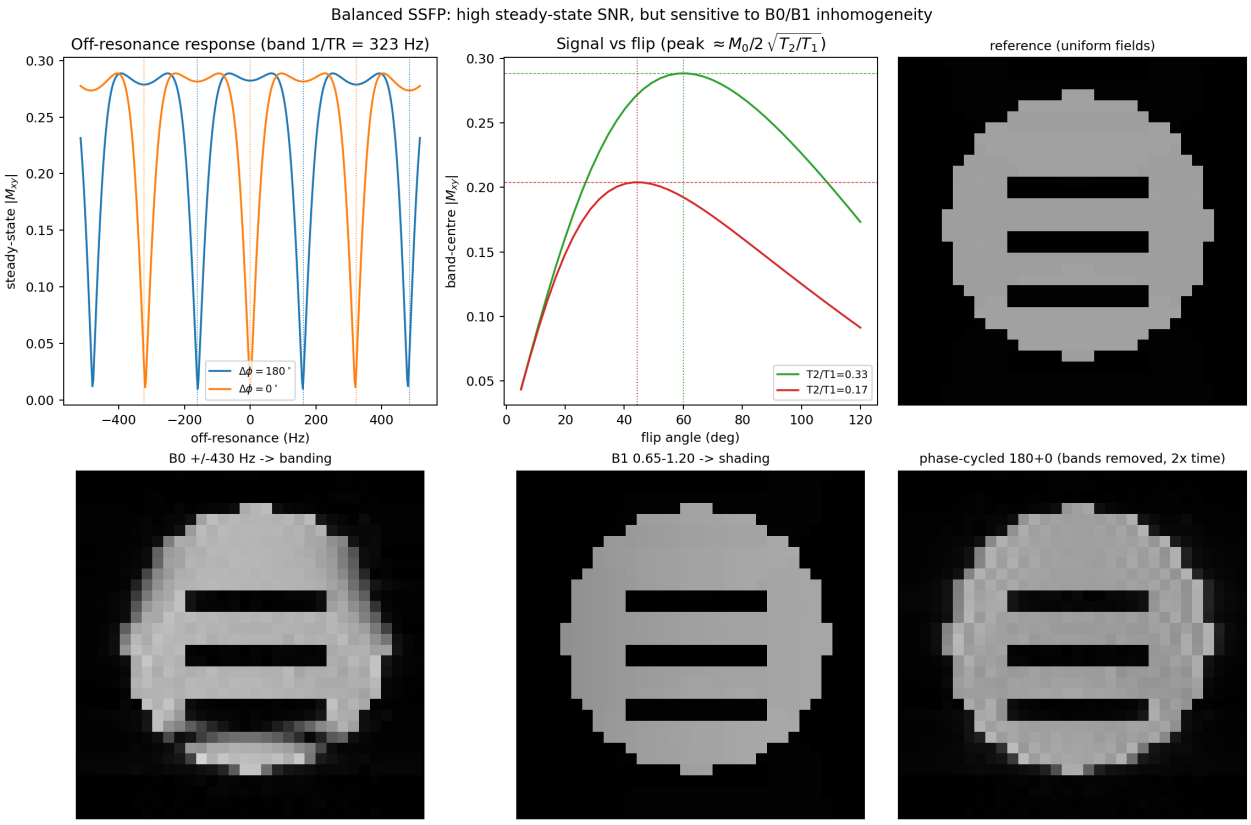


Figure 11.2: Balanced SSFP connects its periodic off-resonance response and flip-angle optimum to image artifacts. B0 variation generates dark bands, B1 variation produces shading, and a complementary phase-cycled pair removes the bands at twice the acquisition time.

$1)/(T_1/T_2+1)$ it reaches $\frac{1}{2}M_0\sqrt{T_2/T_1}$ (`bssfp_optimal_flip_deg` returns $\alpha^\star$)—which is why bSSFP is a favourite at low field, where SNR is scarce. The catch is the flip side of "balanced": each T_R rewinds the *gradient* phase but not the *B0* phase, so the steady state depends on the per-T_R off-resonance precession. Its off-resonance response (`bssfp_steady_state_signal`, computed with the same engine primitives but no gradients) is flat over a passband and then drops to periodic nulls—the bSSFP "dark bands"—spaced $1/T_R$ in frequency. The RF phase increment sets where the passband sits: $180°$ centres it on resonance with nulls at $\pm 1/(2T_R)$; $0°$ shifts it half a band, nulls at $0, \pm 1/T_R$.

In an inhomogeneous B0 those bands cut across the object as signal voids, and B1 (flip-angle) inhomogeneity shades the image. This is the steady-state SNR-versus-robustness trade-off, the imaging cousin of the steady-state SORC result of Section 8.10.1—a steady-state sequence buys sensitivity but pays in off-resonance and field-inhomogeneity sensitivity. The example `plot_bssfp_field_inhomogeneity.py` makes this concrete for a phantom in mildly inhomogeneous fields: it plots the banding profile and the T2/T1 flip response, then images the phantom four ways—uniform (reference), a mild B0 residual ($\sim$ a band across the FOV) that carves a dark banding void, a transmit-B1 gradient that shades the image, and a *phase-cycled* pair (increments $180°$ and $0°$, whose passbands are complementary) combined by maximum, the standard fix that fills the bands back in at twice the scan time.

Run `python examples/plot_bssfp_field_inhomogeneity.py -output bssfp_fields.png` to reproduce Figure 11.2.

Two practical points fall out of running bSSFP honestly. First, the steady state must be established:

`num_dummy_tr` balanced dummy TRs (with an optional $\alpha/2$ "catalyzation" that lands the magnetization near the steady-state cone and suppresses the oscillatory transient) are played before acquisition, and too few of them leaves a transient that modulates the phase-encode lines and blurs or shifts the image—so a few $\times T_1/T_R$ dummies are needed. Second, because the RF phase is cycled, the receiver phase must follow it: each acquired line is demodulated by its transmit phase, without which the alternating pulse phase imprints a $(-1)^{\text{line}}$ modulation on k-space that shifts the image by half the field of view along the phase-encode axis.

11.4.3 Slice-Selective Excitation and 3D Multi-Slice Imaging

`imaging_slice_sensitivity` above is a passive analysis: it uses a hard (rectangular) pulse and relies on the existing B0 inhomogeneity to define the band, with no gradient applied during the RF. Real slice selection instead plays a shaped RF while a gradient is on, so a chosen plane is excited even in a uniform background. `make_slice_selective_excitation` builds this pulse as a windowed-sinc RF train carrying the slice gradient, followed by a refocusing (rephasing) gradient lobe; `simulate_slice_profile` excites a line of spins and returns the through-slice profile, which is a localized band (sharp, unlike the hard-pulse approximation) whose thickness scales as the excitation bandwidth over the gradient.

```
import numpy as np
from spin_dynamics.workflows import simulate_slice_profile, run_multislice_imaging

# Through-slice profile of a 1 ms windowed-sinc 90-degree pulse.
profile = simulate_slice_profile(duration=1e-3, slice_gradient=1.5e7)

# True-3D multi-slice imaging of a 3D volume in a spatially varying (B0, B1) field.
rho = np.zeros((16, 5, 16)); rho[6, 2, 9] = 1.0
volume = run_multislice_imaging(
    rho,
    slice_gradient=1.5e7,
    fov=(0.02, 0.02, 0.02),
    b0_map=b0_volume,        # rad/s, shape of rho
    b1_tx_map=b1_volume,     # relative
    b1_rx_map=b1_volume,
)            # volume.magnitude has shape (nx, n_slices, nz)
```

`run_multislice_imaging` is the true-3D path. A single 3D walker ensemble lives in a 3D `MotionFieldMaps` carrying the actual `(B0, B1)` field, and every slice is excited (its carrier offset to place the slice) and read out through the moving-isochromat engine. Because the slice is selected by *total* off-resonance—slice gradient plus local B0—a nonuniform B0 curves and displaces the excited slice and warps the read-out, exactly as in a real magnet; the example `plot_multislice_halbach_imaging.py` shows this for a structured phantom in a mild Halbach saddle, displaying acquired slices and the 3D reconstruction. `run_multislice_imaging_separable` is a fast approximation that reduces the slice pulse to a 1D through-plane weight and forms each slice with the validated 2D spin-warp workflow; it ignores in-plane field variation (the slice stays flat), trading that fidelity for speed on large volumes. Genuinely Fourier-encoded 3D (slab-select plus a second phase encode reconstructed with `ifftn`) is left for later.

The slice pulse, the moving-isochromat engine, and the phase-encoded imaging kernels all read the same per-voxel physics—off-resonance, transmit/receive B1, relaxation, and density—through the dimension-agnostic `spin_dynamics.fields` layer (`SpatialDomain` and `SpatialFieldMaps`), so the 1D, 2D, and 3D paths share one field representation and one gradient-coupling rule.

11.4.4 Complete Portable Halbach MRI Capstone

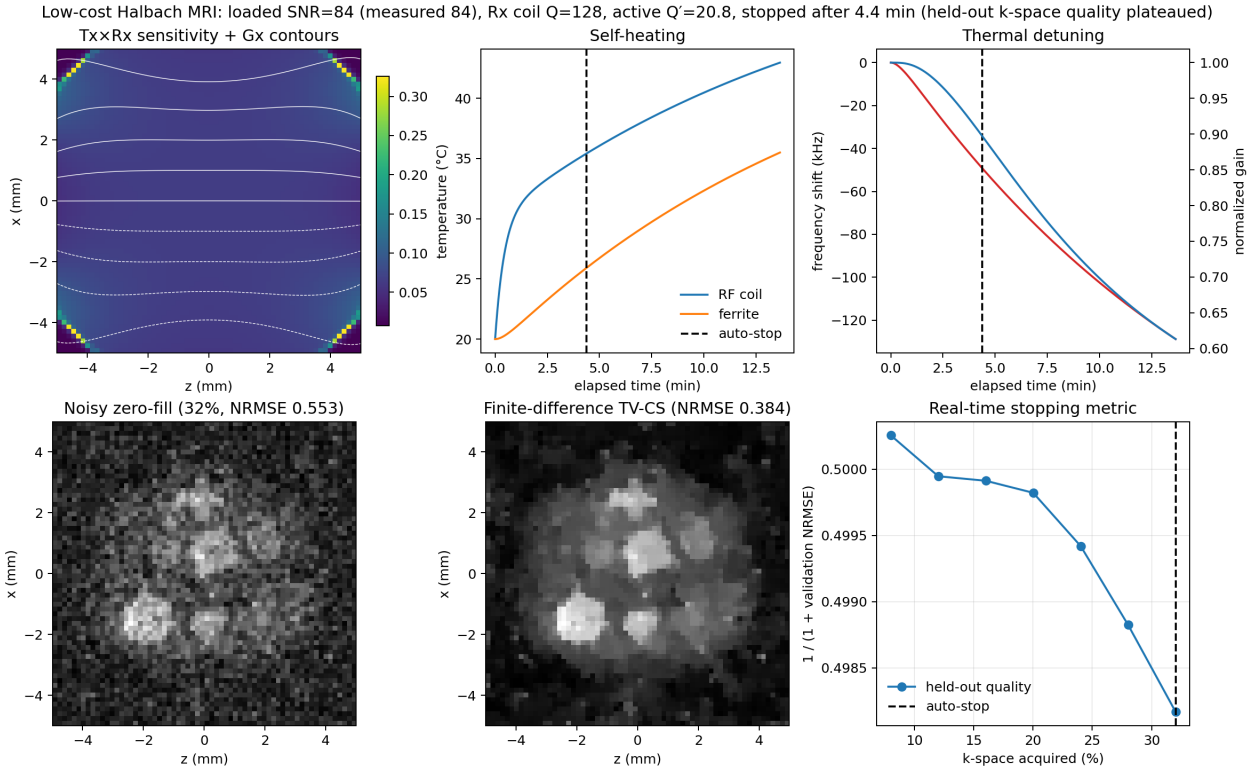
The preceding imaging and field components are deliberately composable, but it is useful to see them closed into one complete design loop. The capstone example

```
python examples\plot_portable_halbach_adaptive_mri.py --matrix-size 64 \
  --output results\portable_halbach_adaptive_mri.png \
  --design-output results\portable_halbach_design_dashboard.png \
  --data-output results\portable_halbach_capstone.npz
```

starts from the eight finite C8 ferrite blocks with deterministic block-to-block remanence mismatch and solves three-dimensional B0, transmit-B1, receive-B1, and nonlinear gradient fields. It then connects the actual transmitter and receiver circuits, PEEC coil impedance, ferrite RF loss, lead/interconnect loss, finite probe bandwidth, circularly polarized NMR coupling, gradient-driver requirements, sample noise, and a thermal network. The same run generates noisy k-space, performs variable-density compressed sensing, and stops acquisition when held-out k-space quality ceases to improve.

This is a complete workflow rather than a collection of isolated plots: the designer summary reports RF and gradient properties, peak RF/gradient drive, ADC gain, system mass, active volume, effective slice thickness, signal bandwidth, and SNR uncertainty. The default case predicts a 91.3 kHz signal span against the measured 100 kHz reference and a single-echo SNR of 84.1 (measured 84; 70.8–95.5 over the measured receiver-noise range). It stops at 32% k-space and retains a visible compressed-sensing advantage over matched-mask zero-fill. The reference full-fidelity study completed in only 15–20 minutes; the optimized 64-by-64 teaching default is normally much faster.

The detailed assumptions, book-calibrated quantities, result tables, and both generated dashboards are collected in `docs/portable_halbach_adaptive_mri.md`. This is the recommended starting point when evaluating a hardware change because every downstream metric is regenerated from the same typed workflow result.



11.5 Field Solvers

`spin_dynamics.fields` computes device fields from first principles rather than assuming a synthetic map. Two families live here: closed-form *analytic* sources — permanent-magnet bars, finite Halbach arrays, and Biot-Savart coils — and structured-grid *numerical* solvers for the effects analytic superposition cannot represent: quasistatic induced-E and eddy currents, and nonlinear magnetostatics with flux-shaping ferrites, saturable iron, and permanent magnets in two and three dimensions. All are mesh-free (uniform grids) and depend only on NumPy; SciPy and (for the 3D solver) pyamg accelerate the larger linear solves when available. The single-sided NMR depth-profiling workflow that consumes these fields is described alongside the analytic sources it is built on.

11.5.1 Analytic magnet and coil sources

The imaging and motion workflows consume `(B0, B1)` maps; `spin_dynamics.fields.magnetostatics` *generates* them from first-principles magnet and coil models (pure NumPy, mesh-free), so a real device field can drive the spin dynamics instead of a synthetic field. The B0 of a uniformly magnetized rectangular bar is the closed-form field of the magnetic charge-sheets on its faces—exact for the nearly linear rare-earth magnets used in single-sided NMR—and a soft-iron return yoke is added by the method of images across a $\mu \to \infty$ plane, which enforces $B_{\parallel} = 0$ at the iron surface. The coil B1 is the Biot-Savart field of straight current segments, and its transverse component is the part perpendicular to the local B0. Finite Halbach dipoles are represented by the lowest-order four-rod approximation: four diametrically magnetized cylinders or square rods arranged around a bore. Their finite length is included by a point-dipole cubature over the magnet volume, which is sufficient for bore-field and fringe-field studies but not a substitute for precision magnet design inside the magnet material.

```
import numpy as np
from spin_dynamics.fields.magnetostatics import (
    nmr_mouse_magnets, circular_loop, sample_magnet_field,
)

bars, yoke = nmr_mouse_magnets(gap=0.012, remanence=1.30)
coil = circular_loop((0.0, 0.030, 0.0), 0.008, axis="y")
x = np.linspace(-0.02, 0.02, 121); y = np.linspace(0.021, 0.045, 121)
fm = sample_magnet_field(x, y, bars, yoke_y=yoke, coil_segments=coil)
```

For the canonical NMR-MOUSE (two antiparallel bars on a yoke) this reproduces the device's hallmarks: $|B_0| \sim 0.25$–0.54 T over the gap, a proton Larmor frequency of ~ 5–23 MHz, and a strong static gradient $G \sim 7$–28 T/m. The example `plot_nmr_mouse_fields.py` shows the field, the gradient, the coil B1, and the depth-resolved sensitive slice.

Run `python examples/plot_nmr_mouse_fields.py -output nmr_mouse_fields.png` to reproduce Figure 11.3.

```
from spin_dynamics.fields.magnetostatics import sample_halbach_dipole_field

fm3d = sample_halbach_dipole_field(
    x_axis, y_axis, z_axis,
    center_radius=30e-3,
    rod_shape="square",
    rod_width=16e-3,
    length=80e-3,
    remanence=1.30,
```

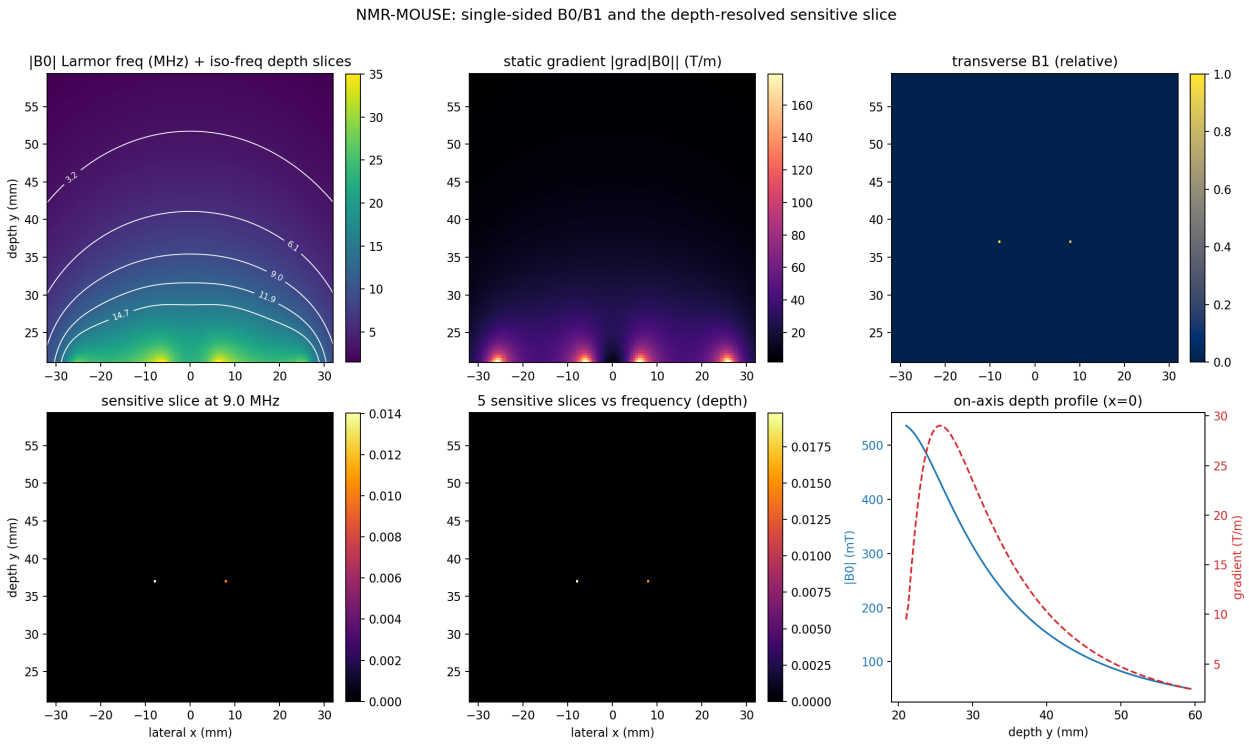


Figure 11.3: A single-sided NMR-MOUSE field model carried from permanent-magnet B0 and its static gradient through the surface-coil B1 map to frequency-selected, depth-resolved sensitive slices. The on-axis profile makes the simultaneous loss of field and gradient with depth explicit.

```
)
```

The 3D Halbach result exposes `b0_vector`, `b0_magnitude`, `b0_gradient`, and `larmor_hz`. The example `plot_halbach_dipole_field.py` plots the mid-plane field, vector direction, uniformity, finite-length axial falloff, and gradient map. These fields also feed the polarization-enhanced NQR transport model in Sec. 8.8.

Electropermanent AlNiCo sources

Electropermanent magnets retain a programmed state without continuous coil power, but AlNiCo material data alone do not determine that state. A finite rod's operating remanence depends on aspect ratio, self-demagnetizing field, partial programming, neighbors, temperature, and pulse history. The EPM API therefore separates nominal `AlNiCoMaterial`, signed retained `RemanenceState`, finite `ElectropermanentRod` geometry, bundle-level `ProgrammingCoil` metadata, and an `EvidenceRecord` tagged as measured, simulated, specified, or inferred.

The first implementation phase is magnetostatic. Arbitrarily oriented rods are integrated by volume cubature of point dipoles; the exact uniformly magnetized finite-cylinder axial expression supplies a convergence reference. Explicit close-packed bundles preserve every rod, while `equivalent_cylinder()` provides a fast area-equivalent reduction that preserves AlNiCo volume and dipole moment but not the detailed near field at the face.

```
from spin_dynamics.fields import (
    RemanenceState, sample_electropermanent_field,
    weinberg_37_rod_bundle,
```

```
)

state = RemanenceState(
    0.42, branch="partial", calibration_id="my-calibration-v1"
)
bundle = weinberg_37_rod_bundle(state=state)
maps = sample_electropermanent_field(
    (x_axis, z_axis), (bundle,), field_direction=(0, 0, 1)
)
motion_maps = maps.to_motion_field_maps()
imaging_maps = maps.to_spatial_field_maps(
    rho=phantom, t1_map=t1, t2_map=t2
)
```

The documented 37-rod preset contains 3.175-mm-diameter, 101.6-mm-long AlNiCo-5 rods, approximately 60 turns of 16-AWG wire, and a nominal 50-μH bundle winding. It defaults to zero retained remanence because the archive does not contain one universally applicable calibrated state. The companion `variable_field_nmr_rod()` preset describes the published 25.4-mm-diameter, 152.4-mm-long rod with 50 turns of 14-AWG wire and 25 μH. Its default effective remanence is explicitly inferred from the reported approximately 150-mT field 1 mm from the face rather than silently equating the operating point with nominal 1.27-T material remanence.

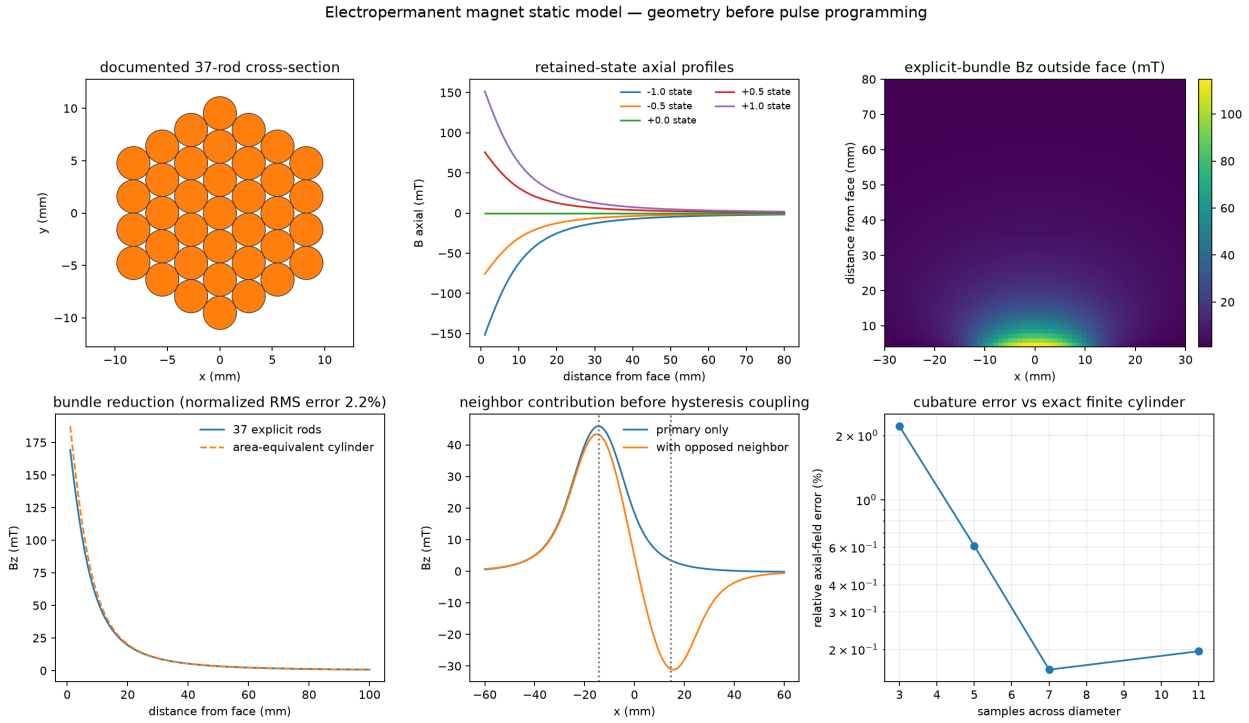


Figure 11.4: Static electropermanent-magnet benchmark. The documented 37-rod geometry is compared with an area-equivalent cylinder; partial retained states, an external B_z map, a neighboring bundle's field contribution, and cubature error against the exact cylinder result remain explicit.

Figure 11.4 is generated by `plot_electropermanent_magnet.py`. The example prints material and geometry provenance, retained fraction of nominal B_r, magnetic moment, surface-field benchmark, equivalent-cylinder error, and neighbor perturbation.

Programming pulse and thermal response. `PulsePowerDriver` integrates the capacitor bank, H-bridge polarity, winding resistance and inductance, passive recovery path, current/voltage limits, and independent lumped coil and driver temperatures. During the drive interval,

$$C\frac{dV_C}{dt} = -pI, \tag{11.1}$$

$$L\frac{dI}{dt} = p(V_C - V_{\mathrm{bridge}}) - I\left[R_{\mathrm{coil}}(T) + R_{\mathrm{series}}\right], \tag{11.2}$$

where $p \in \{-1, +1\}$ is the bridge polarity. At turn-off the capacitor is disconnected and current decays through $R_{\mathrm{coil}} + R_{\mathrm{recovery}}$. The NumPy RK4 path splits the integration at the switching boundary and reports realized current, capacitor and bridge voltage, nominal internal H, electrical losses, temperatures, field impulse, and an energy-balance residual.

```
from spin_dynamics.fields import (
    ProgrammingPulse, RemanenceState,
    archived_igbt_pulse_cases,
    published_demagnetization_calibration,
)

case = archived_igbt_pulse_cases()[0]  # 220 V / 50 us
pulse = ProgrammingPulse(
    220.0, 50e-6, polarity=-1, command_fraction=0.17
)
waveform = case.driver.simulate(times, pulse)
initial = RemanenceState(0.33, branch="positive_saturation")
result = published_demagnetization_calibration().apply(
    initial, waveform
)
```

The three archive comparisons remain configuration-specific: voltage, gate duration, and peak current are measured; the effective inductance and common capacitance/resistance assumptions are inferred because the records do not establish identical windings. The published remanence calibration is similarly conservative. It uses only the positive and negative endpoints and the reported zero crossing near 17 percent duty, carries graphical-extraction uncertainty, and rejects the wrong initial branch or pulse polarity. It is not a general minor-loop model.

Figure 11.5 is generated by `plot_electropermanent_programming.py`.

Return-point memory and coupled programming. `PlayHysteresisModel` represents rate-independent memory as a weighted collection of scalar play operators. For threshold r_i, the hidden coordinate and retained remanence advance according to

$$y_i \leftarrow \max(H - r_i, \min(H + r_i, y_{i,\mathrm{previous}})), \tag{11.3}$$

$$B_r(T) = B_{r,\mathrm{sat}}(T) \sum_i w_i \,\mathrm{clip}(y_i/r_i, -1, 1). \tag{11.4}$$

The construction closes nested reversal loops exactly and wipes an inner loop when the applied field exceeds its enclosing reversal. `ReturnPointState` holds the hidden coordinates and reversal stack, while its public `RemanenceState` remains directly consumable by the static field and imaging adapters. The bundled illustrative AlNiCo thresholds and weights are inferred demonstration parameters, not a measured minor-loop calibration.

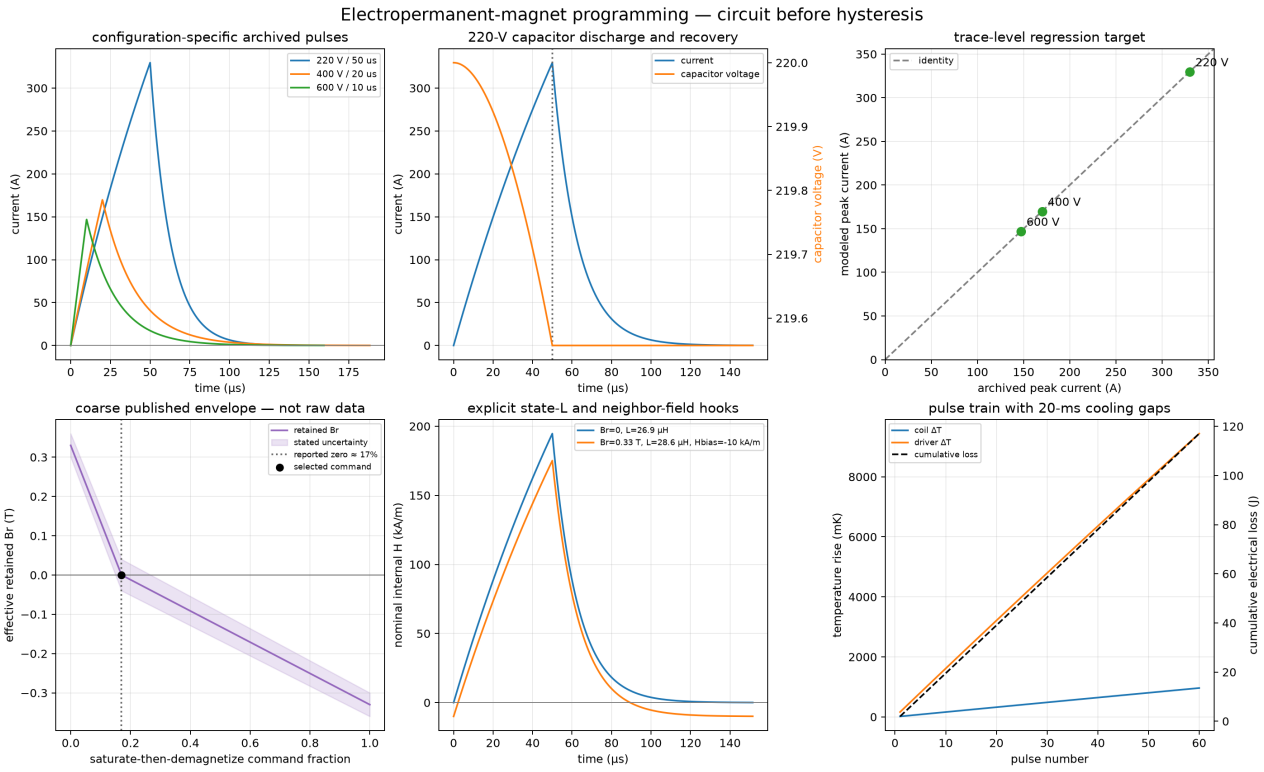


Figure 11.5: Electropermanent programming-pulse diagnostics. Configuration-specific archived waveforms and peak currents are separated from inferred circuit parameters; the protocol-scoped retained-state envelope, state-dependent inductance and neighbor-field hooks, and cumulative pulse heating remain explicit.

`neighbor_coupling_matrix()` evaluates every finite EPM at every other source center per tesla of retained remanence. Optional diagonal $-N/\mu_0$ terms add self-demagnetizing fields. `CoupledEPMProgrammer` then iterates retained state, static self/neighbor bias, state-dependent inductance, and the realized pulse waveform to a remanence tolerance.

```
from spin_dynamics.fields import (
    CoupledEPMProgrammer,
    illustrative_alnico_return_point_model,
    neighbor_coupling_matrix,
)

sources = (target_rod, neighbor_rod)
coupling = neighbor_coupling_matrix(
    sources, self_demagnetizing_factors=(0.02, 0.02)
)
model = illustrative_alnico_return_point_model()
programmer = CoupledEPMProgrammer(
    sources, (driver, driver), (model, model), coupling
)
result = programmer.program(
    0, times, target_pulse, states=initial_return_point_states
)
```

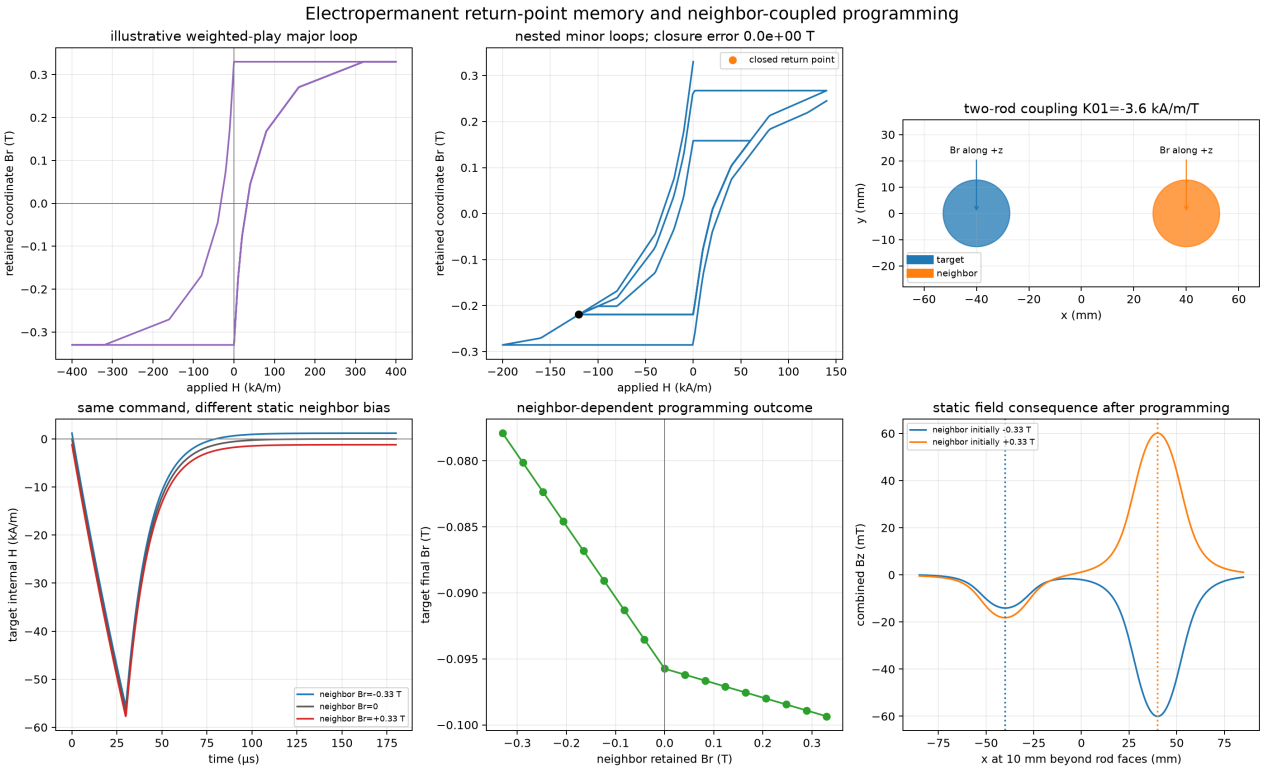


Figure 11.6: Illustrative AlNiCo return-point memory and two-element programming. The major and nested minor loops are separated from geometry-derived side-by-side coupling, neighbor-biased pulse responses, converged retained state, and the resulting external field.

Figure 11.6 is generated by `plot_electropermanent_return_point.py`.

Transient winding cross-talk. `MutualProgrammingCircuit` advances all winding currents while one bridge is commanded. Inactive channels are closed through their recovery resistors, so the coupled circuit and programming fields obey

$$\mathbf{L}(\mathbf{B}_r)\frac{d\mathbf{I}}{dt} = \mathbf{V}_{\text{bridge}} - \mathbf{R}(T,s)\mathbf{I}, \tag{11.5}$$

$$C_q\frac{dV_C}{dt} = -pI_q, \tag{11.6}$$

$$H_i(t) = \sum_j G_{ij}I_j(t) + \sum_j K_{ij}B_{r,j}(t). \tag{11.7}$$

Here G maps winding current to programming field and K maps retained remanence to static internal field. Positive mutual inductance produces an opposing induced current in a closed inactive winding. The result reports every current, applied and induced voltage, field, loss, and temperature history.

`TransientCoupledEPMProgrammer` advances every play-operator state at each time sample and closes state-dependent self inductance with an outer fixed-point iteration. The inactive-channel topology, mutual-inductance matrix, and winding field-leakage matrix remain explicit inputs.

```
circuit = MutualProgrammingCircuit.from_coupling_coefficients(
    (driver_0, driver_1),
    [[0.0, 0.08], [0.08, 0.0]],
    field_coupling_fractions=[
        [1.0, 0.20], [0.20, 1.0]
    ],
)
programmer = TransientCoupledEPMProgrammer(
    sources, (model_0, model_1), retained_coupling, circuit
)
result = programmer.program(0, times, negative_pulse)
print(result.disturbed_indices)
```

Figure 11.7 is generated by `plot_electropermanent_transient_crosstalk.py`. Its coupling values are illustrative until measured matrices are available.

Hybrid array and operating-state synthesis

The array layer groups fixed NdFeB sources and independently programmable AlNiCo rods into hybrid sub-unit objects. An `ElectropermanentArray` then flattens the retained-remanence controls while preserving panel and grid metadata. The reference geometry contains two opposing 3×3 panels, 18 hybrid sub-units, and four AlNiCo controls per sub-unit, for 72 retained-state variables. The hierarchy, 150-mm panel gap, and 40-mm control-region intent are specified by the project evidence; rod sizes, pitch, and the exact within-sub-unit layout remain clearly tagged illustrative until CAD or measured field data are available.

For repeated design solves, `build_field_basis` evaluates magnetostatics once and returns a cached field-basis object. At the sampled control points the vector field is reconstructed as

$$\mathbf{B}(\mathbf{r}) = \mathbf{B}_{\text{fixed}}(\mathbf{r}) + \sum_j B_{r,j}\,\mathbf{b}_j(\mathbf{r}),$$

where $\mathbf{b}_j$ is the field per tesla of retained remanence in element j. Uniform imaging, field-off, and affine transport states share that cached basis and solve a bounded, weighted, regularized least-squares problem. Symmetric per-element remanence limits are enforced directly; SciPy's bounded solver is used when available and a NumPy projected solver provides a dependency-light fallback.

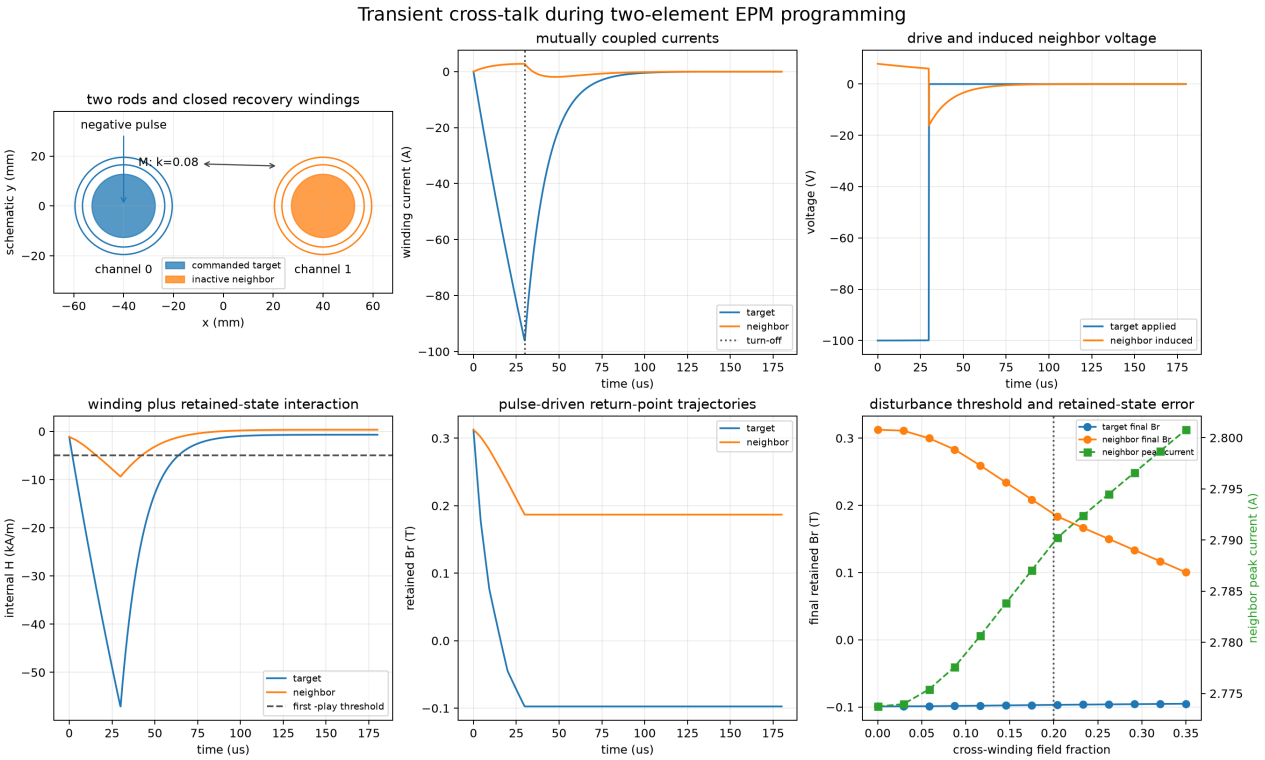


Figure 11.7: Transient programming cross-talk for two EPM elements. A target command induces neighbor voltage and current; winding leakage and retained fields then move both return-point states. The final sweep exposes the neighbor-disturbance threshold.

```
import numpy as np
from spin_dynamics.fields import (
    illustrative_hybrid_epm_array,
    synthesize_field_off_state,
    synthesize_transport_state,
    synthesize_uniform_imaging_state,
)

array = illustrative_hybrid_epm_array(panel_gap_m=0.150)
axis = np.linspace(-0.020, 0.020, 11)
points = np.column_stack((axis, np.zeros_like(axis), np.zeros_like(axis)))
basis = array.build_field_basis(points)

imaging = synthesize_uniform_imaging_state(basis, 2e-3)
field_off = synthesize_field_off_state(basis)
transport = synthesize_transport_state(
    basis, bias_field_t=2e-3,
    gradient_t_per_m=(50e-3, 0.0, 0.0),
)
programmed_array = array.with_remanence(transport.remanence_t)
```

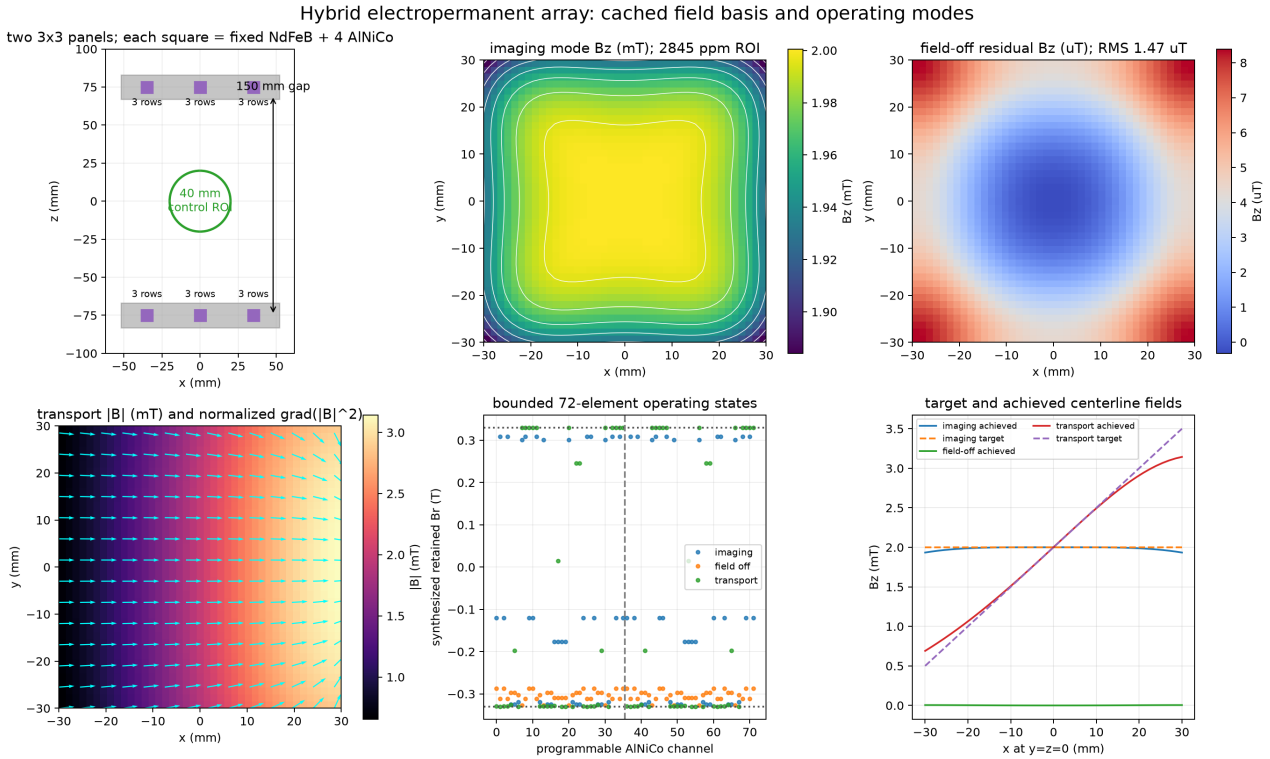


Figure 11.8: Hybrid electropermanent array and three synthesized operating states. The same cached 72-control field basis produces a uniform imaging field, a low-field state, and a directional affine transport field. Maps, retained states, and centerline residuals make saturation and achievable-field errors visible.

Figure 11.8 is generated by `plot_epm_hybrid_array_synthesis.py`. This stage supplies the static system states needed by the therapeutic platform model.

Nonlinear EPM imaging of a tissue phantom

Each retained-remanence state supplies a measured or modeled spatial field profile. After receiver demodulation removes the field at a reference voxel, the signal for state (k) is

$$s_k = \sum_j \rho_j \exp[-i\gamma\tau\,(B_k(\mathbf{r}_j) - B_k(\mathbf{r}_{\mathrm{ref}}))]\,.$$

The global reference subtraction does not remove nonlinear spatial phase. `NonlinearEPMEncoding` stores the resulting dense matrix, and `reconstruct_epm_nonlinear_image` solves it directly with dimensionless Tikhonov regularization and optional nonnegativity. An FFT reconstruction is not valid for these deliberately non-Cartesian field profiles.

```
from spin_dynamics.fields import illustrative_hybrid_epm_array
from spin_dynamics.workflows import (
    build_epm_nonlinear_encoding,
    random_epm_encoding_states,
    run_epm_nonlinear_imaging,
    simple_tissue_phantom,
)

phantom = simple_tissue_phantom(16, field_of_view_m=0.040)
array = illustrative_hybrid_epm_array()
basis = array.build_field_basis(phantom.points_m)
states = random_epm_encoding_states(basis, 384, seed=4)
encoding = build_epm_nonlinear_encoding(
    basis, states, image_shape=phantom.shape,
    phase_encoding_s=300e-6,
)
contrast = phantom.spin_echo_image(
    repetition_time_s=1.2, echo_time_s=0.040,
)
result = run_epm_nonlinear_imaging(
    encoding, contrast, snr_db=35.0, seed=5,
)
```

The bundled `TissuePhantom2D` is a simple elliptical soft-tissue region with one off-center target. Its proton-density, T1, and T2 values are illustrative and are not a clinical tissue-property database. The phantom is intended to exercise contrast, nonlinear encoding, complex acquisition noise, reconstruction, and localization in one reproducible example.

Figure 11.9 is generated by `plot_epm_nonlinear_tissue_imaging.py`. The default 384-state, 16×16-voxel case reports image NRMSE, encoding condition number, field-span range, and target-centroid error.

Image-guided superparamagnetic transport

For a magnetic material volume (V) with low-field SI susceptibility χ, the linear superparamagnetic force is

$$\mathbf{F}_{\mathrm{mag}} = \frac{V\chi}{2\mu_0}\nabla|\mathbf{B}|^2.$$

`SuperparamagneticParticle` stores this magnetic volume separately from the hydrodynamic radius. The default nonlinear path uses $M = M_s\mathcal{L}(3\chi|B|/(\mu_0 M_s))$, which has the same low-field slope but approaches

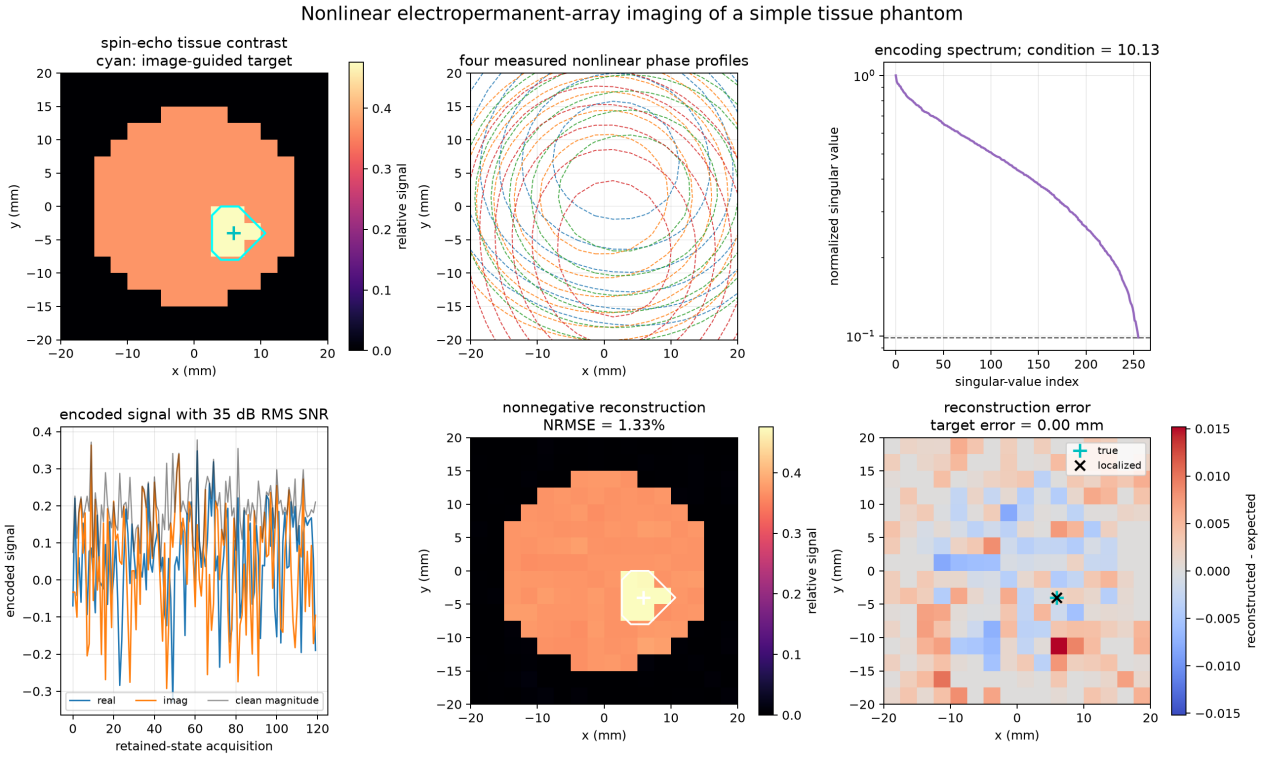


Figure 11.9: Nonlinear EPM imaging of a simple tissue phantom. Four retained-state phase profiles illustrate non-Fourier encoding; the singular spectrum checks conditioning, and the noisy nonnegative reconstruction localizes the off-center target.

the supplied saturation magnetization smoothly. The overdamped magnetic drift is $\mathbf{F}_{\mathrm{mag}}/(6\pi\eta r_h)$, while the Brownian term uses the Stokes–Einstein coefficient $D = k_B T/(6\pi\eta r_h)$.

`magnetic_force_map_2d` differentiates a sampled vector field to obtain $\nabla|B|^2$. `simulate_magnetophoretic_transport` then combines magnetic drift, a constant or callable background velocity, Brownian diffusion, the motion engine's rectangular boundaries, and irreversible entry into a circular capture target.

```
from spin_dynamics.workflows import (
    SuperparamagneticParticle,
    magnetic_force_map_2d,
    simulate_magnetophoretic_transport,
)

force_map = magnetic_force_map_2d(x_axis, y_axis, transport_field)
aggregate = SuperparamagneticParticle(
    magnetic_core_radius_m=12e-6,
    hydrodynamic_radius_m=15e-6,
    magnetic_volume_fraction=0.60,
    volume_susceptibility=1.4,
    saturation_magnetization_a_m=4.5e5,
    fluid_viscosity_pa_s=1.5e-3,
)
transport = simulate_magnetophoretic_transport(
    force_map, aggregate, initial_positions,
    duration_s=75*60, time_step_s=5,
    target_center_m=localized_target,
    target_radius_m=4.2e-3,
    background_velocity_m_s=(2.5e-6, 0.0),
    seed=10,
)
```

Figure 11.10 is generated by `plot_epm_image_guided_transport.py`. Its default particle is an illustrative 30 μm hydrodynamic aggregate, not one nanoparticle and not a clinical dose model. The workflow is dilute and overdamped; it does not yet represent particle interactions, aggregation kinetics, adhesion, branching vasculature, porous tissue permeability, or feedback on the field or flow.

Alternating image-guided controller

`run_epm_image_guided_controller` closes the first system-level loop. Each cycle acquires a fresh noisy nonlinear image, localizes the target, forms the centroid of the uncaptured particle population, synthesizes a transport gradient directed between those centroids, observes an explicit programming dwell, and integrates a transport burst. The controller stops when it reaches the configured capture goal or cycle limit.

```
from spin_dynamics.workflows import (
    EPMTherapyControllerConfig,
    run_epm_image_guided_controller,
)

closed_loop = run_epm_image_guided_controller(
    encoding, contrast, phantom.x_m, phantom.y_m,
    aggregate, initial_positions,
    config=EPMTherapyControllerConfig(
        max_cycles=4,
        capture_goal=0.70,
```

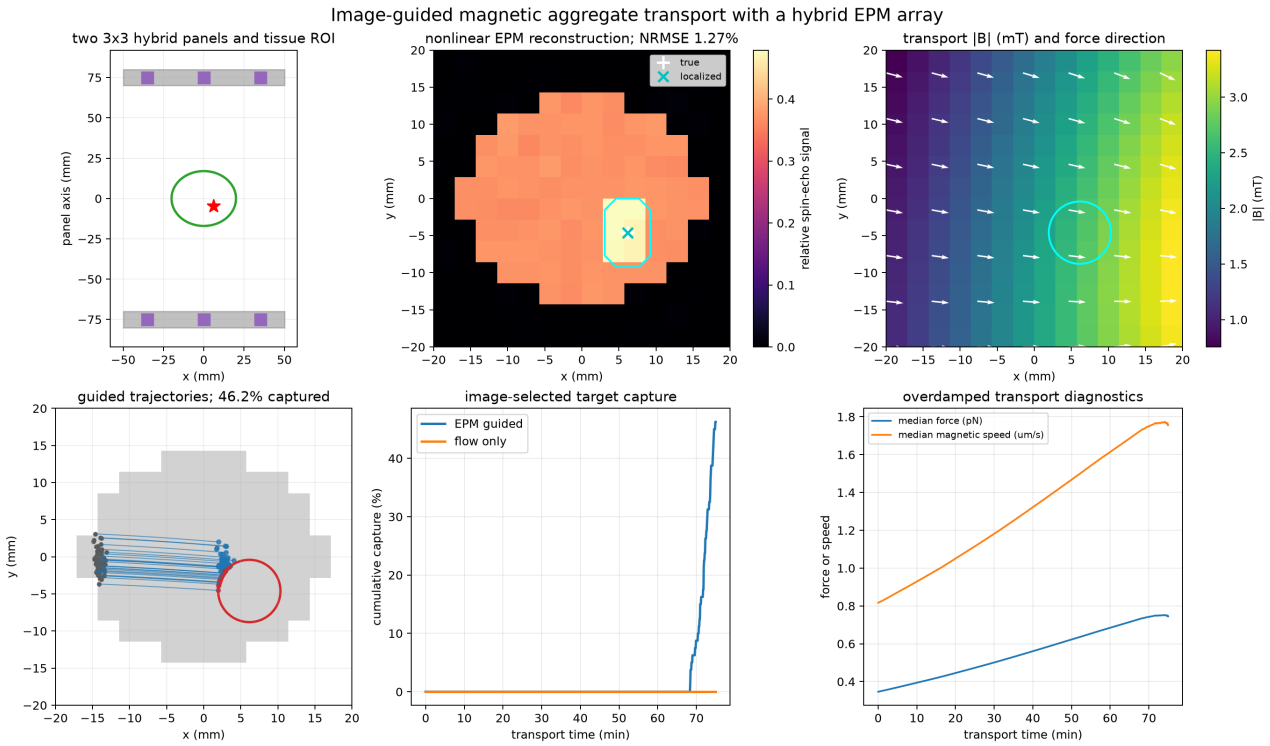


Figure 11.10: Image-guided EPM aggregate transport in the simple tissue phantom. The nonlinear reconstruction selects the target, the fitted affine EPM state sets the force direction, and seeded trajectories compare magnetic guidance with a matched flow-only control. Force, magnetic drift speed, and cumulative target capture remain visible rather than being reduced to a single endpoint.

```
        imaging_window_s=90,
        programming_window_s=0.25,
        transport_window_s=24*60,
        transport_gradient_t_m=0.150,
    ),
    background_velocity_m_s=(2.5e-6, 0.0),
)
```

`ControllerModeInterval` records the wall-clock state machine, and each `EPMTherapyCycleResult` retains its reconstruction, target, feedback direction, synthesized state, force map, particle history, and capture gain. The controller also sums absolute retained-remanence changes across channels and imaging states. This "channel T" total is a programming-effort metric, not electrical energy. Pulse energy, driver stress, and temperature require a calibrated 72-channel programming schedule.

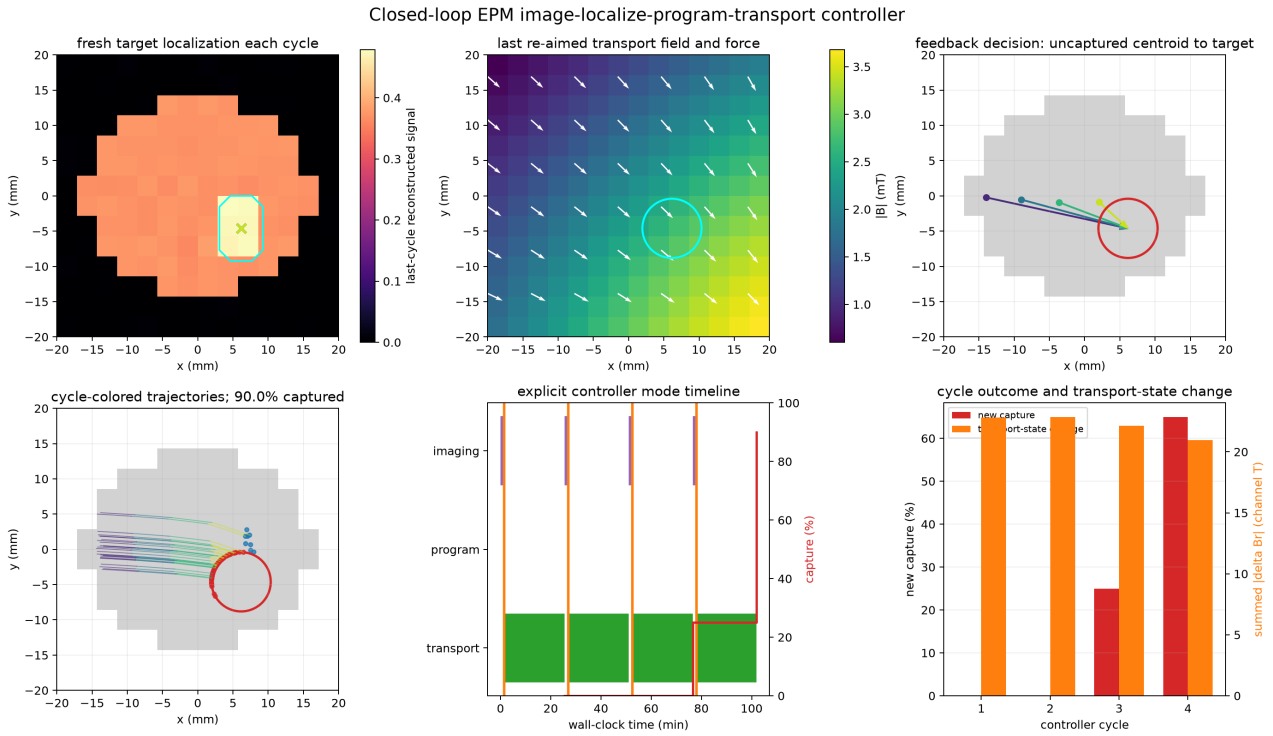


Figure 11.11: Alternating EPM image-localize-program-transport control. Fresh reconstructions update the target, uncaptured-particle centroids re-aim the affine force field, and cycle-colored trajectories show delivery progress. The mode timeline, capture gain, and retained-state change expose the control decisions and their cost surrogate.

Figure 11.11 is generated by `plot_epm_closed_loop_controller.py`. The default illustrative case reaches its capture goal with four re-aimed transport bursts. It still reads the particle centroid and capture mask directly from simulator state; inferring particle concentration from measured images, vascular flow, tissue permeability, adhesion, and release are the next system tier.

Dynamic-inversion central trapping

The controller above deliberately immobilizes particles after target entry. That capture rule is useful for delivery but is not a magnetostatically stable trap. `simulate_dynamic_inversion` instead implements the

pulsed mechanism of Nacev et al. (2015): a uniform field orients a ferromagnetic particle, a short delay follows, and an oppositely directed gradient repels the still-antialigned moment. Cycling the source through (+x,+y,-x,-y) creates an inward average drift without claiming that any static field is stable.

The overdamped state contains position $\mathbf{r}$, body direction $\mathbf{n}$, and magnetic-moment direction $\mathbf{m}/m$:

$$\dot{\mathbf{r}} = \boldsymbol{\mu}_t(\mathbf{n})\nabla(\mathbf{m}\cdot\mathbf{B}) + \mathbf{u} + \boldsymbol{\xi}_t, \qquad \zeta_r\dot{\theta} = (\mathbf{m}\times\mathbf{B})_z + \xi_r.$$

Spheres recover Stokes drag. Blunt rods use separate parallel, perpendicular, and tumbling coefficients from the Tirado–Martinez–de la Torre finite- cylinder expressions. A finite `internal_relaxation_time_s` rotates the moment toward the field without waiting for the body and therefore exposes the failure of a rapidly relaxing superparamagnetic-like moment.

```
from spin_dynamics.workflows import (
    FerromagneticParticle, nacev_2015_sequence,
    simulate_dynamic_inversion,
)

rod = FerromagneticParticle(
    shape="rod", length_m=200e-6, diameter_m=200e-9,
    volume_susceptibility=0.65,
    saturation_magnetization_a_m=1.4e6,
    remanent_magnetization_a_m=1.0e6,
)
trap = simulate_dynamic_inversion(
    nacev_2015_sequence(
        polarizing_field_t=0.5,
        gradient_field_at_center_t=0.2,
        actuator_radius_m=8e-3,
    ),
    rod, initial_positions,
    duration_s=60, target_radius_m=2e-3, seed=3,
)
```

The preset fixes the reported 600 μs polarizing pulse, 5 μs delay, 50 μs gradient, and 60.6 ms element period. Field magnitudes and the inverse-power source radius remain visible inferred inputs because measured coil maps were not published.

`assess_dynamic_inversion_hardware` compares three implementations. Fast coils preserve pulse fidelity but repeat copper and driver loss. EPM-only operation needs polarize, gradient, and field-off retained states every element; a 9.1-minute run therefore requires about 27,027 retained states and 1.95 million channel pulses if all 72 channels change. Because the programming transient is not a calibrated rectangular therapeutic field, EPM-only waveform fidelity defaults to false. A hybrid programs a slow EPM bias/shaping state and uses fast coils for inversion, reducing EPM cycling to setup and teardown while retaining coil loss.

Figure 11.12 is generated by `plot_epm_dynamic_inversion.py`. The current workflow is two- dimensional, dilute, and noninteracting. It does not yet include measured actuator maps, full 3-D focusing, aggregation, adhesion, vascular flow, or a synthesized 72-channel hybrid bias state.

Single-sided depth profiling. Single-sided NMR profiles a sample by depth: an excitation frequency selects the iso-B0 sensitive slice, and the strong static gradient encodes diffusion. The defining feature is that the spins *move through a spatially structured field*—as a molecule diffuses it samples a changing off-resonance set by the real gradient—which is irreducibly spatial and cannot be reduced to a fixed off-resonance distribution. The single-sided workflow module therefore drives the moving-isochromat engine directly with

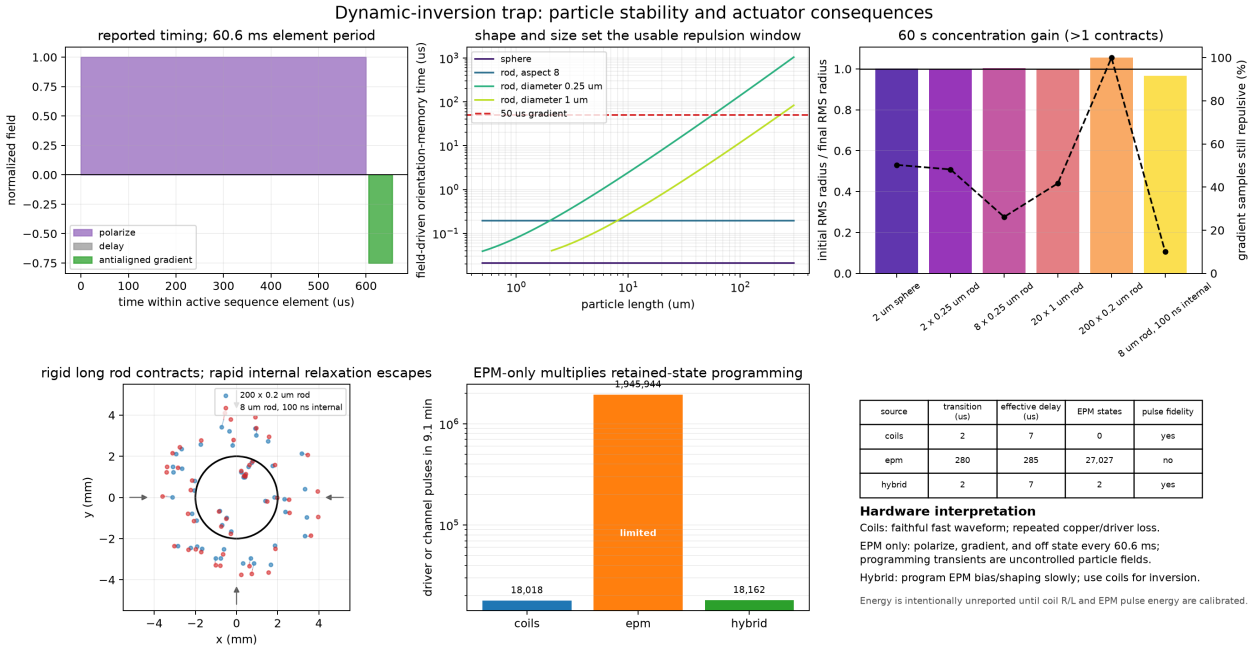


Figure 11.12: Dynamic-inversion stability across particle shapes and sizes. The figure relates pulse timing to Brownian orientation persistence, compares a rigid long rod with a rapidly relaxing-moment control, and counts the coil, EPM-only, and hybrid switching burden. Field magnitudes are illustrative; energy is withheld until actuator pulse energies are calibrated.

the magnet's own B0 map: `mouse_depth_profile` sweeps the carrier frequency so the excited signal traces the depth profile (a density gap shows up as a hole) and the echo decay gives the apparent T_2, diffusion-shortened where the gradient is strong. The companion helper `measure_diffusion_at_depth` runs the CPMG twice with identical initial walker positions (diffusion on and off) so the inhomogeneous-field echo envelope cancels in the ratio, and the attenuation rate $k = \frac{1}{12}\gamma^2 G^2 D\, t_E^2$ gives D using the slice's local gradient. This is a stochastic moving-walker Monte-Carlo whose diffusion sensitivity honestly falls off with depth as the gradient weakens; the engine's constant-gradient physics is validated against the exact Carr-Purcell law. The example `plot_nmr_mouse_depth_profile.py` profiles a layered phantom.

11.5.2 Stream-function gradient-coil design

`spin_dynamics.fields` designs cylindrical MRI gradient coils directly from a desired B_z field. A divergence-free current sheet on a cylinder is represented by a scalar stream function $\psi(\phi, z)$,

$$J_\phi = -\frac{\partial \psi}{\partial z}, \qquad J_z = \frac{1}{r}\frac{\partial \psi}{\partial \phi}.$$

The implementation discretizes the azimuthal component into short filamentary segments and constructs their Biot–Savart sensitivity matrix S, so the sampled field is $B_z = SI$. For target field b, field weights W_f, and regularization α, it solves

$$\min_I \|W_f(SI - b)\|_2^2 + \alpha \|I\|_2^2, \qquad CI = 0.$$

The exact KCL constraint $CI = 0$ closes every azimuthal current column. The regularization path reuses S, reports the field-residual/current-norm L-curve, and selects the maximum-curvature interior sample. Because the unnormalized objective uses tesla and amperes, α has units $\mathrm{T}^2/\mathrm{A}^2$; users should span the visible corner rather than regard any fixed value as universal.

```
import numpy as np
from spin_dynamics.fields import (
    CylindricalWindingSurface,
    build_cylindrical_gradient_system,
    extract_winding_contours,
    linear_gradient_target,
    solve_regularization_path,
    spherical_target_points,
    winding_segments,
)

surface = CylindricalWindingSurface(
    radius=0.08, length=0.18, n_phi=32, n_z=17
)
points = spherical_target_points(0.03, points_per_axis=7)
target_bz = linear_gradient_target(points, (0.0, 10e-3, 0.0))
system = build_cylindrical_gradient_system(surface, points)
path = solve_regularization_path(
    system, target_bz, np.logspace(-20, -9, 23)
)
result = path.selected_result
contours = extract_winding_contours(
    result, current_per_turn_a=1.0
)
segments = winding_segments(contours)
```

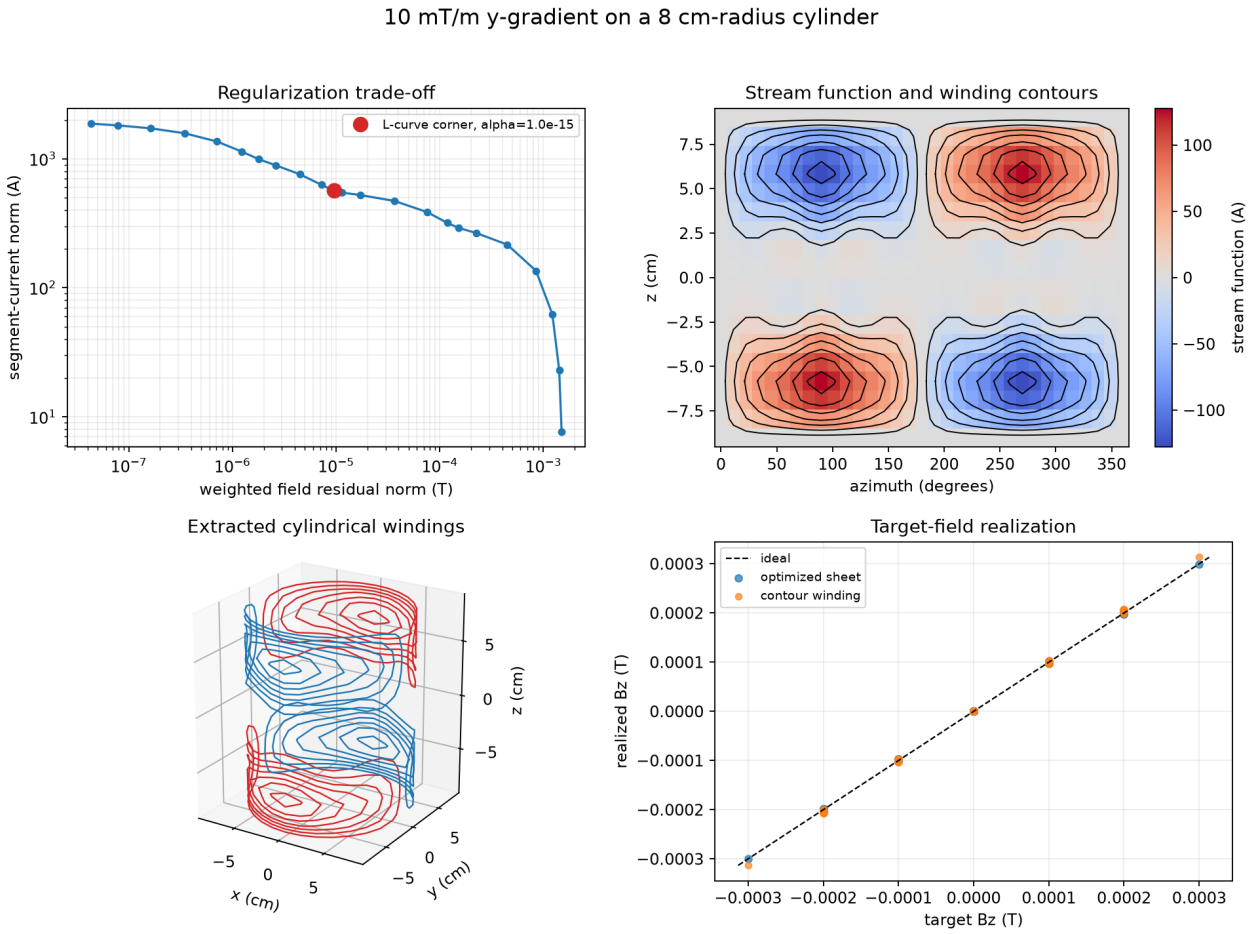


Figure 11.13: A complete cylindrical y-gradient design: L-curve selection, unwrapped stream function and periodic contours, three-dimensional winding paths, and a direct target-versus-realized field comparison. Blue and red paths denote the two contour polarities.

The result includes the optimized segment-current grid, predicted and residual fields, closure and fit diagnostics, and a stream function on the $n_z + 1$ axial cell edges in `result.stream_z`. Its discrete convention is `segment_currents_a == -np.diff(stream_function_a, axis=1)`. `extract_winding_contours` places levels half a turn-current from zero, stitches paths across the periodic $0/2\pi$ seam without requiring Matplotlib, orients them consistently with the optimized current sheet, and maps them to three-dimensional straight segments accepted by `biot_savart`.

Run `python examples/plot_stream_function_gradient_coil.py -output gradient_coil.png` to reproduce Figure 11.13. The companion `plot_gradient_coil_regularization.py` isolates the bias-current trade-off and curvature diagnostic.

Active shielding

An actively shielded design places a second current sheet on a larger concentric cylinder and optimizes the two surfaces together. With target-volume matrix S_t, exterior-surface matrix S_e, and independent KCL

constraints $C_p I_p = 0$, $C_s I_s = 0$, the implemented problem is

$$\min_{I_p, I_s} \left\| W_t \left(S_t[I_p; I_s] - b \right) \right\|_2^2 + \| W_e S_e[I_p; I_s] \|_2^2 + \alpha \left(p_p \| I_p \|_2^2 + p_s \| I_s \|_2^2 \right) .$$

The exterior target is zero. `shield_weights` changes its importance and `surface_regularization_weights` can discourage current on either surface. The result retains separate current grids and stream functions, plus target error, exterior RMS/peak field, current norms, and closure diagnostics. Because the sensitivity matrix depends on geometry rather than the requested target, orthogonal x/y saddle coils reuse the same primary and shield cylinders. The z-gradient rings use a slightly smaller nested layer in the complete set so they do not coincide geometrically with the saddle paths.

```
from spin_dynamics.fields import (
    build_actively_shielded_gradient_system,
    cylindrical_shield_points,
    extract_actively_shielded_winding,
    solve_actively_shielded_gradient_coil,
)

primary_xy = CylindricalWindingSurface(0.05, 0.12, 24, 13)
shield_xy = CylindricalWindingSurface(0.065, 0.15, 24, 13)
outside = cylindrical_shield_points(
    0.072, 0.18, n_phi=24, n_z=17
)
xy_system = build_actively_shielded_gradient_system(
    primary_xy, shield_xy, points, outside
)
x_result = solve_actively_shielded_gradient_coil(
    xy_system, linear_gradient_target(points, (10e-3, 0, 0)),
    regularization=1e-15, shield_weights=0.1
)
y_result = solve_actively_shielded_gradient_coil(
    xy_system, linear_gradient_target(points, (0, 10e-3, 0)),
    regularization=1e-15, shield_weights=0.1
)
```

Run `python examples/plot_actively_shielded_gradient_coil.py -output active_gradient.png` to reproduce Figure 11.14. The continuous current sheets are the inverse solutions; turn extraction quantizes them. Target fidelity and exterior suppression must therefore be recomputed for the extracted contours, and again after leads and crossovers are routed.

Engineering metrics and workflow handoff

`gradient_coil_engineering_metrics` evaluates the realized winding once and returns three grouped reports:

Field	fitted gradient/offset and efficiency per ampere; target RMS error and correlation; exterior RMS and peak field
Electrical	wire length, DC resistance and power, approximate series-equivalent inductance, stored energy, and voltage per current slew
Mechanical	net and per-contour Lorentz force/torque and peak segment force in a supplied static background field

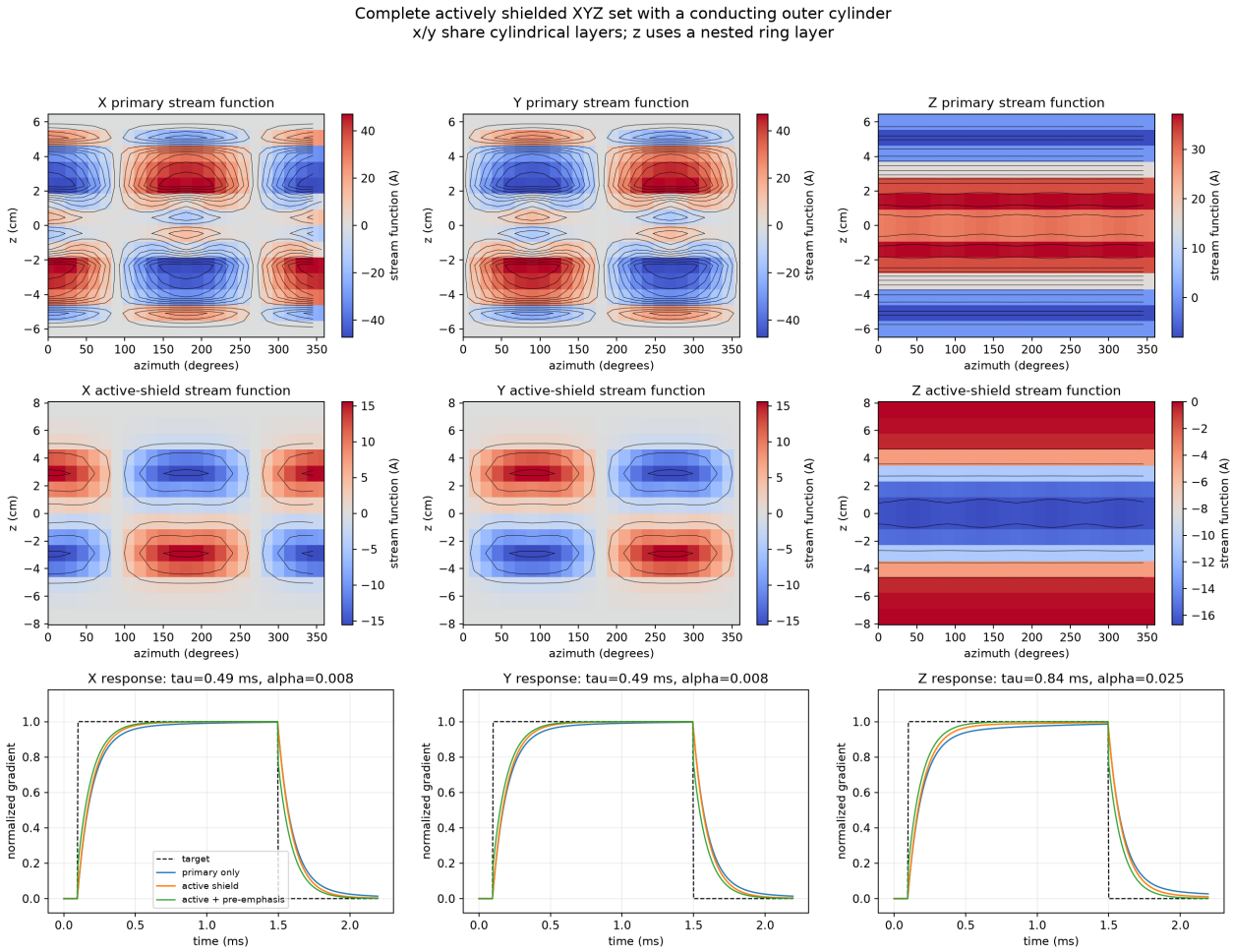


Figure 11.14: Complete actively shielded XYZ gradient set. The first two rows show primary and shield stream functions: x/y are rotated saddle patterns sharing the same cylindrical layers, while z uses nested rings. The bottom row predicts each time-domain response with a conducting outer cylinder, comparing the primary alone, the full active set, and causal pre-emphasis.

Force is integrated as $d\mathbf{F} = I\, d\boldsymbol{\ell} \times \mathbf{B}_0$, with torque about a user-selected origin. The background may be a constant vector or a spatial callback. These are filament-load estimates, not a structural or vibration analysis. Resistance uses the selected conductor material and temperature. Self inductance uses an equal-perimeter circular-loop estimate and mutual terms use the existing Neumann calculation on the actual paths.

```
metrics = gradient_coil_engineering_metrics(
    winding, points, target_field_t=target_bz,
    shield_points=outside, wire_radius=0.5e-3,
    background_field=(0, 0, 3.0)
)

# Existing PEEC and imaging interfaces
conductors = winding_peec_conductors(
    winding, wire_radius=0.5e-3
)
field_map = winding_imaging_field_map(winding, (x_axis, z_axis))
motion_maps = field_map.to_motion_field_maps()
```

```
# Dominant current mode of a thin conducting outer cylinder
shell_mode = cylindrical_shell_eddy_mode(
    outer_result, current_per_turn_a=outer_peak / 5,
    thickness_m=1.5e-3, resistivity_ohm_m=2.82e-8
)
driver = shell_mode.to_gradient_driver(
    winding, gradient_direction=(1, 0, 0), tau_rl=100e-6
)
delivered = driver.apply(commanded, dt)
```

The PEEC handoff returns one `Conductor` per contour, opened at its first sample to define terminals. It deliberately does not invent crossovers or end connections; those paths must be added before treating the loops as one routed series conductor. The eddy adapter passes the combined oriented primary/shield segments to the existing `EddyModes.to_gradient_driver`, yielding the standard pre-emphasis response. For a transverse-capable outer-conductor model, `cylindrical_shell_eddy_mode` extracts a dominant saddle (x/y) or ring (z) current mode on a thin cylindrical shell. Its resistance follows the shell resistivity, thickness and effective current-band width; self and mutual inductance use the actual contour paths. The resulting geometry-derived `time_constant_s` and coupling coefficient feed the same `GradientDriverResponse`. This reduced-order model does not resolve skin depth, apertures, joints, or a dense spectrum of overlapping shell modes. The imaging adapter retains tesla and converts to angular off-resonance in rad/s. Its physical-axis defaults match existing workflows: z in 1-D, (x, z) in 2-D, and (x, y, z) in 3-D.

The inverse solver currently supports regular concentric cylinders. Extracted contours are independent closed loops; manufacturability constraints, automatic routing, arbitrary surfaces, and force/torque constraints inside the inverse solve remain future extensions. Force and torque *evaluation* is available for any realized winding.

11.5.3 Quasistatic E-field and eddy currents

Biot-Savart gives a coil's B-field but not the accompanying E-field — and it is the E-field that drives eddy currents (resistive power loss, B1 perturbation, and gradient field decay) in a conductor. `spin_dynamics.fields.quasistatic` adds the magnetoquasistatic induced field $\mathbf{E} = -\partial\mathbf{A}/\partial t - \nabla\phi$: the vector potential `vector_potential` is computed from the same filament segments (its curl reproduces `biot_savart`), the inductive field is `induced_efield`$= -\mathbf{A}_{\text{unit}}\, dI/dt$, and in a finite conductive sample a Neumann charge correction enforces $\mathbf{J}\cdot\hat{\mathbf{n}} = 0$ so `eddy_currents` returns the eddy current density and the deposited power. Coil generators live in `fields.coils` (`solenoid`, `planar_spiral`, `maxwell_pair`, `cylindrical_shield`).

First-order (Born) limitation. This uses the *primary* field only and ignores the eddy currents' reaction field, so it is valid for low conductivity, thin samples, or skin depth $\gg$ sample size; there is no skin-effect shielding and it over-estimates deposition at high conductivity.

The gradient eddy *dynamics* are a separate, self-consistent calculation. `fields.eddy_modes.EddyModes` models a conducting shield as coaxial rings, builds their resistance, inductance and drive coupling, and eigendecomposes the $\mathbf{L}^{-1}\mathbf{R}$ dynamics into the residual-gradient decay $\sum_k \alpha_k e^{-t/\tau_k}$ — exactly the `eddy_terms` of the M5 `GradientDriverResponse`. So coil + shield geometry now *predicts* the pre-emphasis terms that were hand-specified in Sec. 11.22.3: `EddyModes.to_gradient_driver()` returns a driver the GRAPE PGSE optimizer consumes directly.

```
from spin_dynamics.fields import coils
from spin_dynamics.fields.eddy_modes import EddyModes
```

```
drive = coils.maxwell_pair(radius=0.04, axis="z")
radii, centers = coils.cylindrical_shield(radius=0.06, length=0.3, n_rings=16)
eddy = EddyModes(radii, centers, wire_radius=2e-3, resistivity=1.7e-8)
driver = eddy.to_gradient_driver(drive, tau_rl=1e-4)  # -> M5 GradientDriverResponse
```

Sample loading (reflected impedance). By reciprocity the eddy loss couples back into the coil as an added series resistance $R_{\text{refl}}(\omega) = \omega^2 \int \sigma\,|\mathbf{A}_{\text{unit}}|^2\,dV$ (`reflected_resistance`; frequency-independent geometry factor `geometric_loss_integral`). It grows as ω^2, degrades the loaded $Q = \omega L/R$ and raises the Johnson-noise floor by $\sqrt{R_{\text{total}}/R_{\text{coil}}}$; `coil_loading` sweeps it across frequency given `coil_inductance` and $R_{\text{coil}}(f)$.

The examples `plot_rf_coil_efield.py` (solenoid & planar spiral: B1, |E|, and eddy Joule density in a saline cylinder — note the solenoid's on-axis E null), `plot_gradient_eddy_preemphasis.py` (Maxwell pair + shield: eddy spectrum, gradient droop, and the geometry-derived pre-emphasis that flattens it), and `plot_logging_coil_loading.py` (an inside-out NMR well-logging tool — a solenoid coil in a saturated-brine borehole, 0.5–2 MHz — where the brine loading dominates the coil resistance and collapses Q) demonstrate the solvers. `plot_logging_cpmg_multinuclear.py` extends the logging tool to a full Jasper-Jackson magnet (two opposing SmCo cylinders whose low-gradient sweet spot sets the sensitive shell), runs the asymptotic CPMG echo on the estimated (B_0, B_1) maps to visualize the ^{1}H sensitive region, plots the tuned-probe echo and estimates the per-echo SNR against the loading noise, and adds ^{23}Na (spin-$3/2$, treated with the $(2I+1)$ machinery; resonant at the same frequency where B_0 is $\sim$3.8$\times$ higher, with non-optimal flip angles from the proton-calibrated pulses) to estimate its fractional signal contribution. Design note: `docs/field_solvers_quasistatic.md`.

11.5.4 Coil property extraction: inductance, resistance, Q, and self-resonance

The solvers above give a coil's *fields*; a probe designer needs the coil's *lumped RF properties*. `spin_dynamics.fields.coil` extracts them for a single-layer helical round-wire solenoid — `solenoid_properties` returns a `CoilProperties` with the series inductance (the frequency-independent current-sheet value L_s and the RF value L_{eff}), the AC (skin + proximity) resistance R, the stray capacitance C_p, the unloaded quality factor $Q = \omega L_{\text{eff}}/R$, and the first (quarter-wave) self-resonant frequency f_{res}. These are exactly the numbers the tuned-probe noise model (Sec. 11.5, "Received-Signal Noise") and the matching-network designer consume, so a coil geometry now feeds the SNR budget directly instead of hand-typed L, R, C: `to_probe_params()` returns $\{L, R, C\}$ and `tuning_capacitance()` the tank capacitor $C = 1/(\omega^2 L)$.

```
from spin_dynamics.fields.coil_properties import solenoid_properties

cp = solenoid_properties(diameter=20e-3, length=30e-3, turns=10,
                         wire_diameter=1.0e-3, frequency=10e6)
cp.inductance_effective, cp.ac_resistance, cp.q_factor, cp.self_resonant_frequency
cp.to_probe_params()   # {"L", "R", "C"} -> tuned_probe_output_noise_density
```

The model is the $n = 0$ sheath-helix waveguide method of Corum, with the NIST round-wire self- and mutual-inductance corrections (Knight, Rosa), the Lundin field-non-uniformity correction, and the Medhurst proximity data. Because it is a transmission-line model it stays accurate for electrically long coils, where L_{eff} rises above the quasistatic L_s toward f_{res}. It is a faithful port of Serge Stroobandt's QOIL calculator (GPL-3.0, vendored in `References/`), re-expressed with `scipy.special` Bessel functions and `brentq` root finds; it reproduces QOIL to ~ 6 significant figures. `solenoid_field_inductance` provides an independent cross-check: it sums the Biot-Savart/Neumann partial inductances of the coaxial turns (via `quasistatic.coil_inductance`) and agrees with L_s to within $\sim 10\%$.

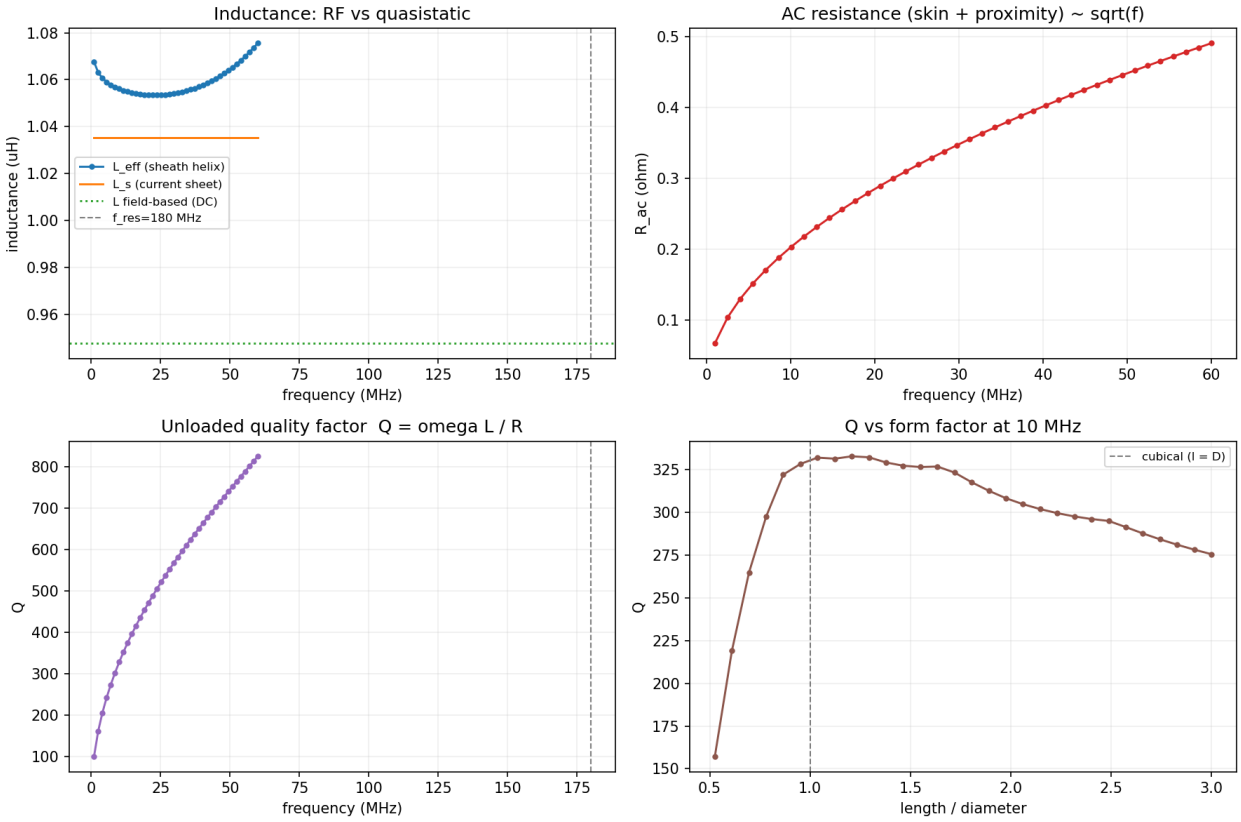


Figure 11.15: A single-layer solenoid carried from geometry to RF design metrics. The panels compare quasistatic and RF inductance, show skin/proximity resistance and unloaded Q versus frequency, and reveal the form-factor optimum at the chosen operating frequency.

Conductor material and temperature. The model is not restricted to copper at room temperature. Presets `ANNEALED_COPPER`, `HARD_DRAWN_COPPER`, `SILVER`, and `ALUMINIUM`—or any custom `ConductorMaterial`—set the resistivity and permeability, and a `temperature` argument evaluates the resistivity (hence R and Q; L, C, and f_{res} are geometric) at the operating temperature. A linear temperature coefficient covers moderate excursions about room temperature; supplying a residual-resistivity ratio ($\mathrm{RRR} = \rho_{293\,\mathrm{K}}/\rho_0$) switches to a Matthiessen model with a residual floor, valid down to cryogenic temperatures. Cooling an OFHC-copper coil ($\mathrm{RRR} = 100$) from 293 to $77\,\mathrm{K}$ cuts R roughly in half and nearly doubles the unloaded Q—the cryoprobe gain. The cryogenic estimate is conservative (the linear phonon term over-predicts the ideal resistivity below $\sim$Debye$/3$), so pass a measured resistivity for a precise cold value.

The example `plot_solenoid_coil_properties.py` sweeps L_{eff}, R, and Q versus frequency (overlaying the field-based inductance and marking f_{res}), shows the classic Q-vs-form-factor peak near a cubical coil ($\ell \approx D$), and closes the loop by feeding the extracted (L, R) into `coil_loading` for the loaded Q and noise penalty against a conductive sample. The field-based generalization to arbitrary geometry is the PEEC solver in the next subsection. Design note: `docs/coil_properties.md`.

Reproduce Figure 11.15 with `python examples/plot_solenoid_coil_properties.py -save solenoid.png`.

11.5.5 Arbitrary-geometry coil solver (PEEC): multi-layer, saddle, and gradient coils

The analytic model above is limited to single-layer air-core solenoids. Real RF coils (multi-layer, saddle, birdcage, surface) and gradient coils have arbitrary wire paths, and `spin_dynamics.fields.coil_peec` extracts their lumped RF properties from first principles by the *partial-element equivalent-circuit* (PEEC) method — the mesh-light approach behind FastHenry/FastCap. A conductor is a polyline centreline plus a round cross-section; the cross-section is tiled into K parallel sub-filaments to resolve the current distribution. Because each sub-filament is a simple series chain, the branch system reduces *exactly* to a $K \times K$ chain-impedance system in which sub-filament k is the whole path swept to cross-cell k and $L_{kk'}$ is the Neumann mutual inductance between two such filaments (reusing `quasistatic.vector_potential`). Solving $Z(\omega) = 1/(\mathbf{1}^{\mathsf{T}}(R + j\omega L)^{-1}\mathbf{1})$ lets the current redistribute with frequency, which *is* the skin effect (own cross-section); a dual thin-wire electrostatic solve on the same geometry gives the self-capacitance and hence the self-resonant frequency. The chain reduction constrains each sub-filament to a *path-constant* current, so it captures only part of the turn-to-turn *proximity* effect; passing `formulation="full"` instead keeps one branch per (segment, sub-filament) — FastHenry's actual system, in which the cross-section current split is a free unknown at every segment — and resolves the proximity loss from first principles at cost $O((MK)^3)$ per frequency. `coil_properties_peec` defaults to `full`; use `chain` when only L, C or the self-resonance matter.

```
from spin_dynamics.fields.coil_peec import helical_solenoid, coil_properties_peec

coil = helical_solenoid(diameter=20e-3, length=30e-3, turns=10,
                        wire_radius=0.5e-3, n_radial=4, n_angular=8)
p = coil_properties_peec(coil, frequency=5e6)
p.inductance, p.ac_resistance, p.q_factor, p.self_resonant_frequency
p.to_probe_params()   # {"L","R","C"} -> same probe-noise / matching pipeline
```

Build any coil from its wire path with `Conductor` / `conductor_from_segments`; `extract_impedance` sweeps $L(\omega)/R(\omega)$ and `current_distribution` returns the cross-section current for the crowding picture. `PEECCoilProperties` mirrors the universal fields of `CoilProperties` (Sec. 11.5.4) so an arbitrary coil drops into the same probe-noise and matching workflow. The solver is validated against the exact Kelvin-function skin-effect ratio (isolated wire), the Neumann kernels, and the QOIL solenoid — its L, self-capacitance and self-resonance agree with QOIL to a few percent (the physical self-resonance, distinct from QOIL's lumped-equivalent C_p fitting capacitance).

Resolution ceiling and its accelerations. The volume solver must tile the cross-section finer than the skin depth, so its $O(K^2)$ cost (in the cell count K) grows as $(a/\delta)^2$ — prohibitive at RF. Two accelerations remove this, benchmarked in `plot_peec_performance.py`. The **surface-impedance backend** `extract_impedance_surface` rings the cross-section boundary with filaments carrying the analytic surface impedance $Z_s = (1+\mathrm{j})\rho/\delta$; its cost is independent of a/δ and its accuracy *improves* with frequency (-3.3% at $a/\delta = 15$, -0.3% at $a/\delta = 151$ with 48 cells), making it ~ 10– $100\times$ faster than a volume solve in deep skin while being more accurate. **Graded meshing** (`grading`> 1) concentrates volume cells at the surface, so the moderate-skin regime converges with far fewer cells (a 1 MHz square bar reaches $\sim 5\%$ at 64 cells versus ~ 250 for a uniform mesh). Inductance, capacitance and self-resonance are geometry-dominated and accurate regardless. FMM/H-matrix acceleration of the $O(K^2)$ chain matrix and magnetic-core coupling remain planned follow-ons, though the two accelerations above keep K small in the regimes that used to force large meshes.

Shielding and dielectric formers. `self_capacitance` accepts a grounded enclosure `shield=GroundedBox(...)` (entered through its 3-D image-charge lattice) and a dielectric former `relative_permittivity`; both raise the coil capacitance and lower the self-resonant frequency. A metal box is also a good conductor *magnetically*: the same shield passed to `extract_impedance(..., ground_plane=...)` / `extract_impedance_surface(..., shield=...)` adds the walls' flux-exclusion image (`magnetic_image_filaments`, six primary wall reflections for a box), which *lowers* L and so *raises* the SRF — partly offsetting the capacitive lowering. The example `plot_shielded_nqr_coil.py` designs a ^{14}N NQR probe (2–3 MHz, coil ID $\approx 1.5''$) on a Teflon former in a grounded aluminium box (one coil end and the box grounded) and plots the properties with and without the box: $L(f)$ and $R(f)$ from the surface-impedance backend with `formulation="full"` (skin + turn-to-turn proximity, no empirical factor), the self-capacitance/inductance and SRF for free/former/box (e.g. $C: 1.5 \to 2.2 \to 3.9$ pF, $L: 4.8 \to 4.2\,\mu$H, net SRF $59 \to 49 \to 39$ MHz, kept above the 10 MHz target), and the Q reduction from box-wall eddy loss. The good-conductor image doubles the tangential field at the wall, so that loss is $R_{\text{box}} = 4R_s \oint |H_{\text{tan,inc}}/I|^2 \,\mathrm{d}A$ ($R_s \propto \sqrt{f}$); dropping the doubling underestimates it by $2\times$.

Single-ended vs differential Q over a ground plane. A `GroundPlane(point, normal)` shield is the single grounded plane (a half-space) below a planar coil, entered as the one method-of-images mirror charge (it reproduces the analytic wire-over-plane $C/\ell = 2\pi\varepsilon_0/\operatorname{acosh}(h/a)$ to a few percent). `self_capacitance` also takes a `potential` profile (array or callable of the arc-length fraction), defaulting to the single-ended ramp $0 \to 1$; `potential=lambda s: s-0.5` is the differential (balanced) drive, antisymmetric about a virtual ground. Over a ground plane the differential coil-to-ground capacitance $C_g = C_{\text{eff}}(\text{plane}) - C_{\text{eff}}(\text{free})$ is several times smaller (the two winding halves' displacement currents to ground cancel in common mode), while L, the copper resistance and the ground-plane eddy loss — all set by the coil *current* — are mode-independent. The plane's effect on L itself is the flux-exclusion image, added by `extract_impedance(..., ground_plane=GroundPlane(...))` (reflect each sub-filament across the plane and reverse its current, giving $L = L_{\text{free}} - M(2h)$, validated against the analytic loop-over-plane mutual); it is set by the winding current and so is the same for both drives. That leaves the capacitive difference, through the board loss $R_{\text{diel}} = \tan\delta\,\omega^3 L^2 C_g$, as the entire single-ended/differential Q split. The example `plot_pcb_coil_ground_mode_q.py` works a PCB spiral on FR4 over copper: the plane lowers L ($\sim$619$\to$216 nH at 0.8 mm, both modes) and the differential drive couples $\sim 7\times$ less to ground ($\sim 8\times$ less dielectric loss), so the differential Q runs higher, widening with frequency — the reason NMR/MRI surface and PCB coils are built balanced.

Loss-model caveats (proximity and eddy currents). Two easy trip-ups when trusting a computed Q. *Proximity:* use `formulation="full"` for any closely-wound coil — both the volume and surface-impedance backends accept it, and the per-segment redistribution it enables *is* the proximity model (FastHenry's own mechanism; no empirical factor). The full volume solve matches an axisymmetric continuum-FEM reference (FEMM 4.2) to 0.1% on a tightly-wound test coil where the reduced `chain` formulation under-predicts the loss, FastHenry's converged graded mesh reads $\sim$20% low, and Medhurst's empirical table reads $\sim$25% high (his measured Q included real-coil losses beyond eddy-current proximity; keep `medhurst_proximity_factor` as a conservative cross-reference). *Radiation:* `radiation_resistance` adds the first-order magnetic-dipole term $R_{\text{rad}} = \eta_0 k^4 |\mathbf{A}_{\text{net}}|^2/6\pi$ from the net vector area of the wire path (reduces to the small-loop $20\pi^2(C/\lambda)^4$, scales as N^2, vanishes for a Maxwell pair); `coil_properties_peec` includes it in Q and the probe parameters by default — negligible at NMR/NQR frequencies, the Q ceiling for large loops at VHF. A `GroundedBox` shield *suppresses* it: for radiation the box is a cavity, and below its lowest resonance (`fundamental_resonance`, the TE_{101} mode) a closed conductor radiates nothing externally, so `shield=` returns 0 below cutoff (the would-be radiated power appears in the separately-modelled wall eddy loss), consistent with the box's electrostatic role. Even the computed R is a theoretical upper bound on

Q — a measured coil is typically ∼1.3–2× lossier (dielectric former, solder/lead resistance, common-mode lead radiation). *Eddy loss in a nearby conductor:* use the *surface-resistance* model ($R \propto \sqrt{f}$) when the skin depth is much thinner than the conductor (a solid metal shield at RF), and the *first-order volume* model (`fields.quasistatic.reflected_resistance`, $R \propto f^2$) only when the conductor is thin compared with the skin depth (a foil, a weakly-conducting sample); applying the volume model to a solid metal box at MHz over-predicts the loss by orders of magnitude. See `docs/coil_peec.md`.

The example `plot_peec_coil_solver.py` shows a two-layer solenoid (L, R, Q, f_{res}, and the cross-section current-crowding map) and a gradient Maxwell pair (L, R, the $\mathrm{d}B_z/\mathrm{d}z$ efficiency from Biot-Savart, and shield eddy modes from `eddy_modes`) — the same solver spanning RF and gradient coils.

Cross-validation against FastHenry, FasterCap, and QOIL. The PEEC solver is validated against three independent references: FastHenry (the M.I.T./FastFieldSolvers partial-element inductance solver) for $L(f)$ and $R(f)$, FasterCap (its boundary-element capacitance solver) for the self-capacitance, and the QOIL sheath-helix model for the self-resonant frequency. `fields.fasthenry_interop` and `fields.fastercap_interop` export a `Conductor` to each tool's input format and, on a Windows install, drive its COM automation server; the example `validate_peec_vs_fasthenry.py` runs the comparisons and live tests run whenever the servers are present (skipped otherwise). Table 11.1 collects the results.

Quantity	Reference	Case	Agreement
Inductance L	FastHenry	1×1 mm bar, 100 mm	$< 0.1\%$
Inductance L	FastHenry	6-turn solenoid, $D = 20$ mm	0.5–0.6%
Resistance R	FastHenry	straight bar, matched uniform mesh	$\leq$2%
Resistance R (full)	FastHenry	6-turn solenoid	3.5–8%
Proximity factor Φ (full)	FEMM continuum FEM	tight 10-turn coil	0.1%
Proximity factor Φ (SIBC-full)	exact two-wire $1/\sqrt{1-(d/s)^2}$	hairpin	∼3%
Self-capacitance C	FasterCap	1 m, 1 mm wire	∼1%
Self-resonance f_{res}	QOIL	10–40-turn solenoids	1–3%

Table 11.1: PEEC solver validation against independent reference solvers. The inductance uses the exact closed-form segment mutual ported from FastHenry's `mutualfil` (Grover's method), which, unlike a numerical quadrature, does not over-couple the vertex-sharing segments of a coil path (a quadrature drifts to 2× as the path is refined). Mesh-matching note: FastHenry silently grades its filaments (adjacent-size ratio 2, `readGeom.c` defaults) — matched comparisons pass `rw=rh=1`.

Why the resistance "diverges" at high frequency — and why it does not. Both PEEC and FastHenry are partial-element solvers: the cross-section must be tiled finer than the skin depth δ, and *both* under-resolve the AC resistance on a coarse mesh. They converge to the same (exact Kelvin) value as the filament count rises. Table 11.2 shows a 1×1 mm copper bar at 1 MHz (skin depth 66 μm, side/$\delta \approx 15$, deep skin): with the meshes *really* matched (uniform filaments both sides, `rw=rh=1` — FastHenry otherwise silently grades with ratio 2) the two solvers track each other to $\leq$1.5% at every mesh and climb together toward the exact value, while the inductance is already converged at the coarsest mesh. For deep-skin resistance, raise the cell count or use the surface-impedance backend. The round-wire cross-section is tiled on a Cartesian grid masked to the disc, so its cells stay square and the resistance converges monotonically; an equal-area polar tiling would diverge as its wedge cells elongate.

The FasterCap capacitance and PEEC `capacitance_to_ground` also both sit ∼10% below the crude analytic $2\pi\varepsilon_0 L/(\ln(L/a) - 1)$, confirming the thin-wire kernel against the reference rather than the approximation; the coil self-capacitance is additionally validated against the Medhurst formula. Design note: `docs/coil_peec.md`.

Filaments $n \times n$	R_{PEEC} (mΩ)	$R_{\text{FastHenry}}$ (mΩ)	(Kelvin ≈ 7.8)
4×4	4.12	4.17	
8×8	6.51	6.58	
16×16	7.77	7.80	
20×20	7.96	7.97	

Table 11.2: AC-resistance convergence versus filament count, 1×1 mm bar at 1 MHz, uniform filaments both sides. The solvers track each other to ≤1.5% and climb toward the exact value together; the inductance is unchanged.

11.5.6 Nonlinear magnetostatics: ferrites and saturable iron

The analytic solvers above superpose permanent-magnet dipoles and Biot-Savart filaments in *free space*; they cannot represent flux-shaping magnetic materials. `spin_dynamics.fields.nonlinear_magnetostatics` adds a structured-grid nonlinear magnetostatic solver for exactly that: a high-permeability RF ferrite that focuses a B_1 field, or saturable iron that shapes and returns a B_0 field. Each nonlinear step's sparse linear system is solved by a SciPy LU factorization (FEMM-style; a NumPy matrix-free conjugate gradient is the fallback when SciPy is absent), with a material's saturation described by a smooth Brauer reluctivity $\nu(B^2) = b_1 + b_2\, e^{b_3 B^2}$ clamped to a physical $\mu_r \geq 1$, and the nonlinear branch tracked by Anderson-accelerated Picard with residual-stall detection. No meshing dependency (structured grid). Two coordinate systems share the material model and solver: `PlanarMagnetostatics` solves the out-of-plane A_z for translationally-invariant (x, y) problems, and `AxisymmetricMagnetostatics` solves the azimuthal A_ϕ for rotationally-symmetric (r, z) problems (solenoid / loop coils with cores). The planar solver is validated against the uniformly-magnetized-cylinder interior field $B_r/2$ and the axisymmetric solver against the analytic solenoid on-axis field.

The example `plot_shim_a_ring_magnet.py` builds a "shim-a-ring": a diametrically-magnetized NdFeB ring inside a concentric soft-iron ring. A uniformly magnetized *tube* produces essentially zero bore field (the outer and inner $\sin\theta$ surface-current sheets cancel); the high-permeability iron ring provides a low-reluctance return path that breaks the cancellation and establishes the transverse bore field, and its *saturation* sets the achievable field — a nonlinear effect analytic superposition cannot capture. The example `plot_logging_ferrite_b1_focusing.py` uses the axisymmetric solver for the complementary RF case, coupled to the same asymptotic-CPMG echo model as `plot_logging_cpmg_multinuclear.py`. It optimizes an inside-out Jasper-Jackson logging coil over two design levers, each swept and traded off against the RF losses:

- **Ferrite core geometry** — the height and thickness of the high-permeability core behind the coil. The efficiency gain is demagnetizing-factor limited (set by the core's aspect ratio, not its permeability), so a short, fat core near the coil beats a coil-length core because it concentrates flux at the sweet-spot midplane.
- **Coil turn spacing** — the axial turn-density profile. Concentrating turns toward the sweet-spot midplane is a strong efficiency lever that the CPMG echo train tolerates despite the B_1 inhomogeneity it creates, so the region-integrated echo signal keeps rising and the optimum is set by practical limits (turn size, voltage), not an interior maximum.

The RF losses are the cost these levers are traded against, not a knob: both levers also boost the coil's coupling to the lossy borehole brine, so the reflected resistance — computed from the solved vector potential over the borehole annulus — rises with the field and offsets much of the naive efficiency gain; the ferrite's own loss-tangent RF loss is included and is negligible next to the brine. The reported figure of merit is

therefore the *net* per-echo SNR and the RF power for a fixed sweet-spot B_1, not the naive reciprocity ratio. The material model and nonlinear scheme are adapted from the author's Motor_Design finite-element solver.

11.5.7 Three-dimensional magnetostatics: the reduced scalar potential

The vector-potential solvers above are two-dimensional — symmetry collapses $\mathbf{A}$ to a single scalar component. General 3D has no such symmetry, and the full curl-curl operator $\nabla \times (\nu \nabla \times \mathbf{A}) = \mathbf{J}$ is singular on a nodal grid (a large gradient null space), which forces edge elements or a gauged block system. `ReducedScalarPotential3D` takes the pragmatic route for magnet- and iron-dominated problems: the *reduced scalar potential*. Writing $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ with $\mathbf{H}_s$ the free-space Biot-Savart field of the coils (so Ampere's law holds for any ψ), Gauss's law $\nabla \cdot \mathbf{B} = 0$ with $\mathbf{B} = \mu(|\mathbf{B}|)(\mathbf{H}_s - \nabla\psi) + \mathbf{B}_r$ reduces to a single scalar, symmetric-positive-definite, variable-coefficient Poisson problem,

$$\nabla \cdot (\mu\, \nabla\psi) = \nabla \cdot \big((\mu - \mu_0)\, \mathbf{H}_s + \mathbf{B}_r\big),$$

solved on the same structured grid with the same Anderson-accelerated Picard driver and the same Brauer material model as the 2D solvers. Permanent magnets enter through the remanence $\mathbf{B}_r$, soft or saturable iron through μ, and coils through $\mathbf{H}_s$; box, sphere, and cylinder region masks paint the geometry.

```
import numpy as np
from spin_dynamics.fields.scalar_potential_3d import ReducedScalarPotential3D
from spin_dynamics.fields.nonlinear_magnetostatics import ndfeb, linear_material

g = np.linspace(-0.09, 0.09, 121)
prob = ReducedScalarPotential3D(g, g, g)
# Two anti-parallel NdFeB bars on a soft-iron yoke (NMR-MOUSE).
prob.add_material(prob.box((-0.02, 0.02), (-0.032, 0.0), (-0.0325, -0.0065)),
                  ndfeb(1.2), remanence_direction=(0, 1, 0))
prob.add_material(prob.box((-0.02, 0.02), (-0.032, 0.0), (0.0065, 0.0325)),
                  ndfeb(1.2), remanence_direction=(0, -1, 0))
prob.add_material(prob.box((-0.02, 0.02), (-0.047, -0.032), (-0.0325, 0.0325)),
                  linear_material(1000.0))
sol = prob.solve(linear_solver="auto")        # AMG on the large grid
bx, by, bz = sol.sample([0.0], [0.0025], [0.0])  # field 2.5 mm above the gap
```

Two implementation points matter. The source term uses $(\mu - \mu_0)\mathbf{H}_s$, not $\mu\mathbf{H}_s$: because $\mathbf{H}_s$ is divergence-free the vacuum part cancels analytically, which makes free space exact ($\psi \equiv 0$, $\mathbf{B} = \mu_0\mathbf{H}_s$) and removes the grid-erratic discrete divergence of the near-singular sampled coil field at the wire. And μ is evaluated as a *stateless* function of ψ — inverting the B-H curve for $\mu(|\mathbf{H}|)$, since $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ is available from the unknown alone — which keeps the $\mu \leftrightarrow \psi$ fixed point inside the Anderson mixing; carrying μ as hidden state fails to converge under partial saturation.

Accuracy and the linear solve. Inside high-permeability material the physical $\mathbf{H} = \mathbf{H}_s - \nabla\psi$ is a near-cancellation of two large quantities, so the field error scales as $\mu_r\,(h/L)^2$; the solver is trustworthy for $\mu_r \lesssim 100$–300, and excellent for permanent magnets ($\mu_r \approx 1.05$). The classical Simkin-Trowbridge total/reduced split does *not* help on this centered nodal grid: the two formulations are an exact linear variable shift apart and yield the identical discrete solution (verified to machine precision), so the residual error is discretization, not floating-point cancellation. The effective cure is grid refinement, made practical by the optional AMG solver. `solve(linear_solver="auto")` uses an exact sparse LU for small grids and switches to pyamg's AMG-preconditioned CG above $\sim 20^3$ unknowns; its grid-independent iteration count scales to $> 10^6$ unknowns (a 121^3 grid solves in seconds, where sparse-LU fill-in cannot reach 64^3). The

design note `docs/reduced_scalar_potential_3d.md` gives the full quantification and the equivalence proof.

The NMR-MOUSE. The example `plot_nmr_mouse_3d.py` is the flagship application: the Blümich NMR-MOUSE, two anti-parallel NdFeB bars on a soft-iron yoke with a surface coil in the gap. Unlike the analytic treatment of Sec. 11.5.1, which adds the yoke by the method of images across a $\mu \to \infty$ plane, the solver treats the magnets and the *finite* iron yoke together, so the yoke's low-reluctance return path is resolved directly and is found to boost the sensor field by $\sim 10\%$ — the effect that motivates the yoke and that free-space dipole superposition cannot capture. With the paper's geometry (26 mm magnets, 13 mm gap, 15 mm yoke) the simulation reproduces the device's hallmarks: $B_0 \approx 0.43$ T at the 2.5 mm sensor (~ 18 MHz ^{1}H; paper 0.41 T, 17.5 MHz), a ~ 14 T/m penetration gradient (paper 10–14), and a near-quadratic lateral $B_{0z}(z)$ across the gap. A useful correctness check for any new geometry: the solver honours the problem's exact symmetries (here B_{0z} even in z, B_{0y} odd) to machine precision.

Driving the workflow with the solved field. The solved field is wired into the single-sided workflow (Sec. 11.5.1) exactly as the analytic magnet is. `ScalarPotentialSolution.to_magnet_field_maps(...)` samples the solved B_0 (and an optional coil B_1) onto a $(\text{lateral}, \text{depth})$ plane and returns the same `MagnetFieldMaps` container the analytic `sample_magnet_field` produces, so a 3-D solve is a drop-in field source. `workflows.SolvedMouseField` wraps a solution, and `simulate_mouse_cpmg/mouse_depth_profile/resonant` now accept either bars (analytic, the unchanged default) or such a field source. The example `plot_nmr_mouse_depth_prof` drives the moving-walker depth-profiling measurement from the 3-D MOUSE solve — magnets and the finite iron yoke — and recovers a buried density gap as a hole in the profile, a higher-fidelity counterpart to the analytic image-yoke profile of Sec. 11.5.1. It also separates the two depth factors: the walker signal is the *intrinsic* slice response, which rises with depth (the slice thickness is pulse-bandwidth/gradient, and the weakening B_0 gradient thickens the slice – a larger excited sample volume), while the *measured* signal weights it by the geometric detection sensitivity $S(d) \sim B_0^2 B_1^2$ (Curie polarization, reciprocity reception, and transmit/receive B_1) computed from the solved fields; S falls $\sim 100\times$ over the first centimetre (surface-coil B_1 dominated), so the measured profile decays with depth as a real single-sided measurement does.

11.5.8 Coupled Thermal Modeling

RF and gradient coils dissipate power and warm up, RF pulses deposit power in conductive samples (SAR), and the resulting temperatures feed back into nearly every quantity the package computes: coil resistance $R(T)$ and Q (hence SNR and the probe transfer functions), Curie-law M_0, and the two-temperature noise models of Sec. 11.6.1. The `spin_dynamics.thermal` package adds a thermal solver alongside the other field solvers to close this loop; the physics options and the phased plan are in `docs/thermal_modeling.md`. The time-scale separation (RF microseconds versus thermal seconds-to-minutes) makes a quasi-static weak coupling exact for practical purposes.

Heat sources. `thermal.sources` converts electromagnetic quantities into duty-cycle-averaged powers: coil Joule heating $I^2R/2$ from the drive current and the coil resistance, RF duty cycles from the `pp tref/aref` arrays, gradient-waveform mean-square power, and sample SAR from the first-order eddy solver (`fields.quasistatic`) as either a volume-integrated power or a per-cell density map. A validation identity ties the two SAR views together: the volume-integrated eddy power equals the circuit-side $P = I_0^2 R_{\text{reflected}}/2$.

Lumped network (Option A). `ThermalNetwork` is a nodal RC network of heat capacities $C = \rho c_p V$ linked by conduction, convection, and grey-body radiation, matching the package's lumped-circuit philosophy. It

solves the steady state (linear, or damped Newton with radiation) and the transient warm-up:

```
from spin_dynamics.thermal import (
    ThermalNode, ThermalLink, ThermalNetwork, convection_conductance,
)

net = ThermalNetwork(
    nodes=[ThermalNode("coil", heat_capacity=2.0),
           ThermalNode("ambient", heat_capacity=None)],  # None = fixed bath
    links=[ThermalLink("coil", "ambient",
                       conductance=convection_conductance(50.0, 4e-3))],
    sources={"coil": 0.6},  # watts, or a DutyCycledSource
)
steady = net.steady_state()          # {"coil": ..., "ambient": ...}
```

Coupling loop (Option B, Phase 2). `ThermalCoupling` closes the feedback: coil resistance rises with coil temperature (skin-limited $R \propto \sqrt{\rho}$ or DC-linear), sample SAR and M_0 track the sample temperature, and the updated powers re-enter the thermal solve. `fixed_point()` returns the converged steady state (and detects thermal runaway when the tempco feedback exceeds the cooling), while `march()` time-steps the warm-up. `probe_updates()` hands back T, R, and Q for an sp mapping, so a cryoprobe's cold-coil / warm-sample state feeds the two-temperature noise density directly.

Electromagnet B_0 sources. For a resistive electromagnet the electrical and thermal problems cannot be separated over a long acquisition:

$$L\frac{dI}{dt} = V - IR(T), \qquad C_{\mathrm{th}}\frac{dT}{dt} = I^2R(T) + P_{\mathrm{loss}} - G_{\mathrm{th}}(T - T_{\mathrm{amb}}), \qquad B_0 = \left(\frac{B}{I}\right)_{\mathrm{coil}} I.$$

`ElectrothermalElectromagnet` integrates these states while the existing conductor model supplies $R(T) = R_{\mathrm{ref}}\rho(T)/\rho(T_{\mathrm{ref}})$. `from_segments()` calculates $(B/I)_{\mathrm{coil}}$ from an arbitrary Biot–Savart winding. The four control modes correspond to the feedback paths of the textbook electrothermal diagram: direct voltage, temperature-compensated voltage $V = I_{\mathrm{set}}R(T)$, current PI feedback, and a direct field lock.

```
from spin_dynamics.fields import coils
from spin_dynamics.thermal import (
    ElectromagnetControl, ElectrothermalElectromagnet,
)

paths = coils.solenoid(radius=0.12, length=0.24, turns=144)
magnet = ElectrothermalElectromagnet.from_segments(
    paths, inductance_h=25e-3, reference_resistance_ohm=0.20,
    heat_capacity_j_per_k=15e3, thermal_conductance_w_per_k=8.0,
)
result = magnet.simulate(
    times, 30e-3,
    control=ElectromagnetControl(
        "field", response_time_s=0.5, voltage_limits_v=(-18, 18)
    ),
)
hardware_b0 = result.uniform_b0()
motion_maps = result.to_motion_field_maps((x_axis, z_axis))
```

The result retains current, temperature, resistance, supply voltage, Joule power, deposited energy, and reference B_0 at every time. The two adapters hand the selected state to the existing uniform-hardware and spatial imaging/motion workflows. Figure 11.16 shows why this coupling matters: constant voltage loses field as copper warms, while all three feedback paths raise the voltage to hold B_0.

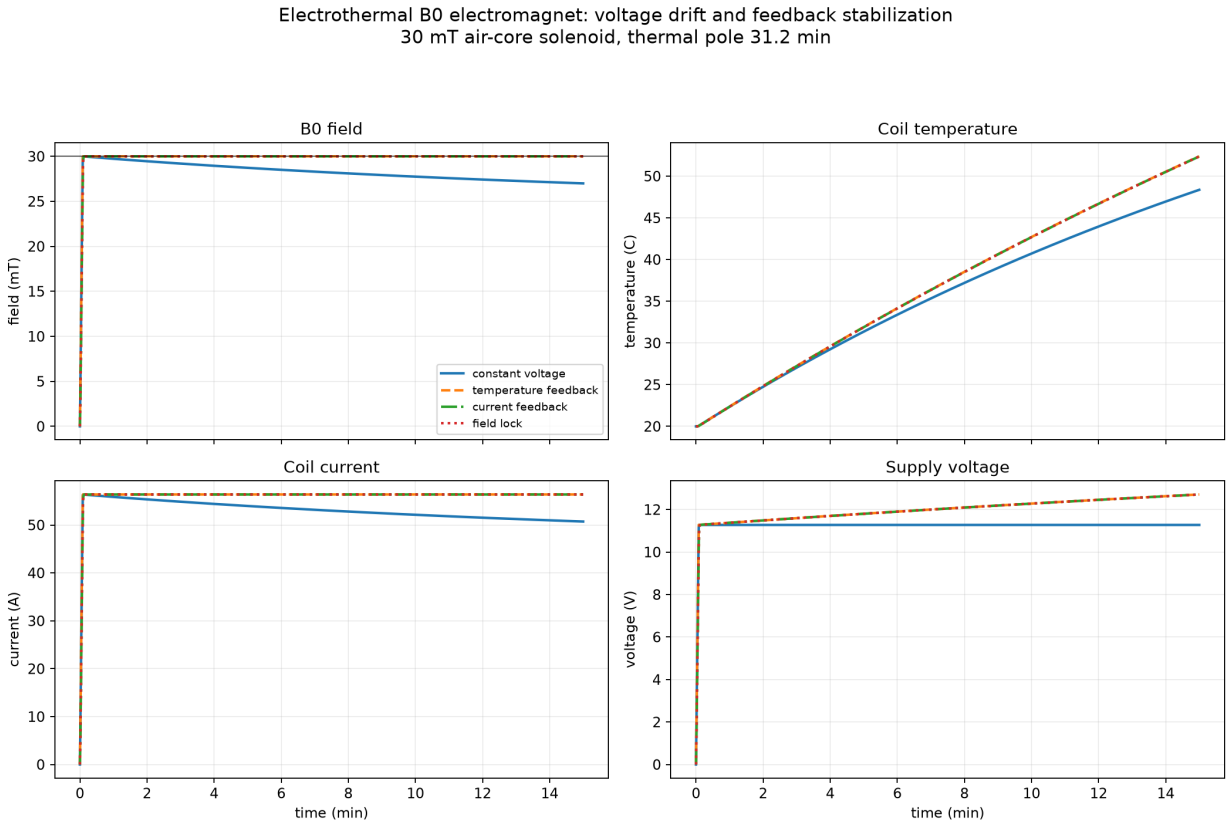


Figure 11.16: Electrothermal warm-up of a 30 mT air-core electromagnet. Constant voltage produces field and current droop as $R(T)$ rises. Temperature compensation, current feedback, and direct field lock hold B_0, at the cost of increased supply voltage and Joule power.

The core model is deliberately reduced order: a uniform coil temperature, one thermal pole, and linear B/I. Ferromagnetic hysteresis and saturation, frequency-dependent core loss, sensor dynamics, detailed regulator ripple, and thermo-mechanical expansion are not inferred. Measured core/eddy loss may be passed as `additional_power_w`; multi-node or spatial temperature models remain available through `ThermalNetwork` and `Conduction1D`.

Conduction profiles (Option B, Phase 3). `Conduction1D` solves the finite-volume heat equation on a slab, cylinder, or sphere with spatially varying properties, mixed boundary conditions, and an optional Pennes blood-perfusion sink $w_b c_b (T_a - T)$ for biological tissue — the internal profile (SAR hot-spot, coil-former stack) the lumped node cannot resolve.

Validation. Every piece is checked against analytic results (single-node exponentials, exact RL steps, electrothermal fixed points, slab/cylinder/sphere uniform-source center rises $qL^2/8k$, $qa^2/4k$, $qa^2/6k$, the perfusion penetration depth $\sqrt{k/w_b c_b}$, and the closed-form self-heating fixed point) and, where a field solution exists, cross-checked against FEMM axisymmetric heat-flow FEA (`validation/femm/`), which agrees to within half a percent of the temperature rise. The example `plot_nmr_mouse_thermal.py` runs a single-sided (NMR-MOUSE) relaxometry exam on perfused tissue: it reports the coupled coil temperature and $R(T)$, the SAR-attributable depth profile of the tissue heating (peak just below the skin, where surface cooling and perfusion compete against the surface-peaked deposition), and the warm-up over a ten-minute exam. The fully quantum measurement-back-action description of spin noise and CFD/flow solving of the convection remain out of scope.

11.5.9 Flowing Samples

Many measurements run on a *moving* sample – inline process monitoring in a pipe, stopped-flow kinetics, hyphenated LC-NMR. The `spin_dynamics.flow` package models how flow changes the measured signal, and the thermal package gains the matching advective-cooling terms; the design and validation are in `docs/flow_modeling.md`. A `FlowModel` describes flow through a cylindrical sensitive region in a `"plug"` (uniform velocity) or `"laminar"` (Hagen-Poiseuille parabolic profile) regime.

Washout. The sensitive volume is uniformly excited at $t = 0$; excited spins then leave and fresh unexcited spins enter, so the acquired signal is multiplied by the fraction $W(t)$ of originally-excited spins still present. Plug flow gives a linear cutoff $W = 1 - t/\tau$ (with $\tau = V/Q$); laminar flow follows the same initial slope to $t = \tau/2$ and then a slow $\tau/(4t)$ tail from the near-wall streamlines. Both share the early-time apparent rate $1/T_2 + 1/\tau$, so flow shortens the apparent T_2 of a CPMG train:

```
import numpy as np
from spin_dynamics.flow import FlowModel, apply_washout

flow = FlowModel(volumetric_flow_rate=3e-6, pipe_radius=4e-3,
                 sensitive_length=25e-3, regime="laminar")
techo = np.linspace(0.0, 1.0, 200)
flowing = apply_washout(flow, techo, np.exp(-techo / 0.4))  # T2 = 0.4 s
```

Inflow polarization. The signal amplitude is set by the longitudinal magnetization the spins built up during transit through the upstream prepolarizer. `inflow_polarization` returns the flux-weighted average, reusing the `spin_dynamics.prepolarization` recovery model: plug flow reduces to the single-transit `prepolarized_flow_state`, and laminar flow integrates over the parabolic profile (slower flow → longer transit → more polarization, saturating at the polarizer equilibrium).

Advective cooling. `thermal.flow_conductance` $G = \rho c_p Q$ is used as a link to a bath at the inlet temperature (steady rise $P/(\rho c_p Q)$; formally the Pennes perfusion sink), and `Conduction1D` accepts a

`velocity` that adds first-order upwind axial advection, whose steady profile matches the analytic advection-diffusion boundary layer $e^{\mathrm{Pe}\,x/L}$.

Everything is validated analytically and by Monte-Carlo (FEMM's heat solver does not model bulk advection): piecewise $W(t)$, the plug-limit equality of the polarization, the advection-diffusion profile to within a percent of span, and independent particle cross-checks of both the washout and the flux-weighted transit distribution. The example `plot_flowing_pipe_nmr.py` shows CPMG washout, apparent T_2 and amplitude versus flow rate, and advective cooling for an inline pipe monitor. Turbulent/well-mixed flow and analyte Taylor dispersion are out of scope.

11.6 Received-Signal Noise

Noise is opt-in and output-referred. High-level workflows accept a `NoiseSpec`, a mapping, or a scalar white-noise standard deviation:

```
from spin_dynamics.noise import NoiseSpec
from spin_dynamics.workflows import run_tuned_cpmg

noisy = run_tuned_cpmg(
    numpts=101,
    noise=NoiseSpec(model="probe", target_snr=25.0, seed=3),
)
```

`model="white"` adds flat complex receiver noise. `model="probe"` uses the probe output noise density where the workflow can supply it, which is important when the received spectrum is filtered by a tuned, untuned, or matched resonator. Result objects preserve clean deterministic fields and add noisy fields plus `NoiseMetadata` so repeated SNR studies remain reproducible.

11.6.1 Spin Noise

Beyond the coil and amplifier, the spins themselves are a noise source: the transverse magnetization of N spins fluctuates with RMS moment $\sqrt{N}\,\gamma\hbar/2$ (all spins, not the Boltzmann excess), with an exponential autocorrelation at T_2 and hence a Lorentzian power spectrum of width $1/(\pi T_2)$ centered on the Larmor frequency (Hoult & Ginsberg 2001; Field & Bain 2010; Annabestani et al. 2015 – see `docs/spin_noise.md` for the full analysis). Two physical models are provided; both derive their coupling strength from the same constant as radiation damping through $R_{n0} = R_{\mathrm{coil}} T_2/(2T_{\mathrm{rd}})$.

Sample properties – geometry, spin density, temperature, nucleus, relaxation – live on a `Sample` object, kept distinct from the detector/coil parameters so the sample and circuit can sit at different temperatures (`sp.T` remains the coil temperature):

```
from spin_dynamics.sample import cylinder_geometry, water_sample

sample = water_sample(cylinder_geometry(2.5e-3, 10e-3), temperature=300.0, t2=0.1)
sample.number_of_spins            # N from spin density and geometry
sample.magnetization_density(1.0) # Curie-law M0 at B0 = 1 T (A/m)
```

Frequency domain (circuit model). The sample presents the Hoult impedance $Z_n(\Delta\omega) = R_{n0}/(1 + j\,\Delta\omega T_2)$ in series with the receive coil. Passing a `SampleCoupling` to the probe noise densities inserts Z_n into the circuit and adds its Nyquist noise at the *sample* temperature. The on-resonance feature emerges from the circuit algebra: a dip at thermal equilibrium (the sample loads the resonant peak), deeper for a cold

sample, flipping to a bump for hot (or hyperpolarized-hot) spins, with radiation-damping line broadening and X_n frequency pulling included:

```
from spin_dynamics.noise import tuned_probe_output_noise_density
from spin_dynamics.spin_noise import sample_coupling_from_sample

coupling = sample_coupling_from_sample(
    sample, field_tesla=0.0235, fill_factor=0.6, inductance=10e-6,
)
pnoise, freqs = tuned_probe_output_noise_density(sp, pp, sample=coupling)
```

Time domain (stochastic source). `simulate_spin_noise` integrates the Bloch equations with a complex Ornstein–Uhlenbeck Langevin force on the transverse magnetization (spin bath at the sample temperature) plus a coil-temperature Johnson force, coupled to the probe through the same radiation-damping feedback state as the deterministic module. The coil-noise realization is shared between the spin drive and the receiver EMF, so the equilibrium dip / hot-spin bump appears in the simulated spectra; it also seeds maser ignition from zero transverse magnetization:

```
import numpy as np
from spin_dynamics.spin_noise import (
    estimate_spin_noise_spectrum,
    simulate_spin_noise,
    spin_noise_source_from_sample,
)

source = spin_noise_source_from_sample(sample, field_tesla=0.0235)
result = simulate_spin_noise(np.arange(0.0, 60.0, 5e-4), probe, source, seed=1)
freqs, psd = estimate_spin_noise_spectrum(result.emf, 5e-4, block_samples=4096)
```

The stochastic model is validated against the analytic linear-response spectrum (`spin_noise_output_psd`), the stationary variance, and the $R(t) = m_{\mathrm{rms}}^2 e^{-t/T_2}$ autocorrelation; `examples/plot_spin_noise_spectrum.py` reproduces the dip/bump phenomenology and a spin-noise-seeded maser. The quantum measurement-back-action description of spin noise (weak vs. strong readout) remains out of scope.

11.7 Non-Inductive Detection: SQUID and OPM Magnetometers

The probe noise model above describes a conventional inductive (Faraday) coil, whose observable is $\mathrm{d}\Phi/\mathrm{d}t$ and whose noise is the coil Johnson noise. At ultra-low field, and for NQR, the enabling detectors are instead *field* sensors—SQUIDs and optically pumped (atomic) magnetometers (OPMs)—which respond to B itself with a frequency-flat noise floor. The `spin_dynamics.detection` package models all three on one axis by referring signal and noise to *magnetic field at the sensor* (T, $\mathrm{T}/\sqrt{\mathrm{Hz}}$). A detector is then a transfer shape $H(f)$ plus a field-referred noise PSD $N^2(f)$, and the matched-filter detected SNR of a field-at-sensor spectrum $S(f)$ is detector-agnostic:

$$\mathrm{SNR}^2 = \int \frac{|S(f)|^2}{N^2(f)}\,\mathrm{d}f.$$

The field $S(f)$ that the precessing sample presents at the pickup is the same for every detector; all detector differences live in N^2. A Faraday coil's field-referred noise *rises* as $1/f$ toward low frequency (its responsivity $H \propto f$ shrinks), so $\int |S|^2/N^2$ reproduces the familiar B_0^2 signal collapse automatically, whereas an untuned SQUID flux transformer has a *flat* N^2. Referring the existing inductive probe density to field via $N^2_{\mathrm{field}} = N^2_{\mathrm{out}}/|H|^2$ leaves the matched-filter SNR unchanged, so the layer reproduces the tuned/ untuned/matched probe numbers exactly (`InductiveCoilDetector`).

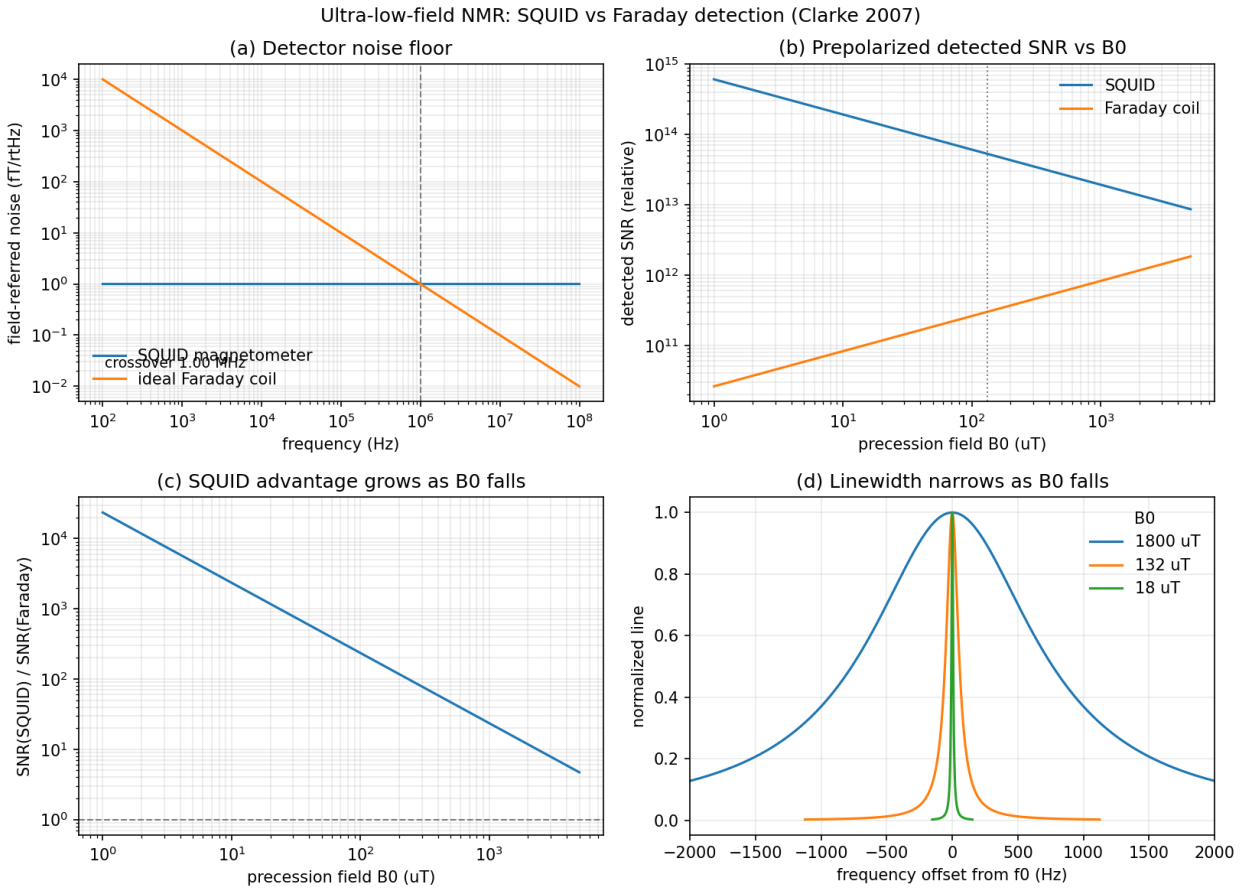


Figure 11.17: SQUID and ideal Faraday detection on a common field-referred scale. The frequency-independent SQUID noise floor wins below the detector crossover; for a prepolarized sample its relative advantage grows as B0 falls while the homogeneous line narrows.

```
import numpy as np
from spin_dynamics.detection import (
    SQUIDMagnetometer, IdealFaradayCoil, detected_field_snr,
)

squid   = SQUIDMagnetometer.berkeley_2007()          # flat 1 fT/rtHz floor
faraday = IdealFaradayCoil(field_noise_T_per_rtHz_ref=1e-15, f_ref_hz=1e6)

f = np.linspace(-500.0, 500.0, 2001)                 # a line at baseband
s = (5.0/np.pi) / (f**2 + 5.0**2)                    # unit-area Lorentzian
snr_squid   = detected_field_snr(s, f, squid).snr    # flat floor
snr_faraday = detected_field_snr(s, f, faraday).snr  # 1/f floor
```

Following Clarke, Hatridge & Mößle (*Annu. Rev. Biomed. Eng.* **9**, 389, 2007), the untuned SQUID floor sits near $1\ \mathrm{fT}/\sqrt{\mathrm{Hz}}$ and is frequency-independent, so it crosses the coil's rising $1/f$ floor at a few MHz: below the crossover the SQUID wins, and with a prepolarized sample (moment set by B_p, not B_0) the detected SNR *grows* as B_0 falls ($\mathrm{SNR} \sim B_0^{-1/2}$ once the line has narrowed to the homogeneous T_2 limit), while the Faraday SNR falls as $B_0^{+1/2}$. The example `examples/plot_squid_ulf_crossover.py` reproduces the noise-floor crossover, the prepolarized SNR-vs-B_0 scalings, and the B_0-proportional line narrowing.

Run `python examples/plot_squid_ulf_crossover.py -save squid_ulf.png` to reproduce Figure 11.17.

Optically pumped (atomic) magnetometers (`OPMMagnetometer`) are field sensors like SQUIDs but reach sub-fT/$\sqrt{\text{Hz}}$ sensitivity without cryogenics, at the cost of a finite atomic-response bandwidth. The vapour-cell spins act as a Lorentzian filter of half-width Δf, so the field-referred noise is flat on resonance and rises as the response rolls off, $N^2(f) = S_0^2\,[1+((f-f_c)/\Delta f)^2]$. A `"serf"` (zero-field) OPM is a low-pass centred at DC—best for zero/ultra-low-field NMR but rolled off, and useless, at MHz; an `"rf"` (Mx) OPM tunes its bias field to place a band-pass on the line f_c, reaching $\sim 0.24\ \text{fT}/\sqrt{\text{Hz}}$ at 423 kHz for the ^{14}N NQR of ammonium nitrate (Savukov et al.; US 7,521,928). Modelling the roll-off is essential: a SERF OPM's field noise at 423 kHz is $\sim f/\Delta f$ (thousands of times) above its DC floor, and the RF-OPM's bandwidth must cover the NQR linewidth or the detected SNR degrades. The example `examples/plot_opm_nqr_detection.py` compares an RF-OPM, a SERF OPM, a SQUID, and a coil on a ^{14}N line and shows the bandwidth-matching penalty.

Where a detector sets *how quiet* the sensor is, a *pickup geometry* (`Gradiometer`) sets *where* it is sensitive. SQUID systems commonly wind the flux transformer as an axial gradiometer—coaxial loops with alternating turns—so a uniform ambient field (and, at second order, a uniform gradient) cancels while a nearby sample still couples. By reciprocity a pickup's receive sensitivity to a source at r equals the field it would transmit there per unit current, so the net sensitivity is the signed sum of loop fields $S(r) = \sum_i w_i B_{\text{loop},i}(r)$, evaluated by reusing the `fields.magnetostatics` loop-field routines. A dipole field and its first and second axial derivatives fall off as $1/r^3$, $1/r^4$, $1/r^5$, so an order-n gradiometer's distant-source sensitivity falls as $1/r^{3+n}$ and it rejects uniform ($n \geq 1$) and first-gradient ($n \geq 2$) ambient noise—the basis of the $\sim 1000\times$ common-mode rejection in Clarke's microtesla MRI. A pickup attaches to a field sensor as `SQUIDMagnetometer(..., pickup=g)` (or `OPMMagnetometer(..., pickup=g)`), and `detected_field_snr_spatial` then routes a distributed sample through the pickup coupling for the signal while ambient sources (`AmbientFieldSource`) couple through the *same* pickup for the noise—so a uniform ambient field (coupling 1 for a magnetometer, 0 for a balanced gradiometer) is rejected exactly as a real gradiometer would. The example `examples/plot_gradiometer_sensitivity.py` maps the reciprocal (excited) region for a magnetometer and first/second-order gradiometers, shows how winding order localizes it, and prints the SQUID detected SNR with and without a uniform ambient field (the magnetometer loses $\sim 1000\times$, the gradiometers are unaffected). Detector-aware GRAPE objectives follow in a later increment (see `docs/non_inductive_detection.md`).

11.8 Diffusion

```
from spin_dynamics.workflows import run_matched_diffusion_q_sweep

sweep = run_matched_diffusion_q_sweep(
    q_values=[20, 50],
    num_echoes=3,
    numpts=21,
    dz=50e-6,
    auto_refine_grid=True,
)
```

The compact matched-probe diffusion workflow is solver-validated through $Q = 2000$. Higher-Q cases remain a stiffness target for the current NumPy matched-probe transient solver.

Matched diffusion CPMG also accepts `absolute_phase`. In this combined path, the diffusion-encoding π pulse and the CPMG refocusing pulses are solved at their laboratory-frame RF phase, while the free-

precession intervals retain the diffusion attenuation:

```
from spin_dynamics.workflows import run_matched_diffusion_cpmg

combined = run_matched_diffusion_cpmg(
    num_echoes=16,
    echo_spacing_seconds=1.0e-3,
    diffusion_time=1.0e-3,
    q_value=50,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 1.0e-3,
        "phase_bins": 16,
    },
)

metadata = combined.absolute_phase
encoding_phase = metadata.encoding_absolute_phase_rad
refocus_phases = metadata.refocus_absolute_phase_rad
```

`metadata.refocus_*` describes the echo-train refocusing pulses. `metadata.encoding_*` describes the diffusion-encoding π pulse, and `pulse_*` fields provide the full RF event list.

The comparison plotting example toggles diffusion and absolute phase independently:

```
python examples\plot_diffusion_absolute_phase_compare.py
python examples\plot_tuned_diffusion_absolute_phase_compare.py
```

The diffusion plotting examples use a narrow default `-dz-um` and enable automatic offset-grid refinement so the displayed decay is not an artifact of discrete-grid rephasing. They print the effective number of offsets after any refinement. Use `-no-auto-refine-grid` only when intentionally testing the rephasing guard. The matched diffusion comparison also prints the matched-probe absolute-phase residual and warns when it is below the plotting tolerance. The current matched pulse-shape solver is often nearly phase-invariant; in that case the plot is a diagnostic limitation rather than evidence for a measurable combined effect. The tuned-probe comparison uses the same diffusion kernel with tuned pulse-shape solves and usually shows a much larger residual, making it the better contrast example for phase-sensitive probe transients.

11.8.1 PGSE and PGSE-Prepared CPMG

For Stejskal-Tanner pulsed-gradient diffusion encoding, use the PGSE workflow helpers. The deterministic backend tracks the effective gradient moment and therefore evaluates

$$b = \int q(t)^2 \, \mathrm{d}t, \qquad q(t) = \gamma \int_0^t G_{\mathrm{eff}}(t') \, \mathrm{d}t'.$$

For a rectangular PGSE pair this reduces to

$$b = (\gamma G \delta)^2 \left(\Delta - \frac{\delta}{3} \right),$$

where δ is the lobe duration and Δ is the leading-edge separation between the two gradient lobes. The physical lobes are scheduled with the same polarity around the refocusing pulse; the 180-degree pulse flips the coherence-frame sign, so stationary spins refocus while diffusive motion attenuates the signal.

```
from spin_dynamics.workflows import run_pgse_moment
```

```
pgse = run_pgse_moment(
    num_echoes=8,
    gradient_amplitude=0.12,      # T/m
    gradient_duration=2.5e-3,     # delta
    diffusion_time=28.0e-3,       # Delta
    diffusion_coefficient=1.2e-9,
    first_echo_time_seconds=56e-3,
    echo_spacing_seconds=8e-3,
    t2_seconds=80e-3,
)
```

`num_echoes=1` is the ordinary spin-echo PGSE acquisition. Larger echo counts represent a PGSE preparation followed by a compact CPMG acquisition. For spatially explicit studies, `run_pgse_walkers` uses the moving-isochromat sequence driver and seeded Brownian walkers. It is slower and stochastic, but can be extended to inhomogeneous field maps, boundaries, and advection:

```
from spin_dynamics.workflows import run_pgse_walkers

walkers = run_pgse_walkers(
    gradient_amplitude=0.05,
    gradient_duration=2.0e-3,
    diffusion_time=16.0e-3,
    diffusion_coefficient=2.3e-9,
    walkers_per_cell=12000,
    seed=123,
)
```

The PGSE tests compare the moment backend to the analytical Stejskal-Tanner formula, and compare the walker backend against the same attenuation through a zero-diffusion walker reference.

11.8.2 Restricted Diffusion and Diffusive Diffraction

Because the walker backend integrates explicit displacements, it can model diffusion restricted by hard walls or exchanged through semi-permeable membranes—physics the closed-form moment backend cannot express. The `boundary` argument of `run_pgse_walkers` accepts the rectangular-box modes `"reflect"`, `"periodic"`, and `"clip"`, or any callable mapping particle positions to confined positions. The helpers `make_circular_reflector(center, radius)` and `make_elliptical_reflector(center, semi_axes)` supply reflecting circular and elliptical pore walls.

For a slab pore of width L along the gradient axis, confining the walkers makes the echo attenuation bend above the free Stejskal-Tanner line $E = \exp(-bD)$: once the diffusion length $\sqrt{2D\Delta}$ approaches L the spins can no longer fully dephase, and the apparent diffusion coefficient $D_{\mathrm{app}} = -\ln(E)/b$ falls below the bulk value D as the diffusion time grows. The example `plot_pgse_restricted_diffusion.py` sweeps several pore widths and diffusion times to show both signatures.

In the long-mixing, narrow-gradient-pulse limit, the signal can become non-monotonic rather than merely bending away from free diffusion. The pore form factor produces diffraction minima whose reciprocal-space positions encode the confining length scale.

Reproduce Figure 11.18 quickly with:

```
python examples/plot_pgse_circular_pore_diffraction.py \
  --method sgp-propagator --output pore.png
```

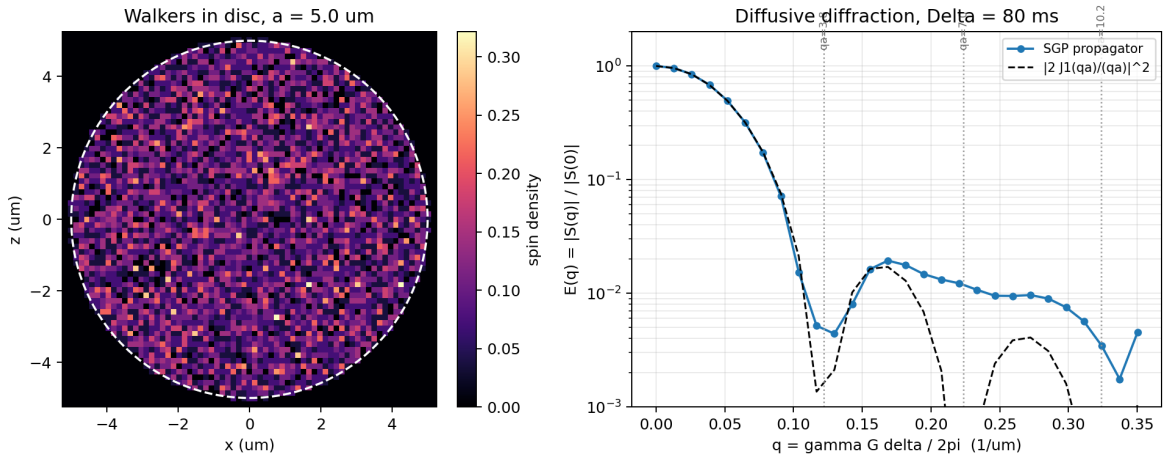


Figure 11.18: Restricted random walkers in a 5 μm circular pore (left) and the resulting PGSE q-space diffraction pattern (right). The propagated echo follows the squared disc form factor through the first minimum; free diffusion would instead decay monotonically toward zero.

Use `walker-sweep` to include finite-pulse effects at higher computational cost.

11.8.3 Large 3D Porous-Rock Walker Challenge

The example `porous_rock_cpmg_walkers.py` is a heavier end-to-end stress test for the moving-walker machinery and the optional JAX/Numba walker backends. It builds a cylindrical rock-core analogue with a multimodal pore-body size distribution, connected throat voxels, constriction-dependent tortuosity, surface-relaxation T_2, and a susceptibility-like internal B_0 map. The sequence is a CPMG echo train with millions of walkers moving through the voxel pore space, rejecting proposed steps into solid voxels.

Before the expensive propagation begins, the script prints geometry-only estimates for the expected distributions:

$$\frac{1}{T_2(\mathbf{r})} = \frac{1}{T_{2,\mathrm{bulk}}} + \frac{3\rho_2}{a_{\mathrm{relax}}(\mathbf{r})}, \qquad D(\mathbf{r}) = \frac{D_0}{\tau(\mathbf{r})}.$$

Here a_{relax} is reduced near walls and in throats, while τ increases in constricted regions. The printed percentiles are a useful analytical check on the final D-T_2 plot: a healthy large run should not collapse to a single D, T_2 point.

The default challenge is intended for CUDA-enabled JAX and writes a four-panel verification figure containing a sampled 3D pore structure, the pore-weighted D-T_2 histogram, the CPMG decay, and the input pore-size distribution:

```
XLA_PYTHON_CLIENT_PREALLOCATE=false python examples/porous_rock_cpmg_walkers.py \
  --backend jax \
  --plot-output results/porous_rock_challenge.png \
  --output results/porous_rock_challenge.npz
```

Use `-estimate-only` to inspect the pore-space, D, T_2, and rough runtime estimates without launching the walker propagation. Use `-grid`, `-z-cells`, `-pores`, `-walkers-per-voxel`, `-num-echoes`, and `-substeps` to scale the calculation down for smoke tests or up for production benchmarks.

In a closed two-dimensional pore the restricted signal becomes non-monotonic. In the narrow-pulse, long-mixing limit ($\delta \ll a^2/D \ll \Delta$) the normalized echo approaches the squared magnitude of the pore form

factor, the *diffusive diffraction* regime of q-space NMR. For a uniform disc of radius a,

$$E(q) \;\longrightarrow\; \left| \frac{2\,J_1(q_{\mathrm{ang}} a)}{q_{\mathrm{ang}} a} \right|^2 ,$$

with minima at the zeros of J_1, i.e. $q_{\mathrm{ang}} a = 3.83,\ 7.02,\ 10.17, \ldots$. Note the factor-of-$2\pi$ ambiguity in the definition of q: the *angular* wavenumber is $q_{\mathrm{ang}} = \gamma G \delta$ (units $\mathrm{rad/m}$, with $b = q_{\mathrm{ang}}^2(\Delta - \delta/3)$), while the *reciprocal-space* (Callaghan) convention uses $q = \gamma G \delta/(2\pi)$ (units $1/\mathrm{m}$, encoding kernel $\exp(i 2\pi q x)$), for which the first disc feature sits near $qa \sim 1$.

```
import numpy as np
from spin_dynamics.motion import make_circular_reflector
from spin_dynamics.workflows import run_pgse_walkers

radius = 5.0e-6
axis = np.linspace(-radius, radius, 21)
xx, zz = np.meshgrid(axis, axis, indexing="ij")
rho = (xx**2 + zz**2 <= radius**2).astype(float)  # uniform disc

walkers = run_pgse_walkers(
    rho=rho, x_axis=axis, z_axis=axis,
    gradient_amplitude=2.0,        # large G reaches the q-space regime
    gradient_duration=0.4e-3,      # short delta (narrow-pulse limit)
    diffusion_time=80.0e-3,        # long Delta >> a^2 / D
    diffusion_coefficient=2.3e-9,
    boundary=make_circular_reflector((0.0, 0.0), radius),
    walkers_per_cell=28, substeps_per_interval=80, seed=2026,
)
```

The example `plot_pgse_circular_pore_diffraction.py` plots the resulting fringes against the Bessel form factor. It offers two methods. The default `walker-sweep` re-runs the confined PGSE simulation at every gradient and so captures finite-pulse blurring of the minima. The optional `-method sgp-propagator` exploits the fact that the walker trajectories are independent of q: it runs the confined diffusion once, records each walker's net displacement Δx over the diffusion time, and evaluates $E(q) = \left|\langle \exp(i q_{\mathrm{ang}} \Delta x) \rangle\right|$ for all q in a single vectorized sum. This is the average-propagator (q-space) method; it is tens of times faster and reproduces the same diffraction pattern in the $\delta \to 0$ limit. Keep the per-substep hop $\sqrt{2D\,\mathrm{d}t}$ well below the pore size so reflection stays accurate.

The inverse problem is exposed separately in `spin_dynamics.workflows.qspace`. Use `qspace_axes_from_real_space` and `pore_form_factor_from_density` to build ideal q-space responses for test phantoms. `reconstruct_qspace_image` directly inverts a complex form factor into pore density, but a conventional diffraction intensity $|F(q)|^2$ inverts to the Patterson/autocorrelation image, not a unique pore. When a loose support is known, `phase_retrieve_qspace_magnitude` performs support-constrained, non-negative HIO/error-reduction phase retrieval to estimate the pore shape from magnitude-only q-space data. The example `plot_pgse_qspace_pore_imaging.p` shows all three panels: the diffraction response, the autocorrelation, and the recovered pore estimate. It also adds a finite-SNR branch by perturbing the normalized q-space intensity with additive Gaussian noise (`-snr` is the zero-q peak divided by the noise standard deviation) and running the same constrained retrieval on the noisy data.

For realistic finite rectangular pulses, use `acquire_pgse_qspace_walkers`. It converts every angular-q vector to the in-plane gradient $\mathbf{G} = \mathbf{q}/(\gamma\delta)$, runs the explicit PGSE walker sequence with a common seed across the grid, and divides by an acquired zero-q echo. The resulting response includes finite-pulse edge averaging and incomplete pore mixing; its `intensity` property removes the small imaginary and negative Monte Carlo components before magnitude-only inversion. `plot_pgse_qspace_walkers.py` validates this

handoff on an elliptical reflecting pore and shows the finite-pulse grid beside its short-pulse limit and recovered pore estimate. As in all walker calculations, convergence must be checked against walker count and boundary substeps.

For slow exchange between two compartments, use `make_semipermeable_plane`. The membrane is an internal line $x = x_m$ or $z = z_m$ inside the rectangular bounds. A walker that crosses the line transmits with probability

$$P_{\text{transmit}} = 1 - \exp(-k\,\Delta t),$$

where k is `exchange_rate`; otherwise the walker reflects from the membrane. This rate form stays consistent when `substeps_per_interval` is changed.

```
import numpy as np
from spin_dynamics.motion import make_semipermeable_plane
from spin_dynamics.workflows import run_pgse_walkers

x = np.linspace(-10e-6, 10e-6, 41)
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))
membrane = make_semipermeable_plane(
    0.0,
    exchange_rate=25.0,  # s^-1 in this seconds-based workflow
    axis="x",
)

walkers = run_pgse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=0.2, gradient_duration=1.0e-3,
    diffusion_time=40.0e-3,
    diffusion_coefficient=2.0e-9,
    boundary=membrane,
    walkers_per_cell=64, substeps_per_interval=16, seed=2026, jitter=True,
)
```

Use `exchange_rate=0` for an impermeable internal wall and `np.inf` for a freely transmitting interface. The outer rectangular boundary still defaults to reflection, so the exchanging compartments remain confined by the sample box.

11.8.4 Stimulated-Echo PGSE (PGSTE)

`run_pgste_walkers` splits the diffusion encoding across three 90-degree pulses, $90 - G(\delta) - 90 - [\text{storage}] - 90 - G(\delta) - \text{echo}$. The second pulse stores one quadrature of the encoded magnetization along the longitudinal axis, so during the (long) storage interval it decays with T_1 rather than T_2. This is why PGSTE is the method of choice for slow diffusion, large pores, and short-T_2 samples such as porous media, low field, and internal-gradient regimes. A spoiler gradient applied during storage crushes the residual transverse coherences. The workflow records an effective selected-pathway phase table and suppresses equilibrium recovery into a contaminating FID, so the surviving stimulated echo follows the Stejskal-Tanner attenuation at *half* the spin-echo amplitude:

$$E(b) \;\longrightarrow\; \tfrac{1}{2}\,\exp\!\left(-\frac{T_s}{T_1}\right)\exp(-bD),$$

where T_s is the storage interval. The argument `diffusion_time` is the leading-edge separation of the two gradient lobes, so $b = (\gamma G \delta)^2(\Delta - \delta/3)$ still holds, and $T_s = \Delta - \delta - 2\,$`encode_delay` $- 2\,$`excitation_duration`.

```
import numpy as np
from spin_dynamics.workflows import run_pgste_walkers

axis = np.linspace(-1.0e-3, 1.0e-3, 64)   # wide slab (see note below)
rho = np.ones((axis.size, 2))

ste = run_pgste_walkers(
    rho=rho, x_axis=axis, z_axis=np.array([-1e-6, 1e-6]),
    gradient_amplitude=0.1, gradient_duration=1.0e-3,  # delta
    diffusion_time=60.0e-3,            # Delta = lobe leading-edge separation
    diffusion_coefficient=2.3e-9,
    t1_seconds=0.4, t2_seconds=15.0e-3,  # storage decays with T1, not T2
    walkers_per_cell=64, seed=2026, jitter=True,
)
ste.phase_cycle.name   # "pgste_stimulated_echo"
```

Use a sample wide compared with the gradient phase wavelength $\pi/(\gamma G\delta)$ so the unwanted stimulated anti-echo dephases spatially; with diffusion present it is suppressed further. The example `plot_pgste_stimulated_echo.py` verifies the half-amplitude Stejskal-Tanner attenuation and contrasts the T_2-limited spin echo with the T_1-limited stimulated echo as the diffusion time grows.

11.8.5 Double Diffusion Encoding (DDE / Double-PGSE)

`run_dde_walkers` applies two refocused PGSE blocks separated by a mixing time, $90 - [G_1\,\text{block}] - \text{mixing} - [G_2\,\text{block}] - \text{echo}$, encoding displacement along `angle1` and `angle2`. Single-PGSE measures the diffusion tensor, so in an orientationally disordered sample of anisotropic compartments it sees only the isotropic orientation average. Sweeping the angle ψ between the two encoding directions instead produces a $\cos 2\psi$ modulation of the echo whose amplitude reports the *microscopic* anisotropy of the local geometry; because it depends only on the relative angle, it survives powder averaging and reveals shape anisotropy that single-PGSE cannot. Paired with `make_elliptical_reflector`, an elliptical pore shows the $\cos 2\psi$ term while an equal-area circular pore does not. The $\cos 2\psi$ term is a higher-order (q^4) effect, so strong diffusion weighting is needed to resolve it in the powder average.

```
import numpy as np
from spin_dynamics.motion import make_elliptical_reflector
from spin_dynamics.workflows import run_dde_walkers

semi_axes = (8.0e-6, 3.0e-6)
x = np.linspace(-semi_axes[0], semi_axes[0], 15)
z = np.linspace(-semi_axes[1], semi_axes[1], 15)
xx, zz = np.meshgrid(x, z, indexing="ij")
rho = ((xx / semi_axes[0]) ** 2 + (zz / semi_axes[1]) ** 2 <= 1.0).astype(float)

dde = run_dde_walkers(
    rho=rho, x_axis=x, z_axis=z,
    gradient_amplitude=1.0, gradient_duration=1.0e-3,
    diffusion_time=12.0e-3, mixing_time=1.0e-3,
    angle1=0.0, angle2=np.pi / 2,          # vary to trace E(psi)
    diffusion_coefficient=2.0e-9,
    boundary=make_elliptical_reflector((0.0, 0.0), semi_axes),
    walkers_per_cell=64, substeps_per_interval=10, seed=2026, jitter=True,
)
```

The example `plot_pgse_double_encoding_elliptical_pore.py` traces $E(\psi)$ for the ellipse and the equal-area circle, both at a single orientation and powder-averaged, and reports the fitted $\cos 2\psi$ amplitude.

11.8.6 Oscillating-Gradient Spin Echo (OGSE)

PGSE varies the diffusion time Δ; `run_ogse_walkers` instead varies the oscillation frequency of a cosine-modulated gradient. Each diffusion-encoding lobe is a waveform $G\cos(2\pi ft)$ of `num_periods` whole periods, applied symmetrically around the refocusing pulse, $90-\cos$ lobe $-180-\cos$ lobe $-\mathrm{echo}$. Because each lobe spans an integer number of periods, its zeroth gradient moment vanishes and stationary spins refocus. The encoding power is concentrated at the angular frequency $\omega = 2\pi f$, so sweeping `oscillation_frequency` maps the diffusion spectrum $D(\omega)$ and reaches the short-diffusion-time regime that ordinary PGSE cannot. The waveform is discretized into `samples_per_period` constant-gradient steps, and the b-value follows the cosine spectrum $b = (\gamma G/\omega)^2 N/f$, which falls steeply with frequency.

```
import numpy as np
from spin_dynamics.motion import make_motion_field_maps_2d
from spin_dynamics.workflows import run_ogse_walkers

x = np.linspace(-2.5e-6, 2.5e-6, 15)         # 5 um reflecting slab along x
z = np.array([-0.5e-6, 0.5e-6])
rho = np.ones((x.size, z.size))

ogse = run_ogse_walkers(
    rho=rho, x_axis=x, z_axis=z,
    fields=make_motion_field_maps_2d(x, z),  # walls = the slab
    gradient_amplitude=0.5, oscillation_frequency=200.0,  # sweep this
    num_periods=2, diffusion_coefficient=2.0e-9,
    walkers_per_cell=160, substeps_per_interval=6, seed=2026, jitter=True,
)
d_app = -np.log(abs(ogse.signal[0]) / rho.sum()) / ogse.b_value  # D(omega)
```

In restricted geometry D_{app} rises from the long-time (tortuosity) value toward the bulk value as the frequency increases, and a smaller pore stays restricted to higher frequency (transition $f_c \sim D/L^2$). The example `plot_ogse_frequency_diffusion.py` sweeps the frequency for free diffusion and two reflecting slab widths.

11.9 Moving Isochromats

The motion layer treats inhomogeneity in the same spirit as static isochromats, but the samples move through two-dimensional B0, transmit-B1, and receive-B1 maps:

```
from spin_dynamics.motion import make_motion_field_maps_2d,
    initialize_ensemble_from_density
from spin_dynamics.sequences.motion import (
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)

fields = make_motion_field_maps_2d([-1, 0, 1], [-1, 0, 1])
ensemble = initialize_ensemble_from_density([[0, 1, 0], [1, 2, 1], [0, 1, 0]])
motion = run_motion_cpmg_sequence(
```

```
    ensemble,
    fields,
    num_echoes=4,
    echo_spacing=1e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
    gradient=(0.0, 0.1),
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=4,
    total_duration=4e-3,
    excitation_duration=20e-6,
    refocusing_duration=40e-6,
    gradient=(0.0, 0.1),
)
```

Lower-level helpers such as `free_precession_with_motion_step` are useful for custom trajectories. The sequence helpers split long intervals into substeps so field maps are sampled along the particle path rather than only at the beginning of an interval.

The same machinery handles deterministic flow as well as diffusion. The example `plot_cpmg_pipe_flow.py` seeds an axisymmetric cylindrical pipe, assigns a laminar Poiseuille velocity profile, computes upstream polarization from residence time in a static magnet, and then runs a downstream CPMG train through spatial transmit/receive coil maps. Increasing the mean flow velocity lowers the initial longitudinal magnetization and adds echo-train dephasing as spins move through the B0 and B1 maps during the sequence.

The engine is not limited to two spatial axes. `make_motion_field_maps` and `initialize_ensemble_from_domain` build one-, two-, or three-dimensional `MotionFieldMaps` and walker ensembles over a `SpatialDomain`, and a `MotionSequenceStep` accepts a gradient vector of matching length, so the same precession/advection/diffusion machinery runs in 3D. This is what the true-3D slice-selective imaging workflow uses; see *Slice-Selective Excitation and 3D Multi-Slice Imaging*.

The motion sequence runners also accept `detuning_waveform`. The callable may return a scalar B0 offset for the whole ensemble or one value per particle. This makes it possible to model slow fluctuating gradients or local bath fields:

```
import numpy as np

fluctuating_gradient = lambda time, positions: (
    1500.0 * positions[:, 0] * np.cos(2 * np.pi * 0.35 * time)
)

udd = run_motion_udd_sequence(
    ensemble,
    fields,
    num_pulses=8,
    total_duration=0.72,
    excitation_duration=0.25e-3,
    refocusing_duration=0.5e-3,
    t1=5.0,
    t2=1.0,
    detuning_waveform=fluctuating_gradient,
```

```
    substeps_per_interval=8,
)
```

This is the more realistic regime where UDD can outperform evenly spaced CPMG: the noise is low-frequency and correlated across time, but varies across the sample so residual phase spread reduces the received signal. In contrast, ordinary unrestricted diffusion in a static gradient usually gives very similar CPMG and UDD attenuation.

11.10 Time-Varying Fields

```
from spin_dynamics.workflows import (
    sinusoidal_field_waveform,
    run_ideal_time_varying_amplitude_sweep,
)

waveform = sinusoidal_field_waveform(num_echoes=16)
sweep = run_ideal_time_varying_amplitude_sweep(
    amplitudes=[0, 0.5, 1.0, 2.0],
    waveform=waveform,
    numpts=101,
    auto_refine_grid=True,
)
```

11.11 WURST

```
from spin_dynamics.workflows import (
    run_ideal_wurst_inversion,
    run_matched_wurst_cpmg,
)

ideal = run_ideal_wurst_inversion(numpts=101)
matched = run_matched_wurst_cpmg(num_echoes=2, numpts=51, num_steps=64)
```

11.12 Nonresonant Field-Reversal Echoes

The `spin_dynamics.nonresonant` package models *nonresonant* NMR, in which nuclear magnetization is manipulated without RF — by suddenly switching and adiabatically rotating applied magnetic fields (Brill *et al.*, *Science* **297**, 369, 2002). Because the manipulation is a field reversal rather than a resonant pulse, it excites spins with an efficiency independent of the Larmor frequency (useful for ex-situ sensors in grossly inhomogeneous fields), but a non-reversible background field — typically Earth's field — survives the reversals and dephases the echoes within a few milliseconds. A non-interacting spin-1/2 is a classical Bloch 3-vector, so the engine propagates an ensemble of isochromats by exact rotations about the instantaneous local field.

The efficiency trick is that a constant-field segment (a "hold", including free precession) is a *single* exact rotation by $\gamma|B|\tau$ about the field direction, with no Larmor-timescale time stepping; only the adiabatic rotations and finite-duration reversals are sub-stepped, and only finely enough to resolve the (slow) field trajectory.

```
from spin_dynamics.nonresonant import (
    NonresonantFieldModel, sample_isochromats,
    csar_sequence, simulate_field_reversal_echoes,
)

model = NonresonantFieldModel(
    coil_a_tesla=2.14e-3, coil_b_tesla=2.14e-3, background_tesla=50e-6)
ens = sample_isochromats(model, 500, b_inhomogeneity=0.25, direction_tilt_deg=18.0)
unit = csar_sequence(model, echo_spacing_seconds=2.5e-3, tau_rev_seconds=8e-6)
result = simulate_field_reversal_echoes(model, ens, unit, num_echoes=256,
                                        t2_seconds=510e-3)
```

The Combined Sudden switching And adiabatic Rotation (CSAR) sequences suppress the background-field dephasing. `csar_sequence` is the 90-degree unit whose echo-to-echo propagator is a π rotation, refocusing every *second* echo (the odd echoes stay near zero and the even echoes carry a $\sim 50\%$ component that decays with T_2 plus a $\sim 50\%$ component with a Bessel-like odd-even modulation set by the finite reversal time). `csar_double_reversal_sequence` gives a 2π rotation that refocuses *both* parities (with an enhanced decay), and `csar_supercycle_sequence` alternates the adiabatic-rotation sense over a four-unit supercycle to suppress the imperfection decay down to the intrinsic T_2 limit. `effective_rotation` extracts the net-rotation axis and angle (the paper's analysis tool), and `sequence_waveform` returns the coil-current drive waveform for plotting.

```
python examples/plot_brill2002_field_reversal_echoes.py --output brill2002.png
```

The example reproduces the paper's key results: the basic reversal sequence decaying within tens of milliseconds from Earth's field, the CSAR train sustaining hundreds of echoes out to $T_2 = 510\,\text{ms}$ with the odd-even modulation, an accurate fitted T_2, and the sequence comparison in which the supercycle reaches the T_2-limited decay. (The 90-degree waveform follows the paper's Fig. 1D directly; the 2π and supercycle sequences here are built from the same primitives and reproduce the same effects, but their exact coil waveforms differ from the paper's specific Fig. 3B/3C implementations.)

11.13 Phase Cycling

The Python API represents phase cycling with `spin_dynamics.phase_cycling.PhaseCycle`. A phase cycle is a scan table: rows are cycle steps, and columns are named logical RF pulse roles or events. For example, the default CPMG/PAP table has one pulse column, `excitation`, and two rows with phases $\pi/2$ and $3\pi/2$. Each row also carries a receiver phase, a branch weight, and an optional label. Branches are combined as

$$\sum_k w_k \exp(-i\phi_{\mathrm{rx},k})\, S_k,$$

with normalization by the sum of absolute receiver-weighted branch weights unless the table disables normalization.

```
from spin_dynamics.phase_cycling import cpmg_two_step_phase_cycle
from spin_dynamics.workflows import run_ideal_cpmg_train

cycle = cpmg_two_step_phase_cycle()
cycle.pulse_names        # ("excitation",)
cycle.branch_weights     # array([0.5+0.j, -0.5+0.j])

train = run_ideal_cpmg_train(num_echoes=8, numpts=51)
train.phase_cycle.name   # "cpmg_two_step"
```

The same table can program any labeled backend-neutral sequence. Each logical pulse name selects RF blocks with the matching label by default; an explicit `pulse_blocks` mapping can select indices, alternate labels, or repeated RF roles. Receiver phase is retained as branch metadata and applied once during combination.

```
branches = sequence.phase_cycle(cycle)
results = [execute(branch.sequence) for branch in branches]
combined = cycle.combine([result.signal for result in results])
```

The phase-cycle columns are not the set of unique RF phase values. They are the named pulse roles whose phase can vary from one scan branch to the next. This keeps the table aligned with sequence events and leaves room for receiver phase and weighting information on each row.

PGSTE walker results expose the same metadata field, but the interpretation is more specific. The `pgste_` constructor returns a one-row table for the selected stimulated-echo pathway with columns `excitation_90`, `store_90`, and `read_90`. The PGSTE workflow does not explicitly simulate and receiver-combine several phase-cycle branches; pathway selection is implemented by the spoiler and by suppressing equilibrium regrowth during storage.

`plot_phase_cycled_stimulated_echo.py` shows the explicit three-pulse phase table on a deterministic inhomogeneous-field ensemble. It compares a single three-90° scan, a read-pulse-only two-step cycle, and a full 64-step table over `excitation_90`, `store_90`, and `read_90`. During storage the example applies physical relaxation: transverse magnetization decays with T_2, while M_z relaxes back toward equilibrium with T_1. The recovered M_z produces a prompt last-pulse FID, and the full receiver signature $-\phi_1 + \phi_2 + \phi_3$ rejects that pathway while retaining the stimulated echo at τ.

11.14 Absolute RF Phase

Finite CPMG train workflows accept an optional `absolute_phase` mapping. The ideal workflow records the laboratory-frame phase schedule while preserving its nominal rectangular pulses. Probe-aware finite trains use the same schedule to solve the tuned, un-tuned, or matched probe waveform for each pulse, discretize the rotating-frame waveform into short segments, and build a separate refocusing matrix for each echo:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=21,
    num_echoes=32,
    absolute_phase={
        "rf_frequency_hz": 0.25 / 200e-6,
        "rf_phase_at_zero_rad": 0.0,
    },
)

phase_step = train.absolute_phase.delta_refocus_phase_cycles
```

For long trains, add `phase_bins` to reuse solved refocusing pulses on a uniform absolute-phase grid:

```
train = run_tuned_cpmg_train(
    num_echoes=256,
    absolute_phase={
        "rf_frequency_hz": 0.03125 / 200e-6,
        "phase_bins": 32,
```

```
    },
)

library = train.absolute_phase.refocus_pulse_library
matrix_phases = train.absolute_phase.refocus_matrix_phase_rad
```

The scheduled phase for every echo remains available as `refocus_absolute_phase_rad`. The quantized matrix phase, bin index, unique phase list, and exported `PulseShapeLibrary` are recorded in result metadata so cache behavior can be audited or reused in downstream analysis. The matched diffusion CPMG workflow uses the same `absolute_phase` mapping and additionally records the diffusion-encoding π pulse in `encoding_*` metadata fields.

The same solved pulse shapes can be inspected without running a full echo train:

```
import numpy as np

from spin_dynamics.pulse_diagnostics import solve_probe_pulse_shape_sweep

sweep = solve_probe_pulse_shape_sweep(
    probe="tuned",
    absolute_phase_rad=2 * np.pi * np.array([0.0, 0.25, 0.5, 0.75]),
    numpts=17,
)

shape = sweep.shapes[1]
drive = shape.drive
quadrature_fraction = shape.quadrature_energy_fraction
library = sweep.pulse_shape_library
```

When sweeping the phase advance between refocusing pulses, keep the absolute phase of the first refocusing pulse fixed if the goal is to isolate echo-to-echo variation. A half-cycle phase advance produces the same transverse rotating-frame pulse shape for a linear tuned probe, while intermediate advances change the rise/fall and quadrature transients and can produce extra attenuation in the matched-filter echo amplitude.

The Mandal-2015 plotting examples expose the diagnostic workflow directly:

```
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned
python examples\plot_mandal2015_echo_modulation.py --probe tuned
python examples\plot_mandal2015_pulse_shapes.py --probe tuned
```

The finite-train plotting scripts accept `-phase-bins N` to exercise the same quantized pulse-cache path used by the workflow API. They also enable automatic offset-grid refinement by default and use `-rephase-action raise`; this keeps demonstration plots from silently using too few isochromats. Pass `-no-auto-refine-grid` only for explicit diagnostic runs.

`plot_mandal2015_pulse_shapes.py` solves a single refocusing pulse at several absolute RF phases and plots $\mathrm{Re}(B_1)$, $\mathrm{Im}(B_1)$, $|B_1|$, and the phase of the nonzero-amplitude segments. It is intended as a debugging and teaching aid: the finite train examples use the same public diagnostics API, pulse-shape solvers, and discretized segments.

11.15 Multi-Frequency Bloch–Siegert Phase and the RWA Limit

Mandal et al., "An ultra-broadband low-frequency magnetic resonance system," *J. Magn. Reson.* 242 (2014) 113–125, used rapid frequency switching in two related ways. Interleaved CPMG slices acquire unwanted

Bloch–Siegert phase from the pulses on every other slice, while paired far-off-resonant pulses can measure that phase intentionally. For a co-rotating nutation frequency ω_1, pulse duration T_{BS}, and offset $\Delta\omega$, their paired-offset result is

$$\phi_{BS}(+\Delta\omega) - \phi_{BS}(-\Delta\omega) \simeq T_{BS}\frac{\omega_1^2}{\Delta\omega}.$$

The workflow `simulate_bloch_siegert_phase_sweep` propagates the real linearly polarized RF field in the laboratory frame, so it retains both circular components. At the paper's Fig. 15 operating point—$f_0 = 1.48$ MHz, $\Delta f = 25$ kHz, and $T_{90} = 102$ μs—the exact simulation reaches 34.3 degrees at $T_{BS} = 400$ μs and refits $T_{90} = 102.4$ μs, reproducing the reported approximately 35-degree endpoint and 102 μs fit. `mandal_multislice_correction` separately implements Eqs. 14–16: for the Fig. 14 spacing of 20 kHz and $T_{90} = 140$ μs, slice 2 acquires a -4.02 degree excitation-phase error, which the prescribed phase correction removes.

The paired experiment also provides a clean teaching example for the limit of the rotating-wave approximation. To second order, a real RF field gives

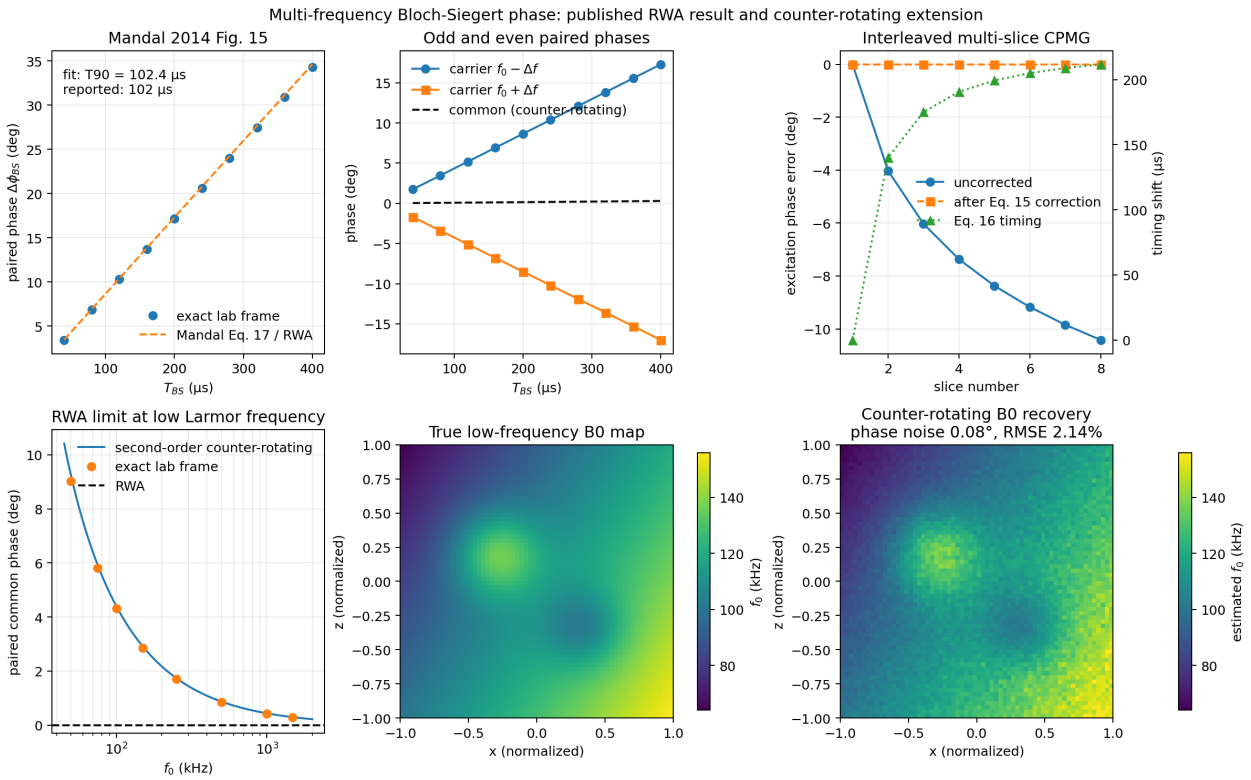
$$\phi_{\mp} = \frac{T_{BS}\omega_1^2}{2}\left(\pm\frac{1}{\Delta\omega} + \frac{1}{2\omega_0 \mp \Delta\omega}\right).$$

The difference $\phi_- - \phi_+$ is the usual B1-sensitive result above. The sum $\phi_- + \phi_+$ cancels identically in the RWA and isolates the counter-rotating term. It grows at low ω_0, making it an observable for low-frequency B0 mapping. The inverse is exposed as `estimate_larmor_hz_from_counter_rotating_phase`; it should not be used near a pole, under strong drive, or when the common phase is comparable to the phase-noise floor without an exact-model fit.

Run the complete literature reproduction and noisy mapping extension with:

```
python examples\plot_bloch_siegert_multifrequency.py \
  --output results\bloch_siegert_multifrequency.png \
  --data-output results\bloch_siegert_multifrequency.npz
```

The default synthetic map adds 0.08 degrees RMS phase noise before inversion and recovers the 45–160 kHz B0 distribution with about 2.1% relative RMSE. This result is intentionally distinct from NQR: the counter-rotating/RWA decomposition here assumes Zeeman-defined NMR coupling, whereas the NQR pulse Hamiltonians retain their linearly polarized laboratory-field convention.



11.16 Radiation Damping

```
from spin_dynamics.workflows import run_radiation_damping_fid

fid = run_radiation_damping_fid(
    probe="matched",
    fill_factor=0.7,
    equilibrium_magnetization=0.8,
    flip_angle=1.0,
    model="circuit",
    detuning=2.0e4,
)
```

Finite tuned and matched CPMG trains accept an opt-in `radiation_damping` mapping:

```
from spin_dynamics.workflows import run_tuned_cpmg_train

train = run_tuned_cpmg_train(
    numpts=51,
    num_echoes=8,
    radiation_damping={
        "fill_factor": 0.7,
        "equilibrium_magnetization": 0.8,
        "model": "circuit",
        "detuning": 2.0e4,
        "apply_during_pulses": True,
    },
)
```

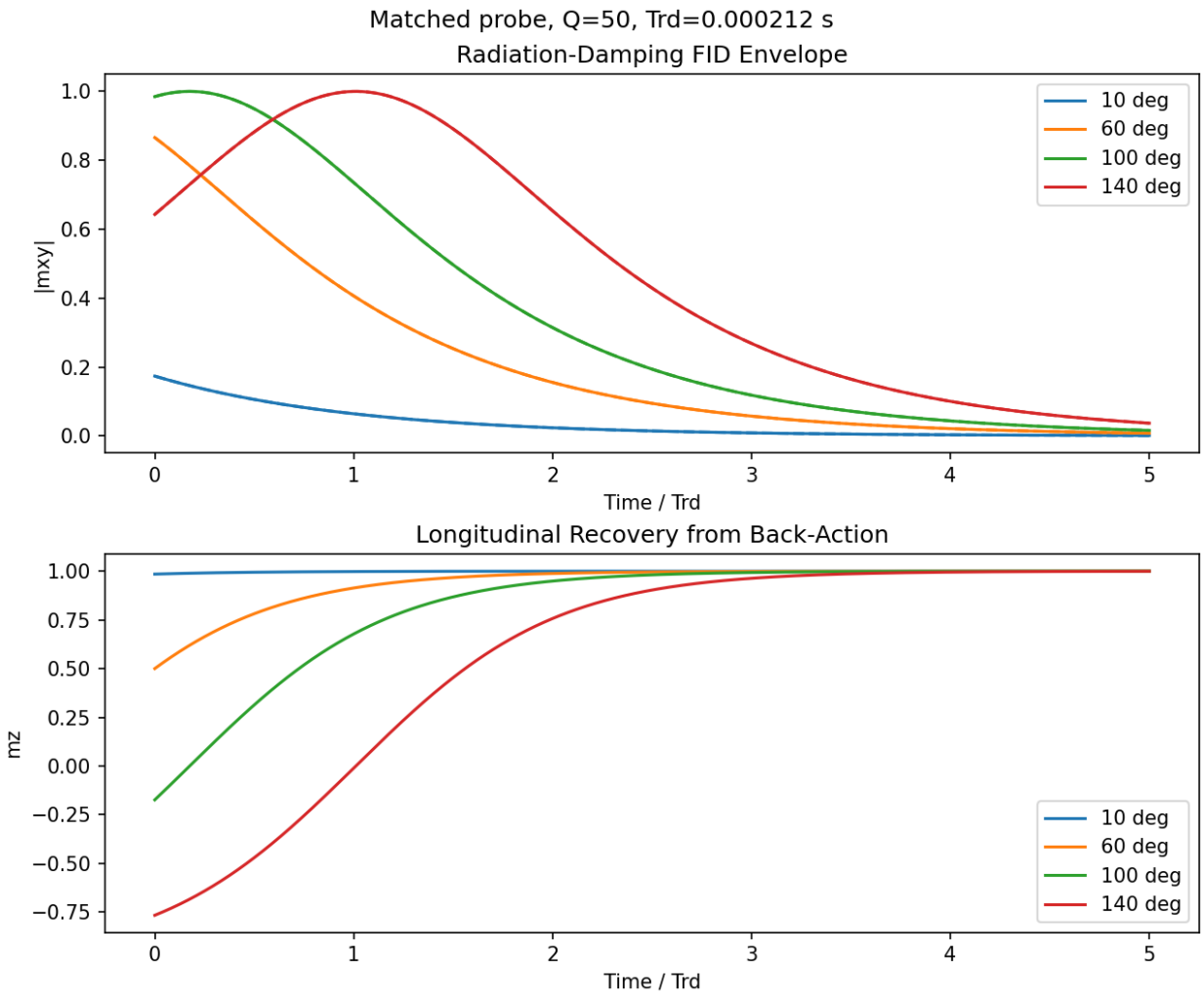


Figure 11.19: Radiation-damping back-action for four initial flip angles, with time scaled by the matched probe's T_{rd}. The feedback drives the transverse FID envelope nonlinearly and returns longitudinal magnetization toward the positive equilibrium direction.

The CPMG train path is deterministic and keeps the radiation-damping feedback inside the finite acquisition model. For strongly inverted samples, `simulate_nmr_maser` uses the same feedback machinery with a pumped longitudinal magnetization; it is intended as an idealized threshold and saturation diagnostic rather than a full spectrometer-control model.

Run `python examples/plot_radiation_damping.py -output radiation_damping.png` to reproduce Figure 11.19.

11.17 Inverse Laplace Analysis

```
import numpy as np
from spin_dynamics.analysis import invert_t2, t2_kernel

echo_times = np.linspace(0.5e-3, 90e-3, 40)
t2_axis = np.logspace(-4, -1, 60)
```

```
signal = t2_kernel(echo_times, t2_axis) @ np.exp(-((np.log(t2_axis) + 4.5) ** 2))

result = invert_t2(
    signal,
    echo_times,
    t2_axis,
    regularization=5e-4,
    regularization_order=2,
)
```

For two-dimensional distributions, the separable forward model is

$$D = K_1 F K_2^{\mathsf{T}},$$

where D is the measured data, F is the unknown distribution, and K_1, K_2 are relaxation or diffusion kernels. Use `invert_t1_t2` or `invert_d_t2` for common NMR cases.

The PGSE D-T2 example ties this analysis layer to a sequence model: it builds the b-axis with `run_pgse_moment`, simulates a PGSE-prepared CPMG echo matrix, and recovers the D-T2 distribution with the SciPy-backed non-negative `invert_d_t2` solver.

11.18 Chemical and Site Exchange

The `spin_dynamics.exchange` namespace adds Bloch-McConnell site exchange, the relaxation-domain counterpart to the diffusion-exchange (DEXSY) walker example. An `ExchangeSystem` holds a tuple of `ExchangeSite` objects and a rate matrix whose entry (i, j) for $i \neq j$ is the first-order rate constant $k_{i\to j}$. Site populations are normalized, and detailed balance $p_i k_{i\to j} = p_j k_{j\to i}$ is checked unless disabled.

```
import numpy as np
from spin_dynamics.exchange import (
    two_site_exchange, simulate_relaxation_exchange_2d, exchange_spectrum,
)
from spin_dynamics.analysis import invert_t2_t2

system = two_site_exchange(
    offset_a_hz=0.0, offset_b_hz=0.0,
    k_ab_hz=8.0, population_a=0.5,
    t2_a_seconds=0.010, t2_b_seconds=0.200,
    labels=("fast", "slow"),
)
```

The kinetic generator X ($\dot{m} = Xm$) is column-conserving, so exchange alone preserves total magnetization. Adding per-site offset precession and transverse relaxation gives the transverse generator used for lineshapes: `exchange_spectrum` integrates the transverse Bloch-McConnell equations and reproduces the slow-exchange (two resolved lines) to fast-exchange (a single population-averaged line) coalescence crossover.

For relaxation exchange spectroscopy (REXSY), the encode-mix-detect model places exchange in a longitudinal mixing interval described by a mixing propagator G:

$$S(t_1, t_2) = \sum_b e^{-t_2/T_{2,b}} \sum_a G_{ba}\, p_a\, e^{-t_1/T_{2,a}}.$$

Inverting $S(t_1, t_2)$ with `invert_t2_t2` yields a T_2-T_2 map: diagonal peaks for spins whose T_2 is unchanged across mixing, and off-diagonal cross peaks whose intensity grows with the exchange rate.

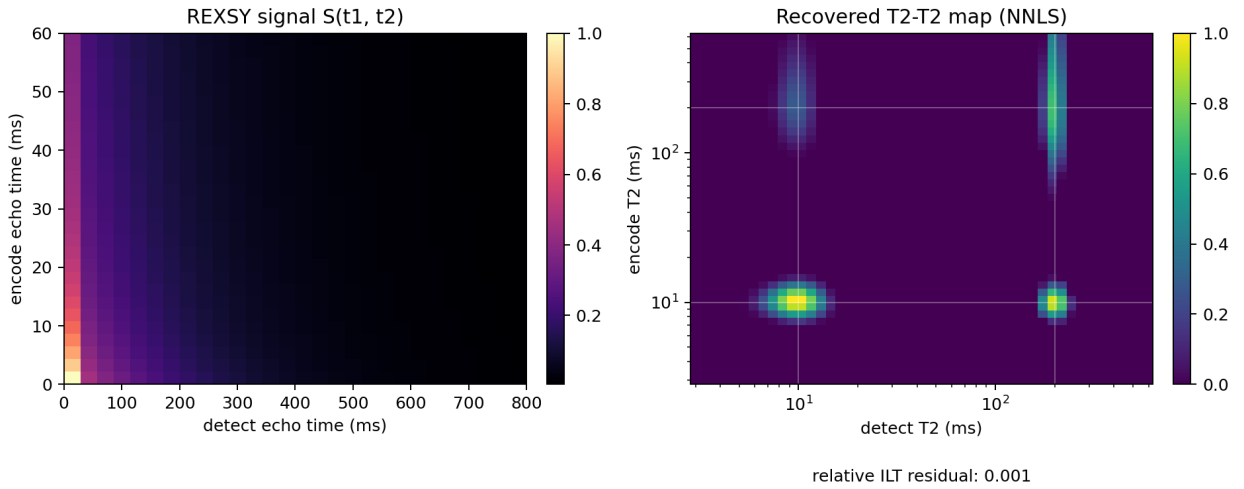


Figure 11.20: A two-site REXSY forward signal and its non-negative inverse-Laplace T_2–T_2 map. Diagonal peaks retain each site's relaxation component, while the symmetric off-diagonal peaks report exchange during the mixing interval.

```
encode = np.linspace(0.0, 0.06, 28)
detect = np.linspace(0.0, 0.8, 28)
rexsy = simulate_relaxation_exchange_2d(system, encode, detect, mixing_time=0.06)

t2_axis = np.logspace(-3, 0, 48)
ilt = invert_t2_t2(rexsy.data, encode, detect, t2_axis, regularization=1e-3)
exchange_map = ilt.distribution  # off-diagonal mass == exchange
```

The `plot_t2_t2_exchange.py` example runs this end to end. The REXSY forward model assumes refocused encode/detect trains and exchange confined to the longitudinal mixing interval; use the transverse engine directly when offset evolution during encoding matters.

Run `python examples/plot_t2_t2_exchange.py -output t2_t2_exchange.png` to reproduce Figure 11.20.

11.19 Internal and Susceptibility Gradients

The `spin_dynamics.susceptibility` namespace generates the static internal field produced by magnetic-susceptibility contrast between a solid matrix and the pore fluid, the inhomogeneity that usually dominates porous-media NMR. Each `CylindricalInclusion` is an infinitely long cylinder perpendicular to the two-dimensional map plane, magnetized by a uniform in-plane applied field. Outside the cylinder the parallel field perturbation is the 2D dipole

$$\Delta B_{\parallel}(\rho,\varphi) = B_0\,\frac{\Delta\chi}{2}\,\left(\frac{a}{\rho}\right)^2\cos(2\varphi),$$

with radius a, distance ρ, and angle φ measured from the applied-field direction. Cylinders superpose in the dilute ($|\Delta\chi| \ll 1$) limit.

```
import numpy as np
from spin_dynamics.susceptibility import (
    CylindricalInclusion, susceptibility_offresonance_map,
```

```
    internal_gradient_distribution, make_susceptibility_field_maps,
)

axis = np.linspace(-50e-6, 50e-6, 161)
grains = [CylindricalInclusion(cx, cz, 12e-6)
          for cx in (-30e-6, 30e-6) for cz in (-30e-6, 30e-6)]

field = susceptibility_offresonance_map(
    axis, axis, grains, b0_tesla=2.0, susceptibility_difference=1e-6,
)
dist = internal_gradient_distribution(field)  # pore-space |g| in T/m
maps = make_susceptibility_field_maps(field)  # drop into motion helpers
```

The off-resonance map is returned in the angular-frequency (rad/s) convention used by `spin_dynamics.motion`, so it feeds the moving-isochromat sequence helpers directly. With no applied gradient, a CPMG echo train of diffusing walkers then decays purely from diffusion in the internal gradient.

The acceleration of that decay with echo spacing (rate $\sim G^2 D\, T_E^2$) is the general diffusion-in-a-gradient effect and is *not* by itself a signature of an internal gradient: a uniform applied or static gradient produces it too. What distinguishes a susceptibility-induced gradient is its field dependence. Because $G_{\mathrm{internal}} \propto \Delta\chi\, B_0$, the diffusion-induced decay rate scales as B_0^2, whereas an applied gradient is set by hardware and is independent of B_0. The broad pore-space gradient *distribution* is a second distinguishing feature; a uniform gradient would be a single value. The `plot_internal_gradients.py` example shows the internal field, the internal-gradient distribution, and the recovered B_0^2 scaling of the diffusion-induced decay rate end to end. Background-gradient-suppressing sequences are implemented in the next section; the internal field produced here is their input.

11.20 Bipolar 13-Interval PGSTE

The `spin_dynamics.workflows.bipolar` module implements the Cotts 13-interval alternating pulsed-gradient stimulated-echo (APGSTE) sequence, the sequence in the Bruker `diff_stebp.gp` program, which measures diffusion while suppressing a constant background gradient g_0 such as the internal gradient of Section 11.19. It is a stimulated echo (three 90 degree pulses) with one 180 degree pulse in each of the two encoding periods, a gradient lobe on each side of every 180, and the applied polarity inverted between the two periods. The 180 pulses refocus the background gradient within each period, and the polarity alternation cancels the applied-times-background cross-term.

Working in the toggling (coherence) frame, the diffusion attenuation in a constant background gradient is

$$\ln E = -D\left(b_{\mathrm{applied}} + g_0\, c_{\mathrm{cross}} + g_0^2\, c_{\mathrm{bg}}\right),$$

and `toggling_frame_moments` returns the three coefficients. For the 13-interval sequence $c_{\mathrm{cross}} = 0$; for an ordinary monopolar stimulated echo it is non-zero, so its apparent diffusion coefficient drifts with g_0.

```
from spin_dynamics.workflows import (
    run_cotts_thirteen_interval_moment, run_monopolar_pgste_moment,
)

c = run_cotts_thirteen_interval_moment(gradient_amplitude=0.1, background_gradient=0.05)
m = run_monopolar_pgste_moment(gradient_amplitude=0.1, background_gradient=0.05)
print(c.cross_term_bias, m.cross_term_bias)  # ~0 vs non-zero
print(c.phase_cycle.name)                   # "diff_stebp"
```

The phase cycle is the 16-step Bruker `diff_stebp` table, built on the package's `PhaseCycle` machinery as `diff_stebp_phase_cycle()`. Its receiver satisfies $\mathrm{ph31} = -(\mathrm{ph1} + \mathrm{ph2} + \mathrm{ph3}) \bmod 4$ (with $\mathrm{ph4} = 0$ on both 180 pulses), selecting the stimulated-echo coherence-transfer pathway $\Delta p = (+1, -2, +1, +1, -2)$ with detection at $p = -1$; the coherence order $p(t)$ this cycle selects is exactly the sign carried by the toggling-frame intervals.

For restricted geometry and finite-pulse effects, `run_cotts_thirteen_interval_walkers` runs the full five-pulse sequence with moving walkers, applying a constant background gradient as a linear off-resonance map. For free diffusion the walker echo reproduces $\exp(-bD)$ with the moment-model b-value, and the apparent diffusion coefficient from a b-value sweep is unbiased by the background gradient. The `plot_bipolar_pgste.py` example traces the wavevectors, the moment-model apparent-diffusion suppression versus g_0, and the walker runner validated against $\exp(-bD)$. The moment model is the free-diffusion b-value for ideal rectangular lobes; trapezoidal ramps need a shaped-gradient extension.

11.21 Optimization Workflows

The optimization package provides compact, plotting-free drivers for phase-program searches and result handoff:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

opt = run_tuned_refocusing_multistart(
    num_segments=3,
    num_starts=4,
    numpts=21,
    max_passes=4,
)
```

Use the optimization examples as diagnostics rather than turnkey production pulse design: they expose refocusing, target-excitation, and inverse-excitation objective behavior while the inverse-cancellation workflow remains under validation.

11.22 GRAPE Optimal Control

The `optimal_control` package generalizes the package's autodiff machinery from a single hand-built objective (the ideal-v0crit refocusing-phase search above) into a reusable gradient-ascent pulse engineering (GRAPE) toolkit: it optimizes the full piecewise-constant RF control waveform of an arbitrary pulse against a state-transfer or propagator/gate fidelity target, using reverse-mode autodiff through a differentiable Hilbert-space propagator chain (`jax.lax.scan` over segment-wise `expm(-i H_seg dt)`, mirroring the `core._jax_kernels` convention that keeps compile time independent of segment count).

Control is parameterized as amplitude and phase per segment (not Cartesian I/Q), because **phase-only control is the primary mode this package targets**: field-deployable NQR/NMR hardware frequently uses switching power amplifiers that cannot vary RF amplitude continuously, so only the phase is agile. Peak-B1 is then an exact box constraint on the amplitude channel in this parameterization (independently box-bounding Cartesian I/Q components would incorrectly permit an effective amplitude above the true hardware limit at the box corners), and fixing amplitude to a hardware-realistic constant and optimizing only phase requires no constrained-optimization machinery beyond the existing bounded L-BFGS-B driver.

```
from spin_dynamics.coupling.systems import coupled_spin_system
```

```
from spin_dynamics.optimal_control import (
    coupled_spin_control_model,
    grape_optimize_phase_only,
    rectangular_seed_phase,
)

model = coupled_spin_control_model(coupled_spin_system([300.0], [[0.0]]))
n_segments = 16
dt = [12.5e-6] * n_segments  # segment duration = hardware control bandwidth

result = grape_optimize_phase_only(
    model,
    rectangular_seed_phase(n_segments),
    fixed_amplitude=5000.0,          # Hz, held constant (phase-only GRAPE)
    dt=dt,
    target=[0.0, 1.0],               # invert |up> -> |down>
    psi0=[1.0, 0.0],
    hamiltonian_batch=training_offset_hamiltonians,  # robust/ensemble objective
)
```

`hamiltonian_batch` turns the objective into a robust/ensemble optimization: the same controls are propagated (via `jax.vmap`) across a batch of `H_drift` variants (e.g. a B0-offset or B1-scale-factor grid) and the per-case fidelities are reduced (mean by default, or worst-case) to a single score – directly reusing the vmap-batched primitive from the acceleration workstream. `examples\grape_broadband_pulse_optimization.py` demonstrates this end to end: a phase-only inversion pulse trained on a coarse B0-offset ensemble and evaluated on a finer held-out grid, where a plain rectangular pulse's worst-case fidelity collapses to zero across a wide offset band while the GRAPE-optimized pulse holds above 0.97.

Validation. The differentiable propagator agrees with two independent, pre-existing implementations to machine precision rather than merely within tolerance: the plain-NumPy reference (bit-identical to the JAX path, as expected for the same arithmetic traced two ways) and the pre-existing `core.rotations` functions `calc_rot_axis_arba4` and `sim_spin_dynamics_exc` (max absolute deviation $\sim 10^{-15}$–10^{-16}, checked for both a single on-resonance pulse and a five-segment off-resonance train). Reverse-mode gradients match central finite differences to a relative error of $\sim 5 \times 10^{-10}$ – well inside the 5×10^{-3} tolerance the existing v0crit autodiff port uses, since this engine differentiates through an explicit $\exp(-iHt)$ rather than the more numerically delicate windowed v0crit objective. In the broadband-pulse example above (16 segments, 5000 Hz nutation, 200 µs total duration, trained on 7 offsets across ± 3500 Hz, evaluated on a held-out 41-point grid), the rectangular baseline's worst-case fidelity is 0.000 – it cannot invert the band edges at all – while the GRAPE-optimized pulse holds 0.974 worst-case and 0.982 mean, at the identical peak B1.

Multistart is not optional. A rectangular (constant-phase) pulse seeded against a perfectly offset-symmetric ensemble sits at an exact zero-gradient stationary point: mirrored offsets' gradient contributions cancel in the ensemble mean, so an optimizer started there reports immediate convergence with no improvement. This is a genuine symmetry, not a bug or an unlucky initial guess – for a constant or antisymmetric-phase pulse the effective rotation axis is confined to the x-z plane with n_x even and n_z odd in offset, so a purely-x figure of merit cannot see any first-order improvement from a symmetric perturbation. Random-restart multistart (`run_grape_multistart`) reliably escapes it – in one tested case a random start reached 0.901 where the rectangular seed's own local search reported 0.780 with zero gradient – so every solver entry point seeds a rectangular baseline alongside genuinely random restarts rather than relying on the rectangular pulse alone.

Scope note: this engine targets a single (or J-coupled) spin-1/2 system with unitary (relaxation-free) propagation. Gradient-waveform control (joint RF+gradient optimization for spatially selective pulses) is now implemented and documented in the next subsection; higher-spin quadrupolar targets are designed as a straightforward extension – the existing `nqr` Hamiltonian builders in place of `coupled_spin_control_model`, same propagation chain – but are not implemented yet.

11.22.1 Gradient control channel and slice-selective pulses

A magnetic-field gradient couples to a spin as a *position-dependent* offset: the local off-resonance of a spin at position r is $g(t)\,r$, an I_z term whose weight is the applied gradient times the position (the `positions @ gradient` rule the motion/imaging engine's `workflows.slice_selective` uses). GRAPE promotes the gradient waveform $g(t)$ to a genuine optimized control channel alongside RF amplitude and phase:

$$H(t) = H_{\text{drift}} + w_1(t)\big(\cos\phi(t)\,H_x + \sin\phi(t)\,H_y\big) + g(t)\,H_{\text{grad}},$$

where $H_{\text{grad}} = 2\pi I_z$ (`gradient_control_operator`). The defining feature that distinguishes the gradient from the RF channel is that a *single* shared waveform $g(t)$ acts through a *position-scaled* operator per ensemble member: for a position ensemble $\{r_k\}$, member k is driven by $g(t)\,r_k\,H_{\text{grad}}$ (`position_gradient_batch` builds the $r_k\,H_{\text{grad}}$ operators). This is a vmap-axis extension of the robust-ensemble primitive, not a new engine.

Because different positions want different outcomes – inverted inside a slice, untouched outside – the objective accepts *per-case* targets: pass `target` as a $(\text{batch}, \text{dim})$ array (one target ket per ensemble member) instead of the shared $(\text{dim},)$ form. Activate the channel by passing `gradient_operator_batch`; set `optimize_gradient=True` (with `initial_gradient` and `gradient_max`) to optimize $g(t)$, or hold it at `fixed_gradient`. The control vector is `concat([amplitude?, phase, gradient?])`.

```
from spin_dynamics.optimal_control import (
    coupled_spin_control_model, gradient_control_operator,
    position_gradient_batch, constant_gradient_seed, grape_optimize,
)

system = coupled_spin_system([0.0], [[0.0]])
model = coupled_spin_control_model(system, include_gradient=True)
h_grad = gradient_control_operator(system)          # base 2*pi*Iz

positions = np.linspace(-1.0, 1.0, 15)
h_grad_batch = position_gradient_batch(h_grad, positions)  # r_k * h_grad
targets = np.stack([[0.0, 1.0] if abs(p) <= 0.3         # invert in-slice
                    else [1.0, 0.0] for p in positions]) # preserve out-slice

result = grape_optimize(
    model, initial_phase, dt=dt, target=targets, psi0=[1.0, 0.0],
    fixed_amplitude=2000.0,                     # phase-only RF
    optimize_gradient=True,
    initial_gradient=constant_gradient_seed(len(dt), gradient=1.0),
    gradient_max=5.0, gradient_operator_batch=h_grad_batch,
)
opt_gradient = result.best_gradient             # the optimized g(t)
```

`examples\grape_slice_selective_pulse.py` demonstrates this end to end: co-optimizing the RF phase and the gradient waveform for a slice-selective inversion, evaluated on a held-out position grid, sharpens the in-slice inversion from ~ 0.73 to ~ 0.90 over a fixed-gradient rectangular baseline while holding out-of-slice

preservation at ~ 0.92, lifting the mean per-position fidelity from 0.62 to 0.92. A conditioning note applies whenever RF and gradient channels are optimized jointly: a raw gradient in hertz-per-position is $O(10^3)$ while phases are $O(1)$, a scale mismatch that stalls L-BFGS-B. Folding a characteristic offset scale into the gradient operator (so the optimized $g(t)$ is itself $O(1)$) restores a well-conditioned problem; the example does exactly this.

11.22.2 Hardware response: probe and gradient-driver dynamics

The pulses above assume the spins see exactly the commanded waveform. Real hardware does not deliver it: a tuned or matched RF probe has finite bandwidth (loaded Q) and a ring-down tail, and a gradient driver has a finite L/R slew plus eddy-current droop. Each is, to excellent approximation, a *linear time-invariant* (LTI) filter between the commanded waveform and the field the spins see. `optimal_control.control_response` models these as one differentiable stage the objective inserts in the loop, between the *command* and the Hamiltonian:

$$u_{\text{delivered}}(t) = \big(\mathcal{L}\, u_{\text{command}}\big)(t), \qquad u = w_1\, e^{i\phi}.$$

Because the whole chain is JAX, autodiff supplies the filter's adjoint for free – no hand-derived circuit gradient. Two consequences make this the physically correct thing to optimize: fidelity is scored on the delivered pulse (*including* the ring-down/eddy tail, which really does rotate the spins), and the box constraints stay on the command, so the optimizer pre-emphasizes only up to the real hardware limits rather than demanding voltages the amplifier cannot produce.

Transmit is filtered *once*, in time, on the rotating-frame envelope: the coil emits one physical $B_1(t)$ that every isochromat sees, differing only by its own detuning (already in the drift), so there is no per-offset transmit filtering. Tuned and matched probes have genuinely different dynamics and are separate classes: a tuned probe is a single resonance, a one-pole envelope lowpass with ring-down time constant $\tau = 2Q/\omega_0 = Q/(\pi f_0)$ (`TunedProbeResponse`); a matched probe adds a matching-network pole and is a distinct two-pole response (`MatchedProbeResponse`); an untuned coil is near-flat (`UntunedProbeResponse`). RF probes filter the complex envelope by an FFT multiply; the gradient driver (`GradientDriverResponse`) is a causal state-space cascade of one L/R slew pole and a shelf per eddy term, whose delivered step is the classic $G(t) = G_\infty[1 - \sum_k \alpha_k e^{-t/\tau_k}]$ with DC gain one (the steady-state gradient equals the command). `eddy_terms_from_step_response` fits the (α_k, τ_k) from a measured step. The command is upsampled by the response's `oversample` factor before filtering, and a ring-down/eddy tail (`TailSpec`: a multiple of the dominant τ, or an explicit window) is appended.

```
from spin_dynamics.optimal_control import (
    coupled_spin_control_model, grape_optimize, TunedProbeResponse,
)

probe = TunedProbeResponse.from_quality_factor(   # ring-down tau = Q/(pi f0)
    f0_hz=1.0e6, quality_factor=120.0, oversample=4,
)
result = grape_optimize(
    model, initial_phase, dt=dt, target=[0.0, 1.0], psi0=[1.0, 0.0],
    fixed_amplitude=6000.0,        # command bound stays on the amplifier
    rf_response=probe,             # fidelity scored on the DELIVERED pulse
)
```

Both `rf_response` and `gradient_response` are optional and default to the ideal (unfiltered) behaviour bit-for-bit; they require a uniform scalar `dt`. `examples\grape_probe_response_pulse.py` shows a band-limited tuned probe cutting a rectangular inversion pulse's delivered fidelity to ~ 0.84, which GRAPE optimizing the command recovers to 1.0; it also illustrates the gradient-driver step response and the eddy

fit. Probe-robust pulses follow by making the filter parameters (tuning, Q, eddy τ_k) the ensemble axis, reusing the robust-ensemble machinery. The design note `docs/optimal_control_hardware_response.md` records the full rationale, including the output-side `ReceiverResponse` for objectives defined on the detected signal.

11.22.3 PGSE diffusion: correct q-vector and detected SNR

The diffusion capstone (`optimal_control.diffusion`) co-optimizes the RF pulses *and* the gradient waveform of a pulsed-gradient spin echo on realistic hardware: a tuned RF probe on transmit *and* receive, a gradient driver with L/R slew plus eddy currents, and an inhomogeneous B_0/B_1 field. It exercises the entire stack – the M2 gradient channel, the M4 hardware-response layer, and a combined (position $\times$ B_0 $\times$ B_1) ensemble – against two objectives:

1. **Correct q-vector**: a functional of the *delivered* gradient. `gradient_moments` returns $q(t) = \int g(\tau)s(\tau)\,d\tau$ (with the coherence sign $s = \pm 1$ that a $180°$ pulse flips), its encode-lobe value and its residual at the echo, matching `workflows.pgse`'s convention. The encode moment must hit a target q^* and the residual must refocus to zero; eddy droop violates both, so the optimizer pre-emphasizes the commanded gradient (bounded) to correct them.
2. **Maximize detected SNR**: `detected_echo_signal` acquires the coherently-summed echo over a readout window, `detected_echo_snr` filters it through the Rx probe and returns the matched-filter amplitude over a fixed noise floor.

`make_pgse_objective` assembles the weighted `value_and_grad` (SNR $- \lambda_q(q_{\text{enc}} - q^*)^2 - \lambda_0 q_{\text{echo}}^2$), propagating with `propagator="expm"` by default – on-resonance spins have $H_{\text{seg}} = 0$ during RF/gradient gaps, whose `eigh` gradient is NaN (the same degeneracy the NQR models hit). `examples\plot_grape_pgse_diffusion.py` runs it in a *grossly* inhomogeneous field (B_0 spread several times the nutation rate). Two effects, both large: (i) a hard $180°$ refocuses only within $\sim \pm$nutation, so the naive echo collapses across the band, while phase-only GRAPE finds a broadband phase-modulated refocusing pulse – the RP2 / symmetric-phase-alternating family – that recovers the echo across the full band for a several-fold detected-SNR gain; and (ii) eddy droop pushes the naive encode moment below q^* with imperfect refocusing, which the gradient pre-emphasis corrects to $\sim 0\%$ error. **Multistart is essential**: a rectangular ($\phi = 0$) seed against a symmetric offset ensemble sits at an exact saddle (mirrored offsets' phase gradients cancel), so a single start never moves the phase – random phase restarts escape it. The design note is `docs/optimal_control_pgse_diffusion.md`.

11.22.4 Detector-aware objectives: optimizing for a flux readout

The PGSE objective scores the detected echo through a tuned-probe `ReceiverResponse` over a scalar noise floor – the inductive-coil picture. A SQUID or OPM (Section 11.7) is a field sensor with a frequency-dependent field-noise floor $N^2(f)$ (flat for a SQUID, a Lorentzian band-pass for an RF-OPM) and no $\mathrm{d}\Phi/\mathrm{d}t$ weighting. `optimal_control.detection_objective` scores the same acquired echo with the field-referred matched filter

$$\mathrm{SNR} = \sqrt{\int \frac{|S(f)|^2}{N^2(f)}\,\mathrm{d}f},$$

so the optimizer shapes the pulse to place signal where the *detector* is quiet. Because $N^2(f)$ is fixed over the acquisition frequency grid (it does not depend on the controls), it is precomputed once with NumPy (`detector_noise_grid`, which also re-centres a band-pass detector's response to baseband) and passed as a static weight into the differentiable objective; a flat $N^2 = \sigma^2$ reproduces `detected_echo_snr` with `noise` $=$

σ (Parseval), so it is a strict generalization. `make_detected_snr_objective` builds the "value_and_grad" over a (B_0, B_1) ensemble; with `per_rf_power` the score is SNR per unit RF amplitude, the efficiency figure of merit that makes the readout actually shape the pulse (raw SNR always rewards more excitation, since out-of-band signal never hurts). The example `examples/plot_grape_detector_aware_pulse.py` optimizes an excitation for a broadband SQUID versus a narrow SERF OPM and shows, by cross-evaluation, that each pulse is best under its own detector.

11.22.5 Axis-matched excitation (AMEX) and CPMG SNR per unit time

`su2_effective_axis` extracts a refocusing cycle's effective rotation axis directly from its GRAPE propagator (validated against `core.rotations.calc_rot_axis_arba4` to machine precision), and `bloch_vector_to_ket` converts that axis to the spin state it is the fixed point of. In an inhomogeneous field the steady state reached after many refocusing cycles is the initial magnetization's projection onto this offset-dependent axis, so exciting directly onto it (axis-matched excitation, AMEX) reaches the asymptotic echo amplitude from the first cycle – provably a fixed point of the cycle – while a conventional excitation must settle through a transient that need not converge cleanly. `examples\grape_cpmg_snr_per_time.py` demonstrates this together with the time cost of long refocusing pulses: it reuses the `optimization.spa` figure of merit `fom_time = echo_spacing / SNR`^{2} (`echo_spacing` includes the pulse length directly, so longer pulses are penalized for producing fewer echoes per unit time), sweeps candidate refocusing-pulse durations, and follows the sequential/separate methodology (refocusing optimized first assuming an idealized excitation, then AMEX excitation designed onto its axis) alongside a joint optimization for comparison – both, and a rectangular pulse at the same peak B1, are reported so the trade-off is decided empirically rather than assumed.

Validation: an ideal axis-matched state is an exact fixed point. Simulating the refocusing cycle's unitary applied repeatedly (the echo-train primitive, `iterate_unitary`) confirms the theory directly: starting from the axis-aligned ket, the ensemble-averaged transverse signal is identical at every one of 25 simulated cycles (agreement to $\sim 10^{-9}$, i.e. no numerically detectable transient), whereas starting from the conventional $+x$ state produces a genuine, unsettled oscillation that has not converged even after 25 cycles for the offset ensembles tested here – not a clean exponential decay to a plateau, since different offsets precess around their own tilted axes at different rates and beat against each other in the ensemble average.

Segments	t_p (µs)	SNR (rect+rect)	SNR (GRAPE+AMEX)	SNR (joint)	fom_time gain
4	32	0.327	0.759	0.805	5.4×
8	64	0.121	0.641	0.703	27.9×
16	128	0.0001	0.762	0.567	4.3×10^4

Table 11.3: CPMG SNR-per-unit-time sweep at fixed peak nutation rate 5000 Hz, 25 refocusing cycles, ± 3000 Hz training ensemble (7 offsets), 41-point held-out evaluation grid, 25 µs free-precession delay on each side of the refocusing pulse. "fom_time gain" is the rectangular-baseline to GRAPE+AMEX ratio of `fom_time` (SNR-per-unit-time improvement) at that duration; the 16-segment entry is dominated by the rectangular pulse's total nutation angle ($\sim 230°$ at this peak amplitude and duration) being badly off a $180°$ refocusing condition – GRAPE's phase modulation recovers a working pulse from what is, at fixed constant phase, an essentially failed one, rather than showing a moderate, generic optimization gain.

The joint optimization (refocusing and excitation phases optimized together against the same asymptotic-signal objective) is not consistently better or worse than the sequential/AMEX approach across this sweep (ahead at 4 and 8 segments, behind at 16) – read as an honest, instance-specific answer rather than a general

preference for one methodology, since with a fixed multistart budget the larger joint parameter space is not uniformly easier to search.

Comparison with prior work. Mandal et al. (2013, *Axis-matching excitation pulses for CPMG-like sequences in inhomogeneous fields*) and Mandal et al. (2014, *Direct optimization of signal-to-noise ratio of CPMG-like sequences in inhomogeneous fields*) are the original sources for both AMEX and the SNR-per-unit-time figure of merit; both use GRAPE or `fmincon` for this same class of refocusing-axis/excitation optimization, and the 2014 paper hand-derives (Appendix A) the analytic gradient that this engine obtains automatically via reverse-mode autodiff. Their $\mathrm{SNR}_{\mathrm{time}} \propto \mathrm{SNR}_{\mathrm{power}}/t_{E,\min}$ is the same physics as `fom_time` (their "SNR" is explicitly a power, i.e. amplitude squared, so $\mathrm{SNR}_{\mathrm{time}} = 1/\texttt{fom\_time}$) – `optimization.spa`'s formula traces to this same result. The symmetry that makes a rectangular (constant-phase) refocusing pulse a zero-gradient stationary point for a symmetric offset ensemble (Section 11.22 above) is their Eq. (5)–(7) (n_x even, $n_y \equiv 0$, n_z odd in offset for constant or antisymmetric-phase pulses), which they exploit deliberately to halve their own search space rather than treat as an obstacle – a natural refinement for a future increment here. Quantitatively, however, this example's offset range (at most $\sim 0.7\times$ the nutation rate) and duration sweep (up to $1.3\times$ the nominal 180° pulse length) are both far milder than either paper's benchmark regime: the 2013 paper uses offsets up to $\sim 10\times$ the nutation rate, the 2014 paper up to $\sim 50\times$, and the 2014 paper's central finding is that SNR per unit time peaks for refocusing pulses 2–$5\times$ the standard 180° duration – a regime this sweep does not reach. That mismatch, not a discrepancy in the physics, is why this example's refocusing-only gain is modest (a few percent) where the papers report gains of several-fold, and why its AMEX-to-ideal-excitation ratio is not close to their reported $\sim 2\times$: mild inhomogeneity leaves a conventional excitation and a simple rectangular refocusing pulse much closer to adequate already. Reproducing their specific benchmark numbers directly would require re-running this example at their offset range and duration sweep, which has not yet been done.

11.22.6 NQR / quadrupolar targets

The GRAPE engine is Hilbert-space-general – it needs only a `ControlHamiltonianModel` (`h_drift`, `h_x`, `h_y`) – so extending it to quadrupolar (NQR) nuclei is a matter of supplying that model, not a new engine. `nqr_site_control_model(site)` builds it for a spin $I = 1, 3/2, 5/2, 7/2, 9/2$ site.

Frame. A piecewise-constant lab/PAS field cannot drive an MHz quadrupolar transition (it is a DC field), so the model is built in the *rotating frame at the RF carrier*, in the energy eigenbasis — exactly the frame the full NQR density-matrix engine uses (Section 8.4). `h_drift` is the rotating-frame static Hamiltonian (`static_hamiltonian_rotating`: diagonal detunings) and `h_x`/`h_y` are the co-rotating RWA drive quadratures extracted from `pulse_hamiltonian`, so `pulse_hamiltonian` $\equiv$ `h_drift` $+ w_1(\cos\phi\,$`h_x`$+\sin\phi\,$`h_y`$)$ and a pulse optimized on this model round-trips through the real simulator to machine precision. The default carrier is the fundamental (strongest) transition; the model operates in the eigenbasis, so the fundamental transition's two eigenstates are the natural inversion endpoints (`nqr_fundamental_states` returns their indices). The single-carrier RWA is faithful only near the fundamental line, so driving a satellite warns (exact satellite pulsing needs non-RWA time-domain propagation, not yet implemented).

The eigh-vs-expm gradient. Quadrupolar spectra are Kramers-degenerate. The default segment propagator diagonalizes H_{seg} with `eigh`, whose reverse-mode gradient carries $1/(\lambda_i - \lambda_j)$ terms that become *NaN* for degenerate eigenvalues. NQR models must therefore pass `propagator="expm"`, which propagates via `jax.scipy.linalg.expm` – its gradient is well-defined under exact degeneracy (agreeing with the `eigh` forward map to $\sim 10^{-9}$ for non-degenerate systems and with finite differences to $\sim 10^{-3}$).

Two robustness axes. Field NQR detection faces two dominant broadenings, each a different ensemble over the shared control waveform:

- **EFG / detuning inhomogeneity.** A distribution of ν_Q (EFG line broadening) detunes spin packets from a fixed carrier; this is a per-case `h_drift` ensemble (`hamiltonian_batch`), identical in structure to the spin-1/2 B0-broadband case. In zero-field NQR the transition frequencies depend only on ν_Q and η, so this is a frequency spread, *not* a powder of orientations (which vary coupling, next item). `examples\grape_nqr_broadband_pulse.py` inverts a ^{63}Cu fundamental line across a ± 25 kHz ν_Q band where a fixed-carrier rectangular pulse's held-out worst-case inversion collapses to ~ 0.01 while the optimized phase-only pulse holds ~ 1.0. `examples\plot_grape_nqr_broadband_spectrum.py` plays the same idea for *excitation*: it optimizes a broadband $90°$ pulse, then Fourier- transforms the excited coherence over a Gaussian ν_Q distribution into a detected spectrum – the rectangular pulse's sinc-like excitation nulls narrow and distort the line, while the GRAPE pulse recovers the true broadened lineshape.
- **Powder orientation.** A crystallite's RF-to-EFG coupling depends on the RF direction in its principal-axis system, so `h_x`/`h_y` vary per orientation while `h_drift` is shared — a per-case *drive-operator* ensemble. Pass `control_operator_batch` (from `nqr_powder_control_batch`); `examples\grape_nqr_powder_pul` inverts a ^{35}Cl line across a powder (held-out worst-case $\sim 0.03 \rightarrow 0.87$).

The two axes compose: passing `hamiltonian_batch` and `control_operator_batch` of equal length makes each ensemble member carry its own $(\texttt{h\_drift}, \texttt{h\_x}, \texttt{h\_y})$, so a flattened *orientation* $\times$ ν_Q product grid optimizes a single pulse robust to detuning *and* orientation at once (no extra machinery – the per-case drift and per-case drive paths already coexist).

Offset-robust excitation for a single-crystal SLSE train. `examples\plot_grape_nqr_slse_excitation.py` optimizes the excitation of a spin-lock spin-echo (SLSE) detection train for a *single crystallite* at a known carrier offset. An off-resonance spin loses signal under a conventional rectangular 90: the refocusing cycle spin-locks only part of the magnetization, so the echo train decays/oscillates to a small steady value. Numerically optimizing the excitation waveform against the *actual* steady echo – the thermal `equilibrium_density` propagated through the full $(2I+1)$ density matrix and read out by `detection_operator` – recovers the on-resonance signal level and a flat, transient-free train. The signal-to-noise gain grows with offset: none on resonance (a rectangular pulse already covers it), rising to ~ 1.3–$1.4\times$ once the offset exceeds the pulse bandwidth ($\gtrsim 2\times$ the nutation rate), where it also beats a flip-angle-optimized rectangular excitation.

This is a single-crystal (known-offset) result and does not extend to a powder. Over a powder the refocusing-cycle response varies strongly and oppositely with orientation (RF coupling and effective axis), one pulse cannot serve a spread of offsets at once, and the conventional rectangular SLSE already navigates the powder with its powder-optimum ("magic-angle", $\sim 119°$ for spin-1 with $\eta \neq 0$) flip – against that flip-angle-optimized baseline an optimized pulse does not improve the powder echo, a genuine negative result reported rather than hidden. The per-orientation refocused-signal ratio is $\min / \max \approx 0.02$ for SLSE versus ≈ 0.26 for the small-tip SORC steady state, so SORC is far more powder-robust to begin with. The clean GRAPE wins for NQR are therefore broadband *excitation* and *inversion* (above), not beating well-tuned *single-coil* powder detection trains.

11.22.7 Tri-axial (multi-coil) SLSE excitation

The module `optimal_control.multi_axis` asks a different hardware-level question: whether additional orthogonal coils improve the powder SLSE signal. It optimizes one rectangular sub-pulse per coil, with

independently adjustable amplitude, phase, delay, and duration for both excitation and refocusing. Sequential versus simultaneous excitation is not imposed; it emerges from the fit. Smooth sigmoid edges render the sub-pulses on a fixed time grid so that delay and duration remain differentiable, and the objective uses the degeneracy-safe `expm` propagator.

`MultiAxisSLSEConfig` sets the number of coils, per-coil B_1 cap, pulse windows, powder grid, echo train, and single- or quadrature-coil receiver. `optimize_multi_axis_slse` then performs bounded multistart optimization of the coherently powder-averaged echo-train energy. Run the comparison with:

```
python examples/grape_nqr_multi_axis_slse.py --save results/grape_multi_axis.png
```

At matched *per-coil* B_1, the optimizer normally recovers most of its gain on moving from one to two axes by driving the crossed coils near quadrature—effectively rediscovering circular polarization (Section 8.9.1). A third independently controlled axis adds little. The comparison reports the additional transmit power and the $\sqrt{2}$ two-channel receiver-noise penalty explicitly; this is a hardware tradeoff, not a free pulse-shaping gain.

Higher-spin transition-selective pulses (drive one line, leave another) and exact lab-frame satellite control are natural follow-ons on the same engine.

12 Examples

The example scripts can be run from `PythonSpinDynamics`; each script also works from inside `examples/` by adding the local source tree to `sys.path` when needed. Treat the chapter as a runnable index rather than a tutorial: the explanatory chapters above describe the model choices, while these commands provide quick checks and reproducible figures. Start with the plain scripts when checking installation and with the plotting scripts when comparing physical effects.

In this chapter. Choose a runnable example by learning goal, then use its module introduction and inline signposts to connect code to the model described here.

12.1 Recommended Learning Paths

The directory is broad, so begin with a short sequence rather than scanning the full command list.

First simulation. Run `experiment_facade_quickstart.py`, then `ideal_cpmg_train.py`, and compare the high-level and direct workflows.

Fields and imaging. Continue with `plot_ideal_imaging.py`, `experiment_imaging_with_coil.py`, and `plot_imaging_inhomogeneity.py`. Then run the complete system study `plot_portable_halbach_adaptive_mr`

Diffusion and transport. Start with `plot_bipolar_pgste.py`, then `plot_pgse_qspace_walkers.py` and `plot_cpmg_pipe_flow.py`.

NQR or ESR. Use `experiment_nqr_auto_model.py` or `plot_esr_pulsed_echo.py` before moving to the optimization and statistical studies.

Each example begins by stating the scientific question, model boundary, important outputs, and a representative command. Comments at major boundaries explain the reasoning behind setup, simulation, diagnostics, and plotting; they are intended as reading guides rather than line-by-line narration.

```
python examples\ideal_cpmg.py --numpts 101
python examples\ideal_fid.py --numpts 101
python examples\ideal_cpmg_train.py --numpts 101 --num-echoes 8
python examples\ideal_time_varying_cpmg.py --numpts 101 --num-echoes 16
python examples\compare_cpmg_fid.py --numpts 101
python examples\export_validation_arrays.py results\validation_arrays.npz --numpts 101
python examples\tuned_probe_cpmg.py --numpts 101
python examples\probe_cpmg_compare.py --numpts 101
python examples\probe_parameter_sweeps.py --numpts 101
python examples\matched_cpmg_ir_train.py --numpts 21 --num-echoes 4 --num-tau 4
python examples\finite_probe_train_sweeps.py --numpts 21 --num-echoes 3
python examples\matched_diffusion_cpmg.py --numpts 21 --num-echoes 3
python examples\porous_rock_cpmg_walkers.py --estimate-only
python examples\received_signal_noise.py --numpts 51
python examples\radiation_damping_fid.py --probe matched --points 401
python examples\radiation_damping_cpmg_train.py --probe tuned --numpts 21 --num-echoes 4
python examples\nmr_maser.py
python examples\heteronuclear_j_editing.py --points 33
python examples\coupled_isochromat_fields.py --points 21
```

```
python examples\diagnose_optimization_backends.py --backend all --numpts 21 --segments 3
python examples\refocusing_gradient_optimization.py --segments 10 --starts 8
python examples\grape_broadband_pulse_optimization.py --segments 16 --starts 24
```

Plotting examples require Matplotlib:

```
python examples\plot_sequence_timeline.py --export-pulseq results\demo.seq --output
    results\demo_sequence.png
python examples\plot_ideal_workflows.py --numpts 201 --output results\ideal_workflows.png
python examples\plot_ideal_imaging.py --pixels 6 --ny 7 --output results\ideal_imaging.png
python examples\plot_custom_imaging_fields.py --pixels 8 --ny 7 --output results\
    custom_imaging_fields.png
python examples\plot_inverse_laplace.py --output results\inverse_laplace.png
python examples\plot_pgse_d_t2.py --output results\pgse_d_t2.png
python examples\plot_phase_cycled_stimulated_echo.py --output results\phase_cycled_ste.png
python examples\plot_probe_parameter_sweep.py --probe tuned --sweep q --output results\
    tuned_q_sweep.png
python examples\plot_probe_cpmg.py --numpts 101 --output results\probe_cpmg.png
python examples\plot_finite_train_workflows.py --numpts 65 --num-echoes 4 --output results
    \finite_trains.png
python examples\plot_diffusion_sweep.py --numpts 65 --num-echoes 3 --output results\
    diffusion_sweep.png
python examples\plot_diffusion_absolute_phase_compare.py --output results\
    diffusion_absolute_phase_compare.png
python examples\plot_tuned_diffusion_absolute_phase_compare.py --output results\
    tuned_diffusion_absolute_phase_compare.png
python examples\plot_time_varying_sweep.py --numpts 51 --num-echoes 12 --output results\
    time_varying_sweep.png
python examples\plot_udd_cpmg_filter.py --pulses 8 --output results\udd_cpmg_filter.png
python examples\plot_sensitive_slice.py --output results\sensitive_slice.png
python examples\plot_multislice_halbach_imaging.py --pixels 16 --slices 5 --output results
    \multislice_halbach.png
python examples\plot_halbach_dipole_field.py --rod-shape square --output results\
    halbach_dipole.png
python examples\plot_nmr_mouse_fields.py --pixels 121 --output results\nmr_mouse_fields.
    png
python examples\plot_nmr_mouse_depth_profile.py --num-depths 13 --num-d-depths 5 --seeds 4
     --output results\nmr_mouse_depth.png
python examples\plot_motion_linear.py --output results\motion_linear.png
python examples\plot_cpmg_pipe_flow.py --output results\cpmg_pipe_flow.png
python examples\plot_motion_diffusion_cpmg.py --output results\motion_diffusion_cpmg.png
python examples\plot_motion_diffusion_udd.py --output results\motion_diffusion_udd.png
python examples\porous_rock_cpmg_walkers.py --backend jax --plot-output results\
    porous_rock_challenge.png --output results\porous_rock_challenge.npz
python examples\plot_radiation_damping.py --output results\radiation_damping.png
python examples\plot_radiation_damping_detuning.py --output results\rd_detuning.png
python examples\plot_radiation_damping_cpmg_train.py --output results\rd_cpmg.png
python examples\plot_bloch_siegert_multifrequency.py --output results\
    bloch_siegert_multifrequency.png
python examples\plot_mandal2015_phase_step_sweep.py --probe tuned --output results\
    mandal_phase_sweep.png
python examples\plot_mandal2015_echo_modulation.py --probe tuned --output results\
    mandal_echo_modulation.png
python examples\plot_mandal2015_pulse_shapes.py --probe tuned --output results\
    mandal_pulse_shapes.png
python examples\plot_nmr_maser.py --output results\nmr_maser.png
```

```
python examples\plot_wurst_flow.py --numpts 41 --num-steps 64 --output results\wurst_flow.
    png
python examples\plot_j_editing_spectrum.py --output results\j_editing_spectrum.png
python examples\plot_j_editing_field_spread.py --output results\j_editing_field_spread.png
python examples\plot_tango_filter.py --target 160 --output results\tango_filter.png
python examples\plot_slic_two_spin.py --j-hz 7 --delta-hz 0.7 --output results\
    slic_two_spin.png
python examples\plot_bpp_relaxation_temperature.py --output results\
    bpp_relaxation_temperature.png
python examples\plot_wall_relaxation_xe.py --output results\wall_relaxation_xe.png
python examples\plot_redfield_water_cpmg.py --output results\redfield_water_cpmg.png
python examples\plot_redfield_nano2_slse.py --output results\redfield_nano2_slse.png
python examples\plot_t1rho_prepolarized_dispersion.py --output results\
    t1rho_prepolarized_dispersion.png
python examples\plot_earth_field_prepolarized_nmr.py --output results\
    earth_field_prepolarized_nmr.png
python examples\plot_optimization_workflows.py --numpts 11 --segments 2 --starts 2 --
    inverse-starts 4 --output results\optimization_workflows.png
python examples\plot_optimization_pipeline.py --numpts 11 --refocusing-segments 2 --
    refocusing-starts 2 --excitation-segments 2 --excitation-starts 2 --inverse-starts 3 --
    output results\optimization_pipeline.png
python examples\plot_grape_cpmg_snr_per_time.py --output results\grape_cpmg_snr_per_time.
    png
python examples\plot_nqr_powder_nutation.py --output results\nqr_powder_nutation.png
python examples\plot_nqr_full_powder_nutation.py --output results\nqr_full_powder_nutation.
    png
python examples\plot_nqr_auto_model_selection.py --output results\nqr_auto_model_selection.
    png
python examples\plot_nqr_population_transfer.py --output results\nqr_population_transfer.
    png
python examples\plot_nqr_slse_offset.py --output results\nqr_slse_offset.png
python examples\plot_nqr_slse_spacing.py --output results\nqr_slse_spacing.png
python examples\plot_nqr_efg_broadening.py --output results\nqr_efg_broadening.png
python examples\plot_nqr_temperature_broadening.py --output results\
    nqr_temperature_broadening.png
python examples\plot_nqr_slse_efg_broadening.py --output results\nqr_slse_efg_broadening.
    png
python examples\plot_nqr_weak_b0_spectrum.py --output results\nqr_weak_b0_spectrum.png
python examples\plot_nqr_polarization_enhancement.py --output results\
    nqr_polarization_enhancement.png
```

Facade examples and configurations exercise the same plan/run interface used by application code:

```
python examples\experiment_facade_quickstart.py
python examples\experiment_imaging_with_coil.py
python examples\experiment_nqr_auto_model.py
python -m spin_dynamics.experiment plan examples\experiment_config_pgse.toml
python -m spin_dynamics.experiment run examples\experiment_config_pgse_walkers_flow.toml -
    o results\pgse_walkers_flow.npz
```

Part V

Evidence and Reference

13 Validation

Simulation credibility is attached to a specific claim, parameter range, and comparison – not to the package as a whole. A passing unit test may show that an API is stable without showing that its physical model is accurate. Conversely, one exact analytical limit may validate an important part of a model without covering every geometry or parameter regime. This chapter therefore separates *validation evidence* from ordinary regression coverage.

The authoritative, machine-readable claims live in `validation/evidence.json`. Each record states the component, claim, evidence type, comparison basis, tested range, metric, tolerance, exact test or script, status, references, and limitations. The generator `docs/generate_validation_matrix.py` validates those records and produces both the detailed Markdown matrix and the compact table included below. CI fails if a reproducer disappears or either generated document becomes stale.

In this chapter. Interpret the evidence levels, locate a claim's exact reproducer, and distinguish physical validation from regression and API coverage.

13.1 Evidence Levels

Evidence letters describe what supports a claim. They are not scores: a level-B parity result and a level-A analytical result answer different questions and are often strongest in combination.

Level	Meaning
A	Analytical identity, exact solution, or limiting case.
B	Parity with the MATLAB or Octave reference implementation.
C	Comparison with an independent numerical tool or independently implemented model.
D	Published benchmark, theory curve, or literature dataset.
E	Comparison with experimental measurements.
R	Regression, invariant, round-trip, or internal cross-backend check.

The current registry has no level-E claim. Making that absence visible is intentional: analytical, reference-code, and external-tool agreement are strong evidence, but they are not substitutes for comparison with measured data.

Level R is deliberately explicit. It is valuable engineering evidence, but it does not independently establish physical correctness. For example, the experiment facade has strong direct-workflow parity and serialization tests; those validate delegation and reproducibility, while the physical credibility comes from the evidence attached to the delegated workflow.

Statuses are similarly scoped. *Validated* means the stated claim has appropriate evidence over the stated range. *Partially validated* means useful independent or analytical evidence exists but important regimes remain. *Regression only* means behavior is pinned down but lacks an independent physical or external comparison. None of these labels implies unrestricted accuracy outside the recorded range.

13.2 Representative Validation Chains

13.2.1 Bloch Kernels and Historical Workflows

The core Python port uses level-B fixtures generated from the active MATLAB or Octave source tree. They cover rotations, echo conversion, finite acquisition, the `arb1O` kernel, probe responses, CPMG/FID assemblies, imaging, and selected optimization results. Algebraic rotation and conservation identities provide complementary level-A evidence. This combination detects both porting errors and violations that could be shared by two implementations.

Fixtures live in `validation/fixtures`; their generation scripts live in `validation/octave`. Matched-probe fixtures that depend on unavailable Octave optimization behavior are generated with MATLAB. The detailed historical inventory and run log remain in `docs/validation_results.md`.

13.2.2 PGSE and Random-Walker Transport

For rectangular PGSE lobes, the deterministic backend is checked against the Stejskal–Tanner result

$$b = (\gamma G \delta)^2 \left(\Delta - \frac{\delta}{3} \right), \qquad \frac{S}{S_0} = \exp(-bD).$$

Moment equality and the logarithmic diffusion slope are level-A evidence. The separately implemented particle engine is then checked for statistical convergence toward the same attenuation (level C), while zero-gradient, zero-diffusion, seed, and boundary checks supply regression evidence. The claim is intentionally limited to the recorded unrestricted/isotropic and ideal-lobe regimes; restricted geometries need their own references.

The pipe-flow layer follows the same pattern. Closed-form plug and Poiseuille residence-time distributions are checked directly, then compared with independent Monte Carlo sampling. This validates prescribed-profile washout and flux-weighted polarization, not CFD or turbulent flow.

13.2.3 Fields, Circuits, and Thermal Models

Magnetostatic solvers are compared with loop-center Biot–Savart fields, symmetry planes, current scaling, and Halbach orientation limits (level A). PEEC models add selected QOIL regressions and optional FastHenry/FasterCap comparisons (level C); their status remains partial because live external tools are not part of ordinary CI and complex/high-frequency geometries are sparsely sampled.

Thermal networks and conduction solvers are checked against RC exponentials, energy conservation, uniform-source slab/cylinder/sphere solutions, perfusion limits, and advection–diffusion solutions. FEMM scripts provide an independent cross-check for selected conduction/network cases. These results do not extend to general 3-D conjugate heat transfer or CFD.

13.2.4 NQR, ESR, and Quantum Models

NQR validation combines angular-momentum identities and closed-form spin-1 and spin-3/2 transition frequencies (level A) with Konnai SORC theory and a published Glickstein melamine enhancement example (level D). ESR checks use exact spin-1/2 resonance, hyperfine-doublet, lineshape, and Hahn-echo limits. These models are marked partial because experimental absolute-signal libraries and broad higher-spin/multi-electron comparisons remain limited.

Small scalar-coupled systems, relaxation superoperators, spin noise, and optimal control use exact operator identities, known decay/statistical limits, finite-difference gradients, backend parity, and selected MATLAB fixtures. The matrix records which of those checks are independent and which are internal.

13.3 Capability Evidence Matrix

The table below is generated from `validation/evidence.json`. Evidence letters are defined in the preceding section; multiple letters indicate complementary checks. The detailed tested ranges, tolerances, reproducers, references, and limitations are published in `docs/validation_matrix.md`.

Capability	Evidence	Status	Recorded claim
Bloch rotations and acquisition kernels	B, A	validated	Core rotations, echo conversion, acquisition, and arb10 propagation reproduce the established MATLAB/Octave implementation.
CPMG, FID, imaging, and probe workflows	B, R	validated	Public ideal, tuned, untuned, and matched workflow assemblies preserve the reference signal and result conventions.
Probe pulse responses and phase programs	B	validated	Rectangular tuned, untuned, and matched pulse responses and selected phase-program helpers match the MATLAB implementation.
Sequence IR, Pulseq interchange, and compiler	R	regression only	Core Pulseq 1.4/1.5 events retain timing and metadata through import, compilation, plotting data, facade execution, and raster-aligned 1.5.0 round trips.
PGSE diffusion encoding	A, C	validated	The deterministic moment backend follows the Stejskal-Tanner b-value and exp(-bD) attenuation, and free walkers converge toward that result.
Moving-isochromat diffusion and boundaries	A, R	validated	Seeded particle motion has the expected diffusion statistics, boundary behavior, RF rotation, and static-gradient refocusing limits.
Flow washout and transit polarization	A, C	validated	Plug and laminar pipe washout and flux-weighted transit polarization follow their analytical distributions and Monte Carlo estimates.
Magnetostatic field solvers	A, D	validated	Biot-Savart coil fields and selected magnet geometries reproduce analytical center-field, symmetry, scaling, finite-length, and far-field limits; the documented AlNiCo electropermanent rod also reproduces its published surface-field benchmark.

Capability	Evidence	Status	Recorded claim
Electropermanent programming pulses and retained state	A, E, R	partial	The NumPy capacitor/H-bridge/series-RLC integrator reproduces the exact underdamped current solution and configuration-specific fits recover the archived 220/400/600-V peak-current examples; empirical remanence updates enforce their calibrated branch, polarity, range, uncertainty, and provenance.
Electropermanent return-point memory and neighbor coupling	A, R	partial	Weighted play operators close nested reversal loops exactly and obey wiping-out, while finite-geometry two-element coupling has the expected symmetry and sign and the self-demagnetizing programming fixed point converges.
Electropermanent transient winding cross-talk	A, R	partial	The multi-winding integrator reduces to the isolated RLC driver at zero coupling, produces the Lenz-law neighbor-current sign for positive mutual inductance, closes the coupled electrical-energy balance, and propagates pulse-driven disturbance into every return-point state.
Hybrid electropermanent array and operating-state synthesis	A, R	partial	The cached fixed-plus-programmable field basis exactly reproduces direct finite-magnet superposition, and bounded synthesis recovers representable targets while reducing the reference field-off residual and preserving a requested transport-gradient sign.
Nonlinear electropermanent-array imaging	A, R	partial	Retained-state fields generate the expected receiver-demodulated phase matrix, and nonnegative regularized inversion reproducibly reconstructs a noisy two-tissue phantom while localizing its off-center target.
Image-guided superparamagnetic particle transport	A, R	partial	The transport workflow recovers the analytical linear superparamagnetic force and Stokes–Einstein limits, smoothly caps magnetization with a Langevin law, differentiates affine fields correctly, and reproducibly integrates seeded flow/diffusion trajectories with target capture.

Capability	Evidence	Status	Recorded claim
Alternating image-guided EPM therapy controller	A, R	partial	The controller executes contiguous image-program-transport mode intervals, re-localizes a known target from independently seeded acquisitions, re-aims each affine transport objective from the uncaptured-particle centroid, preserves prior captures, and stops reproducibly at its capture goal or cycle limit.
Dynamic-inversion ferromagnetic-particle trap	A, D, R	partial	The workflow executes the published polarize-delay-antialigned-gradient timing, recovers spherical Stokes and Tirado finite-cylinder diffusion limits, reproducibly couples rigid-body and magnetic-moment orientation to translation, distinguishes contraction from rapidly relaxing-moment expansion, and makes coil, EPM-only, and hybrid switching burdens explicit.
PEEC coil impedance and parasitics	C, A	partial	PEEC inductance/resistance trends agree with analytical kernels and selected independent QOIL, FastHenry, and FasterCap comparisons.
Thermal networks, conduction, and electromagnets	A, C	validated	Lumped, one-dimensional, axisymmetric, perfused, and advective thermal models reproduce closed-form limits; coupled electromagnets recover exact RL and electrothermal fixed points, and selected conduction cases are cross-checked with FEMM.
NQR Hamiltonians and pulse models	A, D	partial	Spin-1 and spin-3/2 transition conventions, powder weights, selective pulses, and SORC/SLSE limits reproduce analytical and published theory results.
ESR/EPR spectra and pulsed dynamics	A, D	partial	Zeeman resonance, lineshape, hyperfine splitting, relaxation, and Hahn-echo behavior reproduce analytical spin-1/2 limits.
Small scalar-coupled spin systems	A, R	partial	Dense spin operators, scalar-coupling Hamiltonians, and selected low-field editing models satisfy exact operator and spectral limits.
Singlet, triplet, and parahydrogen state primitives	A, R	validated	Embedded spin-pair singlet/triplet projectors, swap symmetry, singlet-order observables, and para/ortho hydrogen mixtures satisfy exact spin-1/2 algebra and physical density-matrix limits.

Capability	Evidence	Status	Recorded claim
Long-lived singlet and hydrogenative PHIP reference workflows	A, R	partial	SLIC preparation/storage/readout obeys its analytical matching and exponential T_S limits, while hydrogenative product order is physical and linear in para excess and pairwise-addition fraction.
Phenomenological and Redfield relaxation	A, D, E, R	partial	Relaxation propagators preserve required invariants and limiting behavior; measured weak-field 35Cl SLSE relaxation is reproduced; the non-diagonal zero-field Redfield model reproduces published transition-resolved 209Bi FVA rate shapes; and RDX NMR/NQR observations recover the published activated barrier and cross-relaxation assignments.
Inverse-Laplace analysis	R, A	regression only	The 1-D and compact 2-D solvers recover known synthetic peaks and obey kernel and non-negativity constraints.
Spin noise and radiation damping	A, R	validated	Equilibrium spin-noise spectra and stochastic dynamics reproduce fluctuation, radiation-damping, Ornstein-Uhlenbeck, and two-bath analytical identities.
Optimal-control propagation and gradients	C, R, B	partial	NumPy/JAX propagators, control gradients, hardware-response primitives, and bounded objectives agree across implementations and with finite-difference limits.
Unified experiment facade	R	regression only	Facade routes reproduce their direct workflow calls and preserve specifications, configs, arrays, and provenance through round trips.
Reproducible results and provenance	R	regression only	Facade archives identify experiment inputs, implementation, numerical environment, randomness, and result content and can verify an exact rerun while remaining backward-compatible with version-1 archives.
Multi-frequency Bloch-Siegert phase and low-frequency RWA limits	D, A, C	validated	Exact lab-frame paired-offset propagation reproduces the Mandal-2014 Bloch-Siegert phase slope and fitted 90-degree pulse length, while the second-order common phase isolates the counter-rotating contribution omitted by the RWA.

Capability	Evidence	Status	Recorded claim
Arbitrary SequenceIR phase cycling	A, R	validated	Named RF roles in a backend-neutral sequence are programmed branch-by-branch and receiver-combined according to an arbitrary phase table.
Inverse-excitation cancellation diagnostics	A, R	validated	Excitation/inverse spectrum pairs report broadband and peak residuals, inverse coherence, and SNR mismatch with correct analytical limiting behavior.
Numba and JAX acceleration	C, R	validated	Accelerated kernels reproduce the NumPy reference for supported batched rotations, propagation, and optimization primitives.
RFI synthesis and rejection	R	regression only	Reference-channel cancellers preserve protected acquisition windows and suppress known synthetic coherent, colored, and impulsive interference.

13.4 Reading and Extending the Evidence

The generated `docs/validation_matrix.md` is the detailed reference. For each capability it gives the precise tested range and tolerance, links to every reproducer, names external tools or literature results, and states what the claim does not cover. `docs/validation_results.md` remains a historical fixture inventory and run log; it is not the current claim registry.

When adding or changing a numerical model:

1. State one falsifiable claim and the parameter range to which it applies.
2. Choose the strongest appropriate independent evidence; do not relabel a regression test as physical validation.
3. Record the comparison metric, numerical/statistical tolerance, exact reproducer, references, and limitations in `validation/evidence.json`.
4. Regenerate the Markdown and LaTeX summaries and commit them with the model and tests.

Run the registry validator directly with:

```
python docs/generate_validation_matrix.py --check
```

Run without `-check` after editing the registry to regenerate two files:

- `docs/validation_matrix.md`
- `docs/generated/validation_summary.tex`

13.5 Test and Fixture Execution

The hosted smoke gate can be reproduced locally:

```
python -m ruff check src tests examples
python docs/generate_api_reference.py
git diff --exit-code docs/python_api/api_reference.md
python docs/generate_validation_matrix.py --check
python -m unittest tests.smoke_tests
```

Run the full Python suite before merging numerical changes:

```
python -m unittest discover -s tests
```

Regenerate dependency-light fixtures with Octave or MATLAB from the `PythonSpinDynamics` directory:

```
matlab -batch "run('validation/octave/generate_basic_fixtures.m')"
matlab -batch "run('validation/octave/generate_imaging_fixtures.m')"
matlab -batch "run('validation/octave/generate_optimization_fixtures.m')"
matlab -batch "run('validation/octave/generate_pulse_fixtures.m')"
```

Matched-probe fixtures require MATLAB toolboxes used by the reference implementation. Octave fixture generation skips matched-probe files when `fmincon` is unavailable. A successful fixture run establishes level-B parity only for the recorded arrays and parameter sets.

14 API Reference

This chapter documents the intended public surface. Lower-level module imports remain available for validation and porting. For application code, start with:

```
spin_dynamics.workflows
spin_dynamics.parameters
spin_dynamics.analysis
spin_dynamics.coupling
spin_dynamics.nqr
spin_dynamics.noise
spin_dynamics.motion
spin_dynamics.experiment
spin_dynamics.sequences
spin_dynamics.sequences.motion
spin_dynamics.prepolarization
spin_dynamics.relaxation
spin_dynamics.optimization
```

In this chapter. Look up the stable public entry points after choosing a model or workflow in the narrative chapters; this chapter is organized for reference, not sequential reading.

Drop into lower-level modules only when a building block is needed directly. Use `spin_dynamics.coupling` for the scoped dense scalar-coupled spin-1/2 tools described in Chapter 6. Use `spin_dynamics.nqr` for the selective pulsed NQR tools described in Chapter 8.

14.1 Experiment and Sequence APIs

Public object or function	Purpose
`Experiment`	Immutable sample/hardware/sequence/acquisition description with `plan()`, `run()`, JSON conversion, and result provenance.
`CPMG`, `CPMGTrain`, `CPMGIRTrain`, `CPMGImaging`	NMR and imaging sequence specifications supported by the facade.
`PGSE`, `PGSEWalkers`	Deterministic and explicit-particle diffusion specifications.
`TransportDomain2D`, `UniformFlow2D`	Physical density/axis domain and optional uniform velocity for walker transport.
`NQRSLSE`, `NQRSORC`, `ESRFID`, `ESRHahnEcho`	NQR and ESR facade specifications.
`load_config`, `save_config`, `load_run`	Reproducible configuration and result-archive I/O.
`SequenceIR`, `SequenceBlock`	Backend-neutral ordered sequence and concurrent event block.
`RFPulse`, `GradientWaveform`, `ADCEvent`	Sampled sequence events in Pulseq-compatible units.
`read_pulseq`, `write_pulseq`	Pulseq core import and signed 1.5.0 export.

`compile_sequence`	Piecewise-constant RF/gradient timeline with exact ADC sample times.
`compiled_to_motion_steps`	Adapter for the moving-isochromat engine.
`plot_sequence`, `sequence_plot_data`	Aligned timeline display and its backend-neutral plotting data.

14.2 Workflow Results

Class	Purpose
`NMRDWorkflowResult`	Condition-by-field BPP dispersion grid with field and frequency axes, rates, times, and spectral-density terms.
`QuadrupolarRelaxationWorkflowResult`	Transition-resolved initial population and coherence decay rates with NQR labels and frequencies.
`CPMGResult`	Asymptotic CPMG spectrum, echo, time vector, SNR, probe label, and phase-cycle metadata.
`CPMGTrainResult`	Finite echo-train spectra, echoes, echo integrals, sequence timing, and phase-cycle metadata.
`CPMGIRTrainResult`	Inversion-recovery finite trains over a `tauvect`.
`MatchedCPMGIRTrainResult`	Matched-probe specialization of the CPMG-IR train result.
`CPMGParameterSweepResult`	Asymptotic Q or mistuning sweeps.
`CPMGFiniteParameterSweepResult`	Finite-train Q or mistuning sweeps.
`ZMagnetizationSweepResult`	Matched-probe z-magnetization Q sweeps.
`IdealTimeVaryingCPMGResult`	Final echo for ideal time-varying-field CPMG.
`ProbeTimeVaryingCPMGResult`	Probe-aware time-varying-field CPMG result.
`IdealCPMGImagingResult`	Ideal phase-encoded CPMG imaging result.
`ProbeCPMGImagingResult`	Tuned or matched phase-encoded CPMG imaging result.
`FrequencyEncodedImagingResult`	Spin-warp / RARE frequency-encoded image, k-space, and per-line echo times.
`SliceSensitivityResult`	Real-space sensitive slice (excited/refocused, B1-weighted) of an excitation in a non-uniform field.
`ImagingEchoFitResult`	Voxel-wise mono-exponential echo-stack fit.
`ImagingNoiseStatistics`	Repeated-trial image-domain noise summary.
`MotionSequenceResult`	Moving-isochromat sequence samples and final particle ensemble.
`HalbachDipoleFieldMaps`	Sampled 3-D finite Halbach dipole field maps, including rods and Cartesian field components.
`ElectropermanentFieldMaps`	Sampled vector, projected, magnitude, gradient, and Larmor maps for EPM rods or bundles, with imaging and motion adapters.
`PulseWaveform`	Realized EPM programming current, capacitor/bridge voltage, internal-field estimate, losses, and lumped temperatures.

`ProgrammingResult`	Protocol-calibrated retained-state transition with initial state, final state, waveform, command, uncertainty, and evidence.
`ReturnPointTrace`	Applied-field, retained-remanence, and complete hidden-state history for a weighted-play propagation.
`CoupledProgrammingResult`	Converged EPM source states, interaction matrix, bias fields, pulse waveform, iteration count, and fixed-point residual.
`ArrayPulseWaveform`	Multi-winding currents, induced voltages, programming fields, losses, temperatures, and coupled electrical-energy residual.
`TransientProgrammingResult`	Time-resolved retained states and converged final EPM sources, including non-target disturbed-element indices.
`ElectropermanentArrayFieldBasis`	Cached fixed vector field and one vector-field column per tesla of retained remanence for every programmable array element.
`ArrayStateSynthesisResult`	Bounded imaging, field-off, transport, or custom target solve with retained states, achieved field, residuals, saturation, and solver diagnostics.
`TissuePhantom2D`	Cartesian proton-density, T1, T2, tissue-label, and off-center-target maps for the illustrative imaging example.
`NonlinearEPMEncoding`	Retained states, projected field profiles, receiver-demodulated phase, and dense nonlinear encoding matrix.
`EPMNonlinearImagingResult`	Clean/noisy acquisitions, nonnegative regularized reconstruction, convergence diagnostics, and image NRMSE.
`SuperparamagneticParticle`	Magnetic volume and susceptibility, Langevin saturation, Stokes drag, and Stokes–Einstein diffusion for a particle or effective aggregate.
`MagneticForceMap2D`	Sampled vector field and bilinearly interpolated $\nabla\|B\|^2$ force precursor on an (x, y) plane.
`MagnetophoreticTransportResult`	Seeded particle histories, force and drift histories, capture times, capture fraction, and force/speed metrics.
`EPMTherapyControllerConfig`	Imaging, programming, and transport timing; field objective; reconstruction settings; capture goal; and cycle limit.
`EPMTherapyCycleResult`	One reconstructed target, feedback direction, synthesized state, force map, trajectory burst, programming effort, and capture increment.
`EPMTherapyControllerResult`	Complete mode timeline, per-cycle localization and capture, stitched trajectories, stopping reason, and total remanence variation.
`FerromagneticParticle`	Sphere or finite-cylinder geometry, remanent moment, anisotropic drag/diffusion, and mechanical/internal orientation-memory scales.

`DynamicInversionSequence`	Polarize-delay-antialigned-gradient timing, direction cycle, inferred inverse-power source geometry, and active duty cycle.
`DynamicInversionResult`	Non-sticky particle/body/moment histories with RMS-radius concentration, retention, escape, and repulsive-phase metrics.
`DynamicInversionHardwareAssessment`	Coil pulses, EPM retained states and channel pulses, transition delay, duty, optional energy/power, and waveform-feasibility flags.
`NoiseMetadata`	Generated received-noise parameters and realized SNR.
`MatchedDiffusionCPMGResult`	Matched diffusion-aware CPMG train.
`MatchedDiffusionQSweepResult`	Matched diffusion Q sweep.
`PGSEMomentResult`	Deterministic PGSE or PGSE-prepared CPMG attenuation from gradient moments.
`PGSEWalkerResult`	Random-walker PGSE signal, sequence samples, and initial particle ensemble.
`PGSTEWalkerResult`	Random-walker stimulated-echo (PGSTE) signal, storage interval, selected-pathway phase-cycle metadata, and initial particle ensemble.
`QSpaceReconstructionResult`	Direct q-space inverse image or autocorrelation with matched real/q axes.
`QSpacePhaseRetrievalResult`	Magnitude-only q-space pore-shape estimate, support mask, and residual history.
`DDEWalkerResult`	Random-walker double diffusion encoding (DDE) signal, encoding angles, and mixing time.
`OGSEWalkerResult`	Random-walker oscillating-gradient spin-echo (OGSE) signal, oscillation frequency, and b-value.
`BipolarPGSTEResult`	Cotts 13-interval bipolar PGSTE moment model: b-value, background cross-term bias, and the `diff_stebp` phase cycle.
`BipolarPGSTEWalkerResult`	Explicit moving-walker bipolar 13-interval PGSTE: acquired echo, moment-model b-value, sequence, and ensemble.
`WURSTInversionResult`	WURST inversion profile and pulse metadata.
`MatchedWURSTCPMGResult`	Matched WURST-CPMG echoes and spectra.
`RadiationDampingFIDResult`	Radiation-damping FID simulation and analytic comparison fields.
`MultiStartOptimizationResult`	Repeated random-start pulse-optimization result.
`RefocusingOptimizationResult`	Bounded fixed-amplitude refocusing phase optimization.
`ExcitationOptimizationResult`	Bounded fixed-amplitude target-excitation or inverse-excitation optimization.
`TunedExcitationInversePipelineResult`	Selected-refocusing to excitation/inverse-excitation pipeline result.
`SLSEResult`	NQR SLSE echo train, local orientation signals, and transition metadata.
`SLSESweepResult`	NQR SLSE selected-echo amplitudes and effective decay estimates across a sweep.

`NQRFIDDistributionResult`	EFG-distribution FID, FFT spectrum, and isochromat metadata.
`SLSEAcquisitionSpectrumResult`	Finite-window SLSE echo acquisition and spectrum, including optional noise and deconvolution metadata.
`WeakB0SpectrumResult`	Static weak-B0 transition spectrum and perturbation-ratio metadata.
`PopulationTransferResult`	NQR perturbation-plus-SLSE signal, reference signal, and normalized difference.
`PolarizationTransferResult`	Polarization-enhanced NQR transport result with sampled trajectory fields, adiabaticity metrics, and ideal enhancement factors.
`ProtonCouplingEstimate`	CIF-based dipole-dipole proton coupling estimate for one quadrupolar nucleus.
`RelaxationExchange2DResult`	Encode-mix-detect T2-T2 relaxation exchange (REXSY) data and longitudinal mixing propagator.

14.3 Prepolarization and Relaxation API

```
from spin_dynamics.prepolarization import (
    PrepolarizedMagnetization,
    longitudinal_recovery,
    field_ratio_equilibrium,
    prepolarized_magnetization,
    prepolarized_state,
    residence_time_seconds,
    prepolarized_flow_state,
    apply_prepolarization_to_parameters,
)

from spin_dynamics.relaxation import (
    BPPRelaxationModel,
    BPPRelaxationRates,
    DipolarRelaxationSource,
    IsotropicLiquidMotionalAveraging,
    PhenomenologicalRelaxationModel,
    RedfieldDipolarRelaxationModel,
    RedfieldEFGRelaxationModel,
    RigidSolidMotionalAveraging,
    spectral_density_lorentzian,
    arrhenius_correlation_time,
    fit_arrhenius_observable,
    bpp_relaxation_rates,
    apply_relaxation_to_parameters,
)
```

Symbol	Purpose
`PrepolarizedMagnetization`	Prepared `m0`, sequence `mth`, and enhancement arrays for handoff to Bloch kernels or moving isochromats.

`prepolarized_state`	Field-ratio-normalized finite-time prepolarization for static samples.
`prepolarized_flow_state`	Same buildup model with residence time computed from path length and speed.
`BPPRelaxationModel`	Arrhenius $\tau_c(T)$, Lorentzian spectral-density, and configurable BPP R_1/R_2 model.
`BPPRelaxationRates`	Temperature, τ_c, $J(0)$, $J(\omega_0)$, $J(2\omega_0)$, R_1/R_2, and T_1/T_2 arrays.
`PhenomenologicalRelaxationModel`	Shared Liouville-space population/coherence damping from supplied T_1/T_2 values.
`DipolarRelaxationSource`	Target–bath dipolar source geometry, gyromagnetic ratios, bath spin, and optional explicit coupling.
`RigidSolidMotionalAveraging`	Fixed-frame dipolar covariance for rigid solids and NQR-style orientation-dependent relaxation.
`IsotropicLiquidMotionalAveraging`	Rotationally averaged dipolar covariance for liquid-state NMR.
`RedfieldDipolarRelaxationModel`	Secular Markovian Liouville-space dissipator built from stochastic dipolar bath sources.
`RedfieldEFGRelaxationModel`	Completely-positive secular dissipator for a fluctuating electric-field gradient.
`fit_arrhenius_observable`	Weighted log-linear activation-energy fit for a positive observable in one activated regime.

14.4 High-Level Workflows

```
from spin_dynamics.workflows import (
    NMRDWorkflowResult, QuadrupolarRelaxationWorkflowResult,
    run_nmrd, run_field_cycling_nmrd,
    run_quadrupolar_relaxation,
    run_arrhenius_quadrupolar_relaxation,
    run_ideal_cpmg, run_tuned_cpmg, run_untuned_cpmg, run_matched_cpmg,
    run_ideal_cpmg_train, run_tuned_cpmg_train,
    run_untuned_cpmg_train, run_matched_cpmg_train,
    run_ideal_cpmg_ir_train, run_tuned_cpmg_ir_train,
    run_untuned_cpmg_ir_train, run_matched_cpmg_ir_train,
    run_tuned_q_sweep, run_matched_q_sweep,
    run_tuned_mistuning_sweep, run_matched_mistuning_sweep,
    run_tuned_finite_q_sweep, run_untuned_finite_q_sweep,
    run_matched_finite_q_sweep,
    run_tuned_finite_mistuning_sweep,
    run_untuned_finite_mistuning_sweep,
    run_matched_finite_mistuning_sweep,
    run_matched_z_magnetization_q_sweep,
    run_matched_diffusion_cpmg, run_matched_diffusion_q_sweep,
    run_pgse, run_pgse_moment, run_pgse_walkers, run_pgste_walkers,
    run_dde_walkers, run_ogse_walkers,
    pgse_b_value, gradient_moment_b_value,
    run_ideal_time_varying_cpmg_final,
    run_tuned_time_varying_cpmg_final,
    run_untuned_time_varying_cpmg_final,
```

```
    run_matched_time_varying_cpmg_final,
    run_ideal_time_varying_amplitude_sweep,
    run_tuned_time_varying_amplitude_sweep,
    run_untuned_time_varying_amplitude_sweep,
    run_matched_time_varying_amplitude_sweep,
    sinusoidal_field_waveform,
    run_ideal_wurst_inversion,
    run_matched_wurst_inversion,
    run_matched_wurst_cpmg,
    run_radiation_damping_fid,
)
```

The main CPMG runners share `numpts` and `maxoffs`. Finite train runners add `num_echoes`, relaxation constants, rephasing controls, optional `auto_refine_grid`, and optional worker controls. Probe-aware train runners also accept `q_value` and `mistuning_offset` where the underlying probe model supports them.

14.5 Imaging API

```
from spin_dynamics.workflows import (
    ImagingFieldMaps,
    make_imaging_field_maps,
    load_imaging_field_maps_npz,
    run_ideal_phase_encoded_cpmg_imaging,
    run_tuned_phase_encoded_cpmg_imaging,
    run_matched_phase_encoded_cpmg_imaging,
    run_t1_encoded_phase_encoded_cpmg_imaging,
    run_spin_warp_imaging, run_rare_imaging, imaging_slice_sensitivity,
    make_slice_selective_excitation, simulate_slice_profile,
    slice_excitation_weights, slice_profile_table,
    run_multislice_imaging, run_multislice_imaging_separable,
    MultiSliceImagingResult,
    reconstruct_image_from_kspace,
    form_imaging_image,
    fit_imaging_echo_decay,
    summarize_imaging_noise_trials,
    TissuePhantom2D,
    NonlinearEPMEncoding,
    EPMNonlinearImagingResult,
    simple_tissue_phantom,
    random_epm_encoding_states,
    build_epm_nonlinear_encoding,
    reconstruct_epm_nonlinear_image,
    run_epm_nonlinear_imaging,
)
```

Prefer the `phase_encoded` names for new code. The shorter compatibility aliases remain available:

```
run_ideal_cpmg_imaging
run_tuned_cpmg_imaging
run_matched_cpmg_imaging
```

14.6 Parameter Constructors

```
from spin_dynamics.parameters import (
    set_params_ideal,
    set_params_ideal_fid,
    set_params_tuned_orig,
    set_params_untuned_orig,
    set_params_matched_orig,
    set_params_tuned_spa,
    set_params_untuned_spa,
    set_params_matched_spa,
    set_params_tuned_jmr,
    set_params_untuned_jmr,
    set_params_matched_jmr,
)
```

These constructors return dataclass instances mirroring MATLAB `sp`, `pp`, and `params` structures. They accept optional `numpts` arguments for lightweight tests and examples while preserving MATLAB defaults when omitted.

14.7 Analysis API

```
from spin_dynamics.analysis import (
    t2_kernel, t1_kernel, diffusion_kernel, laplace_kernel,
    invert_laplace_1d, invert_laplace_2d,
    invert_t2, invert_t1, invert_t1_t2, invert_d_t2, invert_t2_t2,
    default_regularization_strengths,
    estimate_noise_rms_from_snr,
    expected_residual_norm_from_snr,
    select_regularization_1d,
    select_regularization_2d,
)
```

For one-dimensional relaxation distributions, use:

```
invert_t2
invert_t1
```

For separable two-dimensional maps, use:

```
invert_t1_t2
invert_d_t2
invert_t2_t2
```

The `select_regularization_*` helpers choose Tikhonov strengths from an RMS SNR estimate.

14.7.1 Chemical / Site Exchange

The `spin_dynamics.exchange` namespace provides the Bloch-McConnell site exchange model described in Section 11.18:

```
from spin_dynamics.exchange import (
    ExchangeSite, ExchangeSystem, RelaxationExchange2DResult,
    two_site_exchange, exchange_generator, transverse_generator,
    transverse_propagator, mixing_propagator,
    simulate_exchange_fid, exchange_spectrum,
    simulate_relaxation_exchange_2d,
)
```

14.7.2 Internal / Susceptibility Gradients

The `spin_dynamics.susceptibility` namespace provides the internal-field generator described in Section 11.19:

```
from spin_dynamics.susceptibility import (
    CylindricalInclusion, SusceptibilityField, InternalGradientDistribution,
    susceptibility_offresonance_map, make_susceptibility_field_maps,
    internal_gradient_maps, internal_gradient_distribution,
)
```

14.8 Optimization Diagnostics

Optimization helpers expose compact NumPy/SciPy-compatible diagnostics for phase-program searches:

```
from spin_dynamics.optimization import run_tuned_refocusing_multistart

result = run_tuned_refocusing_multistart(num_segments=3, num_starts=4)
```

The plotting-free analysis helpers convert MATLAB-style result cells into score summaries, and the tuned excitation/inverse pipeline carries a selected refocusing candidate into bounded target-excitation and inverse-cancellation tests. The inverse stage is still best treated as a parity and diagnostics surface while strong inverse-pulse cancellation remains under validation.

14.9 Core Numerical API

```
from spin_dynamics.core.echo import (
    calc_time_domain_echo,
    calc_time_domain_echo_arb,
    calc_fid_time_domain,
)
from spin_dynamics.core.rotations import (
    calc_rotation_matrix,
    calc_rot_axis_arba3,
    calc_rot_axis_arba4,
    calc_v0crit,
    sim_spin_dynamics_asymp_mag3,
    sim_spin_dynamics_exc,
)
from spin_dynamics.core.kernels import (
    sim_spin_dynamics_arb7,
    sim_spin_dynamics_arb10,
```

```
    sim_spin_dynamics_arb10_chunked,
    sim_spin_dynamics_arb10_diffusion,
    sim_spin_dynamics_arb10_radiation_damping,
)
from spin_dynamics.core.isochromats import (
    analyze_rephasing,
    check_rephasing,
    estimate_rephase_time,
    recommended_numpts_for_rephasing,
)
```

Core functions are intended for validation, debugging, and continued porting. They are closer to MATLAB's array contracts than the workflow layer.

14.10 Probe and Pulse API

```
from spin_dynamics.probes.tuned import (
    tuned_probe_lp,
    tuned_probe_rx,
    tuned_probe_rx_tf,
    calc_rot_axis_tuned_probe_lp_orig2,
    calc_masy_tuned_probe_lp_orig,
)
from spin_dynamics.probes.untuned import (
    untuned_probe_lp,
    untuned_probe_rx,
    untuned_probe_rx_tf,
    calc_rot_axis_untuned_probe_lp,
    calc_masy_untuned_probe_lp,
)
from spin_dynamics.probes.matched import (
    matching_network_design2,
    find_coil_current,
    find_coil_current_wurst,
    matched_probe_rx,
    calc_rot_axis_matched_probe,
    calc_masy_matched_probe_orig,
    calc_masy_matched_nut,
)
from spin_dynamics.pulses import (
    quantize_phase,
    create_wurst_pulse,
    adjust_untuned_segment_lengths,
    tuned_rectangular_pulse_response,
    untuned_rectangular_pulse_response,
    matched_rectangular_pulse_response,
    matched_wurst_pulse_response,
)
```

14.11 Noise, Motion, and Radiation Damping

```
from spin_dynamics.noise import (
    NoiseSpec,
    NoiseMetadata,
    MatchedFilterSNRResult,
    add_received_noise,
    estimate_matched_filter_snr,
    ideal_noise_density,
    tuned_probe_output_noise_density,
    untuned_probe_output_noise_density,
    matched_probe_output_noise_density,
)
from spin_dynamics.motion import (
    ParticleEnsemble,
    MotionFieldMaps2D,
    MotionFieldMaps,
    make_motion_field_maps_2d,
    make_motion_field_maps,
    initialize_ensemble_from_density,
    initialize_ensemble_from_domain,
    make_semipermeable_plane,
    advect_diffuse_positions,
    move_ensemble,
    apply_free_precession,
    apply_rf_rotation,
    free_precession_with_motion_step,
    receive_signal,
    transverse_b1_magnitude,
)
from spin_dynamics.sequences.motion import (
    MotionSequenceStep,
    MotionSequenceResult,
    make_motion_cpmg_sequence,
    make_motion_udd_sequence,
    run_motion_sequence,
    run_motion_cpmg_sequence,
    run_motion_udd_sequence,
)
from spin_dynamics.fields import (
    SpatialDomain,
    SpatialFieldMaps,
    AlNiCoMaterial,
    RemanenceState,
    ElectropermanentRod,
    ElectropermanentBundle,
    ElectropermanentFieldMaps,
    PulseThermalState,
    ProgrammingPulse,
    PulsePowerDriver,
    PulseWaveform,
    EmpiricalProgrammingCalibration,
    ProgrammingResult,
    ReturnPointState,
    ReturnPointTrace,
    PlayHysteresisModel,
    CoupledProgrammingResult,
```

```
    CoupledEPMProgrammer,
    ArrayPulseWaveform,
    MutualProgrammingCircuit,
    TransientProgrammingResult,
    TransientCoupledEPMProgrammer,
    HybridEPMSubunit,
    ElectropermanentArray,
    ElectropermanentArrayFieldBasis,
    ArrayStateSynthesisResult,
    illustrative_hybrid_epm_array,
    affine_transport_target,
    synthesize_epm_array_state,
    synthesize_uniform_imaging_state,
    synthesize_field_off_state,
    synthesize_transport_state,
    archived_igbt_pulse_cases,
    illustrative_alnico_return_point_model,
    neighbor_coupling_matrix,
    published_demagnetization_calibration,
    variable_field_nmr_rod,
    weinberg_37_rod_bundle,
    sample_electropermanent_field,
    FiniteMagnetRod,
    HalbachDipoleFieldMaps,
    finite_magnet_array_b0,
    halbach_dipole_magnets,
    sample_halbach_dipole_field,
)
from spin_dynamics.fields.magnetostatics import (
    BarMagnet,
    FiniteMagnetRod,
    HalbachDipoleFieldMaps,
    bar_array_b0,
    biot_savart,
    circular_loop,
    finite_magnet_array_b0,
    halbach_dipole_magnets,
    nmr_mouse_magnets,
    sample_halbach_dipole_field,
    sample_magnet_field,
)
from spin_dynamics.workflows import (
    make_slice_selective_excitation,
    simulate_slice_profile,
    run_multislice_imaging,
    run_multislice_imaging_separable,
)
from spin_dynamics.workflows.single_sided import (
    SampleLayer,
    LayeredSample,
    mouse_depth_profile,
    measure_diffusion_at_depth,
    simulate_mouse_cpmg,
    resonant_depth,
)
```

```
from spin_dynamics.sequences import (
    cpmg_pulse_times,
    udd_pulse_times,
    interval_durations,
    toggling_frame_integral,
    dephasing_filter_function,
)
from spin_dynamics.radiation_damping import (
    radiation_damping_time,
    proton_thermal_magnetization_density,
    water_proton_sample,
    hyperpolarized_proton_sample,
    normalized_radiation_damping_weights,
    radiation_damping_probe_from_tuned,
    radiation_damping_probe_from_matched,
    initial_state_from_flip_angle,
    analytic_radiation_damping_envelope,
    simulate_radiation_damping,
    simulate_radiation_damping_fid,
    simulate_nmr_maser,
)
```

14.12 Scalar Coupling API

```
from spin_dynamics.coupling import (
    CoupledSpinSystem,
    CoupledIsochromatEnsemble,
    CoupledIsochromatStep,
    CoupledIsochromatSequenceResult,
    coupled_spin_system,
    coupled_isochromat_ensemble,
    free_precession_step,
    rf_step,
    simulate_coupled_isochromat_sequence,
    spin_operator,
    total_operator,
    product_operator,
    zeeman_hamiltonian,
    secular_j_hamiltonian,
    isotropic_j_hamiltonian,
    rf_hamiltonian,
    equilibrium_density,
    evolve_density,
    expectation,
    j_modulation_curve,
    proton_detected_j_modulation,
    carbon_detected_j_modulation,
    tango_b_filter,
    fit_known_j_spectrum,
    JEditingFitResult,
    simulate_slic_spectrum,
    two_spin_slic_matching_nutation_hz,
    two_spin_slic_transfer_time,
```

```
    SLICSpectrumResult,
)
```

Object	Purpose
`CoupledSpinSystem`	Validated small spin-1/2 system with offsets and scalar couplings in hertz.
`CoupledIsochromatEnsemble`	Ensemble of local B0/B1 field samples for one coupled-spin system.
`free_precession_step`, `rf_step`	Sequence-step builders with optional time-varying B0/B1 overrides.
`simulate_coupled_isochromat_sequence`	Dense coupled-spin sequence propagation over an isochromat ensemble.
`spin_operator`, `total_operator`, `product_operator`	Dense spin-1/2 operator builders.
`zeeman_hamiltonian`	Rotating-frame offset Hamiltonian in radians per second.
`secular_j_hamiltonian`	Weak-coupling $I_z I_z$ scalar Hamiltonian.
`isotropic_j_hamiltonian`	Isotropic scalar Hamiltonian for strongly coupled spin-1/2 systems.
`rf_hamiltonian`	RF nutation Hamiltonian for selected spins.
`equilibrium_density`, `evolve_density`, `expectation`	Minimal dense density-matrix utilities.
`j_modulation_curve`	Sum of cosine J-modulation components.
`proton_detected_j_modulation`	Proton-detected low-field J-editing response.
`carbon_detected_j_modulation`	Carbon-detected $\cos^n$ J-editing response for CH_n groups.
`tango_b_filter`	Ideal TANGO-B scalar-coupling filter envelope.
`fit_known_j_spectrum`	Least-squares amplitudes for supplied J-coupling positions.
`simulate_slic_spectrum`	Spin-lock response scan for dense coupled-spin systems.

14.13 ESR API

```
from spin_dynamics.esr import (
    ESRSpinSystem,
    ESRRelaxationModel,
    ESRDistributionSample,
    NuclearSite,
    ElectronNuclearSystem,
    resonance_frequency_hz,
    resonance_field_tesla,
    effective_g_value,
    simulate_field_sweep,
    simulate_frequency_spectrum,
    static_disorder_grid,
```

```
    simulate_field_sweep_distribution,
    simulate_frequency_spectrum_distribution,
    gaussian_lineshape,
    lorentzian_lineshape,
    flip_angle_duration,
    simulate_fid,
    simulate_hahn_echo,
    electron_nuclear_system,
    diagonalize_hyperfine_system,
    simulate_hyperfine_field_sweep,
)
```

Object	Purpose
`ESRSpinSystem`	Single-electron spin-1/2 ESR system with scalar, diagonal, or full g tensor.
`resonance_frequency_hz`, `resonance_field_tesla`	Convert between field and microwave frequency for one ESR orientation.
`effective_g_value`	Direction-dependent $g_{\mathrm{eff}} = \lvert\mathbf{g}^{\mathsf{T}}\hat{\mathbf{n}}\rvert$.
`simulate_field_sweep`, `simulate_frequency_spectrum`	Single-crystal or powder CW ESR spectra with Gaussian/Lorentzian absorption or derivative display.
`gaussian_lineshape`, `lorentzian_lineshape`	Unit-height absorption profiles and first derivatives.
`static_disorder_grid`	Weighted diagonal g-strain and applied-field-offset samples.
`simulate_field_sweep_distribution`	Field-swept ESR spectrum averaged over static disorder samples.
`flip_angle_duration`	Rectangular-pulse duration for a spin-1/2 flip angle and nutation rate.
`simulate_fid`, `simulate_hahn_echo`	Rotating-frame pulsed ESR FID and Hahn-echo density-matrix simulations.
`ESRRelaxationModel`	Phenomenological Liouville-space T_1/T_2 relaxation model for pulsed ESR.
`NuclearSite`, `ElectronNuclearSystem`	Dense electron-nuclear product-space hyperfine model containers.
`electron_nuclear_system`	Convenience builder for isotropic electron-nuclear hyperfine systems.
`diagonalize_hyperfine_system`	Exact dense energy levels and ESR-active transitions for a hyperfine system.
`simulate_hyperfine_field_sweep`	Field-swept ESR spectrum including isotropic electron-nuclear hyperfine coupling.

14.14 Interference and RFI API

```
from spin_dynamics.interference import (
    AcquisitionMask,
    SampleLabel,
    mask_from_intervals,
    RFIWaveform,
```

```
    tone_waveform,
    am_carrier_waveform,
    chirp_waveform,
    colored_noise_waveform,
    impulsive_waveform,
    UniformPlaneWaveSource,
    MagneticDipoleSource,
    ReferenceCoil,
    coil_voltage,
    reference_matrix,
    scalar_canceller,
    gated_ridge_fir_canceller,
    robust_fir_canceller,
    sparse_reference_canceller,
    frequency_domain_canceller,
    kalman_harmonic_canceller,
    windowed_ridge_fir_canceller,
    adaptive_lms_canceller,
    adaptive_rls_canceller,
    joint_signal_reference_canceller,
    windowed_joint_signal_reference_canceller,
    rfi_suppression_db,
    matched_filter_snr_improvement,
    signal_bias,
    residual_spectral_lines,
    reference_design_diagnostics,
    reference_noise_injection,
    saturation_diagnostics,
    CompensationActuator,
    feedforward_cancel,
    RFIRecording,
    save_rfi_recording,
    load_rfi_recording,
)

from spin_dynamics.nqr import slse_acquisition_mask, sorc_acquisition_mask
```

Object	Purpose
`AcquisitionMask`, `SampleLabel`	Per-sample transmit, ringdown, signal, and baseline labels on one shot clock.
`mask_from_intervals`	Build a generic acquisition mask from second-valued intervals.
`slse_acquisition_mask`, `sorc_acquisition_mask`	Convert NQR SLSE or SORC sequence timing into an acquisition mask with optional baseline windows.
`RFIWaveform`	Real sampled interference waveform with a sample rate.
`tone_waveform`, `am_carrier_waveform`	Narrowband and amplitude-modulated RFI waveform helpers.
`chirp_waveform`, `colored_noise_waveform`, `impulsive_waveform`	Broadband, drifting, and transient RFI waveform helpers.

`UniformPlaneWaveSource`	Far-field RFI source with spatially uniform vector polarization.
`MagneticDipoleSource`	Near-field magnetic-dipole source with $1/r^3$ spatial dependence.
`ReferenceCoil`	Pickup coil with vector sensitivity, position, optional noise, saturation, and NQR leakage.
`coil_voltage`, `reference_matrix`	Simulate one pickup channel or stacked reference matrix for a set of sources.
`scalar_canceller`, `gated_ridge_fir_canceller`	Stationary masked least-squares reference cancellers.
`robust_fir_canceller`	Outlier-robust Huber IRLS reference canceller for impulsive switching-transient pickup.
`sparse_reference_canceller`	Splits the primary into reference-explained RFI and an L1-sparse impulse term, modelling and removing the bursts.
`frequency_domain_canceller`	Per-frequency multi-reference Wiener canceller (WOLA) for frequency-dependent coupling; returns the transfer and coherence spectra.
`kalman_harmonic_canceller`	Reference-free Kalman tracker of drifting narrowband carriers, with mask-aware update freezing.
`windowed_ridge_fir_canceller`	Offline RFI canceller with one regularized reference transfer function per time window.
`adaptive_lms_canceller`, `adaptive_rls_canceller`	Mask-aware adaptive cancellers whose coefficient updates can freeze during signal windows.
`joint_signal_reference_canceller`	Fixed joint fit of reference-derived RFI and a supplied signal basis.
`windowed_joint_signal_reference_canceller`	Offline echo-aware joint fit with windowed reference coefficients and global signal-basis coefficients.
`rfi_suppression_db`	RMS RFI suppression in dB, optionally relative to a known clean signal.
`matched_filter_snr_improvement`	Matched-filter SNR before and after cleanup.
`signal_bias`	Complex gain, amplitude bias, and phase bias of the recovered signal.
`residual_spectral_lines`	Largest residual FFT lines after cancellation.
`reference_design_diagnostics`	Rank and condition number of a tapped reference design matrix.
`reference_noise_injection`	White noise a canceller injects into the cleaned output from noisy references (the "noise figure" of cancellation).
`saturation_diagnostics`	Samples whose magnitude reaches or exceeds a specified front-end threshold.
`CompensationActuator`, `feedforward_cancel`	Active feedforward: a compensation coil subtracts RFI before the ADC, with latency, drive-range, gain/phase, and noise limits.
`RFIRecording`, `save_rfi_recording`, `load_rfi_recording`	Container and ".npz" I/O pairing measured ADC windows with a reconstructed acquisition mask.
`nqr_recording_from_samples`, `slse_/sorc_mask_from_metadata`	Rebuild the gating for a window of ADC samples from echo-spacing metadata (in `spin_dynamics.nqr`).

14.15 NQR API

```
from spin_dynamics.nqr import (
    QuadrupolarSite,
    NQRTransition,
    NQREigensystem,
    spin_matrices,
    quadrupole_frequency_scale_hz,
    quadrupole_hamiltonian,
    zeeman_hamiltonian,
    nqr_hamiltonian,
    diagonalize_site,
    OrientationSample,
    single_crystal_orientation,
    powder_average_grid,
    b0_b1_powder_average_grid,
    b0_powder_average_grid,
    SelectivePulse,
    apply_selective_pulse,
    transition_drive_scale,
    NQRRelaxationModel,
    slse_sequence,
    sorc_sequence,
    simulate_slse,
    simulate_sorc,
    simulate_slse_offset_sweep,
    simulate_slse_spacing_sweep,
    sorc_powder_theory_signal,
    sorc_powder_pathway_signal,
    fid_powder_theory_signal,
    gaussian_efg_distribution,
    temperature_efg_distribution,
    simulate_fid_efg_distribution,
    simulate_slse_acquisition_spectrum,
    simulate_weak_b0_spectrum,
    weak_field_ratio,
    zeeman_frequency_hz,
    simulate_population_transfer,
    PolarizationEnhancedNQRSample,
    CylindricalSampleGeometry,
    LinearTransportMotion,
    HalbachPrepolarizationMagnet,
    ideal_spin1_enhancement_factors,
    simulate_adiabatic_polarization_transfer,
    load_cif_structure,
    dipolar_coupling_hz,
    estimate_proton_dipolar_couplings,
    estimate_proton_dipolar_couplings_from_cif,
    SLSEResult,
    PopulationTransferResult,
    PolarizationTransferResult,
    ProtonCouplingEstimate,
    WeakB0SpectrumResult,
)
```

Object	Purpose
`QuadrupolarSite`	One quadrupolar nucleus in the EFG principal-axis system.
`spin_matrices`	Dense angular-momentum operators for arbitrary integer or half-integer spin.
`quadrupole_frequency_scale_hz`	Spin-dependent Hamiltonian scale for spin-1 and spin-3/2 frequency conventions.
`quadrupole_hamiltonian`	Zero-field quadrupole Hamiltonian in radians per second.
`zeeman_hamiltonian`	Optional weak-B0 perturbation in radians per second.
`diagonalize_site`	Energy levels, transition frequencies, and transition dipole vectors.
`OrientationSample`	One EFG orientation relative to lab RF and optional B0 fields.
`single_crystal_orientation`	One fixed orientation sample.
`powder_average_grid`	Normalized spherical powder quadrature grid.
`b0_b1_powder_average_grid`	Powder grid with correlated static-field and RF-field directions.
`SelectivePulse`	Rectangular selective pulse on one transition.
`apply_selective_pulse`	Embedded two-level RF propagation in the energy basis.
`NQRRelaxationModel`	Phenomenological Liouville-space population/coherence relaxation rates.
`slse_sequence`	Convenience builder for SLSE detection.
`sorc_sequence`	Convenience builder for the strong off-resonance comb sequence.
`simulate_slse`	Selective-pulse SLSE echo train for a fixed orientation or powder.
`simulate_sorc`	Finite-pulse SORC pathway train by default, with optional coherent transient propagation.
`simulate_slse_offset_sweep`, `simulate_slse_spacing_sweep`	SLSE diagnostics versus RF offset or pulse period.
`sorc_powder_theory_signal`, `sorc_powder_pathway_signal`	Infinite-N and finite-N powder SORC responses for Konnai-style comparisons.
`fid_powder_theory_signal`	Spin-1 powder FID pulse-width response used beside SORC pulse-width sweeps.
`gaussian_efg_distribution`, `temperature_efg_distribution`	Static EFG isochromat distributions.
`simulate_fid_efg_distribution`	Time-domain FID and FFT spectrum from an EFG distribution.
`simulate_slse_acquisition_spectrum`	Finite-window acquired SLSE echo and spectrum, with noise and optional deconvolution.
`simulate_weak_b0_spectrum`	Static weak-B0 transition spectrum for spin-1 or spin-3/2.
`weak_field_ratio`, `zeeman_frequency_hz`	Weak-field regime checks based on $\lvert\gamma B_0\rvert/\nu_{\mathrm{ref}}$.

`simulate_population_transfer`	Perturbation pulse plus SLSE detection for 2D NQR diagnostics.
`PolarizationEnhancedNQRSample`	Sample and spin parameters for polarization-enhanced NQR transport.
`CylindricalSampleGeometry`, `LinearTransportMotion`	Detector-volume and conveyor/translation parameters.
`HalbachPrepolarizationMagnet`	Links a sampled Halbach dipole map or constant field to the polarization-transfer model.
`ideal_spin1_enhancement_factors`	Glickstein–Mandal spin-1 enhancement factors from B_{p}, T, and NQR line frequencies.
`simulate_adiabatic_polarization_transfer`	Compute trajectory fields, adiabaticity, residence loss, and ideal transferred NQR signal gains.
`load_cif_structure`	Read fractional coordinates, lattice constants, and symmetry operations from a CIF file.
`dipolar_coupling_hz`	Point-dipole proton–quadrupolar-nucleus coupling estimate for a specified separation and angle.
`estimate_proton_dipolar_couplings`	Find nearby protons around a quadrupolar nucleus in a loaded CIF structure.
`estimate_proton_dipolar_couplings_from_cif`	Load a CIF file and summarize the nearby-proton dipolar coupling scale.

14.16 Optimization API

```
from spin_dynamics.optimization import (
    spa_pulse_list,
    rectangular_refocusing_lengths,
    evaluate_tuned_refocusing_pulse,
    evaluate_untuned_refocusing_pulse,
    evaluate_matched_refocusing_pulse,
    evaluate_ideal_v0crit_refocusing_pulse,
    evaluate_ideal_v0crit_excited_refocusing_pulse,
    evaluate_ideal_time_varying_refocusing_pulse,
    optimize_tuned_refocusing_phases,
    optimize_untuned_refocusing_phases,
    optimize_matched_refocusing_phases,
    optimize_ideal_v0crit_refocusing_phases,
    optimize_ideal_v0crit_excited_refocusing_phases,
    optimize_ideal_time_varying_refocusing_phases,
    run_tuned_refocusing_multistart,
    run_untuned_refocusing_multistart,
    run_matched_refocusing_multistart,
    run_ideal_v0crit_refocusing_multistart,
    run_ideal_v0crit_excited_refocusing_multistart,
    run_ideal_time_varying_refocusing_multistart,
    evaluate_tuned_excitation_pulse,
    evaluate_tuned_inverse_excitation_pulse,
    optimize_tuned_excitation_phases,
    optimize_tuned_inverse_excitation_phases,
    run_tuned_excitation_multistart,
    run_tuned_inverse_excitation_multistart,
```

```
    run_tuned_excitation_inverse_pipeline,
    multistart_to_matlab_results,
    save_multistart_results_npz,
    load_optimization_results,
    summarize_matlab_results,
    select_matlab_result_program,
    analyze_matlab_optimization_results,
    analyze_tuned_inverse_result_pair,
)
```

The optimization layer is useful for compact pulse-design studies and MATLAB-style result inspection. Broad historical `fmincon` parity and strong inverse-excitation cancellation parity remain validation targets.

14.17 Optimal Control API

```
from spin_dynamics.optimal_control import (
    ControlBounds,
    ControlHamiltonianModel,
    GrapeMultiStartResult,
    GrapeOptimizationResult,
    amplitude_phase_bounds,
    assemble_control_bounds,
    average_gate_fidelity,
    bloch_vector_to_ket,
    constant_gradient_seed,
    coupled_spin_control_model,
    eddy_terms_from_step_response,
    export_to_pulse_shape,
    grape_optimize,
    grape_optimize_phase_only,
    gradient_bounds,
    gradient_control_operator,
    make_grape_objective,
    nqr_fundamental_states,
    nqr_powder_control_batch,
    nqr_site_control_model,
    phase_only_bounds,
    position_gradient_batch,
    random_phase_starts,
    rectangular_seed_phase,
    robust_ensemble_fidelity,
    run_grape_multistart,
    state_transfer_fidelity,
    su2_effective_axis,
    # hardware response (probe / gradient driver)
    TunedProbeResponse,
    MatchedProbeResponse,
    UntunedProbeResponse,
    GradientDriverResponse,
    ReceiverResponse,
    TailSpec,
    # PGSE diffusion
    gradient_moments,
```

```
    detected_echo_signal,
    detected_echo_snr,
    make_pgse_objective,
)
```

`coupled_spin_control_model` builds the fixed `ControlHamiltonianModel` (drift plus transverse RF-drive operators) from an existing `coupling.systems.CoupledSpinSystem`; pass `include_gradient=True` to also populate the gradient control operator. Everything beyond Hamiltonian construction requires the optional `jax` extra – reverse-mode autodiff is the entire point of GRAPE, so no finite-difference fallback is provided. `grape_optimize_phase_only` is the primary entry point (fixed amplitude, phase-only control); `grape_optimize` generalizes to joint amplitude+phase control given a peak-B1 bound (`amplitude_max_hz`) and, via `gradient_operator_batch/optimize_gradient`, to the gradient control channel (`gradient_control_operat` `position_gradient_batch`, `constant_gradient_seed`, `gradient_bounds`). `nqr_site_control_model` builds the model for a quadrupolar (NQR) site in the rotating frame (Section 11.22.6), with `nqr_fundamental_states` and `nqr_powder_control_batch` for the inversion endpoints and the per-orientation drive operators; NQR optimizations must pass `propagator="expm"` because the Kramers-degenerate spectrum makes the default `eigh` gradient undefined. `run_grape_multistart` is not optional in practice: GRAPE landscapes are multimodal, so a single-start result is not a trustworthy optimum on its own. The hardware-response classes (`TunedProbeResponse`, `MatchedProbeResponse`, `UntunedProbeResponse`, `GradientDriverResponse`) fold finite probe bandwidth and gradient slew/eddy dynamics into the optimization as LTI actuators (Section 11.22.2); pass them as `rf_response` / `gradient_response` to `grape_optimize`. `ReceiverResponse` is the output-side filter for detected-signal objectives, and `eddy_terms_from_step_response` fits eddy terms from a measured step. `make_pgse_objective` (with `gradient_moments`, `detected_echo_signal`, `detected_echo_snr`) is the PGSE diffusion objective co-optimizing RF phase and gradient waveform for correct q-vector and detected SNR on a tuned probe with eddy currents (Section 11.22.3).

15 Known Gaps

The remaining work is now more architectural than a simple MATLAB-port backlog. The package has broad component coverage; the highest-value improvements make those components compose predictably and make the evidence behind a result easy to inspect.

In this chapter. Understand the present boundary of the public workflows and the specific extensions that remain future work.

Sequence execution. The IR imports, exports, compiles, visualizes, and executes Pulseq core events through the facade on the ideal moving-isochromat engine. Probe-aware and quantum backends still need explicit compiler targets. Pulseq extensions, explicit non-default time shapes, scanner safety limits, and vendor conversion remain outside the executable subset.

Facade breadth. The facade covers the main CPMG, imaging, PGSE, NQR, and pulsed-ESR routes, including random-walker flow. Specialized diffusion, exchange, prepolarization, thermal, RFI, optimization, and coupled-spin workflows still use their direct APIs. They should be added only when their planning rules and provenance fields are meaningful, not as thin name aliases.

Composition and adapters. The shared `spin_dynamics.composition` layer now adapts field solutions, thermal states, flow fields, hardware responses, and compiled sequence timelines onto named SI spatial grids and explicit absolute time axes. It validates channel units and performs spatial and time resampling. Backend-specific execution adapters and richer two-way feedback loops remain future work.

Validation evidence. Broader high-Q matched-probe fixtures, historical MATLAB `.mat` parity, optimization comparisons, paper-level scalar-coupling checks, and experimental NQR/ESR fixtures remain valuable. Validation should be reported by level: unit invariant, analytical limit, reference-code parity, external solver, or experimental comparison.

Provenance and reproducibility. Saved facade runs capture specifications and results, but richer environment, fixture, calibration, input-file digest, and backend metadata would make long-lived studies easier to audit.

Performance and scale. NumPy, Numba, JAX, batching, and benchmark tools exist, but coverage is uneven. Large sparse coupled-spin systems, full 3-D transport/thermal solves, CFD-derived flow, and multi-GPU execution are not general package capabilities.

Scientific scope. Relaxation-aware dense coupled-spin sequence runners, full nonselective quadrupolar propagation, anisotropic multi-electron ESR, and coherent many-spin dipolar networks remain longer-term physics extensions.

New numerical work should first state the model boundary and validation level, then add a small NumPy reference path and fixture or analytical limit. Optional acceleration, facade exposure, visualization, and interchange should follow once the behavior is pinned down.